Browsing and Searching Compressed Documents

Raymond Wan

Department of Computer Science and Software Engineering
The University of Melbourne

Submitted in partial fulfilment of the requirements of the degree of
Doctor of Philosophy (with coursework component)

December 2003

Abstract

Compression and information retrieval are two areas of document management that exist separately due to the conflicting methods of achieving their goals. This research examines a mechanism which provides lossless compression and phrase-based browsing and searching of large document collections. The framework for the investigation is an existing off-line dictionary-based compression algorithm.

An analysis of the algorithm, supported by previous work and experiments, highlights two factors that are important for retrieval: efficient decoding, and a separate dictionary stream. However, three areas of improvement are necessary, prior to the inclusion of the algorithm into a browsing system.

First, in order to accommodate retrieval, the algorithm must produce a dictionary built up on words, rather than characters. A pre-processing stage is introduced which separates the message into words and non-words, along with word modifiers.

Second, the memory requirements of the algorithm prevent the processing of large documents. Earlier work has proposed a solution which separates the message into individual blocks prior to compression. Here, a post-processing stage is proposed which combines the blocks in a series of phases. Experiments show the trade-offs between the number of phases performed and the improvements in compression levels.

Organisations which provide access to an information retrieval system to users are sometimes concerned with the amount of disk space available. But users have a different point of view and may place response time at a higher priority. Indeed, faster computers and network connections translate into plummeting patience levels of users. The last improvement to the compression algorithm is two new coding schemes which replace the entropy coder that was used in previous work. While deploying them sacrifices compression effectiveness, these two mechanisms offer improved efficiency, as shown through experiments.

With the enhancements to the compression algorithm in hand, a technique to efficiently support phrase browsing is presented. Phrase contexts can be searched and progressively refined through the word modifiers. Because of the three changes to the algorithm, phrases are more visually appealing, larger documents can be processed, and response times are improved.

Declaration

This is to certify that

1. the thesis comprises only my original work towards the PhD except where indicates in the Preface,
2. due acknowledgement has been made in the text to all other material used, and
3. the thesis is less than 100,000 words in length, exclusive of tables, maps, bibliographies, and appendices.

Raymond Wan
BSc (University of British Columbia, 1997)

Preface

Publications arising from this thesis.

The general concept of phrase browsing documents compressed with character-based RE-PAIR was introduced as a poster presentation at the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval [Wan and Moffat, 2001b], and then subsequently expanded for the 2001 Symposium on String Processing and Information Retrieval [Moffat and Wan, 2001].

Merging compressed blocks, as covered in Chapter 5, was presented at the 2002 Symposium for Combinatorial Pattern Matching [Wan and Moffat, 2002].

The results from the experiments of Chapter 8 on compression with HTML hyperlinks was presented at the 2001 Australasian Database Conference [Wan and Moffat, 2001a].

Acknowledgements

As expected, completing work of this magnitude is due to the support of many people. As I didn't know anyone in Australia before I arrived here for study, there are a lot of relationships I am thankful for.

The continued support over the years from my supervisor, Alistair Moffat, has made much of this thesis possible. I began this degree in the hope of learning, and fortunately, Alistair has made sure of that from the first day until the last.

The past and present members of Alistair's group have contributed to an excellent support network, and it has been a pleasure to be a part of it. Tim Bell and Owen de Kretser helped me from the very beginning as I struggled with the transition back into university life. Tim's continued maintenance of the "vi" machines, and calmness whenever I brought them to a grinding halt, is also very much appreciated. Encouragement from Anh Ngoc Vo over the years has certainly kept me going from day-to-day. Mike Ciavarella, Mike Liddell, Yugo Isal, Lily Sun, and Tony Wirth have further contributed to my personal life and my studies.

I'm also fortunate to have worked with (or next to) many other people in the department. Hongjian Fan, Kevin Glynn, Bernard Pope, Robert Shelton, Qun Sun, Xiuzhen (Jenny) Zhang, are only some of the people who have shared an office with me at some stage – four offices in all, spread across two buildings! Inga Sitzmann and Lei Zheng also helped complete my network of postgraduates within the department, while working with Linda Stern on several occasions as a tutor has always been both enjoyable and enriching.

Glen Gibb and Leslie Young have been great friends during my stay in Melbourne, and through them, I have met many others. The love and support given to me from Ju Hyung (Juliet) Lee has only grown since we first met. During that time, she's been in the unfortunate position of bearing the brunt of the stress, and I thank her for that.

I'm also thankful to many other friends who live overseas, and which I've had little opportunity to meet while living here. Wilson Lee has always been a close friend, and his advice is something which I've depended on over the years. Joyce Cunningham and Yoshiko Yoshimura provided the motivation to consider further study, while Ed Knorr has always been a mentor to me. Ed's suggestion to do a research degree instead of a coursework one because "I won't regret it" was what started everything here. Now that I'm at the end, I can say that he was right after all.

And last, but most of all, to my parents, for their support and caring. Their patience while their son studied on the other side of the world is much appreciated, as is everything else they've done for me.

Contents

List of Figures

List of Tables

List of Algorithms

Chapter 1

Introduction

I find bookstores irresistible. Recently, a new one opened near my home and while this store was no different from others already in the neighbourhood, I ended up spending half an hour wandering through its bookshelves. I didn't purchase a book that day, but the visit was not wasted. On my next trip, I was in search of a particular title. By then, I knew that it would have the book I wanted, and also its most likely location. I was correct, and ended up purchasing the book with relative ease.

These two separate trips to the bookstore illustrate two methods of approaching a complex object: *browsing* and *searching*. The Australian Concise Oxford Dictionary (third edition) defines browsing as, "[to] read desultorily (going constantly from one subject to another, especially in a half-hearted way)". While this definition resembles my first visit to the bookstore, it is not a complete description of my action. Without a doubt, if I constantly went from one bookshelf to another, I would have received some suspicious looks from the staff. Moreover, *browsing* a store, whether it is a bookstore or a clothing store, may not have anything to do with reading. Intuitively, browsing can also be defined as casually looking at various facets of an object in order to get an overall impression of what it represents. Surprisingly, this definition resembles the one for "browser", also from the same dictionary: "*(Computing)* a software program that enables one to search for and access documents on the World Wide Web". In contrast to browsing, *searching* requires a concise specification of what is being sought; for a book, this could be an author's name, a book title, or a subject area.

As a second example of searching and browsing, Figure 1.1 depicts the two main ways of approaching a university library. Along the top of the figure are the browsing options available to students on their first visit. Some libraries provide a building directory, either as a sign at the entrance, or in the form of a pamphlet. The directory may contain a map, as is the case in the figure, with words indicating the different sections of the library. After looking at the directory, students may choose to browse the bookshelves until finally arriving at a book that piques their interest. Alternatively, the student may arrive at the library because a book is required for an assignment. The assignment's guidelines usually provide information such as a catalogue number or a book title. This information can be entered as a query into a library catalogue interface, similar to the one shown at the bottom of the figure. A set of results is returned by the system in the form of a list of books that satisfy the query. The list may lead the student to the book required, or if that particular book is not available, the student may choose to browse the bookshelf where the subject is covered.

Note that the figure has been somewhat simplified. For example, after the catalogue at the University of Melbourne[1] returns a set of results, a library patron may select a book followed by the option labelled, "Show Items Nearby on Shelf". Books that are physically near the selected book on the bookshelf are displayed to the user in a way that resembles browsing. Searching and browsing are so tightly entwined, we may not realise which action is being performed. A student entering a library, with or without an assignment to do, may jump back and forth between these two methods during a single library visit without noticing. Moreover, Figure 1.1 implies that once a book has been found, the student's retrieval task has been completed. In fact, both browsing and searching occur at multiple levels, and continues after the book of interest has been located. The table of contents and index both provide support for searching, while any part of the book, from the front cover to the brief description on the back, offer chances for browsing.

As for the library's directory, interesting problems are faced when designing it. Including too little or too much information would make the directory either unhelpful, or

[1] `http://cat.lib.unimelb.edu.au/`

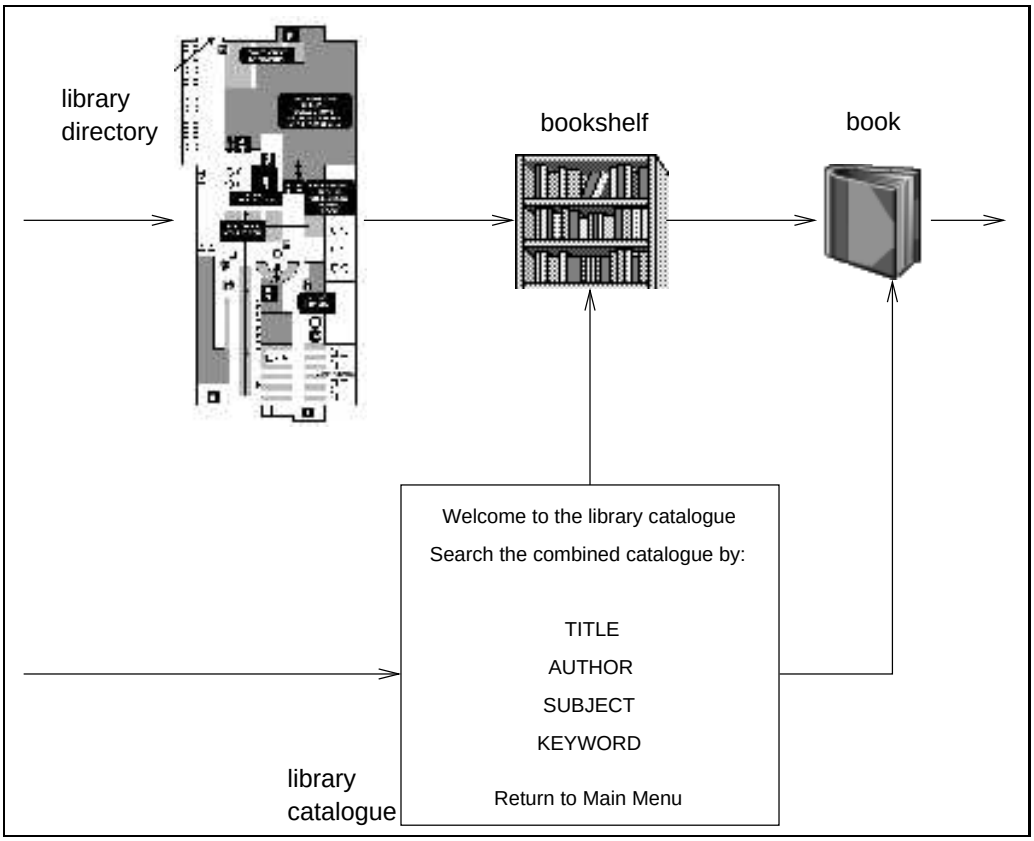


Figure 1.1: The options available to a library patron, with the end result being a particular book. The different levels of browsing are shown along the top, while a search window for a library catalogue is displayed at the bottom.

too cumbersome for library customers. A balancing act is required between the needs for information, and the need to avoid information overload.

The main investigation in this thesis is to explore the compromises required to permit both retrieval and compression of documents when stored in a computer. As with the library's directory, these tasks are in tension. Documents occupy disk space and network bandwidth which need to be minimised for storage or transmission efficiency. Compression is applied to achieve this. However, after a certain point, the document becomes too compressed to offer any information for retrieval. The only solution, then, is to completely decompress the document. Retrieval and compression are elaborated in the following sections.

1.1 Information Retrieval

Information retrieval (IR) is the study of methods for obtaining information from a repository. The repository may contain a single document, or a collection of documents, and can be managed in a variety of ways. The quality of the documents stored might range from a hastily-written memo, to important contracts, and possess a range of requirements for authentication and archiving.

The World Wide Web [Berners-Lee, 1996] is perhaps the largest document collection in existence at the moment, built without any rules for publication. Since almost anyone can create a web page (and probably has!), the quality of the information varies from outright lies to high-quality journal articles. Despite this anarchic nature, the Web is nevertheless an extraordinarily rich source of information.

Another method of obtaining information is through digital libraries [Witten and Bainbridge, 2002], which vary in size, but are typically much smaller than the Web. Just as in a physical library, a digital library is governed by guidelines which specify an acquisition policy, a method of ensuring longevity, and a comprehensive index. Because of their low cost of duplication (for example, on CD), digital libraries of this form offer the possibility of becoming an enormously useful resource in societies without access to conventional libraries.

Both of these examples are applications of IR techniques, and the document repository that lies behind an IR system can vary greatly in size and in the method with which it has been assembled. Perhaps the most common type of repository is a homogeneous collection of documents that might even be physically stored as a single large file. This is the model used in this thesis, a structure that is elaborated on in Chapter 2.

The interactions between a user and a document repository are illustrated in Figure 1.2. At first, the user has an information need, which is entered as a query via the retrieval system's interface. The interface sends the query to the retrieval system for processing. The data repository is accessed and the results from the query are returned by the system for display within the interface seen by the user. The result is usually a summary of the document, or document sections, which satisfy the query. But the actual relevance of the

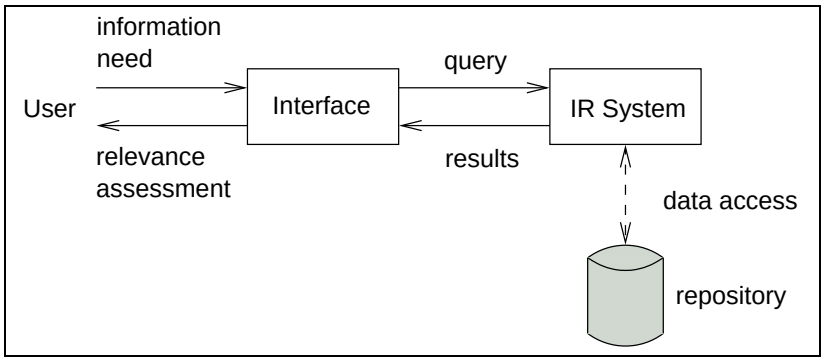


Figure 1.2: Information flow between a user and a repository maintained by an IR system.

result to the original information need of the user is something that only the user can judge. To assist the user to make that decision, the IR system may employ heuristics that attempt to highlight the most likely relevant results, for example, by sorting. The suitability of the results depends on the design of the system, as well as how the information need was represented in the interface.

The quality of a retrieval system is measured in terms of efficiency and effectiveness [Witten et al., 1999]. The efficiency of a retrieval system refers to the time and hardware resources required to provide a response to the user. Effectiveness refers to the accuracy, or correctness, of the result.

If the query is in the form of a single word or a precise phrase, a string matching algorithm such as Boyer-Moore [Boyer and Moore, 1977] can be used to locate sections of documents where that word or phrase appears. The search is performed sequentially from the beginning of each document. The GREP system [Kernighan and McIlroy, 1979] for UNIX operating systems can be used to search across whole directories of documents, and the "search" feature in a word processor is an example of sequential searching within a document.

On the other hand, if a collection of documents needs to be accessed with a variety of queries, information retrieval techniques are required. Data structures should be created to provide efficient, repeated searching. Some facility may also be required to store the data in the repository, while providing simultaneous access from multiple users. Examples of such systems include an encyclopedia CD ROM with a built-in "find" dialog box, or a

search engine for the Web. The purpose of a search engine is to assist users in locating web pages which satisfy their information need. In this scenario, the query is a set of words (or terms) and the result is a list of web pages which the system believes satisfies the query.

Traditional IR systems provide two main methods of querying. A retrieval system that supports *Boolean queries* retrieves documents based on the presence or absence of query terms. Alternatively, a *ranked query* system returns a list of results, ordered according to a heuristic similarity score that is based on the number of query terms that appear in documents, and the frequency with which they appear in that document and in the collection as a whole. Instead of sequentially searching the entire repository for each query term given, these two retrieval techniques are usually implemented using data structures which index the collection. More detailed descriptions of retrieval systems and their implementation can be found in the books of Frakes and Baeza-Yates [1992], Korfhage [1997], Baeza-Yates and Ribeiro-Neto [1999], and Witten et al. [1999]. Recent books which provide more information about digital libraries include Lesk [1997], Witten and Bainbridge [2002] and Borgman [2003].

As with the bookstore and the university library, searching alone may be insufficient to meet all needs, and providing the user with multiple tools can only aid the searching experience. One additional tool that helps is browsing [Palay and Fox, 1981, Baeza-Yates and Ribeiro-Neto, 1999]. In a document repository, the *lexicon* is an ideal candidate for browsing [Kowalski, 1997]. The lexicon is the list of all words found in the collection. An extension to lexicon browsing is *phrase browsing*, where the dictionary used for perusal consists of phrases in the repository, rather than just individual words.

The extraction of phrases from a document can be performed in many ways. The simplest, yet most tedious approach, is to manually select key phrases. If computers are used, then two main methods exist. First, phrases can be formed statistically, based on frequency of word combinations. Second, phrases can be chosen through semantic methods with the help of natural language processing techniques. Wacholder and Nevill-Manning [2001] provide an overview of these and other methods of phrase selection and phrase

browsing, and Chapter 7 considers phrase browsing in greater detail, as well as the related work in the area.

Browsing a document using phrases drawn from it is one of the two topics examined in this thesis. The second is text compression.

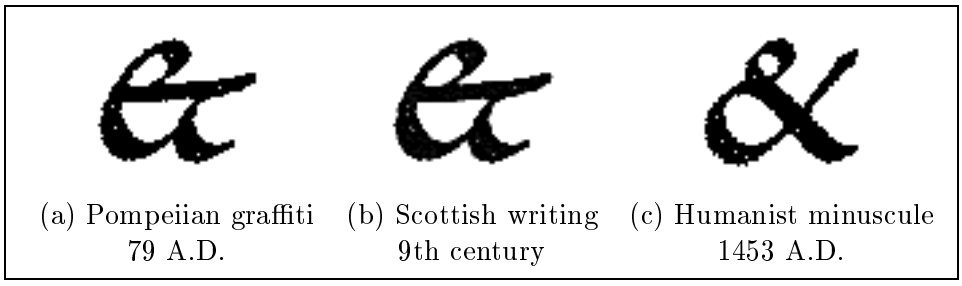
1.2 Text Compression

Text compression is the process of identifying and representing the redundant information in a document in a more economical way, so as to minimise the storage space or transmission cost and time.

The desire to find ways to minimise the amount of writing required to speed message transmissions has been around for centuries in the form of abbreviations and symbols. Shorthand, the art of rapid writing using abbreviations, has been used as far back as 2,000 years ago when debates were recorded within the Roman Senate during the Roman Empire. Around that time, Tiro developed a method of shorthand, and is believed to have invented the ampersand (&) [Kreitzman, 1999], a symbol commonly found today in informal text. Attention was drawn to the symbol during Medieval times when monks used it as a sophisticated substitute for the Latin word, "et", meaning "and" [Todd, 2001]. The symbol is derived from the ligature of the letters "e" and "t", as shown in Figure 1.3. Moreover, the phrase "et cetera", which means "and similar things or people", is often abbreviated now as "etc.", but was further shorted to "&c" up until the end of the 19th century, according to Todd. The word "ampersand" comes from the contraction of the expression "and per se = and", whose literal meaning is: "& by itself is and" [Moore, 1997].

Another symbol found in the English language is the ditto mark (″). The word ditto comes from the Tuscan dialect of Italian, and has been a part of the English language since the seventeenth century [Todd, 2001]. The symbol is used in informal writing as a way to avoid writing the same word or phrase multiple times.

Generally, a fundamental requirement of abbreviations and symbols is to be able to reproduce what they represent – during the Roman Empire, one method was to have



(a) Pompeiian graffiti 79 A.D. (b) Scottish writing 9th century (c) Humanist minuscule 1453 A.D.

Figure 1.3: Changes to the ampersand through time. Figures and captions taken from Caflisch [2003].

multiple people record a dictation and then consolidate the differences afterwards [Pitman, 1891]. In the two millennia since the use of shorthand by the Romans, the compression field has evolved into a highly-studied discipline, primarily because of computers. Manual shorthand is limited to the speed at which text can be spoken. Its purpose is to increase the bandwidth of the person talking, at the cost of writing out the text in full later. Computers have allowed complex methods of compression to be used on large documents at very high speeds for both efficient transmission and storage. Compression with computers is not only fast in comparison, but also less error prone than shorthand. Compression algorithms differ in many ways, including the speed at which they process text and how effectively they can compress a document. Books which cover the different approaches in the field of compression include Storer [1988], Bell et al. [1990], Nelson and Gailly [1996], Sayood [1996], Witten et al. [1999], Salomon [2000], and Moffat and Turpin [2002]. A more detailed discussion of compression is provided in Chapter 2.

1.3 Browsing and Compression

Browsing and compression are two key areas of text management, but serve different purposes. The aim of browsing is to present information about a document in order to help satisfy a user's information need. The purpose of compression is to minimise the size of a document prior to storage or transmission. These two goals are not necessarily compatible with each other. The more compressed a document is, the more difficult it is to search it without completely reversing the transformation. These two areas of text

management are in tension, and the challenge is to find a useful balance point between them.

The aim of this thesis is to investigate the relationship between compression and retrieval, in the context of a homogeneous document collection, and to develop principles that demonstrate how a compromise between the two can be achieved. The emphasis is on phrase browsing, so a more precise objective of this thesis is to consider how a document can be compressed so that phrases drawn from it can still be browsed, and then used as the basis for the retrieval of documents and document fragments. The system described here builds on a compression mechanism called Re-Pair [Larsson and Moffat, 2000]. This thesis investigates aspects of the Re-Pair algorithm that need to be improved in order to provide feasible browsing.

1.4 Structure

This thesis is structured as follows. Chapter 2 presents an overview of compression, beginning with some definitions from information theory, and followed by an explanation of how a compression system is composed of a model, a probability estimation process, and a coder. Algorithms representing these three parts are shown, as well as implementations of the compression systems that are used in this thesis for experimental comparisons. A description of the test data for experiments throughout the thesis is also provided.

The Re-Pair compression process is explained in Chapter 3. Re-Pair is a dictionary-based compression algorithm which builds a collection of phrases [Larsson and Moffat, 2000]. Experimental results are presented that validate the analysis of Larsson and Moffat, and demonstrate Re-Pair's compression efficiency and effectiveness. While previous work has looked at the time and memory space requirements of Re-Pair in the context of the data structures employed, the work in Chapter 3 extends this analysis by considering problems with the algorithm which become evident when compression is not the only goal. An overview of four proposed changes to the algorithm is given, as well as the reasons for choosing Re-Pair for phrase browsing, despite the problems noted. The four changes are explained in detail in the four following chapters.

The original RE-PAIR algorithm selects phrases based on symbol pair frequencies. In Chapter 4, three new methods of selecting phrases for RE-PAIR are given as alternatives to the one used in the implementation of Larsson and Moffat [2000]. The addition of these alternatives provides better quality phrases for phrase browsing, while ensuring that they are properly aligned on word boundaries. Experimental results show the difference in compression effectiveness and the improvement in the types of phrases available to the user.

The amount of memory used by RE-PAIR limits the size of the documents it can process. The solution employed by Larsson and Moffat [2000] is to partition large documents into blocks and then process each block independently. Unfortunately, this approach separates the dictionary into disjoint components, making it difficult to browse as a whole. Chapter 5 proposes a block merging scheme whose primary goal is to combine the separate dictionaries into a single one. As a secondary goal, the block merging scheme improves the compression effectiveness of RE-PAIR by removing redundancies between blocks. Experimental results are given that show the usefulness of this technique.

The coder accompanying the RE-PAIR algorithm is equally important for efficient browsing, and is the subject of Chapter 6. Since the original intention of RE-PAIR was good compression effectiveness, Larsson and Moffat [2000] selected an entropy coder to couple with the parser. However, a static coder that sacrifices some compression effectiveness for retrieval efficiency has the potential to reduce the amount of waiting time experienced by a user and support searching operations better. An experimental comparison between the new coder proposed and a standard Huffman coder concludes the chapter.

The last enhancement to RE-PAIR is the addition of a phrase browsing interface and a random access decoding tool, described in Chapter 7. While Nevill-Manning et al. [1997] shows how a phrase browsing mechanism can be based on the compression algorithm SEQUITUR [Nevill-Manning and Witten, 1997], inherent differences between the two compression algorithms, coupled with the choices made in the previous chapters yield a system with different capabilities. This chapter describes the tool and the interface which allow

the user to browse phrases and, when a phrase of interest has been found, quickly access the contexts in which the phrase occurs. An analysis of the phrase browser is conducted with respect to efficiency, effectiveness, phrase quality, and related work. The SEQUITUR algorithm is described earlier in Chapter 3.

Chapter 8 considers a problem which is separate from the work covered in previous chapters. Instead of large document collections, this chapter investigates how compression can reduce bandwidth requirements for interactive web browsing. Information contained in web server logs are exploited in order to provide an experimental framework based on the HTML files.

Finally, Chapter 9 concludes this thesis by returning to its main theme. This final chapter looks at how RE-PAIR and the additions to RE-PAIR combine to form a phrase browsing system for homogeneous document collections of up to a gigabyte or more. Future directions for this work are also suggested.

To complete the thesis, Appendix A lists notation and Appendix B describes the Porter stemming algorithm, employed in Chapter 4.

Chapter 2

Text Compression

Compression is the fundamental area of text management which recognises and removes redundancies, and represents the remaining content in a more economical way so that the underlying information is retained. The compressed document is then transmitted or stored for later use. The receiver of the compressed document must reverse the transformation in order to obtain the original. The reduction in document size usually comes at the expense of another factor, such as processing time for the conversion.

This chapter provides some background information on compression, and is structured as follows. Section 2.1 presents fundamental theory which forms the foundation of all compression methods. Section 2.2 gives an overview of compression systems, and how they are divided into its three parts: model, probability estimator, and coder. Section 2.3 describes models and probability estimators in the context of three classes of algorithms prevalent in text compression: dictionary-based, statistical, and block sorting. Section 2.4 gives insight into static and entropy coding techniques. The focus of this chapter shifts to practice in Section 2.5, when some complete compression systems are described. Details of the framework for the experiments performed throughout this thesis are covered in Section 2.6. Finally, this chapter concludes in Section 2.7 with a discussion of the effect compression has on searching.

2.1 Fundamentals in Compression

Some definitions and background about the area of information theory as the foundation for compression are required as a first step. We suppose that a *message* is created by some *source*. Whether a message is created for transmission or storage for archival purposes, its recipient is the *receiver*. The *channel* is the medium between the source and the receiver, and can be a communications line or a disk drive. Between the source and the channel is the process which performs compression: the *compressor*. Its counterpart is the *decompressor*, employed by the receiver to reconstruct the message.

The smallest unit in the message is a *symbol*. The set of all possible symbols available to the source for building messages is the *alphabet*, denoted as Σ. The symbols in the alphabet depend on the source and the type of messages being transmitted. For example, if the source only produces telephone numbers, then an alphabet which includes the 10 digits and the hyphen is sufficient. The size of this alphabet (denoted as $|\Sigma|$) is 11. In this thesis, all messages are encoded using the Latin-1 character set [International Organization for Standardization, 1998], an extension to the ASCII character set [American National Standards Institute, 1997]. The ASCII alphabet consists of 128 symbols, including the 26 letters of the English alphabet in both cases, the 10 digits, and a range of punctuation marks and non-printing characters. The Latin-1 alphabet adds an additional 128 symbols to ASCII, to permit the encoding of text in most western European languages. The symbols in a message encoded with Latin-1 are also called *characters*.

Each symbol in a message conveys some information to the receiver. The amount of information conveyed depends on the probability of the symbol occurring in the message. Since a highly probable symbol offers little surprise to the receiver, the information content is small. On the other hand, an unlikely symbol has a high information content. If α represents a symbol in the alphabet, then the relationship between its information content, $I(\alpha)$, and its probability, $P(\alpha)$, was defined by Shannon [1948] as:

$$I(\alpha) = -\log_2 P(\alpha)\,. \tag{2.1}$$

In Equation 2.1, any base can be selected for the logarithm. However, since compression is usually performed with computers based on the binary counting system, a base of two is a convenient choice, and the information content is measured in units of bits. For example, in a message representing a sequence of throws of a six-sided dice, the probability of any value appearing is 1/6. The information content of each of the 6 events is $-\log_2(1/6) = 2.58$ bits.

If the information content of all of the symbols in some alphabet Σ are averaged, then the *zero-order entropy* of the source is obtained [Shannon, 1948]:

$$H = \sum_{\alpha \in \Sigma} P(\alpha) \cdot I(\alpha) = -\sum_{\alpha \in \Sigma} P(\alpha) \cdot \log_2 P(\alpha)\,. \tag{2.2}$$

If the probabilities are derived from a specified message, rather than the source, then the quantity described in Equation 2.2 is the per-symbol zero-order *self-information* of the message. Shannon demonstrated that the entropy is the lowest limit (in bits per symbol) with which a message from the source can, on average, be coded.

The definitions for information content and entropy depend on two factors. First, they assume that the probability of the symbols in the alphabet can be obtained. While other definitions of entropy exist, this definition assumes a Markov source that generates the symbols in a message independently and identically distributed. That is, the value of a symbol does not affect the values of neighbouring symbols. Of course, this is not true for the letters that appear in an English message, for example. Despite these requirements, the definition of entropy has served as a guide for work in compression for many decades. A more thorough coverage of information theory can be found in books such as Cover and Thomas [1991].

2.2 Compression Systems

Many types of compression systems exist in order to satisfy a range of needs. A system is *lossless* if the decoder is able to recreate the original message exactly. A system that does

not achieve this goal is *lossy*. Lossy compression is commonly found in the audio and video compression fields, since changes to the message that go unnoticed by the human senses are considered acceptable. Some work has been conducted in the area of lossy compression for text. Nielsen et al. [1997] considered the lossy compression of HTML documents by transforming them to improve bandwidth, provided they rendered correctly within a web browser. Katajainen et al. [1986] applied a syntax-directed compression scheme to Pascal source code. As a higher priority was given to the program source code, other free-form text such as comments and extraneous whitespaces were modified as necessary. Despite these counterexamples, text compression is usually taken to imply lossless compression, because a missing character or word from a sentence can greatly alter the meaning of the sentence.

Whether a compression system is tailored for audio, video, or text, it can usually be divided into three tasks: modelling, probability estimating, and coding [Rissanen and Langdon, Jr., 1981]. The model receives the message from the source and accumulates knowledge about it. The definition of entropy earlier was based on independently occurring symbols, which is not true for most messages. If the probability a symbol occurs is based on neighbouring symbols, it is during the first phase of the compression system that this relationship is captured. Next, the probability estimator assigns probabilities to symbols in the message to form the model, usually based on the observed frequencies derived from the part of the message processed. Finally, the coder maps the output of the probability estimator to a set of codewords based on the *channel alphabet*, which in this thesis is assumed to be binary.

As shown in Figure 2.1, these three components are situated before (and after) the channel, forming the compressor (and decompressor). The decompressor reverses the operations performed by the compressor to obtain the original message. In a compression system, these three parts may each exist in isolation, or be tightly bounded together, with no apparent division between them.

Each component of the compression system operates *statically*, *semi-statically*, or *adaptively*. A static algorithm makes use of fixed, pre-determined statistics to process

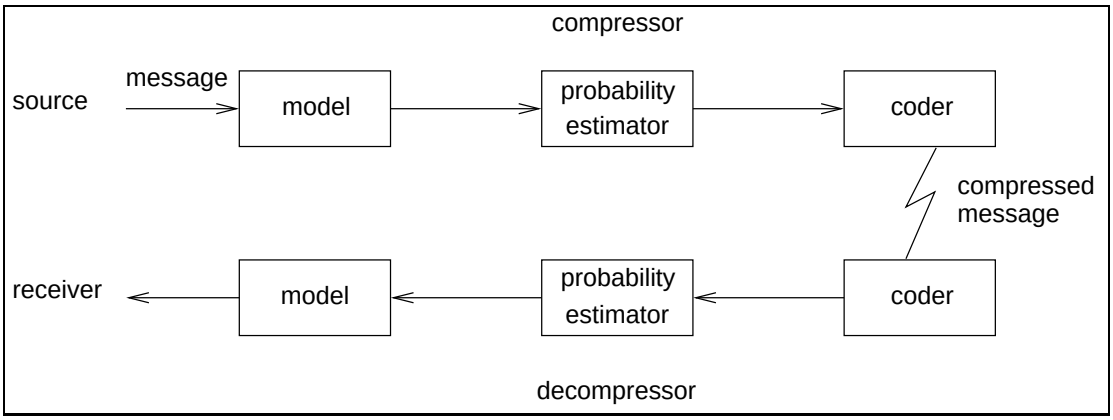


Figure 2.1: The separation of a compression system into a model, probability estimator, and coder.

the input. These statistics are presumed to be available at both ends of the channel, so that explicit transmission is unnecessary. The drawback to such a regime is that it is only suitable when the static probabilities match those found in the input. When there is a mismatch, compression effectiveness degrades.

On the other hand, an adaptive algorithm learns about the input throughout the compression process, and typically commences with bland statistics. As compression progresses, these statistics are updated, in an attempt to match those possessed by the message. Like static algorithms, adaptive ones have the advantage of requiring only a single pass. Furthermore, adaptive mechanisms are suitable for any type of message.

Semi-static algorithms encode the message in two passes. The first pass collects information about the message. That knowledge is then applied in the second pass. The primary difference between a static algorithm and a semi-static equivalent is that the information that exists on both sides of the channel in the static approach must be explicitly transmitted by the semi-static one. Thereafter, the process is the same.

Closely related to these three algorithm classifications are the categories *on-line* and *off-line*. An on-line algorithm processes a symbol, and then immediately commits it to the channel as a stream of bits before reading in the next symbol. An off-line mechanism has the luxury of examining a significant block of the input before sending anything to the decoder, at the cost of buffering symbols in memory.

```
how␣much␣wood␣could␣a␣woodchuck␣chuck␣
if␣a␣woodchuck␣could␣chuck␣wood␣
as␣much␣wood␣as␣a␣woodchuck␣would␣chuck␣
if␣a␣woodchuck␣could␣chuck␣wood
```

Figure 2.2: The "Woodchuck" message. A space is indicated by a "␣" character. Linefeeds have been inserted to improve readability.

In the next two sections, models and coders are discussed in greater detail through sample algorithms. Coverage of probability estimators is absorbed into the description of models, because of their close relationship.

2.3 Modelling

The model analyses and learns about the message, and in the process, identifies the redundancies that allow the message to be represented in an alternate, yet cost-effective manner. Three different models are considered in this chapter: dictionary-based, statistical, and block sorting.

A short passage has been chosen as the example for most of this thesis. The "Woodchuck" tongue twister from Opie and Opie [1959, pg. 30] is shown in Figure 2.2. All words in the message have been case folded, and all punctuation has been removed. In addition, all whitespace characters have been collapsed into a single space, as indicated in the figure as "␣". While linefeeds are used in the figure to improve display, they are not considered to be part of the message.

Dictionary-Based Models

Dictionary-based models (or *macro encoding* mechanisms [Storer and Szymanski, 1982]) seek to represent the input message in a more cost effective way by building a dictionary of phrases. Sections of the message are substituted by references to the dictionary's entries. Compression is achieved if the cost of expressing the dictionary is less than the total number of symbols it replaces.

Algorithm 2.1: Algorithm for LZ77.

input : input message $\mathcal{M}$, window size N
output : encoded message $\mathcal{M}'$

```
1  prev_offset ← 0; offset ← 0
2  i ← 0; ω ← λ
3  for i < |M| do
4      appendString(ω, M[i])
5      offset ← findLongestMatch(ω, M[i − N] ... M[i − 1])
6      if offset = 0 then
7          output ⟨prev_offset, |ω| − 1, M[i]⟩ to M′
8          prev_offset ← 0
9          ω ← λ
10     else
11         prev_offset ← offset
12     i ← i + 1
13 return (M′)
```

A static dictionary-based model would hard-code the phrases in the compressor and the decompressor. The dictionary can also be built semi-statically or adaptively so that each compressed message has an associated dictionary. In this case, either the dictionary must be explicitly sent to the decompressor, or enough information must be transmitted for the decompressor to reconstruct the dictionary. The overall effectiveness of a dictionary-based model depends on the frequency with which dictionary phrases occur, and their lengths.

This section focuses on two dictionary-based models, both of which operate adaptively and on-line. The first is the LZ77 algorithm by Ziv and Lempel [1977], which creates a dictionary by maintaining a window of recently seen symbols. Each symbol in the window is considered to be the beginning of a phrase of arbitrary length. Sequences (or strings) of symbols in the message are replaced by phrase references.

Algorithm 2.1 presents the algorithm for LZ77. The input and output messages are represented as $\mathcal{M}$ and $\mathcal{M}'$, respectively. The size of the window is denoted as N. As each symbol is read, it is appended to the string ω, which is initialised to the empty string λ at the beginning of the algorithm on line 2. On line 5, a search for ω is performed on the dictionary of phrases for the set of strings that have ω as a prefix. A match is indicated

by a distance offset backwards, counting from the end of the window. When ω cannot be found, an offset of 0 is returned from the search, and the string ω is encoded as a 3-tuple of the form $\langle d, l, s \rangle$. The location in the window of the last successful match is specified by d, while the length of this match (one less than the length of ω) is specified by l. The last symbol in ω that caused the mismatch is transmitted directly as s. As a special case, when the string ω only contains one symbol, α, the novel symbol is replaced with the tuple: $\langle 0, 0, \alpha \rangle$.

The size of the window is restricted to some value N, and as more of the message is processed, symbols at the beginning of the window are removed so that the window is kept recent. The eradication of old symbols and the addition of new ones give the illusion of a window sliding over the message. So, mechanisms based on LZ77 are sometimes referred to as "sliding window" compression schemes. For efficiency reasons, N is usually chosen as a power of 2.

Table 2.1 illustrates the LZ77 algorithm with an example. The sample text is formed from part of the "Woodchuck" message, "woodchuck␣chuck␣if". Each line in the figure indicates one 3-tuple. The dictionary window is indicated by a shaded box, while ω is placed in a framed box. The window is $N = 8$ symbols in length. In row (a) of the table, the window is initially empty, and the tuple $\langle 0, 0,$w$\rangle$ is transmitted, to get the first character through. Later, in row (c), the second occurrence of the letter "o" creates an ω two characters in size. At the second to last step in the table, ω is 7 characters in length and the longest match is 6 characters from the end of the window. At this stage, the first two characters in the message, "wo", have departed from the window.

In terms of memory, the LZ77 algorithm requires at least space proportional to N in order to hold the sliding window. However, since searching for ω in the dictionary can be the most costly operation in the algorithm, additional data structures are used to improve efficiency. Some basic data structures include sorted lists and binary search trees. Other possible mechanisms include tries (also called multi-way tries [Sedgewick, 1997]) and hash tables. A trie is a tree where each node has an outdegree equal to $|\Sigma|$. When searching for a string ω, reading each character results in one transition from the current node to one

	Tuple	Message
(a)	$\langle 0, 0, \text{w} \rangle$	woodchuck␣chuck␣if
(b)	$\langle 0, 0, \text{o} \rangle$	wo odchuck␣chuck␣if
(c)	$\langle 1, 1, \text{d} \rangle$	wood chuck␣chuck␣if
(d)	$\langle 0, 0, \text{c} \rangle$	woodc huck␣chuck␣if
(e)	$\langle 0, 0, \text{h} \rangle$	woodch uck␣chuck␣if
(f)	$\langle 0, 0, \text{u} \rangle$	woodchu ck␣chuck␣if
(g)	$\langle 3, 1, \text{k} \rangle$	woodchuck ␣chuck␣if
(h)	$\langle 0, 0, \text{␣} \rangle$	w oodchuck␣ chuck␣if
(i)	$\langle 6, 6, \text{i} \rangle$	wo odchuck␣ chuck␣i f
(j)	$\langle 0, 0, \text{f} \rangle$	woodchuck␣chuck␣i f

Table 2.1: Dictionary-based compression using the LZ77 algorithm. Each row illustrates the string ω being encoded and its associated 3-tuple. On the right, the shaded region is the current window, which grows until a fixed size and then "slides" across the text. The unshaded region is the section of text being processed (ω).

of its $|\Sigma|$ child nodes. A hash table can be implemented so that the first few characters of each phrase are used to calculate a position in the table. The data structure chosen must also take into account the fact that symbols leave the window as well as join it. So, phrases are continually added and removed from the data structure.

In comparison, the decoder is simpler than the compressor. While a sliding window must be maintained, string searching is not required. Each tuple in the compressed message indicates explicitly where in the window to copy phrases from, and the number of symbols to copy. Hence, decoding is extremely quick. In fact, the fast decoding time is a trait shared by most dictionary-based algorithms. Reconstructing the message by directly copying strings from the dictionary is attractive from a pragmatic point of view.

Some variations on the LZ77 algorithm include LZR and LZSS. The LZR scheme [Rodeh et al., 1981] removes the size of the sliding window so that all of the message seen so far is used as phrases. Bell [1986] designed LZSS to remove the restriction on constant tuple sizes by mixing the output stream with characters and back pointers. A bit flag was used to indicate whether a literal character or a back pointer exists in the stream. This work was taken further by Fenwick [1993], who investigated other lengths of bit flags.

The second dictionary-based algorithm discussed in this chapter is the LZ78 algorithm

Algorithm 2.2: Algorithm for LZ78.

```
   input   : input message M
   output  : encoded message M'
 1 prev_position ← 0; position ← 0
 2 i ← 0; ω ← λ
 3 for i < |M| do
 4     appendString(ω, M[i])
 5     position ← searchDictionary(ω, D)
 6     if position = 0 then
 7         output ⟨prev_position, M[i]⟩ to M'
 8         add ω to D
 9         prev_position ← 0
10         ω ← λ
11     else
12         prev_position ← position
13     i ← i + 1
14 return (M')
```

of Ziv and Lempel [1978]. Like LZ77, LZ78 is adaptive, operates on-line, and replaces sections of the message with pointers to the dictionary. However, rather than using a sliding window of symbols as a phrase book, an explicit dictionary is built. Algorithm 2.2 lists the algorithm for LZ78. As before, symbols are read one at a time and appended to the string ω. If ω exists in the dictionary D, then the next input symbol is considered. But if ω cannot be found, then it is replaced with a 2-tuple, $\langle p, s \rangle$. The last successful match for a phrase (that is, ω without its last character) is specified by p as a position in the dictionary. The last symbol in ω is encoded explicitly as a literal, s. Whenever a new phrase has been found, it is added to the dictionary on line 8 of the algorithm. Dictionary entries are indexed from 1 so that a new symbol, α, is encoded as $\langle 0, \alpha \rangle$. The dictionary is initially empty.

A sample application of the LZ78 algorithm on the same message as Table 2.1 is shown in Table 2.2. Each row in the table shows a section of text being encoded as a 2-tuple (indicated by the column marked "Tuple"). The position in the dictionary of each new string is shown in the next column, followed by the progress of the compression. The shaded region represents text that has already been seen, while the framed box draws

	Tuple	Current dictionary	Message
(a)	⟨0,w⟩	1 → w	woodchuck␣chuck␣if
(b)	⟨0,o⟩	2 → o	wo odchuck␣chuck␣if
(c)	⟨2,d⟩	3 → od	wo od chuck␣chuck␣if
(d)	⟨0,c⟩	4 → c	wood chuck␣chuck␣if
(e)	⟨0,h⟩	5 → h	woodc huck␣chuck␣if
(f)	⟨0,u⟩	6 → u	woodch uck␣chuck␣if
(g)	⟨4,k⟩	7 → ck	woodchu ck␣chuck␣if
(h)	⟨0,␣⟩	8 → ␣	woodchuck␣chuck␣if
(i)	⟨4,h⟩	9 → ch	woodchuck␣ch uck␣if
(j)	⟨6,c⟩	10 → uc	woodchuck␣ch uc k␣if
(k)	⟨0,k⟩	11 → k	woodchuck␣chuc k␣if
(l)	⟨8,i⟩	12 → ␣i	woodchuck␣chuck␣i f
(m)	⟨0,f⟩	13 → f	woodchuck␣chuck␣i f

Table 2.2: Dictionary-based compression using the LZ78 algorithm. Each row illustrates the string ω being encoded, with the generated 2-tuple in the first column. The position in the dictionary and the equivalent string of the new tuple is shown in the second column. At the end of each row is the progress of the algorithm. The shaded region indicates text that has already been processed, while the unshaded region is the section of text being processed (ω). Dictionary strings grow one character longer at each step.

attention to the string ω. Due to the short example, the message is encoded only as strings of length 1 or 2. Since the dictionary gradually grows, the repetition of the string "chuck␣" is not captured within the text shown, and more pointers are required than the equivalent LZ77 parsing. Also, in row (k), note that the letter "k" is encoded as a literal, even though it is the second time it has been seen. The first time it was encountered, it was part of a string of length 2.

In comparison to the LZ77 algorithm, LZ78 searches for phrases in an explicit dictionary, rather than a sliding window of phrases. The relevance of the dictionary to the text being compressed is no longer limited by a window size. Even so, in practice, a limitation on memory usage is imposed on LZ78 so that the amount of space is bounded. Also, while phrases are explicitly added into D, note that the addition of a phrase does not give assurance that it will be used.

The same data structures described for LZ77 can also be employed by LZ78 for main-

taining phrases. But since phrases are never deleted from the dictionary, one complication inherent in LZ77 is eliminated by LZ78. The LZ78 algorithm, though, requires the dictionary to be maintained by both the decompressor and the compressor. As a result, decoding with LZ78 is usually slower than LZ77.

In LZ78 implementations, several approaches exist when dealing with a full dictionary. First, the dictionary can be deleted, and re-constructed from scratch. Second, a least recently used policy can be employed, so that the oldest phrases are removed to make space for new ones [Tischer, 1987]. Alternatively, if every symbol in the alphabet is used to initialise the dictionary (such as the case with LZW by Welch [1984]), then it can be kept static for the remainder of the message. Because of this, the output from the LZW algorithm only contains pointers to phrases. Combinations of these three policies are possible; for example LZW rebuilds the dictionary if compression degrades significantly.

Statistical Models

Instead of building a dictionary, statistical models directly estimate the probability for each symbol in the message. As an example, suppose symbols are words and the following sentence fragment has to be completed with a word, "I purchased a book from the ...". Many plausible guesses exist, but most of them would be sources of books, such as "bookstore" and "Internet". This conclusion can be drawn from the information provided by the preceding words.

Generalising the problem to symbols, rather than words, statistical models combine with probability estimation modules in order to gather statistics and provide estimates for each symbol as it is encountered. Restricting the context to the preceding symbols permits messages to be processed in a single pass. An *order-K* context uses the previous K symbols to predict the current symbol. By definition, an order-0 context does not use any preceding symbols, while an order of minus-one assigns the same probability to every symbol. The Latin-1 standard can be thought of as a very simple compression system based on a fixed minus-one order context. An order-1 statistical model predicts each symbol in the context of one immediately preceding symbol. Intuitively, the higher the

order, the more accurately the model is able to predict symbols. However, if the size of the alphabet is $|\Sigma|$, then an increase in the context by one symbol results in an increase in memory usage by a factor of as much as $|\Sigma|$, and in general, for an order-K context, the amount of memory required is potentially proportional to $|\Sigma|^{(K+1)}$.

The Prediction by Partial Matching (PPM) algorithm [Cleary and Witten, 1984] is one example of a statistical model. In order to conserve memory, instead of creating all possible contexts initially, PPM records contexts as they are encountered in the message. If a symbol cannot be predicted in the current context, then the algorithm reduces the context by one symbol and tries again, until a context can be found.

Implementations of the PPM algorithm are typically parameterised in terms of the maximum order K. The encoding of a symbol α is first attempted using the previous $k = K$ symbols. If α has not occurred in this context, then a special *escape symbol* is transmitted to indicate that a context of length $k - 1$ should be attempted. This is repeated until α can be estimated based on a previously seen context, or when $k = -1$ and the symbol is encoded as a literal.

Several variations on the PPM algorithm exist, differing primarily in the probability assigned to the escape symbol. The probability given to the escape symbol is related to the *zero frequency problem* [Witten and Bell, 1991]. The zero frequency problem refers to the dilemma of assigning a probability to an event that has not occurred. Since there is a chance it could occur later, novel events that have not yet occurred must still be assigned non-zero probabilities.

With respect to PPM, the probability of the escape symbol is an instance of the zero frequency problem since the escape symbol is emitted when a symbol has not occurred in the current context. Several methods related to this problem have been explored. Method A of PPM increases the count of each context by 1 for the escape probability. Method B subtracts 1 from the counts of every context so that an event is considered to have occurred only if has happened twice. The remaining counts are allocated to the escape symbol. Methods A and B were described by Cleary and Witten [1984]. Method C [Moffat, 1990] assigns a probability of $d/(n + d)$, where d is the number of distinct

symbols seen in the current context, and n is the total number of symbols encountered in that context. Method D [Howard, 1993], estimates the probability of the escape symbol as $d/(2n)$. Implementations of PPM which make use of the latter two methods generally achieve better compression effectiveness than variants using methods A and B [Witten et al., 1999]. Cleary and Teahan [1997] looked at PPM with an unbounded context.

An example of the PPM algorithm is shown in Table 2.3. The sample message is again "`woodchuck␣chuck␣if`". A row in the table represents the processing of a character in the message. The message is in the last column, with the shaded region representing text that has already been seen. The framed box indicates the character under consideration. In this example, the maximum order is fixed at $K = 2$. So, an attempt is made to first predict each character using the two immediately preceding characters. The columns labelled "Order" and "Context" indicate the order and context that is finally used to predict the current character. Novel characters are always encoded using a -1 order model. In the example, four of the letters in the second occurrence of "`chuck`" can be predicted using a second order model.

The amount of memory required by the PPM algorithm is higher than dictionary-based algorithms, due to the number of contexts that have to be maintained. Typical implementations use several megabytes of memory for data structures. Bell et al. [1990] showed how an implementation of PPM would use a trie of context nodes, with each node representing a unique context of symbols. Nodes are interlinked with *vine pointers* so that when a shorter context is needed, it can be reached in $O(1)$ time. Even so, PPM tends to execute slower for both encoding and decoding in comparison to LZ77 and LZ78 because of the overhead of maintaining the context tree, and the fact that PPM operates on a per symbol basis. It is widely accepted that the combination of PPM with an arithmetic coder (to be discussed below), is one of the best general-purpose compression systems[1]. Bell et al. [1990] summarise work by others (including Shannon [1948], Rissanen and Langdon, Jr. [1981], and Bell and Witten [1987]), and show that an equivalent statistical model

[1]The Canterbury Corpus web site at `http://corpus.canterbury.ac.nz/` compares a wide range of compression systems.

	Order	Context	Message
(a)	-1	-	[w]oodchuck␣chuck␣if
(b)	-1	-	w[o]odchuck␣chuck␣if
(c)	0		wo[o]dchuck␣chuck␣if
(d)	-1	-	woo[d]chuck␣chuck␣if
(e)	-1	-	wood[c]huck␣chuck␣if
(f)	-1	-	woodc[h]uck␣chuck␣if
(g)	-1	-	woodch[u]ck␣chuck␣if
(h)	0		woodchu[c]k␣chuck␣if
(i)	-1	-	woodchuc[k]␣chuck␣if
(j)	-1	-	woodchuck[␣]chuck␣if
(k)	0		woodchuck␣[c]huck␣if
(l)	1	c	woodchuck␣c[h]uck␣if
(m)	2	ch	woodchuck␣ch[u]ck␣if
(n)	2	hu	woodchuck␣chu[c]k␣if
(o)	2	uc	woodchuck␣chuc[k]␣if
(p)	2	ck	woodchuck␣chuck[␣]if
(q)	-1	-	woodchuck␣chuck␣[i]f
(r)	-1	-	woodchuck␣chuck␣i[f]

Table 2.3: Statistical modelling using PPM. Each row indicates the order and context used to predict the current character. The progress of the algorithm is presented at the end of each row, with the current character placed in a box. The shaded region indicates the characters that have been processed.

exists for any dictionary-based algorithm which selects phrases in a greedy fashion. This relationship was further discussed by Bell and Witten [1994]. So, a greedy dictionary-based model cannot outperform a statistical one.

Block Sorting

The last modelling technique considered in this chapter is the block sorting mechanism devised by Burrows and Wheeler [1994]. Also called the Burrows-Wheeler transform (or BWT), the algorithm differs from dictionary-based and statistical ones in that it applies a transformation to the message to improve compression. A subsequent move-to-front mechanism is invoked in order to underscore the locality created by the transformation, and eventually, a zero-order coder is applied.

The block sorting transform is best explained through an example. Figure 2.3 demon-

woodchuck	uckwoodch	c h
oodchuckw	woodchuck	c k
odchuckwo	huckwoodc	d c
dchuckwoo	ckwoodchu	h u
chuckwood	oodchuckw*	k w*
huckwoodc	dchuckwoo	o o
uckwoodch	chuckwood	o d
ckwoodchu	kwoodchuc	u c
kwoodchuc	odchuckwo	w o
(a)	(b)	(c)

Figure 2.3: Example of the block sorting algorithm with the word "woodchuck". The n strings (for a block of n characters) are obtained by permuting the original word as shown in (a). Then, the strings are sorted in (b), with the first letter of the word indicated by the asterisk ("*"). The characters indicated in the grey column are transformed using MTF before being transmitted to the receiver. Decoding is shown in (c).

strates the algorithm with the string, "woodchuck"[2]. The first step in block sorting is to partition the message into blocks so that each one can be processed independently in an off-line manner. For each block of n symbols, a set of n strings is created, starting from each symbol in the block. When the end of the string is reached, the string wraps-around to the beginning. The n strings for the example string are presented in Figure 2.3(a). The block of strings is then sorted by comparing from the second-to-last symbol until the beginning of the string, in a reverse lexicographic order. This is shown in Figure 2.3(b). Sorting into context order gives a good indication about what characters appear after a particular conditioning sequence. For example, the first two strings of Figure 2.3(b) show that in this string, the character "c" only forms a context for "h" and "k". The sequence of last characters of the block of sorted strings is shown with a shaded column in the figure. This string of characters is encoded for transmission. The encoder notifies the decoder as to the location of the symbol at the beginning of the original message. In this example, it is the "w" in the fifth position of the string, as indicated by the asterisk (*).

The decoder receives the permuted string, and sorts it. In Figure 2.3(c), the sorted string on the left is placed next to the received unsorted string, shown to its right. The

[2]See other sources, such as Witten et al. [1999, pg. 66], for an example of block sorting using a longer message with more redundancy.

sorted string of received characters is identical to the column of second-to-last characters in Figure 2.3(b). That is, each character in the sorted string forms a one-symbol context for the character to its right. Moreover, in both strings, the relative order of characters which have the same value is preserved. The second "o" in the sorted string (seventh character) is the same character as the second "o" in the unsorted string (last character). One symbol is decoded by examining a character from the received string, then the same character in the sorted version, and finally, the character it forms a context with, to its right, as illustrated in Figure 2.3(c). In this example, the first character in the unsorted string is the "w" in position 5. This character is the same as the "w" at the end of the sorted string, which forms a context for the second "o" in the unsorted string.

The difference between the transmitted string and the original message is that the permuted string has localised character occurrences. In the first two strings of Figure 2.3(b), the characters "h" and "k" appear in the context of "c". Even if the message was longer, only the few character that appear after a "c" would cluster in this region.

To exploit the localised clustering, a move-to-front (MTF) transformation is then applied. The transformation translates each symbol in the string to a number that indicates how many distinct symbols have been observed since this one was last seen. A symbol that was recently seen gets translated into a small value. In a cluster of symbols where there is little variation, sequences of small numbers result. In the end, the permuted string becomes a list of numbers with a skewed distribution of mainly small values, and it is that sequence that is passed to a coder. If the seven distinct characters in "woodchuck" are initially ordered alphabetically, then the MTF values for the permuted string is $\{3, 4, 3, 6, 7, 7, 7, 5, 3\}$. If the original message was longer, then clusters of symbols in the permuted sequence would generate runs of small MTF numbers. As an alternative, Wirth and Moffat [2001] compressed the output from block sorting directly using other schemes, without applying the MTF transform.

Generally, systems which implement block sorting with MTF yield compression ratios close to PPM due to their exploitation of symbol contexts [Cleary and Teahan, 1997]. As the most time consuming operation in BWT is string sorting, others have considered ways

of improving this phase of the algorithm [Sadakane, 1998, Seward, 2000, 2001]. Several other authors have explored refinements to improve compression effectiveness [Deorowicz, 2000, Fenwick, 1996a,b].

2.4 Coders

The model and probability estimator identifies the redundant information in the message and produces a *reduced message*. This reduced message must then be coded to achieve the desired transmission across the channel. Based on the description of models presented earlier in this chapter, the reduced message may contain components of a 3-tuple from a LZ77 implementation, a stream of symbols indicating which PPM contexts were chosen, or MTF values for a message permuted with block sorting.

The coder obtains the reduced message, the set of distinct symbols in the reduced message (D), and the probabilities with which these symbols occur (P). From these three pieces of information, the coder creates a set of *codewords* (C) which map each symbol in the reduced message to a string of letters in the channel alphabet. Compression has been achieved if the set of codewords (collectively called *codes*) chosen produce an output message shorter than the original message. Furthermore, if the codes are selected so that no codeword is a prefix of any other codeword, then the codes are *prefix-free*. Prefix-free codes ensure that the compressed message is uniquely decodable, without the need to buffer codewords.

Two classes of coders are presented next. Methods in the first class assign codewords statically, based solely on a symbol's value within the alphabet. The second class of coders are entropy coders, which use the supplied symbol probabilities to produce a compressed message whose length is intended to be close to the self-information of the reduced message.

Static Coders

Static coders map each symbol in the reduced message to a particular codeword, without acknowledging the probabilities with which the symbols occur. The simplest scheme is a

x	unary	binary	gamma	delta
1	0	000	0	0
2	10	001	100	1000
3	110	010	101	1001
4	1110	011	11000	10100
5	11110	100	11001	10101
6	111110	101	11010	10110
7	1111110	110	11011	10111
8	11111110	111	110000	11000000
9	111111110	—	110001	11000001

Table 2.4: Some sample static codes for the values 1 to 9. The prefix of the gamma and delta codes are shaded in grey.

unary code, which encodes the symbol x as a codeword made up of $x-1$ one-bits, and terminated by a zero bit. Unary codes are biased toward smaller values of x, since the codeword for x is exactly x bits in length.

Another straightforward coding mechanism is binary, which allocates a codeword of length $\lceil \log_2 N \rceil$ bits for each x, for some upper limit N. Unlike unary coding, binary coding assumes a uniform distribution of the symbols and a definite range. Some examples of binary coding are the ASCII and Latin-1 character sets, which encode each symbol in 7 and 8 bits, respectively. The second and third columns of Table 2.4 list the unary and binary codes for the values 1 to 9, respectively. The binary codes in this table are based on an upper limit of $N = 8$. Because of this, no binary code is possible for $x = 9$.

The fourth column of the table shows the codewords produced by gamma coding [Elias, 1975]. Gamma codes offer a compromise between the skewed distribution assumed by unary coding, and the uniform distribution of binary coding. A gamma code has two parts, a prefix and a suffix. The prefix, shown shaded in the fourth column, uses a unary code to specify the binary length of the symbol. The suffix part specifies the exact symbol in binary within that order of magnitude. For the symbol x, the prefix specifies $1+\lfloor \log_2 x \rfloor$ in $1+\lfloor \log_2 x \rfloor$ bits, while the suffix indicates $x - 2^{\lfloor \log_2 x \rfloor}$ in $\lfloor \log_2 x \rfloor$ bits. The total length of the gamma code for x is thus $1 + 2\lfloor \log_2 x \rfloor$ bits. If the prefix is coded using a gamma code, and the suffix in binary, then the delta codes result, also described by Elias. The last column of Table 2.4 presents some delta codes, with the prefix again highlighted. The

length of a delta code for a symbol x is $1 + 2\lfloor \log_2 \log_2 2x \rfloor + \lfloor \log_2 x \rfloor$ bits.

There are two benefits of using Elias codes instead of unary and binary codes. In comparison to unary codes, Elias codes are shorter for all but a very small number of values of x. And while the length of binary codes are even shorter, binary codes are assigned based on a known range, which might not be a reasonable restriction. On the other hand, Elias codes and unary codes can represent arbitrarily large values. All four codes are prefix-free.

Huffman Coding

Entropy coders assign codes derived from symbol probabilities that are calculated directly from symbol frequencies in the reduced message, or through other means. Either way, they are assumed to be true for the symbols being processed. Entropy coders that decide on codewords based on the observed frequency of symbols in the reduced message can do so using either semi-static or adaptive mechanisms. Two entropy coding regimes are considered in this section: semi-static Huffman coding, and adaptive arithmetic coding.

Huffman coding is a famous entropy coding technique [Huffman, 1952]. This algorithm assigns codewords so that frequent symbols obtain shorter codewords than rare symbols. The Huffman algorithm calculates codewords incrementally, by first placing each symbol of the alphabet in a *package*, and then by repeatedly merging the two packages with the lowest combined probabilities, until only one package remains. Whenever two packages are combined, the probabilities are summed. Every symbol has an associated codeword length which is incremented by one every time the symbol participates in a merge. When the algorithm concludes, the set of lengths are used to assign codewords to each symbol.

Figure 2.4 explains the Huffman algorithm through an example. Let the set of packages be D, the associated probabilities of each symbol in the alphabet be P, and their corresponding codeword lengths L. In the example, the alphabet consists of the four symbols $D = \{\{\text{w}\}, \{\text{x}\}, \{\text{y}\}, \{\text{z}\}\}$, with respective probabilities $P = \{0.1, 0.2, 0.2, 0.5\}$. The codeword lengths are all initialised to 0 so that, $L = \{0, 0, 0, 0\}$. First, the two packages with the lowest probabilities are merged in Figure 2.4(b). Since the packages {x}

D	P	L
w	0.1	0
x	0.2	0
y	0.2	0
z	0.5	0

(a) Initial packages.

D	P	L
w	0.3	0
x		1
y	0.2	0
z	0.5	0

(b) Combine w with x.

D	P	L
w		2
x	0.5	2
y		1
z	0.5	0

(c) Combine wx with y.

D	P	L
w		3
x	1.0	3
y		2
z		1

(d) Combine wxy with z.

Figure 2.4: The merging of packages to build a set of Huffman codes. Lightly shaded packages are merged and their codeword lengths incremented by 1.

and {y} have the same probabilities, either one can be combined with {w}. The selected packages, shown shaded, are merged and their codeword lengths are incremented by 1. Once the package {z}, is processed in Figure 2.4(d), the final set of codeword lengths is $L = \{3, 3, 2, 1\}$.

Next, a set C of prefix-free codewords is assigned to each symbol based on L to complete Huffman's algorithm. One set of codewords that satisfy this condition is $C = \{000, 001, 01, 1\}$. Note that the assignment of codewords is not unique. If there are n packages, then there are $n - 1$ merge operations. Each merge operation is equivalent to appending a 0 bit to one package and a 1 bit to the other, once L has been calculated. As the algorithm does not state the bit a package receives, there are 2^{n-1} sets of Huffman codes. Moreover, as shown in the example, when multiple packages share the same probability, ties are broken arbitrarily.

An alternative method to the Huffman code construction described is the bottom-up method which employs a binary tree [Johnsonbaugh, 1993, Salomon, 2000, Cormen et al., 2001]. Figure 2.5 shows a tree for the codewords C that were assigned using the lengths L from the example of Figure 2.4. Symbols are placed at the leaves of the tree. In this graph representation, symbol lengths translate to depths of symbols in the tree, so that each edge signifies a bit in the codeword. A bit stream is decoded by traversing the tree

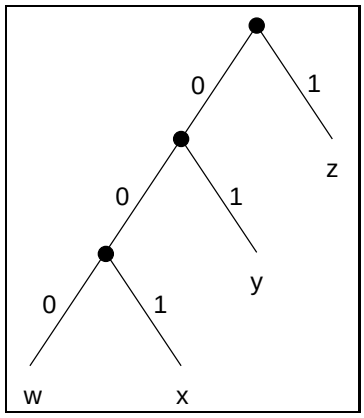


Figure 2.5: Visualising Huffman codes as a binary tree.

starting from the root, so that a bit permits one edge transition. When a leaf has been reached, the symbol at that leaf is output and the process begins again from the root. While visualising the decoding of Huffman codes with a tree is helpful, Moffat and Turpin [1997] presents a more efficient method for decoding which replaces tree construction by both the encoder and the decoder with a set of arrays. The primary requirement is that the codes must be *canonical*. A set of Huffman codes are canonical if they are ordered lexically when sorted by length.

According to the codewords of Figure 2.4, the reduced message "yzzw" would be coded in 7 bits as 0111000. Unfortunately, this bit stream alone is not enough for the decoder to recreate the original message. A *prelude* is required which indicates the alphabet of the reduced message (D). Details about the construction and transmission of the prelude for Huffman coding is described by Turpin and Moffat [2000].

Other variations to the Huffman algorithm include length-limited coding [Larmore and Hirschberg, 1990, Liddell and Moffat, 2002] and adaptive Huffman coding [Vitter, 1987, Knuth, 1985]. In length-limited coding, codewords are assigned so that none of them are longer than L_m bits, for some upper limit L_m. Adaptive Huffman coding differs from the semi-static paradigm described above in that the probabilities of the symbol are modified while the message is processed. Moreover, like the PPM algorithm described earlier, the zero-frequency problem re-emerges since a probability must be assigned to novel symbols.

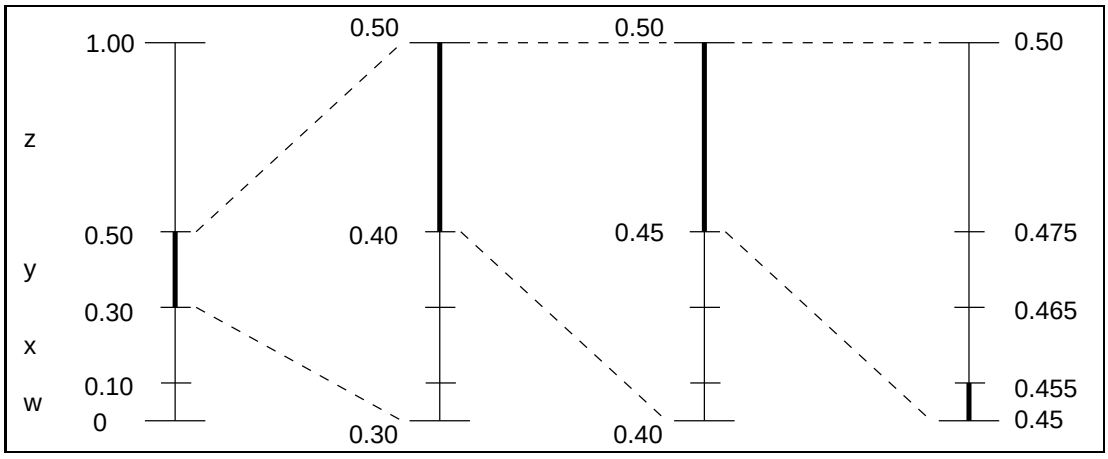


Figure 2.6: The division of intervals when arithmetic coding is applied to the message "yzzw", for the four symbol alphabet, $\{w, x, y, z\}$. The intervals are broken down further with each symbol encountered.

Arithmetic Coding

All of the coding techniques described so far have one trait in common. Regardless of the value of a symbol, or its probability in the message, the symbol must be coded in at least one bit. Arithmetic coding by Rissanen [1976] and Witten et al. [1987], and later revised by Moffat et al. [1998], eliminates this restriction by coding sequences of symbols at a time.

Figure 2.6 explains the algorithm through the sample message "yzzw", using the symbol probabilities used earlier. At first, the interval [0, 1) is divided according to the cumulative probabilities of D. As each symbol in the message is encountered, the section corresponding to the symbol is further divided. In the example, after "y" has been processed, the interval [0.3, 0.5) is divided into four parts based on the symbol probabilities. Once the last symbol is read, the interval [0.45, 0.455) has been isolated. In binary, with the decimal removed, this interval is [0111001, 0111010001). The message can be encoded by specifying any number within this interval. One sequence that meets this requirement is 011101 (or, 0.453125).

As with Huffman coding, the prelude must be sent to the decoder. Furthermore, in order for the decoder to mimic the encoder, the probabilities of the symbols need to be

transmitted as well. Then, the decoder operates by dividing and selecting intervals in the same order as the encoder. Several other simplifications have been made. For example, an actual implementation would use integers instead of real numbers so as to avoid any rounding error. Also, intervals must be scaled when they become too narrow. Moffat and Turpin [2002] provides a detailed discussion about the implementation issues associated with arithmetic coding.

Generally, arithmetic coding performs better than other coding techniques for highly skewed probabilities – especially when the self-information of any of the symbols is less than one bit. In fact, the compression effectiveness attained by arithmetic coding is better able to approach the self-information of the message than any other coding technique. However, in comparison to other coding methods, arithmetic coding is relatively slow for both encoding and decoding, since an interval has to be divided for each symbol in the message.

2.5 Compression in Practice

Three compression systems are explained next, in the context of the algorithm descriptions presented above. These systems, two of which are readily available on the Internet, are used throughout this thesis as baseline comparisons. The three implementations are GZIP, PPMD, and BZIP2.

The dictionary-based system GZIP [Gailly, 1993] employs the DEFLATE algorithm [Deutsch, 1996], which combines a variant of the LZ77 algorithm with Huffman coding. The DEFLATE algorithm partitions a message into blocks, and processes a block at a time for efficiency reasons. The DEFLATE algorithm uses 1-tuples for literal symbols and 2-tuples for references into the dictionary, and neither type of tuple specifies the next symbol. The sliding window size is fixed at 32 KiB[3]. The DEFLATE algorithm assumes that the message is encoded in bytes, so that literal symbols fall within the range of 0 to 255. Literal symbols and phrase lengths are encoded using one set of Huffman codes,

[3]In this thesis, 1 KiB = 2^{10} bytes = 1,024 bytes; 1 MiB = 2^{20} bytes = 1,048,576 bytes; 1 GiB = 2^{30} bytes = 1,073,741,824 bytes. [IEC Technical Committee (TC) 25, 2000].

while distances are encoded with another set. For each block, DEFLATE chooses between static or semi-static Huffman codes. If semi-static Huffman codes are selected, then the prelude is situated at the beginning of the compressed block. A publicly available library called ZLIB [Gailly and Adler, 1995] is available, which allows DEFLATE to be compiled into any program. The default compression effectiveness of GZIP can be improved by adding the `-9` option. The `-9` option forces GZIP to search more exhaustively for a longer match. All experiments with GZIP in this thesis use version 1.3.2 with the `-9` option.

The PPMD system by Alistair Moffat (University of Melbourne) couples a PPM mechanism based on escape method D with an arithmetic coder. Two parameters can be set by the user to alter the program's behaviour. The first parameter specifies the maximum context, while the second one sets a limit on the amount of memory that can be used. Context nodes are created by PPMD until the memory limit is reached. If this happens while a message is being processed, then all context nodes are deleted, and compression continues with a fresh trie. Experiments in this investigation make use of a seventh order model with a memory limit of 255 MiB, both chosen to favour compression effectiveness.

The BZIP2 program [Seward, 1996] combines block sorting with a Huffman coder. The block sorting mechanism divides the message into blocks ranging from 100,000 to 900,000 bytes in size, depending on the value of a command line argument. In this thesis, the program parameter `-9` is always given so that a block size of 900,000 bytes is used. The documentation that accompanies the program states that the memory required by BZIP2 for encoding with this block size is around 7.4 MiB, while decoding requires about 3.6 MiB. A compression library similar to ZLIB called LIBBZIP2 is available from the BZIP2 web site. The version of BZIP2 used in this thesis is 1.0.2. Details of the implementation of BZIP2 are given by Seward [2000] and Seward [2001].

Two additional programs are purely entropy coders which presume a zero-order model, and hence that all modelling processes have already been allowed for. Input to these coders is a sequence of integers. The SHUFF system [Moffat and Turpin, 1997, Turpin and Moffat, 2000] is a semi-static Huffman coder that compresses an integer sequence in blocks in two passes. The end of a block is determined either by an explicit size from the user, or when

a special end-of-block symbol has been found in the input stream. The second coding mechanism is an arithmetic coder called UINT, developed as part of a package of software to demonstrate the attributes of arithmetic coding by Moffat et al. [1998].

With the exception of the PPMD system,[4] the remaining four programs are publicly available for download, with source code, from web sites mentioned in their respective bibliography entries at the back of this thesis.

2.6 Experimental Framework

Experiments are performed in this thesis in order to validate the compression and retrieval ideas presented. All tests are carried out on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache running Debian Linux. Programs implemented for this thesis are written in the C programming language and compiled with version 2.95.4 of the GNU `gcc` compiler with the `-O3` optimisation flag. Compression systems have been evaluated in terms of efficiency and effectiveness. Unless otherwise stated, compression times are expressed in CPU seconds, averaged over three trials. CPU seconds reflects the amount of time the computer spent on the program, but excluding the time spent accessing files, for example. Due to the small variations in execution times, an average calculated from three runs was sufficient for most of the experiments.

Compression ratios throughout the investigation are primarily expressed in one of two different ways. If the original message is composed of 32-bit integers, then the ratios are expressed in bits per symbol (bps). However, for most of this thesis, the message is encoded in Latin-1 and the units employed are bits per character (bpc).

The test data that was chosen for this investigation included the Large Canterbury Corpus[5] (`LCC`) and data from the Text REtrieval Conference[6] (TREC). The Large Canterbury Corpus includes three files: `E.coli`, `world192.txt`, and `bible.txt`. The `E.coli` file consists of the complete genome of the E.coli bacterium and only contains the four

[4] One publicly available PPM system which employs escape method D is at `http://www.cs.waikato.ac.nz/~singlis/`.

[5] URL: `http://corpus.canterbury.ac.nz/`

[6] URL: `http://trec.nist.gov/`

characters a, c, g, and t, with no whitespace. The file `world192.txt` is the CIA world fact book, while `bible.txt` is the King James version of the bible. The sizes of these three files are listed in Table 2.5.

Three data files were drawn from the news articles of disks 1 and 2 of TREC's TIPSTER collection [Harman, 1995]. The largest data file is `NEWS`, which is composed of news articles from the Wall Street Journal (`WSJ`) from 1987 to 1992, as well as articles from the Associated Press (`AP`) from 1988 and 1989. All of the news articles have been marked up with the Standard Generalized Markup Language (SGML) [International Organization for Standardization, 1986]. Due to the wide range in years covered by the articles, and the fact that the SGML tags differ between `WSJ` and `AP`, there are some discontinuities of style caused by the concatenation. However, the homogeneous data collection of news articles permits a data file of about 1,000 MiB in size to be processed. Small excerpts from `WSJ` and `AP` are shown in Figure 2.7. The remaining two data files from TREC are `WSJ508` and `WSJ20`. The `WSJ508` file includes all of the Wall Street Journal articles from TREC, which is 508 MiB in total. A smaller data file, `WSJ20`, consists of just the first 20 MiB of `WSJ508`.

The three compression systems, GZIP, BZIP2, and PPMD, were applied to these 6 data files. The results are presented in Table 2.5 and Table 2.6. Since SHUFF and UINT are coding mechanisms and not general-purpose compression systems, experiments with them are detailed later in Chapter 3 with different data files.

Table 2.5 shows the exact file sizes of the 6 files and the compression effectiveness attained with the compression programs. Both GZIP and BZIP2 were executed with the `-9` option, while PPMD used a seventh order model and 255 MiB of memory in order to favour compression effectiveness. For all 6 files, the statistical model of PPMD consistently performed the best, followed by BZIP2, and then GZIP. This order is expected, given their respective models and coders, as well as the amount of memory available for them, and the consequential use of very large blocks by PPMD.

Table 2.6 examines the compression experiments with respect to time in CPU seconds, averaged over three trials on the test machine discussed earlier. In almost every scenario,

```
<DOC>
<DOCNO> WSJ870324-0001 </DOCNO>
<HL> John Blair Is Near Accord
To Sell Unit, Sources Say</HL>
<DD> 03/24/87</DD>
<SO> WALL STREET JOURNAL (J)</SO>
<IN> REL
TENDER OFFERS, MERGERS, ACQUISITIONS (TNM)
MARKETING, ADVERTISING (MKT)
TELECOMMUNICATIONS, BROADCASTING, TELEPHONE,
TELEGRAPH (TEL) </IN>
<DATELINE> NEW YORK </DATELINE>
<TEXT>
 John Blair &amp;amp; Co.  is close to an
agreement to sell its TV station
```

(a) Excerpt from WSJ (WSJ20 and WSJ508).

```
<DOC>
<DOCNO> AP890101-0001 </DOCNO>
<FILEID>AP-NR-01-01-89 2358EST</FILEID>
<FIRST>r a PM-APArts:60sMovies 01-01 1073</FIRST>
<SECOND>PM-AP Arts:  60s Movies,1100</SECOND>
<HEAD>You Don't Need a Weatherman To Know '60s Films Are
Here</HEAD>
<HEAD>Eds:  Also in Monday AMs report.</HEAD>
<BYLINE>By HILLEL ITALIE</BYLINE>
<BYLINE>Associated Press Writer</BYLINE>
<DATELINE>NEW YORK (AP) </DATELINE>
<TEXT>
 The celluloid torch has been passed to a new
generation:  filmmakers who grew up in the 1960s.
```

(b) Excerpt from AP.

Figure 2.7: Two excerpts from the NEWS data set.

GZIP required the least amount of time for both encoding and decoding, followed by BZIP2, and then PPMD. The notable exception is the compression of E.coli, where GZIP required the most time for encoding. This phenomenon is due to the small set of characters in the file, which made looking for the longest match more time consuming.

The tables demonstrate a common trade-off in compression between efficiency and effectiveness. Generally, the best compression effectiveness comes at the cost of execution

Filename	File size	GZIP	BZIP2	PPMD
E.coli	4,638,690	2.240	2.158	2.034
world192.txt	2,473,400	2.333	1.584	1.454
bible.txt	4,047,392	2.326	1.671	1.564
WSJ20	20,971,520	2.908	2.078	1.656
WSJ508	533,196,049	2.941	2.105	1.637
NEWS	1,048,491,001	2.963	2.145	1.674

Table 2.5: Compression ratios for the Large Canterbury Corpus, WSJ20, WSJ508, and NEWS using GZIP, BZIP2, and PPM, expressed in bits per character. File sizes are indicated in bytes.

Filename	Encoding (s)			Decoding (s)		
	GZIP	BZIP2	PPMD	GZIP	BZIP2	PPMD
E.coli	14.0	4.4	3.6	0.2	1.5	3.6
world192.txt	0.6	2.1	3.3	0.1	0.7	3.4
bible.txt	1.8	3.4	4.4	0.1	1.1	4.5
WSJ20	6.4	17.9	31.0	0.8	6.2	31.5
WSJ508	170.0	447.3	800.9	20.4	155.8	821.5
NEWS	323.3	871.5	1,600.8	40.3	304.3	1,641.0

Table 2.6: Compression times for the Large Canterbury Corpus, WSJ20, WSJ508, and NEWS using GZIP, BZIP2, and PPM. Times are reported in CPU seconds on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache, averaged over three trials.

time. These results also indicate why these three programs were selected for this chapter. Besides demonstrating the three types of models mentioned earlier, the GZIP system was chosen for its speed, while PPMD was chosen for its compression effectiveness. The BZIP2 system demonstrates the compromise in time and space between the other two systems. Throughout this thesis, these two tables are frequently referred to as benchmarks for these compression systems and the test data.

Finally, note that GZIP is particularly fast for decoding, a trait that is common with most dictionary-based algorithms, and one that forms the foundation for much of this thesis, as explained in the next section of this chapter.

2.7 Effect of Compression on Browsing

The central theme in this thesis is the exploration of the compromises required in order to accommodate both compression and browsing. The compression systems surveyed in

this chapter can reduce a message to less than a fifth of its original size. However, several aspects of their underlying algorithms preclude any form of retrieval, let alone browsing, without performing a complete decompression.

The support for searching a compressed message is important, since time and space could be saved while still being able to obtain information from the compressed representation. The solution offered by the ZGREP script [Free Software Foundation, 2001], is to decompress a reduced message created by GZIP temporarily, and then search this copy with a standard pattern matcher. However, while such a solution is convenient for the user, the solution is somewhat unsettling given that the compressed message is fully decoded. The source of the problem is the compression algorithms themselves.

The main drawback to the dictionary-based and statistical models presented is their adaptive, on-line nature. In order to retrieve from an arbitrary point in a message processed with either LZ77, LZ78, or PPM, everything before it must be decoded. In the case of block sorting, only the block where the point of interest is located must be decoded. So, extending this solution to the other algorithms, creating more blocks would improve retrieval feasibility. Generalising the solution, *synchronisation points* could be inserted throughout the compressed message so that retrieval from a location in the compressed stream only requires a full decode from the last synchronisation point. However, since synchronisation points indicate a location in the compressed message where the model is re-built, as the frequency of these points increase, compression can be expected to suffer.

Furthermore, while compression systems are measured in terms of effectiveness and efficiency, retrieval systems are usually interactive. That is, users are more concerned with response times and expect a short waiting period for results to appear. The importance of access time provides another reason why a full decode of the compressed document is discouraged.

In addition to these two points, coding algorithms present a third problem, which is byte-alignment. Even though the channel alphabet is based on the binary system, computers access data in bytes consisting of 8 bits each. The low-level operations on bits which are intrinsic to static and entropy coding algorithms also affect both retrieval and

decoding times.

These three problems are considered in the remainder of this thesis, beginning with Chapter 3. At the heart of this investigation is the RE-PAIR algorithm of Larsson and Moffat [2000], which is described next. The algorithm is dictionary-based and has many similarities with LZ77 and LZ78. However, RE-PAIR possesses other qualities that make it ideal for browsing. The chapters that follow Chapter 3 build on the algorithm and address the three problems posed here.

Chapter 3

Compression by Recursive Pairing

Re-Pair is a dictionary-based compression algorithm developed by Larsson and Moffat [2000]. Re-Pair operates off-line and commences the compression process only after the entire message has been seen. This is in contrast to dictionary-based algorithms such as LZ77 and LZ78, described in Chapter 2, which build dictionaries on-line.

Off-line compression algorithms have gained attention recently for two reasons. First, the amount of memory available in computers has steadily increased in recent years. Earlier on-line algorithms were able to operate with just enough memory to perform the compression, but not enough to also hold the entire message. With more memory to use, off-line algorithms which examine an entire message before starting compression offer an alternative to on-line algorithms. Second, decompression occurs more frequently than compression in retrieval systems. In these circumstances, an underlying off-line compression mechanism that operates slower than its on-line counterparts but ensures fast decompression may be preferred. As with on-line dictionary-based algorithms, off-line ones build a dictionary of phrases. Occurrences of these phrases in the message are replaced by references to them. However, off-line dictionary-based algorithms are faster because the model does not have to be updated throughout decompression.

Re-Pair transmits the dictionary explicitly as a separate stream to the receiver instead of incrementally like LZ77. This results in two outputs: the dictionary, and a sequence of references to entries in the dictionary. These two parts are coded independently and transmitted to the decompressor, Des-Pair, as two distinct streams in order to reconstruct

the message. As is discussed later in this chapter, it is RE-PAIR's separation of the message into two separate streams that makes it ideal for phrase browsing.

The remainder of the chapter is structured as follows. Section 3.1 provides an overview of the RE-PAIR algorithm. While recent computers have more memory available for off-line algorithms to use, memory is still limited and Section 3.2 gives a detailed analysis of the data structures used by RE-PAIR. These data structures provide a balance between memory space and execution time. Section 3.3 describes how the two outputs of RE-PAIR are represented as coded sequences. Section 3.4 gives experimental results for the implementation of RE-PAIR used, and Section 3.5 describes other algorithms similar to RE-PAIR. Finally, Section 3.6 discusses the features of RE-PAIR that make it attractive for fast searching and retrieval, and the issues that prevent it from being used for that purpose.

3.1 The Re-Pair Model

RE-PAIR reduces the length of a message by replacing repeated pairs of symbols with a new symbol which does not appear in the message. Initially, the most frequently occurring pair of characters is located and all occurrences of it are replaced with a new symbol. Each replacement reduces the length of the message by one. The new symbol is added to a dictionary, together with a record of how it should be expanded. This process is repeated, and the next most frequently occurring pair of symbols (now including any new symbols introduced by RE-PAIR) is replaced. The process terminates when no pair of adjacent symbols occurs more than once in the message. The source of the name "RE-PAIR" arises from the recursive pairing nature of the algorithm.

In order to describe RE-PAIR in greater detail, some more precise definitions and notations are necessary. The alphabet (Σ) of the original message are also called *primitives*. RE-PAIR creates and adds *phrases* to the dictionary. The set of all phrases is denoted as ρ.

Primitives and phrases are collectively called *symbols*, and are denoted using, for example, α, β, δ, and γ. If it is important to indicate that a symbol α is a phrase

introduced by Re-Pair and not a primitive, then it is circled, ⓐ. If the order in which the phrase was added to the dictionary is important, then the phrase is instead indicated by a circled number, ①.

Every symbol α has a generation $g(\alpha)$ which indicates how deep it is in the hierarchy of paired symbols. By definition, the generation of any primitive is zero. The generation of a phrase α is one more than the higher generation of its two components.

The outputs of Re-Pair are a dictionary, and a sequence of references to entries in the dictionary. The dictionary contains all of the primitives and phrases composed out of them by the pairing process. Since every phrase consists of two symbols from a lower generation (either shorter phrases or primitives), the dictionary resembles a hierarchy of phrases. To underscore this fact, the dictionary generated by Re-Pair is called the *phrase hierarchy*, denoted as $\mathcal{P}$. The list of pointers to symbols in the phrase hierarchy is called the *reduced sequence*, and is denoted as $\mathcal{S}$.

A sample application of Re-Pair is shown in Table 3.1. The sample message used is part of the "Woodchuck" message of Figure 2.2. The initial message is shown in the first entry of the first column. Each subsequent row of the table indicates the identification of the most frequently occurring pair of symbols in the previous row, and the replacement of all occurrences of it with a new symbol. The new phrase added to the phrase hierarchy as a result of that set of replacements is shown in the second column. For each phrase, the frequency with which it occurs in the message is shown in the third column, and the generation of that phrase is shown in the fourth column. As expected, the sequence of replacements performed by Re-Pair is in decreasing frequency. In this example, the generation numbers of the phrases are in increasing order, but this is not always the case.

Several phrases from the same generation may occur with equal frequencies. The order in which they are chosen depend on the data structures used by Re-Pair. Discussion of the method for breaking ties appears in Section 3.2.

After the last replacement in the table, no pair of symbols occurs more than once. Re-Pair then terminates, and the final sequence (last entry in the first column) is encoded. The phrase hierarchy is also a necessary part of the coded message, which includes all of

Sequence	Phrase hierarchy	f	$g(\alpha)$
how␣much␣wood␣could␣a␣woodchuck␣chuck			
how␣mu①␣wood␣could␣a␣wood①uck␣①uck	① → c h	3	1
how␣mu①␣woo②coul②a␣wood①uck␣①uck	② → d ␣	2	1
how␣mu①③oo②coul②a③ood①uck␣①uck	③ → ␣w	2	1
how␣mu①③oo②coul②a③ood①u④␣①u④	④ → c k	2	1
how␣mu①③⑤②coul②a③⑤d①u④␣①u④	⑤ → o o	2	1
how␣mu①③⑤②coul②a③⑤d⑥④␣⑥④	⑥ → ① u	2	2
how␣mu①⑦②coul②a⑦d⑥④␣⑥④	⑦ → ③ ⑤	2	2
how␣mu①⑦②coul②a⑦d⑧␣⑧	⑧ → ⑥ ④	2	3

Table 3.1: A sample application of RE-PAIR on part of the "Woodchuck" message. Each line represents the replacement of all occurrences of a pair of symbols. The effect on the sequence and the phrase hierarchy is shown, along with the frequency of the symbol pair, and the generation number of the replacing phrase.

the second column as well as a description of the set of primitives.

The Decompressor

The RE-PAIR decompressor (nicknamed, for good reason, DES-PAIR) is simpler than the compressor. After decoding the stream representing the phrase hierarchy, the entire structure is built within memory so that each phrase requires two words of memory to indicate its two components.

A symbol in the reduced sequence is decoded by expanding the phrase hierarchy until the string of primitives that it represents are found. A recursive traversal for each symbol in the reduced message can be slow, depending on the size of the phrase hierarchy and the generation of each symbol. A more economical approach is to decode every phrase in the phrase hierarchy first, so that a decoding table could be made. Then, for each symbol in the sequence, each decoded symbol can be copied directly from the decoding table to the output stream. This method is significantly faster, and is similar to the LZ78 algorithm discussed in the previous chapter. A third approach, which strikes a balance between the other two, maintains a buffer of recently decoded symbols for later use. When a symbol is decoded, the buffer is consulted first before it is recursively expanded.

Table 3.2 presents some experimental results which compare the differences in time

Method	Time (s)	Memory (MiB)
Recursive expansion	4.3	11.5
Fully expanded symbols	2.2	30.2
Buffer of recent symbols	2.3	11.8

Table 3.2: The trade-offs between decoding time and maximum memory usage of Des-Pair using the three approaches on `WSJ20`. The third method uses a buffer of 256 KiB.

and memory usage of these three methods. The test data is `WSJ20`, which was compressed in experiments described later in Section 3.4. The compressed message has 90 primitives, 310,486 phrases, and 1,904,118 references. The longest phrase was composed of 1,257 primitives, and there were 21 generations in the phrase hierarchy. The third method listed in Table 3.2 employs a 256 KiB buffer of recently expanded symbols. Decompression time was measured in CPU seconds, averaged over 3 trials. Memory is stated in MiB, and represents the maximum memory consumed during execution.

Expanding the phrase hierarchy before decoding the sequence was only slightly faster than employing a buffer of recently expanded symbols. The difference in time is expected to be larger if the reduced sequence had more symbols. That is, the cost of fully expanding symbols is recovered when more symbols are decoded. Recursively expanding the phrase hierarchy with each symbol requires more time, but less memory. The difference in memory between the first and the third methods is equal to the size of the buffer. Also, note that the amount of extra memory required by the second method is equal to the size of the document due to the recursive nature of the phrase hierarchy.

The results show that the creation of a modest buffer offers a compromise between execution time and memory usage. This approach has been adopted in the implementation of Des-Pair for later experiments.

3.2 Analysis of Re-Pair

As with any compression system, implementation choices in Re-Pair allow a range of trade-offs between memory space and processing time. The three different approaches to the implementation of Des-Pair is one example of the choices available. A similar

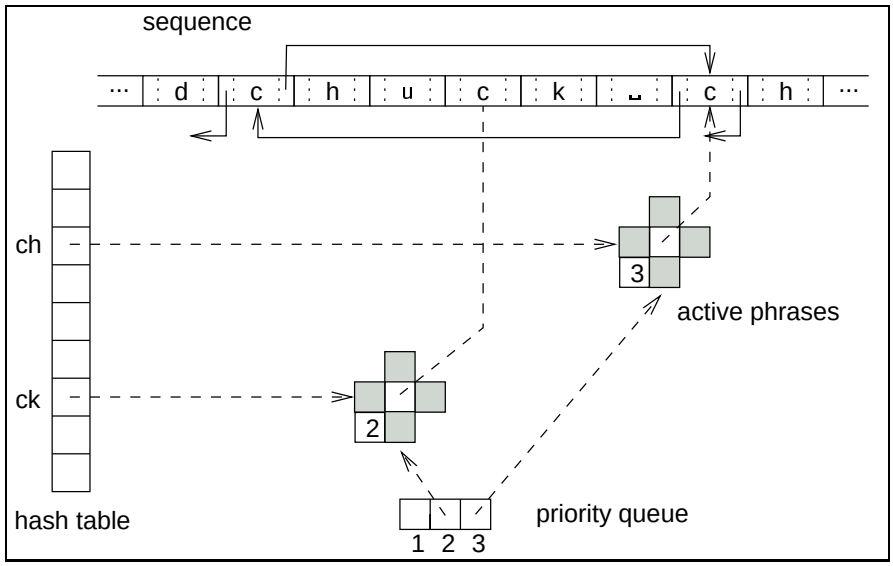


Figure 3.1: The main data structures used by Re-Pair. The active pair pointers for one occurrence of the pair "ch" are shown. Since the list of active pairs is a circular list, the "next" pointer of the last "ch" leads to the first "ch". The shaded areas indicate pointers used for the two sets of double-linked lists, one pair to link items in the hash table and one pair to link items in the priority queue structure.

balance is required by Re-Pair between memory space and execution time. This section discusses data structures described by Larsson and Moffat [2000], and the trade-offs that they allow.

Data Structures

The three main data structures used by Re-Pair are shown in Figure 3.1, with part of the text of Table 3.1 used as an example. The purpose of the data structures is to locate *active pairs* and *active phrases* efficiently. An active pair is a pair of symbols in the sequence that is under consideration for replacement. Likewise, an active phrase is a potential replacement for all active pairs which have the same left and right components. For example, if the pair $\alpha\beta$ appears 10 times in the sequence, each of the occurrences is an active pair, and the 10 active pairs are associated with a single active phrase. Active phrases are required when considering a set of replacements, while active pairs are required for a particular replacement.

The first data structure is an array of *sequence nodes*, shown along the top of Figure 3.1. Initially, each sequence node holds a primitive from the message so that for a message of n primitives, n sequence nodes are required. During compression, a pair of adjacent symbols is replaced with a single symbol. Each sequence node contains a symbol and two pointers. The pointers serve two purposes. Active pairs are formed by taking pairs of adjacent sequence nodes so that every node (except for the first and last ones) can be a left component and a right component of an active pair. The first purpose of the sequence node pointers is to form a circular double-linked list of active pairs so that each node points to the left components of the previous and next equivalent active pairs. In Figure 3.1, the first "ch" active pair shown is the second of three that appear in the "Woodchuck" message. When the active phrase "ch" is chosen, each occurrence of "ch" can be replaced in $O(1)$ time. As active pairs are replaced, gaps appear in the sequence. The second purpose of the node pointers is to allow each gap to be traversed in $O(1)$ time, as illustrated in Figure 3.2. Four snapshots of the sub-sequence of Figure 3.1 are shown at various points during the recursive pairing process. As gaps appear in the sequence array, the pointers at either end of each gap lead to the other end of the gap. The nodes within a gap are unused, and are coloured grey.

At the centre of Figure 3.1 are active phrase nodes. An active phrase node contains a pointer to its active pairs, as well as a count of the number of active pairs. Each active phrase needs to be locatable either by its active pair count or its components. To accomplish this, each node is placed on two circular double-linked lists. One list permits fast look-up of symbol pairs through a hash table, with collisions resolved by chaining. The other list is placed in a priority queue data structure implemented as an array. Each slot in the priority queue corresponds to active phrases of a given number of active pairs. The number of slots in the priority queue is equal to the number of active pairs associated with the active phrase with the most active pairs. The active phrases in a list at a given priority queue slot are placed so that the oldest active phrase node is at the head of the list. Active phrase nodes are attached to the hash table and priority queue using circular double-linked lists, so as to allow $O(1)$ time addition and removal from their respective

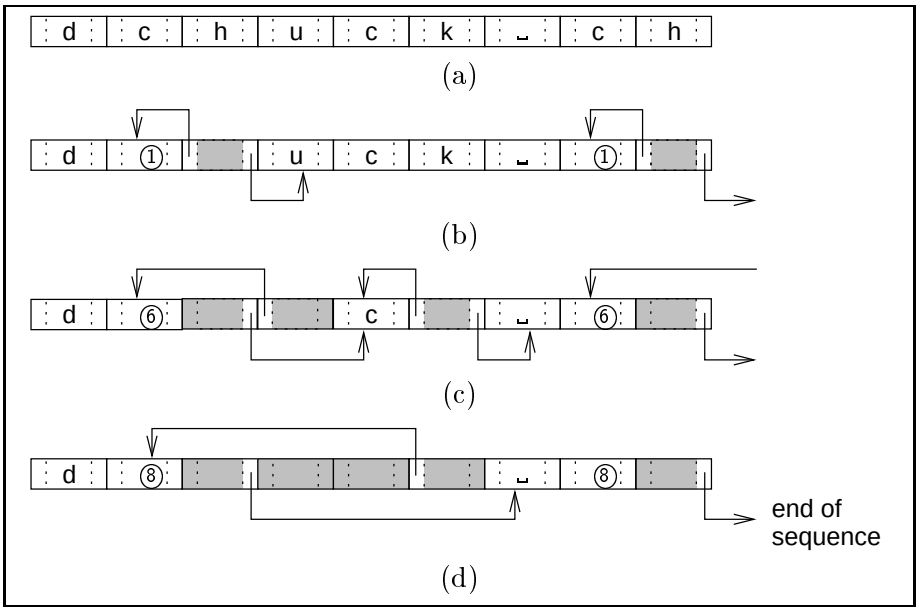


Figure 3.2: The re-using of sequence node pointers to jump over gaps created during the RE-PAIR process. For simplicity, pointers indicating active pairs are not shown. The parts of a sequence node which are no longer used have been shaded.

lists, once they have been located.

A fourth data structure, not shown in the figure, records the active phrases that have been added to the phrase hierarchy. Each phrase in the phrase hierarchy contains pointers to its two component phrases and its generation number.

Space Analysis

The amount of space used by the four main data structures of RE-PAIR is linear in the length of the message. To see this, suppose that a message of n primitives is to be processed. The greatest number of active phrases throughout the processing of the message is p. The size of the hash table is fixed at h slots. The most active pairs an active phrase can have is $\lfloor n/2 \rfloor$, which is equal to the maximum number of slots for the priority queue. The most active pairs occur when every primitive in the message is the same. In this case, there are $n-1$ pairs of primitives, but only half of them are made active because RE-PAIR prevents the existence of overlapping pairs. For example, when there are three consecutive symbols with the same value, only one active pair exists. Overlapping pairs

Data structure	Space used
sequence	$3n$
active phrases	$6p$
hash table	h
priority queue	$\lfloor n/2 \rfloor$

Table 3.3: The amount of memory used by the most important data structures in Re-Pair, in words. The initial message has n symbols. The most active phrases in memory during compression is p. The hash table has h slots.

are elaborated further below.

The amount of memory used by the main data structures and the active phrase nodes in words is summarised in Table 3.3. Since an active phrase node is larger than a phrase in the phrase hierarchy, the space occupied by an active phrase node can be re-used by its corresponding phrase in the phrase hierarchy. So, the memory required for the phrase hierarchy is not shown in Table 3.3. Initially, as many as $n - 1$ active phrases may be created, each with one active pair. However, as none of them would be candidates for replacement, Re-Pair would then terminate. Indeed, in order to minimise memory usage, Larsson and Moffat [2000] showed how only useful active phrases that have at least two active pairs each would be created by performing an additional pass through the message.

While the exact relationship between p and n cannot be determined theoretically, empirically, it can be shown that n is the dominant term for average text. To investigate the relationship, Re-Pair was used to process `WSJ20`. (Details of these experiments appear in Section 3.4.) For `WSJ20`, $|\Sigma| = 90$, $|\rho| = 310{,}486$, and $n = 20{,}971{,}520$. Figure 3.3 shows the number of active phrases in memory after a certain number of phrases have been added to the phrase hierarchy by Re-Pair for `WSJ20`. The number of phrases that have been added to the phrase hierarchy (horizontal axis) was sampled at regular intervals. The total number of active phrases peaks at 482,370 after 13,137 phrases have been created in the hierarchy. Afterwards, the number of active phrases decreases until the last replacement, which creates symbol number 310,486, deletes the last active phrase. The size of the priority queue is initialised to the most frequent symbol, which for `WSJ20`, occurred 466,082 times.

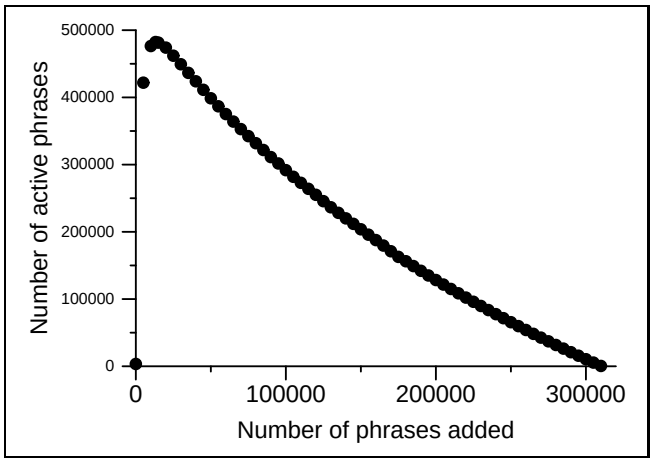


Figure 3.3: The relationship between the number of active phrases and the number of phrases added to the phrase hierarchy for `WSJ20`.

Using the above values for n and h, if $p = 482{,}370$, then n is easily the most important factor with respect to memory usage. The sequence nodes occupy the most space out of all the data structures. Therefore, Re-Pair requires $3.5n + O(p)$ words of memory.

The memory space analysis presented is slightly different from the one given by Larsson and Moffat [2000] due to minor changes to the data structures used. For example, Larsson and Moffat limited the size of the priority queue to $\sqrt{n}$ words of memory so that the last list in the priority queue contains active phrase nodes which have $\sqrt{n}$ or more active pairs. This final list is again ordered so that the least recently created active phrase node is at the head of the list. However, to locate the next active phrase on the list with the most active pairs would require a linear search through the list. Instead, a larger priority queue was chosen to avoid this linear search. Larsson and Moffat also showed a method of periodically compacting the sequence nodes if memory becomes a serious constraint. By reclaiming the space occupied by the gaps, no new active phrase nodes need to be created after the replacement process has begun. Each compaction operation requires an additional pass over the sequence node array to locate the gaps and an update of the pointers of each sequence node.

A shorter priority queue and sequence node compaction reduces the amount of memory used by Re-Pair by using additional execution time. Neither of these approaches were

incorporated into the implementation of Re-Pair for this thesis because a higher priority was given to execution time. Furthermore, if longer messages are processed which require more memory than available, then an alternative method is employed, which is described in Chapter 5.

Time Analysis

The data structures described in the last section allow a message to be processed efficiently. With the data structures described, the Re-Pair algorithm is shown in Algorithm 3.1.

An analysis of the time complexity of the algorithm is as follows. In the first step, an initial pass over the sequence is performed which considers every symbol as a left and a right component with its two immediately adjacent symbols. Active phrases are created if necessary and inserted into the hash table. Then, in step 2, a pass is performed over the hash table to insert active phrases into the priority queue lists.

The dominant step of the algorithm is the recursive pairing loop that begins at step 3. Each replacement reduces the length of the sequence by one symbol. During a single replacement, the most costly steps involve adding, removing, and updating active phrases. By using double-linked lists in the hash table and priority queue, all of these operations execute in $O(1)$ expected time. Therefore, Re-Pair runs in time linear to the length of the message.

Furthermore, the number of active phrases created by replacing the k active pairs of $\alpha\beta$ can never exceed k. That is, after the priority queue has been initialised in step 2, it does not enlarge in size. Also, once an active phrase has been placed on the priority queue list at slot k, it is guaranteed to be used by Re-Pair for pairing or moved to a lower slot in the priority queue. That is, Re-Pair is guaranteed to terminate.

The algorithm also stipulates how active phrases with equal frequency counts are handled. A "least recently used" policy is enforced by adding active phrases to the end of the priority queue lists. This ensures that active phrases are processed first in decreasing frequency, and then in the order in which they were encounterd by Re-Pair. An exception to this rule are generation 1 active phrases. Generation 1 active phrases are created in

Algorithm 3.1: The RE-PAIR algorithm.

1. Scan the message and create up to $n-1$ active pairs. Create active phrases as necessary and add them to the hash table.
2. Add every hashed active phrase with at least two active pairs to the appropriate position in the priority queue. Active phrases that have less than the minimum number of active pairs are deleted.
3. Recursively pair symbols as follows until no pair of adjacent symbols occur twice or more in the sequence.
   a) Retrieve the first active phrase node from the list located at the last priority queue slot that is in use. This node corresponds to the active phrase, $\alpha\beta$, which currently has the most active pairs. Let the replacement be $\textcircled{\gamma}$.
      i. Locate the next active pair for $\alpha\beta$ and its context, $\delta\alpha\beta\delta$.
      ii. Decrement the active phrase frequency counts of $\delta\alpha$ and $\beta\delta$ and move them one priority queue list to the left. Remove any active phrase with a frequency count of 0.
      iii. Replace $\alpha\beta$ with $\textcircled{\gamma}$.
      iv. Increment the active phrase frequency counts of $\delta\textcircled{\gamma}$ and $\textcircled{\gamma}\delta$ and insert them to the end of the appropriate priority queue lists. Create new active phrases if necessary.
   b) Add the phrase $\textcircled{\gamma} \rightarrow \alpha\beta$ to the phrase hierarchy.
   c) Remove the active phrase $\alpha\beta$ from the hash table and priority queue.
   d) Remove all active phrases with counts less than 2.
4. Transmit the phrase hierarchy and the reduced sequence using an appropriate representation.

step 1 but are added to the priority queue in step 2 from the first slot in the hash table to the last.

Problems with Re-Pair

There are two problems with the RE-PAIR algorithm which were not discussed by Larsson and Moffat [2000]. They arise in certain isolated cases. They appear because of RE-PAIR's handling of overlapping pairs. Overlapping pairs occur when a sequence of length 3 or

more of a single symbol appear in the message. During the initial scanning, the second of two overlapping pairs is not considered as an active pair. For example, in the string "$a_1a_2a_3$" (where the subscripts are used to distinguish between occurrences of "a"), "a_1a_2" is the only active pair. A false count would result if these three primitives were considered as two active pairs.

However, suppose a section of a sequence appears like this: $\delta\alpha\beta_1\underline{\beta_2\beta_3}\delta$. Every pair of symbols is an active pair except for the underlined pair because of the three consecutive β symbols. If the replacement $\textcircled{\gamma} \rightarrow \alpha\beta$ is made next, then the sequence becomes $\delta\textcircled{\gamma}\underline{\beta_2\beta_3}\delta$. According to the algorithm, after the replacement, two new active pairs are considered: $\delta\textcircled{\gamma}$ and $\textcircled{\gamma}\beta_2$. But with β_1 replaced, $\beta_2\beta_3$ should be considered an active pair. While the algorithm looks for new active pairs by examining the immediately preceding and following symbols to a replacement, when overlapping pairs are concerned, this example shows that a look-ahead of two symbols from where the replacement was made is required to ensure that an active pair is not missed.

Unfortunately, longer sequences of consecutive symbols may require more look-ahead. Suppose there are five consecutive symbols as in the following section of a sequence: $\delta\alpha\beta_1\underline{\beta_2\beta_3}\overline{\beta_4\beta_5}\delta$. Two pairs of symbols are inactive, as shown. As before, suppose $\alpha\beta$ is replaced, resulting in: $\delta\textcircled{\gamma}\underline{\beta_2\beta_3}\overline{\beta_4\beta_5}\delta$. The consecutive sequence of four β's now only contains one active pair. After the replacement, the sequence should have been examined up to four symbols after $\textcircled{\gamma}$ so that the active pair $\beta_3\beta_4$ gets replaced by the two active pairs $\beta_2\beta_3$ and $\beta_4\beta_5$. The amount of look-ahead required, though, depends on the number of consecutive symbols. In the implementation of Re-Pair used, a look-ahead of just two symbols is performed, and when more consecutive symbols appear in a message, active pairs are missed. However, a missed active pair causes a problem only if it would have been replaced. Suppose in the above example, $\beta_5\delta$ is replaced by @. The sequence then becomes $\delta\textcircled{\gamma}\underline{\beta_2\beta_3}\beta_4@$. Three β's remain with only one active pair among them, which is correct. Even though the active pair $\beta_4\beta_5$ was missed, it was unimportant because of a replacement involving β_5.

The problem of consecutive symbols causing missed active pairs that should have been

replaced generally occurs for only whitespace in average text. In the 20 MiB of `WSJ20` mentioned earlier, only one active pair was missed which could have been replaced. The problem may be more serious for pathological cases and binary data, such as graphic files. The word-based parsing for Re-Pair described in the next chapter make this situation even more rare, though still possible.

The second problem with the Re-Pair algorithm is the appearance of duplicate phrases caused by consecutive symbols. After the replacement of $\alpha\alpha$ with $\textcircled{\gamma_1}$, three consecutive symbols may appear multiple times in the sequence, requiring another replacement. However, based on what pairs of consecutive symbols are active and which are inactive, two phrases may result. The first replacement might be $\textcircled{\gamma_2} \rightarrow \alpha\gamma_1$ and the second might be $\textcircled{\gamma_3} \rightarrow \gamma_1\alpha$. These two phrases are identical if they are expanded to primitives. This problem is also not handled by the implementation of Re-Pair used in this thesis, but a resolution is discussed in Chapter 5.

3.3 Coding for Re-Pair

The phrase hierarchy and the sequence produced by the Re-Pair model need to be coded separately before transmission. The coding requirements of each are different because of how they are used. The phrase hierarchy needs to be fully decoded due to the interdependency between symbols across generations. On the other hand, it is possible that only small sections of the sequence are required during retrieval. Any part of the sequence can be displayed as long as the phrase hierarchy is already available. This decoding requirement for the sequence is set aside for the moment and is considered in Chapter 6.

The Phrase Hierarchy

Larsson and Moffat [2000] discuss a variety of methods for encoding the phrase hierarchy. The best method, chiastic slide followed by interpolative coding, is used in the implementation of Re-Pair considered in this thesis.

Interpolative coding [Moffat and Stuiver, 2000] encodes a sorted list of non-uniformly

distributed integers by first encoding the median, and then recursively representing the two half lists of values which are less than or greater than the median. Since the number of possible values is restricted by the endpoints of the intervals, the medians within each interval can be encoded within the range established by its endpoints instead of the entire list. This allows shorter codes to be used when large number of values cluster together.

The two components of each phrase in the phrase hierarchy form a two-dimensional space of approximately $(|\Sigma| + |\rho|)^2$ in size. To apply interpolative coding each phrase is mapped to a single number using its left (l) and right (r) components. The method used is called the chiastic slide, denoted as $\chi(l, r)$.

Chiastic slide values are calculated incrementally starting from the first generation of phrases. The way that generations are defined ensures that each phrase has at least one of its components in the immediately preceding generation, as otherwise it would have been placed in an earlier generation. To calculate the chiastic slide values for phrases in generation i, the two-dimensional space where at least one component is in generation $i - 1$ is enumerated. The set of primitives is also interpolative coded, but does not need to be mapped using the chiastic slide. Instead, they are encoded using their underlying values. For example, the primitives of the text message of Table 3.1 are encoded as a sorted list of their Latin-1 representations.

Figure 3.4 shows how chiastic slide is used to enumerate the primitives and the first generation of phrases for the example of Table 3.1. On the left is a list of all possible left components and along the bottom is a list of all possible right components.

The set of all possible first generation phrases (unshaded region) can be calculated once all of the primitives have been sorted and encoded. The five boxed numbers in this region represent the five phrases from the first generation. This sorted list $[0, 13, 45, 46, 114]$ is then interpolative coded.

The encoding of the first generation of phrases is a special case since only one previous generation (the primitives) exists. The encoding of subsequent generations all resembles the encoding of the second generation (shaded region). Phrases in the second generation must have either both components in the first generation (dark grey region) or one com-

left component (l)																	
oo	15	9	19	29	39	49	59	69	79	89	99	109	118	125	130	133	134
ch	14	8	18	28	38	48	58	68	78	88	**98**	108	117	124	129	132	131
ck	13	7	17	27	37	47	57	67	77	87	97	107	116	123	128	127	126
d␣	12	6	16	26	36	46	56	66	76	86	96	106	115	122	121	120	119
␣w	11	5	15	25	35	45	55	65	75	85	95	105	114	113	112	111	**110**
w	10	20	39	56	71	84	95	104	111	116	119	120	104	103	102	101	100
u	9	19	38	55	70	83	94	103	110	115	118	117	94	93	92	91	90
o	8	18	37	54	69	82	93	102	109	**114**	113	112	84	83	82	81	80
m	7	17	36	53	68	81	92	101	108	107	106	105	74	73	72	71	70
l	6	16	35	52	67	80	91	100	99	98	97	96	64	63	62	61	60
k	5	15	34	51	66	79	90	89	88	87	86	85	54	53	52	51	50
h	4	14	33	50	65	78	77	76	75	74	73	72	44	43	42	41	40
d	3	**13**	32	49	64	63	62	61	60	59	58	57	34	33	32	31	30
c	2	12	31	48	47	**46**	**45**	44	43	42	41	40	24	23	22	21	20
a	1	11	30	29	28	27	26	25	24	23	22	21	14	13	12	11	10
␣	0	10	9	8	7	6	5	4	3	2	1	**0**	4	3	2	1	0
		0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
		␣	a	c	d	h	k	l	m	o	u	w	␣w	d␣	ck	ch	oo

right component (r)

Figure 3.4: Example of chiastic slide for the example of Table 3.1. The chiastic slide values for the first two generations of phrases are shown in boxes.

ponent in an earlier generation (light grey regions). Since it is more likely that one of the two components of a phrase is from a different generation, the chiastic slide assigns values starting from where one of the components is zero up to the top right corner of the dark grey region. The sorted list [98, 110] is used to represent the second generation of phrases; they are assigned symbol numbers 16 and 17.

In order to assign chiastic slide values for the current generation, i, the size of the shaded region is required. This region is bounded by the total number of primitives and phrases up to and including the immediately preceding generation, g_{i-1}, and up to but excluding it, g_{i-2}. The size of the shaded region is thus $g_{i-1}^2 - g_{i-2}^2$. In the example, to encode the second generation of phrases, $i = 2$, $g_1 = 16$, $g_0 = 11$, and $g_1^2 - g_0^2 = 134$.

The formula used to calculate the chiastic slide, adapted from Larsson and Moffat, is shown in Equation 3.1. The first two cases are the light grey regions, while the last two are used for the dark grey region. While calculating the chiastic slide requires multiplication and squaring operations (and division and square roots for decoding), many of the terms are calculated per generation and not per phrase.

$$\chi(l,r) = \begin{cases} 2l(g_{i-1} - g_{i-2}) + g_{i-1} - 1 - r & \text{for } l < g_{i-2} \\ (2r+1)(g_{i-1} - g_{i-2}) + l - g_{i-2} & \text{for } r < g_{i-2} \\ (l(2g_{i-1} - l)) + g_{i-1} - 1 - r - g_{i-2}^2 & \text{for } g_{i-2} \leq l \leq r \\ (r(2g_{i-1} - r - 2)) + g_{i-1} - 1 + l - g_{i-2}^2 & \text{for } g_{i-2} \leq r < l \end{cases} \tag{3.1}$$

In order to use interpolative coding with the chiastic slide, the phrase hierarchy needs to be sorted twice. First, the phrases are sorted by generation number, since they were added to the phrase hierarchy by decreasing frequency and not increasing generation. Second, for each generation, the phrases are assigned values using the chiastic slide, and are sorted by these values before being interpolative coded. As shown in Figure 3.4, after the primitives and phrases have been assigned chiastic slide values and sorted by them, they are enumerated starting from 0 (shown along the two axes). A final pass through the

sequence is also required to ensure it uses the re-numbered phrase hierarchy, rather than the one that corresponds to decreasing pair frequency.

The encoding of the phrase hierarchy resembles a context-free grammar in Chomsky normal form (CNF) [Chomsky, 1959]. In CNF, every production rule consists of a non-terminal which generates either two other non-terminal, or a single terminal. The encoding of phrase "ch" represents ⑭ → ②④ while the primitive "c" is represented as ② → "c". It is for this reason that each entry in the phrase hierarchy can be considered to be a rule. The significant differences between a Re-Pair phrase hierarchy and a more general CNF is that the phrase hierarchy is used to generate a single message using the sequence as a starting rule whereas a context-free grammar in CNF is usually expected to generate a language with more than one string in it.

The Reduced Sequence

The other output of the Re-Pair process is the reduced sequence. As the primitives and phrases are enumerated from 0 at the end of the chiastic slide encoding, references in the reduced sequence are just integers which refer to a rule in the phrase hierarchy. Since the purpose of Re-Pair is to remove redundancy in the message, no pair of symbols appears more than once in the reduced sequence. An entropy coder using a zero-order model on the symbols in the sequence captures the great majority of the remaining structure. Certainly, using a higher order model would be counter productive. The semi-static Huffman coder and the arithmetic coder described in the last chapter can be used without further modification to the sequence.

3.4 Experiments

Experiments were conducted on the Large Canterbury Corpus and `WSJ20` on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache machine, described in the last chapter. Larger test data could not be compressed because of the amount of memory used by Re-Pair. The next chapter demonstrates how this limitation can be overcome by

Filename	$\lvert\Sigma\rvert$	$\lvert\rho\rvert$	$\lvert\mathcal{S}\rvert$	Number of generations	Longest phrase (in primitives)
E.coli	4	67,368	652,665	20	1,800
world192.txt	94	55,473	212,648	19	432
bible.txt	63	81,229	386,095	19	548
WSJ20	90	310,486	1,904,118	21	1,257

Table 3.4: Statistics from experiments with Re-Pair.

```
< DOC>•<DOCNO>␣WSJ 870 324 -0001␣</DOCNO>•<HL>␣
John␣Blair␣ Is␣ Near␣ Accord•To␣Sell␣ Unit,␣ Sources␣Say
</HL>•<DD>␣03/2
4/87</DD>•<SO>␣WALL␣STREET␣JOURNAL␣(J)</SO>•<IN>␣
REL •TENDER␣OFFERS,␣MERGERS,␣ACQUISITIONS␣(TNM)•↩
MARKETING,␣ADVERTISING␣(MKT )•TELECOMMUNICATIONS,␣↩
BROADCASTING,␣TELEPHONE,␣TELEGRAPH␣(TEL
)␣</IN>•<DATELINE>␣NEW␣YORK␣</DATELINE>•<TEXT>•␣␣␣
John␣Blair␣&amp;␣Co.␣ is␣close␣to␣ an␣agreement␣to␣
```

Figure 3.5: Expanding the first 20 symbols of the WSJ20 sequence into primitives. For the purposes of display, • indicate linefeeds and long phrases are broken at the locations indicated by ↩.

partitioning the message into blocks. For all experiments described in this chapter, each message is processed as a single block.

Some statistics from applying Re-Pair on these files are shown in Table 3.4. As an example of the phrases formed by Re-Pair, the beginning of WSJ20 is presented in Figure 3.5.

The compression ratios achieved by Re-Pair on these four files are shown in Table 3.5. The last three columns of this table represent compression ratios for bzip2, gzip, and ppmd, taken from Table 2.5 on page 41. Recall that gzip and bzip2 were executed with the -9 option, while a seventh order model for ppmd was used to ensure the best possible compression. Execution of ppmd was limited to 255 MiB of memory, so as to effectively remove the restriction on its memory usage and reduce the chance of requiring the model to be re-constructed. For the results reported in Table 3.5, the model was not re-built by ppmd. While Re-Pair has the potential to use more memory than any of these compression systems, they are reasonable benchmarks due to their widespread popularity.

Filename	Re-Pair			gzip	bzip2	ppmd
	Phrase hierarchy	Total using shuff	Total using uint			
E.coli	0.108	2.086	2.081	2.240	2.158	2.034
world192.txt	0.312	1.624	1.597	2.333	1.584	1.454
bible.txt	0.290	1.763	1.749	2.326	1.671	1.564
WSJ20	0.223	1.774	1.770	2.907	2.078	1.656

Table 3.5: Compression ratios for the Large Canterbury Corpus and WSJ20 using Re-Pair. Total compression ratios for Re-Pair include the phrase hierarchy and the coded sequence with either shuff or uint. Compression results from the last three columns have been taken from Table 2.5.

Filename	Encoding			Decoding		
	Re-Pair	shuff	uint	Des-Pair	shuff	uint
E.coli	16.2	0.3	1.1	0.4	0.1	1.2
world192.txt	9.1	0.1	0.4	0.3	<0.1	0.4
bible.txt	15.0	0.3	0.9	0.4	0.1	1.0
WSJ20	88.5	1.6	6.7	2.5	0.4	7.5

Table 3.6: Compression and decompression times for the Large Canterbury Corpus and WSJ20 using Re-Pair and Des-Pair in seconds. Times have been averaged over three trials.

Compression effectiveness using Re-Pair is better than gzip in all cases and varies with bzip2, depending on the file. This is an expected result given the differences in the amount of memory used. ppmd achieves better compression than Re-Pair, but as mentioned in the last chapter, statistical models can be expected to outperform dictionary-based ones. The arithmetic coder gives better compression than the Huffman coder. Note also that the phrase hierarchy is small compared to the original message and the reduced sequence, occupying less than 20% of the final message for all of the files.

Compression and decompression times for Re-Pair are shown in Table 3.6 in CPU seconds, averaged over three trials. Even though the coding of the phrase hierarchy is logically separate from the Re-Pair process, in the implementation of Re-Pair used, the coding of the phrase hierarchy is tightly coupled with Re-Pair, and so, the execution times of Re-Pair and Des-Pair both include the encoding and decoding times of the phrase hierarchy.

Comparing the compression times of Re-Pair with the other programs (Table 2.6), Re-Pair takes more time even if entropy coding is not taken into account. However,

decoding with the two dictionary-based systems, DES-PAIR and GZIP, are fast. Of the two, GZIP is consistently faster. Furthermore, the encoding and decoding times are higher for arithmetic coding compared to Huffman coding. Because of this speed difference and the very slight difference in compression, the entropy coder used by experiments in later chapters is the Huffman coder. The downside of this decision is the slightly worse compression, as shown in Table 3.5.

3.5 Related Work

Other than operating off-line, several other differences exist between RE-PAIR and the dictionary-based algorithms of Chapter 2. First, RE-PAIR explicitly separates the dictionary from the sequence of references, producing two streams that together, make up the compressed message. Second, every primitive and phrase must be used at least once in order to appear in the RE-PAIR phrase hierarchy. In contrast, phrases may be added to the dictionaries formed by LZ77 or LZ78, but never referred to.

Storer and Szymanski [1982] showed that, in a dictionary-based compression algorithm, determining the phrases that would achieve the best compression for a given message is NP-hard. Therefore, like LZ77 and LZ78, many dictionary-based compression algorithms use heuristics for phrase construction. Bentley and McIlroy [2001] considered a pre-processor to compression which identifies long repeated strings appearing far apart.

Wolff [1975] described a program called MK10 for identifying breaks in text in the absence of any whitespaces. The system operates similar to RE-PAIR by identifying and recursively replacing pairs of symbols. However, unlike RE-PAIR, the message is scanned from left to right and replacements are made after a pre-determined threshold has been met (for example, 10 occurrences). Later, Wolff [1977] extended this work by selecting the most frequent pair of symbols after one complete scan of the message. However, since the primary goal of Wolff was the segmentation of text, little attention was given to the compression effectiveness or efficiency of MK10.

The incremental encoding of Rubin [1976], like RE-PAIR, uses frequency of digrams (pairs of symbols) and operates through recursive pairing for compression. However, more

emphasis is placed on conditions for program termination rather than a compact method of storing the dictionary. Some of these conditions include phrase frequency and phrase length. There were also few details given on an efficient implementation with respect to time or memory space. However, as an indication of the amount of time and space that such an algorithm would require, Rubin [1976] noted that the methods used have "fairly high computation time and large storage requirements".

Manber [1997] used digrams for searching in compressed text. Even though Latin-1 is a character set of size 256, half of it is unused by most English documents. The remaining 128 values were assigned to frequently occurring pairs of primitives and used to compress a message. The search pattern was similarly compressed so that a general-purpose string matching algorithm could be used to locate occurrences of the pattern within the compressed message. The problem of overlapping pairs is avoided by ensuring that a symbol which is a left component of one pair of symbols cannot be a right component of another pair. Earlier, Gage [1994] devised a technique called Byte-Pair Encoding (BPE) which resembles the algorithm by Manber, except that searching was not considered.

More recently, the Off-Line mechanism of Apostolico and Lonardi [2000] showed how an annotated trie for a message could be used to locate candidates for a dictionary. The phrases in the dictionary were created directly from substrings of the message without referring to other phrases in the dictionary. Central to Off-Line is the computation of a *gain measure*. The gain measure determines the next best phrase to add to the dictionary using criteria such as substring length, substring frequency, and pointer costs to all occurrences. Three methods of dictionary encoding were employed, one of which stored the phrases as literal text in an external dictionary. Turpin and Smyth [2002] extended this work by looking at other methods of computing the gain measure.

The Ray system [Cannane and Williams, 2001] also compresses a message off-line, but through multiple passes so that the message itself does not need to be stored in memory. It builds a dictionary by recursively replacing diagrams, similar to Re-Pair. Each pass over the message is roughly equivalent to a generation in Re-Pair. The Ray algorithm consists of three steps: accumulation of frequencies, the identification of candidates, and

replacement of symbol pairs. The first step is only performed once since the sequence's statistics are updated as replacements are made. The remaining two steps are repeated until either no pair of symbols appears twice or more, or a preset limit on generations has been reached. Pairs of symbols of equal frequency were handled differently than RE-PAIR. If the three symbols $\alpha\beta\delta$ appear in the sequence and step 2 is being performed, $\alpha\beta$ is identified as a candidate for replacement only if α occurs with an equal or higher frequency than β. If this is not the case, then this pair of symbols is skipped, and the next pair, $\beta\delta$, is considered. The RAY dictionary is encoded using Elias and Huffman codes.

Finally, operating on-line instead of off-line is the SEQUITUR algorithm of Nevill-Manning and Witten [1997]. The SEQUITUR algorithm builds a dictionary resembling a grammar, but unlike RE-PAIR, a rule can generate more than two symbols. During compression, as each message symbol is read, the grammar is updated according to two rules: *digram uniqueness* and *rule utility*. Digram uniqueness ensures that no pair of adjacent symbols appear more than once in the grammar. Digram uniqueness effectively adds phrases to the dictionary as symbols appear in the sequence (rule $\mathcal{S}$). The creation of rules can cascade from rule $\mathcal{S}$, and results in a hierarchical grammar. Rule utility prohibits any rule from being used only once. When a rule created earlier ends up appearing once in the right-hand sides of all of the rules, it is expanded at the location it is used and then removed from the grammar. Adherence to rule utility results in the right-hand sides of rules becoming longer than just digrams. Rule S is transmitted to the receiver while the rest of the grammar is sent implicitly using back-pointers, similar to a LZ77 implementation. Also, Nevill-Manning and Witten [2000] compared on-line and off-line techniques for SEQUITUR by using phrase heuristics such as most frequent, longest, and most compressive.

An example of SEQUITUR is shown in Table 3.7 using the same "Woodchuck" message as the one from the example of Table 3.1. Each row of the table indicates the current rule $\mathcal{S}$, the SEQUITUR rule that was violated, and the action required to fix the grammar. The first row shows the first time a rule violation has occurred. If a rule already exists when digram uniqueness is enforced, no new rule is added to the dictionary, as shown with the

Rule $\mathcal{S}$	Rule enforced	Action applied to grammar
how␣much␣wood␣could␣	digram uniqueness	Add ① → d␣
how␣much␣woo①coul①a␣w	digram uniqueness	Add ② → ␣w
how␣much②oo①coul①a②o	digram uniqueness	Add ③ → ②o
how␣much③o①coul①a③	rule utility of rule ②	Expand ②: ③ → ␣wo Remove rule ②
how␣much③o①coul①a③o	digram uniqueness	Add ④ → ③o
how␣much④①coul①a④dch	digram uniqueness	Add ⑤ → ch
how␣mu⑤④①coul①a④d⑤uck␣ch	digram uniqueness	Use rule ⑤
how␣mu⑤④①coul①a④d⑤uck␣⑤u	digram uniqueness	Add ⑥ → ⑤u
how␣mu⑤④①coul①a④d⑥ck␣⑥c	digram uniqueness	Add ⑦ → ⑥c
how␣mu⑤④①coul①a④d⑦k␣⑦	rule utility of rule ⑥	Expand ⑥: ⑦ → ⑤uc Remove rule ⑥
how␣mu⑤④①coul①a④d⑦k␣⑦k	digram uniqueness	Add ⑧ → ⑦k
how␣mu⑤④①coul①a④d⑧␣⑧	rule utility of rule ⑦	Expand ⑦: ⑧ → ⑤uck Remove rule ⑦

Table 3.7: A sample application of the SEQUITUR algorithm on the message used in Table 3.1. Message symbols are read one at a time and appended to rule $\mathcal{S}$, shown in the first column. Each row represents the moment when one of the two rules of SEQUITUR (digram uniqueness or rule utility) need to be enforced. The SEQUITUR rule that is violated and the action performed to enforce it are shown in columns 2 and 3, respectively.

third occurrence of "ch". After phrase ③ is added to the dictionary, phrase ② is used only once. Phrase ② is expanded in phrase ③ and removed from the dictionary because of rule utility. At the end of this example, the rules sent to the receiver are $\mathcal{S}$, ①, ③, ④, ⑤, and ⑧.

While the SEQUITUR algorithm differs from RE-PAIR in the above points, a system for phrase browsing has also been developed called PHIND [Nevill-Manning et al., 1997]. The methods used by PHIND, discussed later in Chapter 7, are similar to the ones for RE-PAIR, but differences emerge due to their underlying compression mechanisms.

3.6 Retrieval with Re-Pair

In Chapter 2, a wide range of algorithms for modelling were presented. This chapter adds another dictionary-based model to the list, which is more suited for both retrieval and browsing.

The most important characteristic of the RE-PAIR algorithm is that recursive pairing based on frequency is more appropriately implemented in a semi-static manner, so that the entire message is available. As a consequence of this, the compressed message is composed of two separate streams, which each has its own advantages. The phrase hierarchy is small and occupies only 3% of the space taken by the original message. Moreover, the phrase hierarchy contains a significant amount of information about the document. The hierarchical structure of the phrase hierarchy means that it describes the many relationships symbols form with lower and higher generation symbols. The phrase hierarchy is not only crucial for decompression, but appears suitable for phrase browsing, as well. The second stream is the reduced message of references to the dictionary.

Searching the reduced sequence has been examined by Manber [1997] and Shibata et al. [1999], who based their work on algorithms similar to RE-PAIR. While Manber modified the pattern prior to searching, Shibata et al. presented two approaches. First, they expanded all corresponding encodings of the pattern before applying a pattern matching algorithm. Second, they used the Knuth-Morris-Pratt automaton [Knuth et al., 1977] for the pattern to aid the search. In addition to searching, the reduced sequence can be decoded from any reference, provided the phrase hierarchy has been processed already.

At the conclusion of the last chapter, three requirements were mentioned which a compression algorithm should satisfy in order to be adapted for retrieval. First, the algorithm should allow decoding from an arbitrary point in the compressed message, which is satisfied by the structure of the two streams from RE-PAIR. In contrast, since algorithms like PPM and block sorting rely on symbol contexts, a random point in the compressed representation cannot be decoded without examining everything before. Alternatively, a compressed message made with block sorting can be pre-processed in-memory so that an indexing structure is available for searching [Ferragina and Manzini, 2000]. The LZ77 and LZ78 algorithms are both dictionary-based like RE-PAIR, but they were designed to be adaptive and so, combine the dictionary and the references into a single stream.

The second requirement is that decoding should be fast so that the amount of time that a user of a retrieval system waits is minimised. While RE-PAIR is slower than GZIP for

decoding, a comparison between the results of Table 3.6 and Table 2.6 show that RE-PAIR is noticeably faster than BZIP2 and PPMD.

Finally, the alignment of codewords to bits rather than bytes would facilitate more efficient decoding. Experiments in this chapter compressed the reduced sequence with SHUFF and UINT. Even though Huffman coding was advocated after examining the decoding times of SHUFF and UINT, an alternative coding mechanism which is able to decode the sequence faster may be possible which sacrifices some compression effectiveness. As for the phrase hierarchy, the good compression effectiveness attained with interpolative coding make the lack of any byte-alignment acceptable.

There are, however, four changes to RE-PAIR before it can be included into a retrieval system. First, the use of frequency alone to select phrases for the phrase hierarchy presents problems for retrieval. In a retrieval system, it is expected that the query terms match the primitives or phrases in the phrase hierarchy. The solution to this problem requires the attention to shift to how RE-PAIR can be forced to select words and phrases which resemble the ones displayed to the user in the interface.

The second change deals with the problem of memory space required by RE-PAIR. While RE-PAIR operates in space linear in the length of the message, the constant factor is quite high. Assuming four bytes per word, compressing `WSJ20` requires 240 MiB for the sequence nodes alone. In order for a retrieval system to be useful, files one or even two orders of magnitude larger may need to be processed using reasonable amounts of memory.

The third improvement is with the entropy coding applied to the sequence. While phrase browsing is performed solely with the phrase hierarchy, searching for those phrases involves the reduced sequence. By using an entropy coder, if appearances of a phrase are being sought, the reduced sequence needs to be sequentially decoded. Instead, a coding mechanism is required which allows efficient searching through the reduced sequence.

Finally, an interface is required which allows users to browse phrases and then locate and examine the contexts in which they occur.

Solutions to each of these four problems are investigated separately in the next four

chapters, in the order they have been listed. Chapter 4 describes alternatives to Re-Pair's method of selecting phrases by first aligning phrases to whitespaces, and then by using a word-based pre-processing scheme using punctuation as a guide. Chapter 5 shows how a message can be partitioned into blocks by Re-Pair so that larger messages can be compressed. Compressed blocks are merged in order to produce a single phrase hierarchy for browsing. Chapter 6 examines alternatives to entropy coding the entire reduced message to allow fast searching for phrases. And finally, Chapter 7 describes phrase browsing and an interface which ties the above components together.

Chapter 4

Selecting Phrases for Browsing

The previous chapter showed how Re-Pair reduces a message by repeatedly selecting active phrases, replacing their corresponding active pairs with new symbols, and adding these new symbols to the phrase hierarchy. Active phrases are chosen in order of decreasing number of active pairs, with ties broken by choosing the active phrase created first.

Re-Pair's approach to phrase construction uses primitives as the smallest building block in the phrase hierarchy. In the previous chapter, primitives were defined as the Latin-1 representations used in the message. However, people do not view messages as characters, but as clusters of alphabetic characters which form words with whitespace characters in between. A user that browses phrases drawn from a Re-Pair phrase hierarchy should be presented with meaningful words rather than arbitrary strings of characters that might not align with word boundaries. In order to accomplish this, Re-Pair needs to treat whitespace and non-whitespace characters differently.

This chapter considers the problem of how Re-Pair can build a phrase hierarchy made up of useful phrases for browsing. The policy used by Re-Pair to select phrases based on decreasing frequency alone is insufficient for this purpose. Three alternative methods are considered which complement Re-Pair's phrase selection heuristic.

The remainder of this chapter is structured as follows. In Section 4.1, an overview of information retrieval and compression systems which operate on words rather than characters is provided, along with the definition of "word" that is used in this chapter. Then, the three methods of extending Re-Pair are examined. In Section 4.2, the first

method adds rules to Re-Pair in order to align words to the word boundaries implied by the non-words that surround them. The second method introduces a word-based pre-processing phase which explicitly separates a document into two streams: one for words and the other for non-words. In Section 4.4, a third method combines the first two methods by using the word-based pre-processing step with a new set of rules for punctuation marks. Section 4.5 compares these three methods, and shows the impact of the changes through experiments. Finally, the chapter concludes with a summary of the three methods described.

4.1 Forming Words from Characters

Re-Pair, as described in the last chapter, views documents at the character level. However, people read and understand documents as a sequence of words. In a retrieval system, the query terms that a user provides and the results that the system returns should be in the form of words. Just as index-based retrieval systems build indexes using words, so should a phrase browsing system using Re-Pair.

Information retrieval systems generally parse their input into three different classes of symbols: word symbols, inter-word symbols, and special processing symbols [Kowalski, 1997]. Documents are treated as an alternating sequence of word symbols and inter-word symbols (or non-word symbols). Some retrieval systems also make use of special processing symbols for separating documents [Witten et al., 1999] or for indicating meta-data, such as SGML tags.

Symbols are encoded individually by a compression system which assume a memoryless source. Because the characters that form words in a document do not appear independently, word-based mechanisms have also been employed for compression. A word-based compression system requires choices to be made with respect to how the document is parsed. When a document is parsed as words, similar to a retrieval system, suitable definitions of words, non-words, and special processing symbols (if any) are required. These different symbol types may exist in the reduced message as different streams, or as a single, unified stream. Finally, some methods of parsing a message into words replaces

the words with references to a lexicon (or dictionary) that also needs to be encoded and transmitted.

Bentley et al. [1986] devised a word-based parsing scheme which separated a message into two alternating streams: one of alphanumeric symbols and another of non-alphanumeric symbols. A move-to-front scheme (MTF) followed by Huffman coding was used to encode word references. Moffat [1989] undertook experiments on the Bentley et al. scheme, and also extended character-based PPM to word-based PPM and found improvement in compression effectiveness with a first-order model, but little additional improvement with a second-order model. An alternating sequence of words and non-words was also used, with a 20 character limit on each. Horspool and Cormack [1992] investigated word-based compression for LZ78, where the lexicon was transmitted implicitly by using an escape mechanism to indicate the appearance of a new word. They also investigated first-order context modelling by conditioning a word based on the parts-of-speech (noun, verb, article, adjective, or other) of the preceding word. The parts-of-speech of words were determined through dictionary look-up. Words that did not appear in the dictionary were assigned to the category that would achieve the best compression. Isal and Moffat [2001] devised a compression scheme that combined block sorting with a word-based model. Words and non-words were limited to 12 characters each and a single lexicon was produced by adapting the *spaceless words* approach of de Moura et al. [2000]. The spaceless words approach combines the word and non-word lexicons into a single lexicon. During decoding, a space character is added after every word. If a word is followed by an explicitly coded non-word, then no space is added and the non-word is decoded in its place instead. Isal and Moffat [2001] reported a compression ratio of 1.69 bpc for `WSJ20` when combining word-based block sorting with a suitable coder.

Nevill-Manning and Witten [1997], parsed semi-structured documents into words so that SEQUITUR could be applied to references to words in the word lexicon. In semi-structured data, words may be conditioned on other words that are not immediately adjacent. As a trivial example, a closing tag in an SGML message appears because of an opening tag earlier in the message. SEQUITUR used heuristics based on the amount

of gain that could be obtained from a distant word prediction. Experiments showed that SEQUITUR attained better compression effectiveness than other systems, including GZIP and a PPM implementation using escape method C, for a sample semi-structured document. Compression effectiveness further improved when the heuristics were employed.

The changes to RE-PAIR described in this chapter reflect previous work in word-based compression. However, some different choices were made because the goal of this investigation is to balance phrase browsing with compression. Before discussing word-based compression for RE-PAIR, some definitions are required. A word character is any alphabetic or numeric character in the Latin-1 character set. In order to accommodate the SGML test data used in this thesis, the three characters "<", ">", and "/" are also considered to be word characters. Any other symbol, including whitespace and punctuation marks, are non-word characters. No special processing symbols are defined at this time. A sequence of contiguous word (or non-word) characters is a *token*. A document is viewed as an alternating sequence of word tokens and non-word tokens.

The original version of RE-PAIR described by Larsson and Moffat [2000] and in Chapter 3 is called character-based RE-PAIR. This distinction is required in order to differentiate it from three additional word-aware versions of RE-PAIR that are described in the next three sections.

4.2 Word-aligned Re-Pair

The first method of processing a document as words is called *word-aligned* RE-PAIR. Word-aligned RE-PAIR augments rules to the part of the RE-PAIR algorithm that decides which pairs of symbols are permitted as active pairs.

Figure 4.1 provides the motivation behind this variation of RE-PAIR. The figure expands the phrases generated by character-based RE-PAIR in Table 3.1 on page 48 into all primitives. The phrases in Figure 4.1 are listed in the order in which they are added to the phrase hierarchy. Three of these eight phrases contain the space character, a non-word symbol. While browsing phrases with non-word symbols is visually unappealing, a more noticeable problem is its effect on the choice of phrases made by RE-PAIR. Since

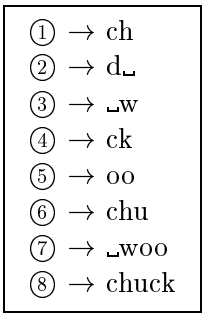


Figure 4.1: Expanding the phrases from the example of Table 3.1 into all primitives.

the phrases "␣w" and "d␣" in Figure 4.1 occur with a higher frequency in the message than "wo" and "od", the likelihood of the word "wood" from being formed as a phrase is reduced. The problem is more obvious with longer documents and is caused by all non-word symbols, not just the space character.

Word-aligned Re-Pair solves this problem by introducing the four rules shown in Table 4.1. As each phrase is added to the phrase hierarchy, it is labelled as a word symbol (W), a non-word symbol (N), or a mixture (M). The primitives (characters) are also labelled as "W" or "N" during initialisation. When two adjacent symbols are being considered as potential active pairs, the word-aligned rules are consulted. The second and third columns represent the classification of the two symbols under consideration. The first and fourth columns represent the surrounding context in which that pair of symbols occurs in the sequence. An "X" indicates that the classification is unimportant. If the pair of symbols satisfies any one of the four rules, then it can become an active pair. The fifth column of the table indicates the category of the corresponding symbol after it has been added to the phrase hierarchy. For example, when an active phrase representing active pairs composed of two non-word symbols is added to the phrase hierarchy, it is also classified as a non-word symbol. The last column provides examples which satisfy each rule, with boxes representing the two symbols that are being considered as an active pair.

The alignment of words is caused by the fourth rule. The fourth rule ensures that a word symbol can only be combined with a non-word symbol, forming a mixed symbol, if both of them are *complete*. A complete word is a sequence of one or more word symbols,

Before-left	Left	Right	After-right	Result	Example
X	N	N	X	N	woodchuck␣␣chuck␣␣wood
X	W	W	X	W	woodchuck␣␣chuck␣␣wood
X	M	M	X	M	woodchuck␣␣chuck␣␣wood
N | M	W	N	W | M	M	woodchuck␣␣chuck␣␣wood

Table 4.1: The rules used by word-aligned RE-PAIR for deciding when a pair of symbols can become active pairs. If a pair of adjacent symbols satisfy any of the four rules, then it becomes an active pair.

such that on both sides of it, there are only non-words. A similar definition applies to non-words. As there is only one such rule, words can only be left components of mixed symbols, and non-words can only be right components. The first three rules simply allow longer phrases of the same type to be formed.

These four rules are applied at two locations in the RE-PAIR algorithm of Algorithm 3.1. They are used when the message is initially scanned (step 1), and after an active pair replacement is made and the frequency counts of the newly introduced symbol with its adjacent neighbours is incremented (step 3(a)iv). This ensures that the active pair counts for active phrases only include active pairs that satisfy one of these rules. During recursive pairing, even though the sequence changes from the time when an active pair is identified to when it is replaced, the classification of active pairs remains correct. The first three rules classifies pairs of symbols regardless of their contexts. More importantly, changes in the sequence do not affect pairs classified using the fourth rule. Suppose a pair of symbols is classified as a mixed pair when the symbol preceding the left component is a non-word symbol. Later, when this pair is replaced, the preceding non-word symbol may have become part of a mixed symbol. But since all mixed phrases start with a word symbol and end with a non-word symbol, the fourth rule still holds for this active pair.

Table 4.2 shows word-aligned RE-PAIR applied to the same "Woodchuck" message, with the third column showing the expanded phrases. The expansion of phrase ⑥ is the word "wood", as desired. The pair "d␣" does not exist as a phrase because, in this message, "d" does not appear by itself as a complete word. As another example, Figure 4.2 shows the expanded phrases generated by word-aligned RE-PAIR when it is applied to

Sequence	Phrase hierarchy	Expanded phrases
how␣much␣wood␣could␣a␣woodchuck␣chuck		
how␣mu①␣wood␣could␣a␣wood①uck␣①uck	① → c h	→ ch
how␣mu①␣wo②␣could␣a␣wo②①uck␣①uck	② → o d	→ od
how␣mu①␣wo②␣could␣a␣wo②①u③␣①u③	③ → c k	→ ck
how␣mu①␣④②␣could␣a␣④②①u③␣①u③	④ → w o	→ wo
how␣mu①␣④②␣could␣a␣④②⑤③␣⑤③	⑤ → ① u	→ chu
how␣mu①␣⑥␣could␣a␣⑥⑤③␣⑤③	⑥ → ④ ②	→ wood
how␣mu①␣⑥␣could␣a␣⑥⑦␣⑦	⑦ → ⑤ ③	→ chuck

Table 4.2: Applying word-aligned Re-Pair to the same brief "Woodchuck" message used in Table 3.1 on page 48.

the complete "Woodchuck" message (see page 18). The word-alignment rules ensure that word symbols and non-word symbols continue to grow until they are surrounded by symbols of a different type. Then, mixed symbols are created by having complete word symbols combine with complete non-word symbols on the right. In the figure, every symbol represents phrases that are either part of an English word, a complete English word, or an alternating sequence of complete words followed by complete non-words. In this example, the longest non-word symbol is only one symbol long (the space character, ␣), so complete non-word phrases that are longer than one primitive do not exist in the phrase hierarchy.

In a phrase hierarchy with $|\Sigma|$ primitives and $|\rho|$ phrases, word-aligned Re-Pair requires an extra $(|\Sigma| + |\rho|)$ words of memory compared to character-based Re-Pair. Every primitive and phrase in the phrase hierarchy is assigned a category. Since the categories can be represented in 2 bits, an implementation which conserves memory is possible which only uses an extra $2(|\Sigma| + |\rho|)$ bits. Categories could also be assigned to each symbol in the sequence. However, for a message of length n, this approach requires n words of memory, which is typically larger than $(|\Sigma| + |\rho|)$. Furthermore, augmenting categories to the sequence is unnecessary because the category of a symbol in the sequence is the same as its category in the phrase. The phrase "wood" is a word symbol (W), regardless of the context in which it appears. When an occurrence of "wood" is considered as a component of an active pair, then its adjacent neighbours in the sequence are checked.

(W)	① → ch	(M)	⑯ → wood␣
(W)	② → wo	(W)	⑰ → could
(W)	③ → od	(M)	⑱ → could␣
(W)	④ → ck	(W)	⑲ → if
(W)	⑤ → chu	(W)	⑳ → as
(W)	⑥ → wood	(W)	㉑ → mu
(W)	⑦ → chuck	(M)	㉒ → a␣woodchuck␣could␣
(M)	⑧ → a␣	(M)	㉓ → if␣
(W)	⑨ → ld	(M)	㉔ → as␣
(W)	⑩ → woodchuck	(W)	㉕ → much
(M)	⑪ → chuck␣	(M)	㉖ → a␣woodchuck␣could␣chuck␣
(W)	⑫ → uld	(M)	㉗ → chuck␣if␣
(M)	⑬ → woodchuck␣	(M)	㉘ → wood␣as␣
(M)	⑭ → a␣woodchuck␣	(M)	㉙ → much␣
(W)	⑮ → co	(M)	㉚ → chuck␣if␣a␣woodchuck␣could␣chuck␣

Figure 4.2: Expanded phrases generated by word-aligned Re-Pair on the complete "Woodchuck" message, in the order in which they were added to the phrase hierarchy. In front of each phrase, its word-aligned category is shown. No phrases in this example are non-words (N).

There are two advantages to word-aligned Re-Pair. First, the alignment performed by word-aligned Re-Pair operates transparent to the decompressor. The augmented rules only affect the compressor. Second, word-aligned Re-Pair allows compound words like "woodchuck" to be split up into its constituent words, provided they exist elsewhere in the message by themselves.

Experimental results with word-aligned Re-Pair are provided at the final section of this thesis. However, problems with word-aligned Re-Pair exist, and are addressed in the next section by word-based Re-Pair.

4.3 Word-based Re-Pair

Word-aligned Re-Pair creates phrases that are more useful than those of character-based Re-Pair, by aligning phrases to word boundaries. However, several problems remain in the phrase hierarchy which require a different solution. First, two words used in the complete "Woodchuck" message cannot be found in the phrase hierarchy of Figure 4.2: "how" and "would". Each word exists in the message only once and while "how" can be

located by performing a direct search on the reduced sequence using primitives, "would" cannot be found since it has been broken into two phrases: ② and ⑫. Second, punctuation was removed in the complete "Woodchuck" message, but if it was included, then the phrase "wood␣" is treated differently from "wood?". To a user, both phrases have the same meaning, except the second one marks the end of a sentence. Furthermore, case information of letters was removed from the message, but word-aligned Re-Pair would have treated "Wood␣" as being a different phrase from "wood␣".

That is, while word-aligned Re-Pair is an improvement over character-based Re-Pair, some problems remain. The essence of the problems is that word-aligned Re-Pair does not force a message into words, but merely applies a set of rules only when a decision is required. With respect to the first problem above, since "how" and "would" each appears once in the message, no decision was required by Re-Pair, resulting in neither word appearing in the phrase hierarchy. Instead, a word-based scheme is required, similar to the ones used by other compression systems described earlier. Word-based compression requires an additional processing stage where a message is divided into word tokens and non-word tokens. The processing can occur as the message is compressed, or it may complete before compression begins.

The pre-processing used by word-based Re-Pair needs to satisfy two criteria in order to strike a balance between the requirements of retrieval and compression. First, the phrases produced by Re-Pair after the pre-processing should improve compared to ones created by the character-based and word-aligned Re-Pair variants. For example, phrases should be aligned on word boundaries. Second, like other compression algorithms, compression must be lossless. Figure 4.3 shows a structure that satisfies these two conditions.

In Figure 4.3, processes are shown as rectangles, while the flow of data are indicated as lines. Each process is further elaborated later in this section. Initially, the message is separated into two streams of tokens: one for words and the other for non-words. The word tokens are then case folded and stemmed. Lexicons for words and non-words are maintained separately and updated as the message is processed. The word references are then compressed by Re-Pair. The punctuation flags, shown as dashed lines, are explained

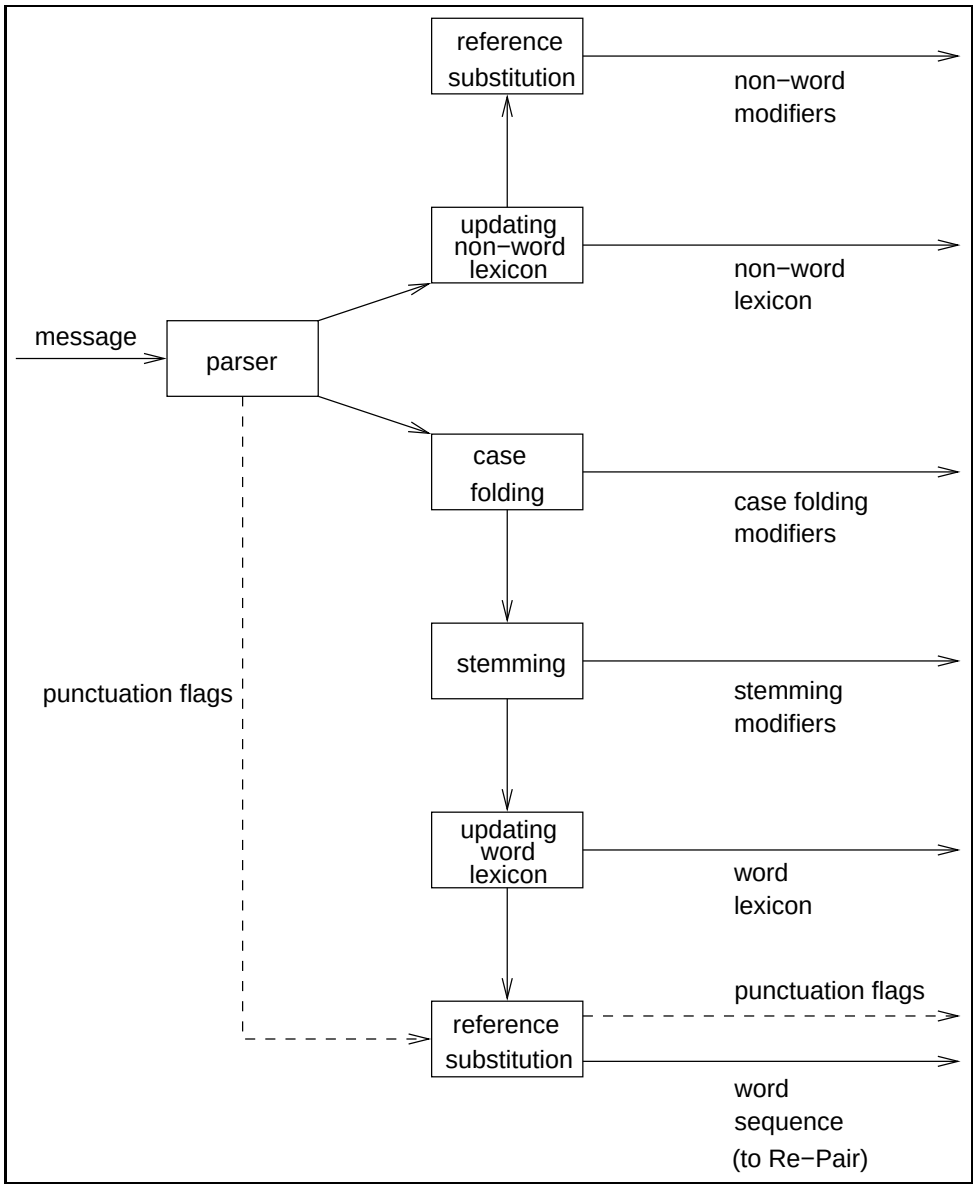


Figure 4.3: The pre-processing stage used by word-based Re-Pair to separate a message into non-words (top path) and case folded, stemmed words (bottom path). Punctuation-aligned Re-Pair, described in Section 4.4, also follows a similar scheme, except for the addition of punctuation flags for Re-Pair, shown as dashed lines.

in Section 4.4. All of these steps are performed by the pre-processor, dubbed Pre-Pair, to emphasise its use before Re-Pair.

Separating the Message

The message is parsed as an alternating sequence of word and non-word tokens. A token is at most 16 characters in length. For longer words (and non-words), a special zero-length token exists in both types of tokens. For example, if a word of length 21 characters is encountered, then it is split so that two words of lengths 16 and 5 are encoded, with a zero-length non-word in between. This parsing scheme is similar to the one described by Moffat [1989].

Furthermore, to accommodate the SGML test files used by this thesis, the definition of words is extended. The "<" character specifies the start of a new word token while the ">" character specifies the end of one, regardless of the context in which they occur. This definition ensures that SGML tags are tokens, regardless of whether or not there is whitespace surrounding them.

Case folding

After a word token has been identified, it is case folded to lower case. Case folding is a technique commonly used by retrieval systems for constructing and accessing an index. It permits a user to enter query terms without considering whether the case of the words being given to the system matches the case of the indexed terms. However, in retrieval systems, the index is separate from the message that it indexes. So, the case folding can be a lossy transformation because it need not be reversed. In contrast, the case folding done by Pre-Pair needs to be reversible since Re-Pair is used for constructing phrases for both browsing and compression. Therefore, a modifier is produced for each word token when case folded. When the modifier is applied to the case folded token, the original word is produced.

Since any character in a word might be in upper case and so require folding, a modifier of at least 16 bits is required, with each bit corresponding to a character position. A bit value of "1" means that the corresponding source character is in upper case. The bit flags are in reversed order with respect to the characters in the word token. That is, a word token with only the first character in upper case results in a case folding modifier with

the least significant bit turned on. This ensures that all word tokens with only the initial character in upper case have the same case folding modifier, regardless of word length. Furthermore, an extra bit is used to indicate the special case when every character is in upper case, so that the case folding modifier is 17 bits in length. While encoding a token, every character is checked. When decoding, as soon as no other bits in the modifier are set to "1", the rest of the word is not processed.

Reversible case folding is also used by the Length Index Preserving Transform (LIPT) of Awan and Mukherjee [2001]. LIPT is a compression pre-processor that uses a static dictionary of lower case words. As most words that contain upper case characters either have only the first character in upper case or all of the characters in upper case, special symbols are inserted into the message to indicate this. For other combination, individual characters are flagged if they are in upper case. PRE-PAIR operates similar to LIPT except that the case folding modifiers of PRE-PAIR exist as a stream separate from the words. Also, PRE-PAIR assigns a bit flag to every character that is processed while LIPT adds a character modifier only if an upper case character is found.

Stemming

After the word token is case folded, it is stemmed. Stemming, or conflation [Frakes, 1992] is a technique used in information retrieval to remove word suffixes to produce its root form. For example, "searches", "searching", and "search" all stem to the word "search". This allows users to query a retrieval system without worrying about entering the correct ending for a word. Moreover, stemming also reduces the number of entries in the lexicon.

Several authors have proposed stemming algorithms [Lovins, 1968, Dawson, 1974, Porter, 1980]. Of these, the Porter stemming algorithm is used in this chapter, based upon the recommendation of Kowalski [1997].

All stemming algorithms conflate a word by applying steps which successively remove endings. A step is applied if the word matches a set of conditions. Some conditions include the length of the word, the characters in the suffix, or the characters before the suffix. The algorithms do not consider the semantic meaning of the word, and each

Original word	Root word obtained	Expected root word
character	charact	character
frequency	frequenc	frequent
retrieval	retriev	retrieve
searchable	searchabl	search
skies	ski	sky
pies	pi	pie

Table 4.3: Some unexpected results from the Porter stemming algorithm. Words are listed in the first column. The root form obtained by the algorithm is shown in the second column, while the third column displays the root form a person most likely would provide.

word in a document is stemmed independently of adjacent words. This is acceptable for most information retrieval systems since the stemmed word is used internally to search through an index. However, these results would appear unexpected to a user. Table 4.3 shows a few examples from applying the Porter stemming algorithm. In most cases, the root word obtained is not an English word. Of particular interest, the word "skies" is transformed to the unrelated word "ski", instead of "sky", through the removal of the ending "es". Nevertheless, the benefits of stemming outweigh the costs, and provided all related words and no unrelated words stem to the same string most of the time, no problems are caused. As words are combined to form longer words during browsing, it is expected that neighbouring words make up for any important contexts lost during stemming.

The stemming algorithm used by Re-Pair contains minor changes to the original stemming algorithm by Porter. Details of the changes are provided in Appendix B. The American spelling assumed by the implementation is retained in order to accommodate the spelling used by the test data.

As with case folding, compression implies a requirement to be able to losslessly reverse any stemming performed. A stream of stemming modifiers is generated, with each word token having a corresponding modifier. Appendix B lists the steps to the algorithm and shows that a modifier of length 23 bits is sufficient to indicate how to unstem a word. Each step (and sub-step) is assigned bit positions in the modifier, independent of other steps. While it might have been possible to reduce the number of bits used by the modifier

by combining steps, separating the steps keeps the variant used by RE-PAIR as close to the original as possible.

Since stemming reduces the length of a word, an alternative design for PRE-PAIR would apply the 16 character limit on word tokens after stemming. However, this approach was not taken for two reasons. First, PRE-PAIR would be less efficient because a buffer of characters which varies in length would have to be maintained. Second, the one-to-one correspondence between word tokens and stemming modifiers could no longer be guaranteed. Reversing stemming not only increases the length of a stem, but changes characters within it as well. So, if a long word is split across two word tokens, it may be incorrect to assign the stemming modifier to only the second word token. The consequence of this decision is that words longer than 16 characters would not be stemmed, even if their stemmed form is less than the limit.

There are two reasons for incorporating case folding and stemming into word-based RE-PAIR. During phrase browsing, these transformations group words that are derived from the same root word to improve usability, as shown later in Chapter 7 when the phrase browser is presented. Also, both case folding and stemming help reduce the number of entries in the word lexicon. The effect these two transformations have on the number of distinct word tokens is shown in Table 4.4 for the test file `WSJ20`. A detailed discussion of the experiments with `WSJ20`, including compression times and compression ratios, is provided later, in Section 4.5. Some of those results are presented here to complete the discussion on case folding and stemming. Case folding and stemming do not have to be used together. However, because the implementation of the Porter stemming algorithm used by word-based RE-PAIR only stems lower case words; words with endings in upper case are untouched. This occurred often in `WSJ20` since certain sections of every news article was entirely in upper case.

Lexicon statistics when the application of case folding and stemming is varied are shown in Table 4.4. In all cases, PRE-PAIR identified 3,356,915 word tokens. Both case folding and stemming decrease the number of unique word tokens, as shown in the first column. The average length of a word token in the message is 4.92 characters. Note that

	Unique tokens	Average length in lexicon (chars)
word lexicon		
no transformations	74,686	7.31
case folding only	60,748	7.38
stemming only	50,412	6.26
case folding and stemming	39,638	6.27
non-word lexicon	1,399	7.68

Table 4.4: Statistics from PRE-PAIR which indicate the effects of case folding and stemming on the word lexicon. Transformations to word tokens do not affect the non-word lexicon, which is also shown. The test data used was `WSJ20`.

this value is smaller than the average token lengths in the lexicons, as shown in the second column of Table 4.4. As one would expect, in an English document, short words occur more frequently than long ones. The small averages imply that extending the limit of 16 characters per token would achieve little benefit for document collections such as `WSJ20`.

Case folding and stemming transformations have no effect on the non-word lexicon, whose size is shown in the last row of Table 4.4. The number of entries in the non-word lexicon is much less than the word lexicon. The average length of a non-word token in the message is 1.34 characters, much less than the 7.68 characters in the non-word lexicon.

While modifiers for case folding and stemming can be stored together, they have been separated and placed into their own individual streams. This separation ensures a one-to-one correspondence between a word token and its associated modifiers. This approach highlights an important point regarding how modifiers are used. While users usually search for words, it would be difficult to envision a user searching for a particular case folding modifier. Instead, when a user has found a word token of interest, the position k of the word token in the message needs to be determined. Then, to reverse case folding and stemming, the system must retrieve the modifiers at position k in the respective streams. Experiments with several compression systems were applied later in this chapter in order to process the two streams in the context of this requirement. But first, the next section provides more details about the two lexicons.

Encoding the Lexicons

After a word token is case folded and stemmed, it is located in the word lexicon and replaced with an ordinal number indicating its position in the lexicon. If the word token is novel, then it is replaced with the number representing the next available position, and the lexicon is updated. The pointer back to the lexicon is appended to the output word sequence. After Pre-Pair completes, its lexicon is written as a separate file while the stream of word sequence numbers is then passed to Re-Pair, to produce the *reduced word sequence*. Finally, the reduced word sequence is compressed, to yield the *compressed word sequence*.

In contrast to the word sequence, the stream of non-word tokens are processed without any stemming, case folding, or application of Re-Pair. Also, the non-word sequence is called the *non-word modifier* stream. That is, the symbols in this stream are considered modifiers which transform word tokens by appending non-word symbols to them.

The Pre-Pair system keeps each lexicon in memory with a splay tree [Sleator and Tarjan, 1985]. A splay tree is a binary search tree that is rotated after every node access or modification to ensure that the most recently used node is at the root of the tree. The number of entries in the lexicon is equal to the number of unique case folded, stemmed words in the document. Heinz et al. [2002] describes an alternative data structure for maintaining lexicons called a *burst trie*.

Three methods of compressing the lexicons exist. The first method involves the application of a transformation called *front coding* [Gottlieb et al., 1975]. Front coding is a technique often used to encode a sorted list of strings. Many adjacent strings in a sorted list have similar prefixes. Front coding exploits this fact by encoding every word in the lexicon as a triple made up of two numbers and a substring. The first number indicates the number of characters the prefix of the current word matches to the prefix of the previous word (0 if nothing matches). The second number is the remaining length of the current word, without the length of the matching prefix. The substring of the remainder of the word is the last part of the triple. Recall that in word-based Re-Pair, word and non-word tokens are limited to 16 characters in length. Therefore, 8 bits are sufficient for indicating

the two numeric triples of each word.

The second method of lexicon encoding uses one of the compression systems described in Chapter 2. The lexicon is written in sorted order, with each token preceded by one byte, indicating its length. In the experiments described shortly, this is called literal encoding. Then one of the following three compression systems are applied: GZIP, BZIP2, and PPMD. Similar to the experiments in the previous chapter, GZIP and BZIP2 were executed using the `-9` option in order to favour compression effectiveness. Likewise, PPMD used 255 MiB of memory and a seventh order context. The third scheme combines the first two by applying one of these compression systems to the output of the front coder. Nevill-Manning and Witten [1997] also used PPM after front coding for compressing SEQUITUR's word lexicon.

Table 4.5 and Table 4.6 show the compression effectiveness of the word lexicons with respect to itself and with respect to the size of `WSJ20`, respectively. Case folding and stemming have been varied along with the three compression strategies. The two views of the results are necessary because these transformations affect the size of the lexicons. As a benchmark, the tables also provide compression ratios when literal coding is used without any additional compression system. Compression ratios are reported in bits per character (bpc) with the best result in each table highlighted.

When literal coding is not coupled with any compression mechanism, the size of the word lexicons are shown in the tables in the rows marked "none", under the first columns. Case folding and stemming both decrease the amount of space occupied. In all four combinations, GZIP achieves the best compression when no front coding is applied. Of course, a smaller word lexicon means that some compression effectiveness must be paid for by additional modifier streams, as described later in this chapter. Once front coding is used, compression with any of the three systems surveyed is better than applying the same program by itself. In particular, in all cases, front coding with BZIP2 offers the best compression effectiveness.

Table 4.7 shows the effect of applying these compression mechanisms on the non-word lexicon. On the left, the compression results are reported with respect to the size of the lexicon; on the right, they are presented with respect to the size of `WSJ20`. According to

	Literal coding	Front coding
none	8.000	3.518
GZIP	3.411	2.054
BZIP2	3.770	1.771
PPMD	3.442	1.864

(a) No case folding or stemming

	Literal coding	Front coding
none	8.000	3.370
GZIP	3.289	1.928
BZIP2	3.714	1.677
PPMD	3.416	1.751

(b) Case folding only

	Literal coding	Front coding
none	8.000	3.958
GZIP	3.857	2.512
BZIP2	4.233	2.202
PPMD	3.926	2.363

(c) Stemming only

	Literal coding	Front coding
none	8.000	3.801
GZIP	3.701	2.341
BZIP2	4.130	2.085
PPMD	3.905	2.226

(d) Case folding and stemming

Table 4.5: Comparing the use of front coding with several compression systems on the word lexicon of `WSJ20` while varying the use of case folding and stemming. Results are in bits per character with respect to the size of the lexicon.

	Literal coding	Front coding
none	0.237	0.104
GZIP	0.101	0.061
BZIP2	0.112	0.052
PPMD	0.102	0.055

(a) No case folding or stemming

	Literal coding	Front coding
none	0.194	0.082
GZIP	0.080	0.047
BZIP2	0.090	0.041
PPMD	0.083	0.042

(b) Case folding only

	Literal coding	Front coding
none	0.140	0.069
GZIP	0.067	0.044
BZIP2	0.074	0.038
PPMD	0.068	0.041

(c) Stemming only

	Literal coding	Front coding
none	0.110	0.052
GZIP	0.051	0.032
BZIP2	0.057	0.029
PPMD	0.054	0.031

(d) Case folding and stemming

Table 4.6: Comparing the use of front coding with several compression systems on the word lexicon of `WSJ20` while varying the use of case folding and stemming. Results are in bits per character with respect to the size of `WSJ20`.

the table on the right, the non-word lexicon is only 0.005 bpc when output as literal text, while the word lexicon is at least 0.110 bpc when case folding and stemming are both applied. Again, front coding with BZIP2 provided good compression effectiveness at 0.001

	Literal coding	Front coding
none	8.000	2.925
GZIP	2.632	1.464
BZIP2	2.588	1.446
PPMD	2.385	1.373

(a) Relative to the lexicon

Literal coding	Front coding
0.005	0.002
0.002	0.001
0.001	0.001
0.001	0.001

(b) Relative to `WSJ20`

Table 4.7: Comparing the difference between the use of front coding with several compression systems on the non-word lexicon of `WSJ20`. Results are in bits per character.

bpc. Other regimes are also able to attain similar compression ratios for the non-word lexicon. These tables do not mention the time required for compressing the lexicons. Later in this chapter, detailed experiments with `WSJ20` show that both lexicons are small in size compared to the other streams. Processing the lexicons contributes little to the overall compression and decompression times.

4.4 Punctuation-aligned Re-Pair

The pre-processing step of Pre-Pair solves many of the problems that existed with character-based Re-Pair and word-aligned Re-Pair. A word in the message is no longer broken into more than one phrase. Moreover, all words, even infrequent ones, are in the word lexicon and are therefore primitives in the phrase hierarchy.

One minor problem remains, which is related to the treatment of punctuation marks. In English, punctuation marks such as commas and full stops separate phrases or sentences from each other. Word-based Re-Pair ignores these punctuation marks and allows phrases to span over them. A minor change to word-based Re-Pair, called punctuation-aligned Re-Pair, encourages phrases to end with punctuation marks.

Punctuation-aligned Re-Pair identifies a set of punctuation marks as special processing symbols. The set of six punctuation marks are: the exclamation mark (!), the full stop (.), the question mark (?), the comma (,), the colon (:), and the semicolon (;). The first three punctuation marks set off a sentence from the surrounding text, while the remaining three separate an independent clause, a phrase, or a subordinate clause from the rest of the

sentence. During parsing with Pre-Pair, these punctuation marks are identified without understanding the context in which they occur. Because of this, Pre-Pair treats a full stop in an abbreviation as if a sentence had just ended. All of these punctuation marks are categorised as *separating punctuation marks*, in contrast to *enclosing punctuation marks* [Greenbaum, 1996]. Enclosing punctuation marks, such as parentheses, highlight a section of text usually through a pair of delimiters.

While other punctuation marks could have been identified, adding too many would have degraded the performance of Pre-Pair. Moreover, Greenbaum [1996] presented results from a study of American texts which consisted of writing totalling 72,000 words. The documents were all samples of "journalism, learned writing, and fiction". The results showed that the six punctuation marks used by punctuation-aligned Re-Pair accounted for 95.9% of all punctuation marks. Full stops and commas alone contributed to 91.8% of the punctuation marks found. In comparison, these six punctuation marks make up 98.0% and 61.9% of the punctuation marks in `bible.txt` (Large Canterbury Corpus, described in Chapter 2) and `WSJ20`. The percentage of punctuation marks for `WSJ20` is lower, even though this value has already excluded SGML tags. A few lines from `WSJ20` were provided in Figure 2.7(a) on page 40. As the figure shows, other punctuation marks such as hyphens and slashes appear, which do not segment sequences of words into phrases.

In the same way that word-aligned Re-Pair modifies character-based Re-Pair, punctuation-aligned Re-Pair augments a set of rules to word-based Re-Pair which dictate what active pairs can be chosen. These rules are shown in Table 4.8. Every symbol in the word sequence is assigned a category. Either it is a word that is not followed by any of the 6 punctuation marks (W), or it is (P). The table gives the four ways these two symbol types, shown under the first two columns of the table, can combine. If the combination is permitted, and subsequently replaced, the new symbol obtains a category as shown in the third column. The fourth column of the table gives some examples in which the symbol combinations may occur using variations on "Melbourne University", an unofficial name for The University of Melbourne.

Whether or not a word is followed by a punctuation mark, it can become an active

Left	Right	Result	Example
W	W	W	Melbourne University
P	P	P	M. U.
W	P	P	Melbourne University.
P	W	Not allowed	In Melbourne, universities

Table 4.8: The rules used by punctuation-aligned Re-Pair for deciding when a pair of symbols can become active pairs. An active pair is allowed if any of the first three rules apply; an active pair is disallowed if the last rule applies.

pair with symbols of the same category. If it is replaced, the new symbol inherits the same category as its components. The rule in the second row permits abbreviations and words in a series, separated by commas, to form. However, symbols of two different types can only combine if the right component is a symbol followed by a punctuation mark. The reversed combination (fourth row in the table) is disallowed. Note that these rules do not pair symbols based on the categories of the surrounding symbols. While word-aligned Re-Pair forced complete words and non-words to be formed by examining the contexts in which symbols occur, punctuation-aligned Re-Pair is less restrictive. For example, in the second rule of Table 4.8, two symbols with P categories can combine regardless of the classification of the symbol to the left of the left component. Punctuation-aligned Re-Pair merely encourages word tokens to combine to the left if they are followed by punctuation marks. This design choice recognises the fact that phrases can form without punctuation marks, so confining phrases to exist only between punctuation marks would be inappropriate.

There are three other differences between word-aligned and punctuation-aligned Re-Pair. First, a mixture (M) category is unnecessary since anything that combines with a word token terminated by a punctuation mark is a (P) token. Second, the symbol categories are stored with the sequence and not with the phrase hierarchy. Unlike word-aligned Re-Pair, the category of a word token depends entirely on the presence of a punctuation mark after it. Therefore, maintaining the categories in the phrase hierarchy is not possible. As a result, for a message of n symbols, n extra words of memory are required to implement punctuation-aligned Re-Pair. Since only one bit is ever used, this

approach is wasteful, and a more careful implementation is possible which operates on the bit level. Then, only n extra bits of memory would be needed. Finally, the information about punctuation marks has been separated from the word tokens by Pre-Pair. In order to implement punctuation-aligned Re-Pair, punctuation flags (shown as dashed lines in Figure 4.3 on page 82) need to be passed to Re-Pair along with the word sequence. For each word token, the presence or absence of a punctuation mark following it is indicated by a flag. These flags are only used by Re-Pair, and the output structure of punctuation-aligned Re-Pair remains identical to that of word-based Re-Pair.

Figure 4.4 shows the difference between the four versions of Re-Pair with respect to the type of phrases created. The document used is `bible.txt`, chosen because the SGML markup in the `NEWS` file, and the files that are derived from it, would clutter the example. After being compressed, the first 20 symbols have been expanded into primitives for each version of Re-Pair. Since character-based and word-aligned Re-Pair do not remove non-word characters, the space and linefeed characters are represented as "␣" and "•", respectively, in the figure. Case folding and stemming were used for word-based and punctuation-aligned Re-Pair, with a space added between each word to facilitate display.

Since character-based Re-Pair forms phrases using only frequency, word boundaries are not adhered to, as shown in Figure 4.4(a). For example, in the fourth line, the word "waters" spans the first and the second phrases of that line. Also, some phrases commence with non-words. Word-aligned Re-Pair handles both of these problems by using its rules to guide the recursive pairing. While a phrase may contain part of a word, no word spans across phrases, as shown in Figure 4.4(b). Also, all phrases start with a complete word and end with a complete non-word. Alternatively, they can also contain all word characters, or all non-word characters. A phrase categorised as a mixture cannot begin with a non-word character.

The phrases from word-based Re-Pair in Figure 4.4(c) have been case folded and stemmed. Since the smallest atomic units are word tokens, word-based Re-Pair ensures that no word spans across one or more phrases. Moreover, non-word characters do not appear in the expanded phrases anymore. While case folding and stemming have

```
In␣the␣ beginning␣ God␣ created␣the␣ heaven␣and␣the␣earth
.␣And␣the␣ earth␣was␣ without␣form,␣and␣void ;␣and␣darkness␣was␣
upon␣the␣face␣of␣the␣ deep .␣And␣the␣ Spirit␣of␣God␣ moved␣
upon␣the␣face␣of␣the␣water s.␣•And␣God␣ said,␣Let␣there␣be␣
light :␣and␣there␣was␣ light
```

(a) Character-based Re-Pair

```
In␣the␣beginning␣ God␣ created␣the␣ heaven␣and␣the␣ earth.␣
And␣the␣earth␣ was␣without␣ form,␣and␣void;␣ and␣darkness␣was␣
upon␣the␣face␣of␣the␣ deep .␣ And␣the␣Spirit␣of␣God␣ moved␣
upon␣the␣face␣of␣the␣waters.␣• And␣God␣said,␣Let␣ there␣be␣
light:␣ and␣there␣was␣ light.␣•
```

(b) Word-aligned Re-Pair

```
in the begin  god creat  the heaven and the earth
and the earth was  without form and void  and dark  was
upon the face  of the deep  and the spirit of god  move
upon the face of the water  and god said let there be  light
and there was  light  and god  saw the  light  that it was
```

(c) Word-based Re-Pair with case folding and stemming

```
in the begin  god  creat  the heaven and the earth
and the earth was  without form  and void  and dark was
upon the face  of the deep  and the spirit of god  move
upon the face of the water  and god said  let there be light
and there was  light  and god  saw the  light
```

(d) Punctuation-aligned Re-Pair with case folding and stemming

Figure 4.4: Expanding the first 20 symbols of `bible.txt` into primitives, after being compressed with character-based Re-Pair, word-aligned Re-Pair, word-based Re-Pair, and punctuation-aligned Re-Pair. For the purposes of display, • and ␣ are used to indicate linefeeds and spaces, respectively, in (a) and (b). Non-words are not shown in (c) and (d).

transformed some of the words, they still resemble the original words, when compared with Figure 4.4(a). Word-based Re-Pair ignores the location of punctuation marks. Punctuation-aligned Re-Pair rectifies this problem by encouraging word tokens followed by a punctuation mark to only pair with a symbol to its left. For example, comparing word-based Re-Pair with punctuation-aligned Re-Pair, in the second line of Figure 4.4(d), "without form, and void" is no longer accepted as a phrase. Also, while word-based Re-Pair created the phrases "and god said let there be" and "light" in the fourth line, punctuation-aligned Re-Pair separated these seven words according to the locations of

	Symbol range	Distinct values	Number of symbols	Self-information
case folding modifier	0 to 65,536	72	3,356,915	1.227
stemming modifier	0 to 7,356,416	346	3,356,915	2.292
non-word modifier	0 to 1,398	1,399	3,356,915	1.750
word sequence	0 to 39,637	39,638	3,356,915	10.272
reduced word sequence	0 to 235,929	218,822	1,582,065	15.829

Table 4.9: Statistics about some of the streams from PRE-PAIR, and the reduced word sequence from RE-PAIR on the `WSJ20` file. The column headed self-information is the zero-order self-information for the frequency distribution of the symbols in that file, measured for each file in bits per symbol relative to the size of that file.

the comma and the colon.

4.5 Experiments

The compression efficiency and effectiveness of the three methods of word-aware document parsing were compared with each other and with character-based RE-PAIR through experiments. Partial results from these experiments have already been shown in Section 4.3. In all of these experiments, `WSJ20` was used as test data and the test machine was a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache.

Before the four versions of RE-PAIR are compared, a detailed look at the various streams of word-based RE-PAIR are examined. Section 4.3 showed compression results for the word and non-word lexicons of `WSJ20`. For both of these streams, front coding followed by BZIP2 offered the best compression effectiveness. Statistics about the remaining four streams produced by PRE-PAIR are listed in Table 4.9.

The word sequence is compressed with RE-PAIR, by treating the word sequence symbols as primitives. A hierarchy of phrases is constructed for browsing, as well as a reduced word sequence. In Chapter 3, experiments showed that the phrase hierarchy can be effectively stored by transforming it with chiastic slide, and then encoding it with interpolative coding. Statistics about the reduced word sequence of `WSJ20` are also given in Table 4.9.

The three streams of modifiers are stored as 32-bit integers. The number of distinct symbols in any of these streams is less than the word sequence. Also the self-information

	SHUFF	GZIP	BZIP2	PPMD	RE-PAIR with SHUFF
case folding modifiers	0.232	0.200	0.142	0.229	0.125
stemming modifiers	0.388	0.583	0.457	0.520	0.394
non-word modifiers	0.320	0.367	0.261	0.362	0.224

Table 4.10: Compression effectiveness of the modifiers from word-based RE-PAIR for WSJ20.

of these streams is lower than the word sequence.

The modifiers are also compressed, but since a phrase hierarchy is not required for browsing, compression systems other than RE-PAIR can also be applied. Results obtained by applying five compression systems to WSJ20 are presented in Table 4.10. The compression mechanisms employed include the zero-order Huffman coder SHUFF, GZIP, BZIP2, PPMD, and RE-PAIR. The SHUFF program processed the modifiers as a single block, while GZIP and BZIP2 were executed with the -9 option in order to favour compression effectiveness. The PPMD system operated with a seventh order model and a limit of 255 MiB of memory. The three modifiers were also processed with RE-PAIR as a single block, with SHUFF used to encode the sequence, also as a single block. While GZIP, BZIP2, and PPMD compress the modifiers as individual bytes, both RE-PAIR and SHUFF treat each 32-bit integer as a symbol.

The compression results from Table 4.10 show the compression results in bits per character, relative to the size of WSJ20. The best compression ratio for each modifier is highlighted. While the case folding and non-word modifiers are most effectively processed by RE-PAIR (with SHUFF), SHUFF alone provided the best compression ratio for the stemming modifiers. Moreover, BZIP2 also gave better compression effectiveness than SHUFF for the same modifiers as RE-PAIR. Both RE-PAIR and BZIP2 are better at capturing the repetition in the modifier streams.

The time required to process these streams with the five systems are reported in Table 4.11, with the fastest encoding and decoding time for each stream highlighted. The SHUFF system operated the fastest for both encoding and decoding, partly due to the simplistic zero-order model that it is based on. The BZIP2 program was particularly slow for encoding the non-word modifier, but otherwise yielded consistent encoding and

	SHUFF	GZIP	BZIP2	PPMD	RE-PAIR with SHUFF
Encoding time					
case folding modifiers	0.4	20.4	4.8	4.0	9.5
stemming modifiers	0.4	51.0	4.6	5.0	13.3
non-word modifiers	0.4	17.1	70.6	5.3	10.4
Decoding time					
case folding modifiers	0.1	0.2	1.3	4.6	0.1
stemming modifiers	0.2	0.3	1.9	5.5	0.3
non-word modifiers	0.1	0.3	3.4	5.9	0.2

Table 4.11: Compression times for the modifiers produced by word-based RE-PAIR on WSJ20, reported in CPU seconds and averaged over three runs each.

decoding times. Decoding with the dictionary-based systems (GZIP and RE-PAIR) are fast, and in one case, gave the same decoding time as SHUFF.

These results show that a combination of SHUFF alone and RE-PAIR coupled with SHUFF seem to offer the best compression effectiveness for the modifiers. In addition, both systems offer fast decoding times, an important factor to consider when designing a retrieval system. While these two systems satisfy the needs of compression, they are inappropriate for retrieval for other reasons. These reasons are related to how these streams are used in the browsing system, a problem which is covered later beginning in Chapter 6. The method used for encoding the reduced sequence and the modifiers is revisited at that time.

Table 4.12 presents some statistics when RE-PAIR is applied to either the original WSJ20 file of 20,971,520 characters (character-based and word-aligned RE-PAIR) or to the word sequence of 3,356,915 symbols (word-based and punctuation-aligned RE-PAIR) produced from PRE-PAIR. The results for character-based RE-PAIR are taken from Table 3.4 on page 63. Word-aligned RE-PAIR produces a larger phrase hierarchy than character-based RE-PAIR, but with less generations and a longer reduced sequence. Many phrases in the word-aligned phrase hierarchy include a phrase "α" in one generation, and then the phrase "α␣" in the next generation. This effect is also evident in the phrase hierarchy of Figure 4.2 on page 80.

As Table 4.12 shows, word-based and punctuation-aligned RE-PAIR are similar, but

Method	$\|\Sigma\|$	$\|\rho\|$	$\|\mathcal{S}\|$	Generations	Longest phrase (in primitives)
character-based	90	310,486	1,904,118	21	1,257
word-aligned	90	343,520	1,912,566	19	1,269
word-based	39,638	196,291	1,582,065	15	215
punctuation-aligned	39,638	191,924	1,634,652	13	215

Table 4.12: Statistics from experiments with the four variants of Re-Pair with `WSJ20`. The first two methods are applied to `WSJ20` directly, where the characters are primitives. The other two methods are used on the word sequence produced by Pre-Pair.

both produce a shorter reduced sequence and a smaller phrase hierarchy than character-based and word-aligned Re-Pair. Also, the maximum phrase generation between the two is 15. The identification of word tokens with Pre-Pair allow contexts of characters to be found during pre-processing, before Re-Pair is used.

The last column of the table indicates the longest phrase of each method, measured in primitives. The longest phrase for word-based Re-Pair is 215 symbols, which expands to 1,325 characters in length. This is a measure of stemmed, case folded words with no non-words. Without these two transformations, the same phrase also has more primitives than the longest phrase for character-based Re-Pair: 1,421 characters.

The compression effectiveness of the four versions of Re-Pair is listed in Table 4.13. The test file is `WSJ20` and compression ratios are given in bits per character relative to the original file. The results for character-based Re-Pair are from Table 3.5 on page 64. Compression levels for four variants of word-based Re-Pair are presented, which each differ by whether or not case folding and stemming are applied. Punctuation-aligned Re-Pair imposes rules on Re-Pair which prevents certain phrases from forming. However, the three modifier streams are unchanged. In the table, only results for punctuation-aligned Re-Pair with both case folding and stemming are shown. All lexicons are encoded with front coding followed by bzip2. The reduced word sequence and the three streams of modifiers are encoded using shuff.

The best compression achieved is by word-based Re-Pair, when no word transformations are applied. Compression levels degrade as case folding and stemming are included. Note that the rules for stemming assume that the word tokens have been case folded.

	Word lexicon	Phrase hierarchy	Reduced (word) sequence	Case folding modifiers	Stemming modifiers	Non-word lexicon	Non-word modifiers	Total
character-based	-	0.223	1.551	-	-	-	-	1.774
word-aligned	-	0.201	1.533	-	-	-	-	1.734
word-based								
no transformations	0.052	0.154	1.320	-	-	0.001	0.320	1.708
case folding only	0.041	0.152	1.285	0.232	-	0.001	0.320	2.031
stemming only	0.038	0.145	1.264	-	0.380	0.001	0.320	2.148
both	0.029	0.143	1.230	0.232	0.388	0.001	0.320	2.343
punctuation-aligned								
both	0.029	0.138	1.254	0.232	0.388	0.001	0.320	2.362

Table 4.13: Comparison of the compression ratios achieved by the four versions of Re-Pair for `WSJ20`, in bits per character.

Hence, the stemming modifiers occupy 0.380 bpc or 0.388 bpc, depending on whether or not the word tokens have been case folded. When both transformations are used, a compression effectiveness of 2.343 bpc is attained. Even if the best selection of compression mechanisms is chosen for the modifiers (see the highlighted numbers in Table 4.10), the compression ratios drops by 0.203 bpc to 2.140 bpc. So, despite the fact that word-based Re-Pair and punctuation-aligned Re-Pair produce a smaller phrase hierarchy, character-based Re-Pair and word-aligned Re-Pair achieve better compression effectiveness.

One reason for the poorer compression levels of word-based and punctuation-aligned Re-Pair is that separating the message into several independent streams disturbs the contexts in the message. One possible direction for future work is to encode the modifier streams more carefully. Symbols in these streams do not appear independently of neighbouring symbols, and are even conditioned based on symbols from another stream. For example, if the first letter of a word is in upper case, the word is probably the beginning of a sentence. One would expect the few words that follow to be in lower case. Moreover, one of the non-word characters that immediately precede it is probably a sentence ending punctuation mark. And, since the stems of words correspond to their parts-of-speech (for example, "-ing" is associated with verbs), and these categories restrict the order of words in a sentence, it is expected that some correlation exists between stemming modifiers. This observation means that the encoding of the symbol in position k of the case folding modifiers can be conditioned by preceding symbols as well as symbols in the other modifier streams, or the word sequence.

Table 4.13 also shows that word-aligned Re-Pair achieves slightly better compression than character-based Re-Pair, even though the former adds more phrases to the phrase hierarchy. This result shows that character-based Re-Pair's use of frequency alone as a heuristic does not always choose the best phrases. A heuristic which forms complete words before combining with complete non-words to the right improves compression effectiveness over character-based Re-Pair.

The compression effectiveness of the four versions of Re-Pair are presented in Figure 4.5 as a graph, with the compression results for `WSJ20` with gzip, bzip2, and ppmd

(see Table 2.5 on page 41) indicated as horizontal lines. As a further guide, but not shown in the figure, Isal and Moffat [2001] used a word-based block sorting mechanism and a Huffman coder on `WSJ20` and reported compression effectiveness of 1.69 bits per character. Word-based and punctuation-aligned RE-PAIR both performed case folding and stemming. All four versions of RE-PAIR have been broken down by stream. Word-based and punctuation-aligned RE-PAIR both achieve slightly worse compression than BZIP2, but better than GZIP. However, both of these versions of RE-PAIR require more memory than either BZIP2 or GZIP. Even when neither case folding nor stemming are used with word-based RE-PAIR, the compression effectiveness of 1.708 is still slightly worse than the 1.656 bpc offered by PPMD.

Other than compression effectiveness, the second factor to consider when evaluating compression systems is execution time. All times reported below have been averaged over 3 trials. Table 4.14 displays the encoding and decoding times of character-based RE-PAIR (from Table 3.6 on page 64) and word-aligned RE-PAIR. The rules embedded within word-aligned RE-PAIR reduces the number of active phrases and active pairs that character-based RE-PAIR keeps. As a result, word-aligned RE-PAIR creates a longer reduced sequence, but encodes the message in less time. However, no difference exists for the other program execution times shown in Table 4.14.

The time required to compress `WSJ20` by word-based RE-PAIR, with case folding and stemming, is shown in Table 4.15. If both case folding and stemming are not used, the parsing performed by PRE-PAIR reduces to 7.8 seconds. Since the only difference between word-based RE-PAIR and punctuation-aligned RE-PAIR is the way phrases are selected, the only times affected are those related to RE-PAIR. Punctuation-aligned RE-PAIR takes 16.3 and 1.1 seconds to encode with RE-PAIR and SHUFF and 0.9 and 0.3 seconds to decode with DES-PAIR and SHUFF. Similar to word-aligned RE-PAIR, the augmented rules to punctuation-aligned reduces the amount of time to encode the word sequence.

Finally, Figure 4.6 shows the difference in phrases between word-aligned RE-PAIR and word-based RE-PAIR by expanding the first 20 symbols of `WSJ20` into primitives. As a reference point, the same section of `WSJ20` was presented in Figure 3.5 on page 63.

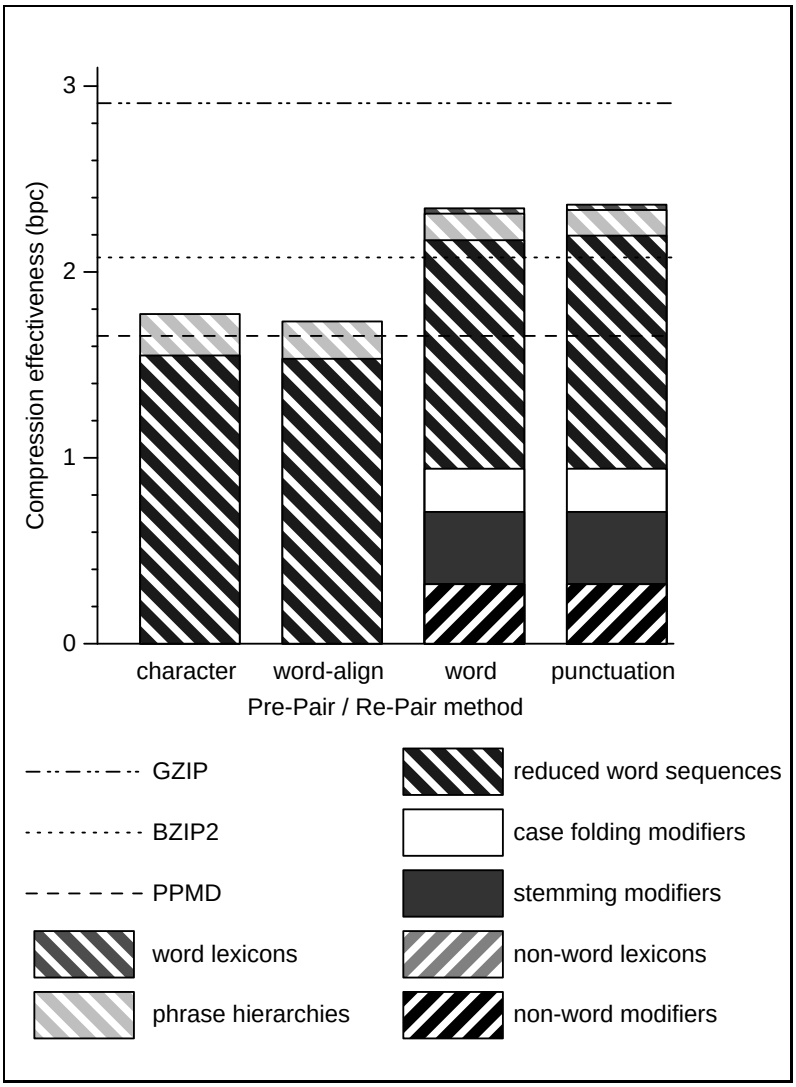


Figure 4.5: Comparing the compression ratios of the four variants of Re-Pair for `WSJ20`, broken down by stream. Also shown are compression results from Chapter 2 using gzip, bzip2, and ppmd.

Punctuation-aligned Re-Pair is not shown because, for this section of text, only a small difference with word-based Re-Pair exists. An extensive comparison of phrases between the four versions of Re-Pair was discussed earlier with `bible.txt` (Figure 4.4). Figure 4.6 presents the phrases for `WSJ20` for completeness, since most of this thesis uses `WSJ20`, or files similar to `WSJ20`, as test data.

The same comparisons that were made for Figure 4.4 also apply to Figure 4.6. However, a couple of points of interest exists in Figure 4.6. Note that SGML tags are treated

	Character-based Re-Pair	Word-aligned Re-Pair
Encoding time		
Re-Pair	88.5	61.2
shuff	1.6	1.6
Total	90.1	62.8
Decoding time		
shuff	2.5	2.5
Des-Pair	0.4	0.4
Total	2.9	2.9

Table 4.14: Compression times for character-based Re-Pair and word-aligned Re-Pair for `WSJ20`. Times have been averaged over three trials and are given in CPU seconds.

Stream	Compression system	Encoding time	Decoding time
Pre-Pair		14.2	2.8
word lexicon	front coding and bzip2	0.1	0.1
word sequence			
	Re-Pair or Des-Pair	19.6	0.9
	shuff	1.1	0.2
case folding modifiers	shuff	0.3	0.1
stemming modifiers	shuff	0.3	0.1
non-word lexicon	front coding and bzip2	0.1	0.1
non-word modifiers	shuff	0.3	0.1
Total		36.0	4.4

Table 4.15: Compression and decompression times for word-based Re-Pair in CPU seconds. Case folding and stemming have been performed and front coding followed by bzip2 is used for encoding the lexicons. Times have been averaged over three trials.

as word tokens, and all of them have been case folded. Also, the word "telecommunications" on line 5 of Figure 4.6(b) is too long and has been broken in two. However, after breaking it, no stemming rules could be applied, so this word appears unchanged. If no maximum word length was imposed, the word "telecommunications" would have stemmed to "telecommun".

```
<DOC>•<DOCNO>␣ WSJ8703 24 - 0001␣</DOCNO>•<HL>␣
John␣Blair␣ Is␣Near␣ Accord•To␣Sell␣ Unit,␣
Sources␣Say</HL>• <DD>␣ 03/ 24 /87</DD> •
<SO>␣WALL␣STREET␣JOURNAL␣(J)</SO>•<IN>␣ REL•
TENDER␣OFFERS,␣MERGERS,␣ACQUISITIONS␣(TNM)•↩
MARKETING,␣ADVERTISING␣(MKT)•
TELECOMMUNICATIONS,␣BROADCASTING,␣TELEPHONE,␣↩
TELEGRAPH␣(TEL)␣</IN>•<DATELINE>␣NEW␣YORK␣↩
</DATELINE>•<TEXT>•␣␣␣ John␣Blair␣&amp;␣Co.␣
```

(a) Word-aligned Re-Pair

```
<doc> <docno> wsj870324 0001 </docno> <hl> john blair
is near accord to sell unit sourc say </hl> <dd> 03/24/87
</dd> <so> wall street journal j </so> <in> rel
tender offer merger acquisit tnm market advertis mkt
telecommunicatio ns broadcast telephon telegraph tel↩
</in> <dateline> new york </dateline> <text>
john blair amp co is close to an agreement to sell its
tv station advertis represent
```

(b) Word-based Re-Pair with case folding and stemming

Figure 4.6: Expanding the first 20 symbols of WSJ20 into primitives, parsed with word-aligned Re-Pair and word-based Re-Pair. For the purposes of display, • is used to indicate a linefeed and long phrases have been broken at the points indicated by ↩.

4.6 Phrases for Browsing

Three additional heuristics for selecting phrases have been proposed in this chapter which extend the capabilities of character-based Re-Pair. Experiments have been performed which measured the difference in compression effectiveness achieved and compression times. However, the initial motivation for embarking on this investigation has been to create phrases that are suitable for phrase browsing.

From Figure 4.4 and Figure 4.6, the most visually appealing phrases are those of word-based Re-Pair and punctuation-aligned Re-Pair. Case folding and stemming reduce the size of the lexicon and allow users to browse phrases without considering the case and stems used in the document. The separation of non-words from the message permits browsing without the clutter caused by them. While not appearing in the phrase hierarchy, non-word tokens which contain punctuation marks are used by punctuation-aligned Re-Pair

Output from PRE-PAIR	**word lexicon**	word sequence	
Purpose	phrase browsing		searching for symbols
Model	front coding and BZIP2	RE-PAIR	
Coder		**phrase hierarchy** chiastic slide and interpolative coding	**reduced word sequence** SHUFF

(a) Words

Output from PRE-PAIR	**non-word lexicon**	**non-word modifiers**	**case folding modifiers**	**stemming modifiers**
Purpose	skipping to symbols			
Model	front coding and BZIP2	SHUFF	SHUFF	SHUFF
Coder				

(b) Non-words and modifiers

Table 4.16: The seven streams produced by PRE-PAIR and RE-PAIR. The purpose of each stream, and the compression methods employed in this chapter for these streams are also shown. Entries marked "SHUFF" are replaced in Chapter 6 with systems better suited to those specific tasks.

to guide the choice of phrases made.

There are six streams created by the message pre-processor, PRE-PAIR. After RE-PAIR compresses the word sequence, a phrase hierarchy and a reduced word sequence are created. In Table 4.16, these seven streams (in boldface), their purpose, and the method employed in this chapter for compressing them are shown. Further details about how these seven streams are used by a phrase browsing system are given in Chapter 6 and Chapter 7. In Chapter 6, the four instances of "SHUFF" are replaced by compression systems that are more suited to the intended purposes of the reduced word sequence and the three modifier streams. The encoding methods for the lexicons and the phrase hierarchy remains unchanged for the remainder of this thesis.

Phrase browsing can be performed with only the word lexicon and the phrase hierarchy to determine the words used in the message and the relationships between them. Together, these two represent just 0.167 bits per character (punctuation-aligned RE-PAIR, from

Table 4.13), or just over 2% of the original document size, yet provide a very good guide to the context of the document. The lexicons are relatively small, and can be compressed using methods other than front coding with BZIP2.

The word lexicon and word phrase hierarchy are used for browsing. But when the context in which the phrases occur are required, then the compressed word sequence (1.254 bpc) can be used. The 3 streams so far are only an approximation of the document; words in the phrases need to have the case folding and stemming reversed. Also, the non-words within phrases have to be re-inserted. If punctuation-aligned RE-PAIR is used for compression, then the original document can be obtained by decoding all of the streams and applying a post-processor to combine them. For retrieval, though, only a portion of the modifiers and the non-word sequence may be required. If a one-to-one correspondence between a word and its case folding, stemming, and non-word modifiers is ensured, then a phrase can be reproduced during retrieval by skipping to precise locations in each of these streams. In this chapter, the Huffman coding used to encode the modifiers prevent this operation from occurring without decoding the entire document. However, this issue and the problem of efficiently searching for phrase contexts in the compressed word sequence are considered later, in Chapter 6. Also, more details about how a user browses phrases and searches for the contexts in which the phrases occur is examined in Chapter 7.

But first, in the next chapter, a different problem is considered. So far, RE-PAIR has been used to compress a 20 MiB document. Ideally, larger documents should be compressed and browsed and some ways of achieving this is discussed next.

Chapter 5

Block Merging for Re-Pair

Phrase browsing techniques become more important as document size increases. When character-based Re-Pair was introduced in Chapter 3, the largest document processed by Re-Pair was a 20 MiB document, `WSJ20`. Limitations on computer memory prevent significantly larger documents being handled by Re-Pair. The space analysis for Re-Pair (Section 3.2, from page 52) showed that its memory usage is linear to the length of the message. However, the analysis also showed that the constant factor is high. For example, a 100 MiB message would require more than 1 GiB of memory for the sequence alone.

While the goal of Pre-Pair was to align words in a document for the purpose of browsing, both word-based and punctuation-aligned Re-Pair also reduce the memory requirements of Re-Pair. Pre-Pair separates a message into multiple streams, including a word sequence stream. In the previous chapter, it was shown that the parsing of `WSJ20` by Pre-Pair yielded roughly one word token for every 5 characters in the message for Re-Pair. The `WSJ20` document of 20 million symbols (characters) is transformed into a word sequence of just over 3 million symbols (references to the word lexicon). Moreover, the shorter input into Re-Pair has a beneficial effect on the overall compression time, even through the set of primitives increases in size. When character-based Re-Pair is applied to `WSJ20`, 88.5 seconds is required (Table 3.6, page 64). However, word-based Re-Pair requires only 19.6 seconds (Table 4.15, page 104). (These times exclude the costs in parsing and entropy coding of the sequences and modifiers, which together add 1.6 and

16.4 seconds back on to the execution costs of character-based and word-based RE-PAIR, respectively.)

In this chapter, the problem of using RE-PAIR on longer messages is considered. The solution proposed involves an initial partitioning of the message into blocks, followed by the processing of each block by RE-PAIR. The division of a message into b blocks presents problems for browsing and compression. The challenge is to provide post-processing steps for RE-PAIR that merge multi-block outputs, thereby solving these problems.

The remainder of this chapter is structured as follows. The creation of blocks using RE-PAIR is discussed in Section 5.1. The subsequent merging of blocks occurs in three phases. The first phase, described in Section 5.2, merges the b phrase hierarchies into a single one. Section 5.3 describes phase 2, which employs the newly formed phrase hierarchy to identify pairs of symbols for further replacement. Then, Section 5.4 outlines the third phase, which re-examines the sequence to identify pairs of symbols that have not yet been represented by any symbol in the phrase hierarchy. In order to simplify the discussion in this chapter, character-based RE-PAIR is used for demonstrating the three phases. Section 5.5 explores the changes required to each of these three phases in order to accommodate punctuation-aligned RE-PAIR. Section 5.6 provides experiments that demonstrate the efficiency and effectiveness of block merging. Section 5.7 describes how text can be appended to a compressed document by extending the block merging technique. Section 5.8 concludes the chapter by discussing the trade-offs that exist with block merging.

5.1 Creating Blocks

To sidestep memory limitations, many compression systems partition large messages into blocks, and then process each block independent of neighbouring blocks. The GZIP compression system implements an input buffer whose size is fixed for all documents, limiting the maximum block size. On the other hand, PPMD and BZIP2 create blocks according to options supplied by the user. The PPMD implementation used as a reference point in this thesis ends the current block when the maximum amount of memory specified by the user

has been exhausted, and compression is simply restarted from a "know nothing" state. The BZIP2 system allows the user to specify one of a set of block sizes.

The BZIP2 approach is also adopted by RE-PAIR for large messages. An explicit input block size is supplied by the user, which specifies that the message be divided into b blocks, with all blocks equal in size, except for the last one. Each block is then compressed independently using RE-PAIR, to produce b phrase hierarchies and b reduced sequences.

The forceful separation of a message into blocks for compression presents three problems for browsing and compression. First, browsing multiple phrase hierarchies simultaneously is difficult, and as cumbersome as physically browsing multiple books at the same time. Second, since RE-PAIR selects phrases from each block without any information gained through the compression of previous blocks, it is likely that partitioning a uniform file produces shorter phrases. The maximum generation of symbols in the phrase hierarchies would be less than if the file is compressed as one block. Lastly, not only does the quality of phrases worsen, but compression degrades. As the block size decreases, more blocks are produced. Across any two blocks, information is duplicated in the phrase hierarchies and in the preludes in the final entropy coded sequence.

The effect blocks have on phrase quality and compression is verified by experiments using the `WSJ508` file described in Chapter 2. Recall that the file is 508 MiB in size and that the `WSJ20` from earlier experiments was derived from the first 20 MiB. In the experiments, `WSJ508` was compressed using character-based RE-PAIR and block sizes ranging from 1 MiB to 32 MiB. Coding of the sequence was handled by the Huffman coder, SHUFF, with each block entropy coded independently of others. The results from these experiments are shown in Table 5.1 and Figure 5.1.

Table 5.1 shows that as the block size increases, the overall quality of the phrase hierarchy improves since RE-PAIR is able to find longer phrases. In the fourth column, the total number of symbols in all of the phrase hierarchies decreases as the block size increases. Finally, in the last column, the time for RE-PAIR to process the same file (excluding entropy coding), averaged over three trials, increases with the block size. Since SHUFF performs roughly the same amount of work irrespective of block size, the time taken

Block size	Length of longest phrase	Maximum generation	Number of symbols in phrase hierarchies	Re-Pair time (s)
1 MiB	3,094	25	16,189,378	1,969.9
2 MiB	3,094	27	13,474,410	2,033.7
4 MiB	3,094	29	11,367,932	2,092.2
8 MiB	4,970	28	9,668,774	2,171.1
16 MiB	4,970	29	8,306,278	2,275.6
20 MiB	4,970	31	7,948,461	2,323.3
32 MiB	4,970	31	7,215,594	2,448.0

Table 5.1: Statistics from applying character-based Re-Pair on `WSJ508` by partitioning with block sizes ranging from 1 MiB to 32 MiB. The second and third columns list the length of the longest symbol (in number of primitives) and the maximum generation of all of the blocks, respectively. The fourth column tabulates the total number of primitives and phrases across all blocks. The last column shows the execution times of Re-Pair, excluding the entropy coding of the sequence.

for coding remains the same.

Figure 5.1 depicts compression effectiveness as a function of block size. The division of the output bits between the phrase hierarchy, the compressed sequence prelude, and the compressed sequence codes are also shown. Three horizontal lines indicate the compression effectiveness of gzip, bzip2, and ppmd on `WSJ508`, taken from Table 2.5 on page 41. Note that a comparison between these three systems and Re-Pair is unfair given that Re-Pair is using more memory. Of the four systems, gzip performs the worst, even with the `-9` option, since it creates a context window of only 32 KiB. With the `-9` option, bzip2 creates blocks of 900 KiB, which is comparable to the smallest 1 MiB block size of Re-Pair shown in the graph. The ppmd system with 255 MiB of memory and a seventh order model achieves the best compression overall. Clearly, compression effectiveness suffers when smaller block sizes are used by Re-Pair, because symbols in the phrase hierarchy and codewords in the compressed sequence prelude are duplicated.

In order to minimise the problems with phrase browsing and compression effectiveness when Re-Pair processes a message in blocks, a block merging scheme called Re-Merge has been designed as a post-processing step. Re-Merge operates in phases, as shown in Figure 5.2. These phases can be arranged into different configurations, yielding five methods labelled method A to method E, as shown. The only required phase is phase 1, which is applied immediately after Re-Pair. Then either of two alternative second

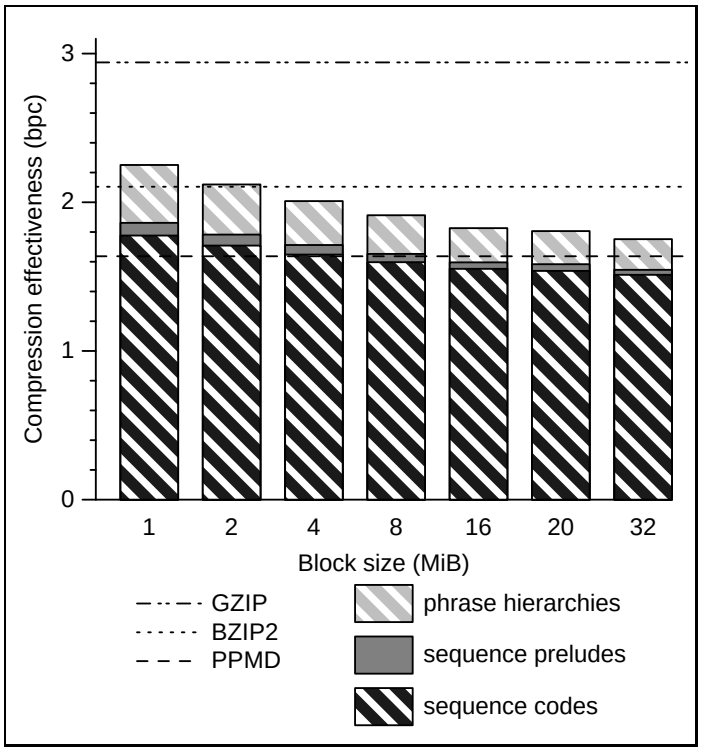


Figure 5.1: Compression ratios from applying character-based RE-PAIR on `WSJ508` by partitioning with various block sizes. Also shown are results from applying the GZIP, BZIP2, and PPMD compression systems.

phases is used, phase 2a or phase 2b. If phase 3 is required, it is performed last. The five combinations of these phases represent the trade-offs that exist between execution time of RE-MERGE, compression effectiveness, and phrase quality, as detailed in experiments later.

Before discussing each phase, the notation introduced in Chapter 3 needs to be extended. As before, the number of primitives and phrases in a phrase hierarchy are $|\Sigma|$ and $|\rho|$, respectively. The phrase hierarchy is denoted as $\mathcal{P}$, and the sequence as $\mathcal{S}$. The total number of primitives and phrases in the phrase hierarchy can be represented as $|\Sigma| + |\rho|$ or $|\mathcal{P}|$. The number of symbols in the sequence is $|\mathcal{S}|$. Let $\mathcal{P}_1$ and $\mathcal{P}_2$ represent initial phrase hierarchies before any phase, and $\mathcal{P}'$ represent the final phrase hierarchy after the same phase. Their corresponding sequences are $\mathcal{S}_1$, $\mathcal{S}_2$, and $\mathcal{S}'$. When a symbol α is expanded, it becomes a string of primitives with a length equal to $|\alpha|$. In a string, a vertical bar (|)

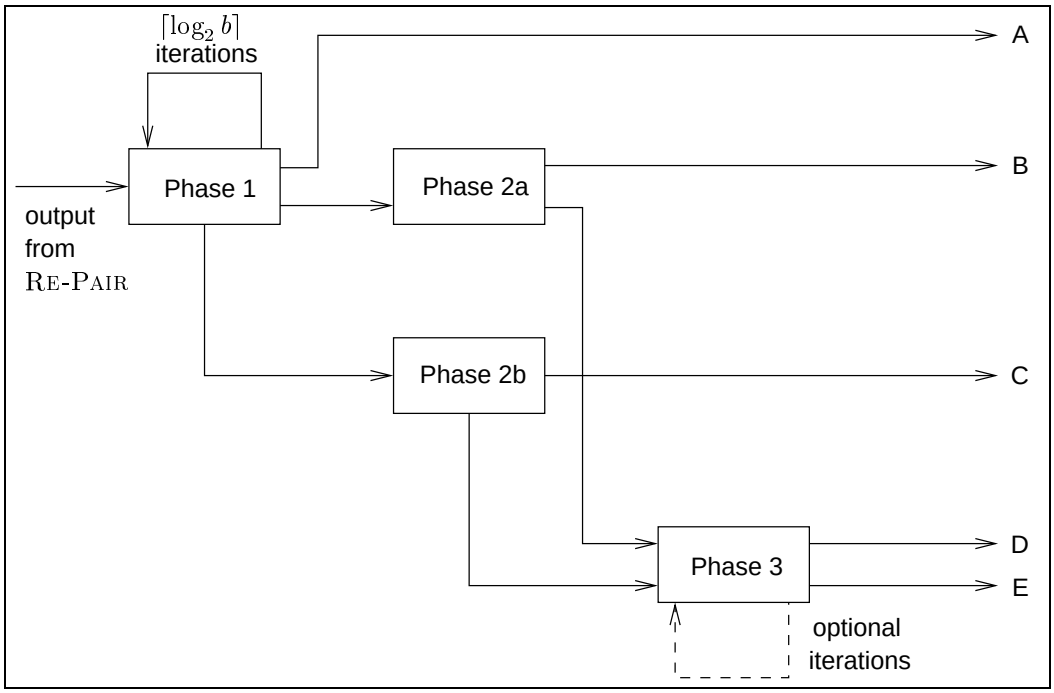


Figure 5.2: The three phases of Re-Merge and the combinations in which they are applied. Two alternatives to the second phase of Re-Merge exists, labelled phase 2a and phase 2b. Five combinations of the Re-Merge phases are possible, labelled A to E, on the right. The two outputs of Re-Pair are required by every Re-Merge phase, even though they may not be modified by every phase. So, a line represents both streams. Entropy coding of the reduced sequence is performed when Re-Merge has completed every necessary phase.

is used to indicate the division between the expanded left component and the expanded right component (for example, "wood|chuck").

The next three sections describe the three phases of Re-Merge in detail.

5.2 Merging Phrase Hierarchies

The first phase of Re-Merge carries out a pair-wise merging of blocks via a series of passes. Each pass halves the number of blocks, meaning that for an initial message of b blocks, $\lceil \log_2 b \rceil$ passes are required. During each pass, the union of the pair of phrase hierarchies, $\mathcal{P}_1$ and $\mathcal{P}_2$, is formed to produce $\mathcal{P}'$. After the two phrase hierarchies are merged, the sequences $\mathcal{S}_1$ and $\mathcal{S}_2$ are modified to ensure that each symbol refers to $\mathcal{P}'$ and not their respective phrase hierarchies. Afterwards, $\mathcal{S}_1$ and $\mathcal{S}_2$ are concatenated together.

Initially, phase 1 decodes both phrase hierarchies. They are then merged one generation at a time, with duplicate phrases that have equal left and right components removed. Then, each symbol is expanded into a string of primitives. The phrase hierarchy is sorted by these strings, producing a temporary phrase hierarchy, $\mathcal{P}^*$. With the strings sorted, duplicate symbols that exist across and within generations, but with different components, can also be identified and removed. Recall from page 58 that when a message (or in this case, a block) is processed by Re-Pair, duplicates may occur within the phrase hierarchy which have different components. For example, one symbol may expand to "a|aa" while another symbol may expand to "aa|a". The merging of two phrase hierarchies occurs in two steps in order to reduce the size of $\mathcal{P}^*$.

The Re-Pair decompressor, Des-Pair, also expands phrases into primitives. However, since symbols are expanded into primitives only as needed, Des-Pair uses a buffer of recently decoded symbols as a trade-off between execution time and memory space (Table 3.2, page 49). The needs of phase 1 of Re-Merge differs from that of Des-Pair. In order to sort the strings for $\mathcal{P}^*$, ideally, all strings have to be available for sorting at once, and therefore all need to be expanded beforehand. But simply decoding every symbol could potentially occupy memory proportional to the length of the original message. Phase 1 of Re-Merge (and subsequent phases which require the phrase hierarchy to be expanded) take an alternative approach which expands symbols starting from the last generation of symbols. Only symbols which are not used as a component of any other symbol are expanded. As a symbol is expanded, all symbols which it is composed of, either directly or indirectly, refer to the substrings of the expanded string instead. This approach ensures that a minimum amount of memory is taken up by the strings, while all of them are available for sorting and other operations in later Re-Merge phases.

When two or more symbols in $\mathcal{P}^*$ have been found which expand to identical strings, the one with the lower generation is retained. When a symbol has one of its components removed, it must reference the retained duplicate as a component instead. The symbol's generation number also decreases accordingly. As a result, a cascading effect occurs, and symbols that refer to a removed symbol, either directly or indirectly, shift to lower

generations. The overall result is the reduction of gaps in the phrase hierarchy. After the removal of duplicates, the symbols in the phrase hierarchy are sorted in generation and chiastic slide value order to form $\mathcal{P}'$, for the interpolative coding stage.

Figure 5.3 shows an example of Re-Merge phase 1. The following message is compressed with Re-Pair using blocks of 17 characters: woowoodwoodywoody$\overline{\text{woodywood}}$. Two blocks are created by Re-Pair, as shown with the underline and overline. Their respective lengths are 17 characters and 9 characters. When Re-Pair is applied to these two blocks independently, two sets of phrase hierarchies and sequences are produced, as shown in Figure 5.3(a) and Figure 5.3(b). In the example, the symbols in each of these phrase hierarchies are expanded immediately only for illustrative purposes. To the right of each expanded symbol is its generation. The two phrase hierarchies are merged so that duplicates with the same components are removed. At this stage, the algorithm expands each symbol into primitives and sorts each string, producing $\mathcal{P}^*$. As Figure 5.3(c) shows, the duplicate "wood" exists and the higher generation of the two is removed. This affects the fourth generation phrase "woody" by shifting it into the third generation. Figure 5.3(d) shows the final merged phrase hierarchy, $\mathcal{P}'$, sorted in generation and chiastic value order. Below the phrase hierarchy is $\mathcal{S}'$, created from the concatenation of the two initial sequences and renumbered so that it refers to the final phrase hierarchy.

In order for $\mathcal{P}^*$ to be formed, symbols that are not referred to by a higher generation symbol are expanded. This is depicted by Figure 5.4 for the example of Figure 5.3. In Figure 5.4, the symbols have been rearranged into a directed acyclic graph, with a level in the graph corresponding to a particular generation. The symbols with the highest generation are at the top of the figure, while primitives are lined up along the bottom. Directed edges are added from each phrase to its two components, except for the primitives which have no edges emanating from them.

The symbol expansion by Re-Merge phase 1 starts from the highest generation symbol with the greatest chiastic slide value in that generation. The equivalent in the hierarchical graph representation is to commence from the top of the graph, and then proceed toward the primitives, a level at a time. As Figure 5.4 shows, the order within a level is

$\mathcal{P}_1$		$\mathcal{P}_2$		$\mathcal{P}^*$		$\mathcal{P}'$	
$0 \rightarrow$ d	0	$0 \rightarrow$ d	0	d	0	$0 \rightarrow$ d	0
$1 \rightarrow$ o	0	$1 \rightarrow$ o	0	o	0	$1 \rightarrow$ o	0
$2 \rightarrow$ w	0	$2 \rightarrow$ w	0	o|d	1	$2 \rightarrow$ w	0
$3 \rightarrow$ y	0	$3 \rightarrow$ y	0	o|o	1	$3 \rightarrow$ y	0
$4 \rightarrow$ o|o	1	$4 \rightarrow$ o|d	1	w	0	$4 \rightarrow$ o|d	1
$5 \rightarrow$ w|oo	2	$5 \rightarrow$ w|o	1	w|o	1	$5 \rightarrow$ o|o	1
$6 \rightarrow$ woo|d	3	$6 \rightarrow$ wo|od	2	w|oo	2	$6 \rightarrow$ w|o	1
$7 \rightarrow$ wood|y	4			wo|od	2	$7 \rightarrow$ w|oo	2
				~~woo|d~~	~~3~~	$8 \rightarrow$ wo|od	2
				wood|y	4	$9 \rightarrow$ wood|y	3
				y	0		
$\mathcal{S}_1 = [\ 5\ 6\ 7\ 7\ ]$		$\mathcal{S}_2 = [\ 6\ 3\ 6\ ]$				$\mathcal{S}' = [\ 7\ 8\ 9\ 9\ 8\ 3\ 8\ ]$	
(a)		(b)		(c)		(d)	

Figure 5.3: The merging of two phrase hierarchies using RE-MERGE phase 1. Below the two initial phrase hierarchies and the final phrase hierarchy are their respective sequences. Vertical bars (|) indicate the division between the left and right components of an expanded symbol. Also shown is the intermediate phrase hierarchy, $\mathcal{P}^*$, and the removed duplicate, "woo|d". Symbols from every phrase hierarchy have been expanded into primitives, with each symbol's generation shown on the right.

unimportant, since no symbol can depend on other symbols within the same generation. With the directed graph, expansion into primitives can be summarised as first searching level-by-level to find a symbol to expand, skipping those that have already been expanded. Once a node without a directed edge leading to it has been identified, all of its descendents are "decoded" in a depth-first fashion by making each descendent refer to the higher generation node. In the example, when the fourth generation symbol, "wood|y" is first encountered, it is expanded. Then, all symbols that refer to "wood|y", either directly or indirectly, are "expanded" by making the lower generation symbols refer to parts of the expanded string and by recording the length of the substring. Note that in this example, in order to expand the entire phrase hierarchy, only two symbols, "wood|y" and "wo|od" have to be explicitly expanded. Finally, note that after $\mathcal{P}^*$ has been sorted and duplicates removed, some symbols may be deleted even though their expansion is needed by a lower generation symbol. Because of this, it is important that symbols in $\mathcal{P}^*$ are not actually removed during phase 1, but simply ignored when $\mathcal{P}'$ is built.

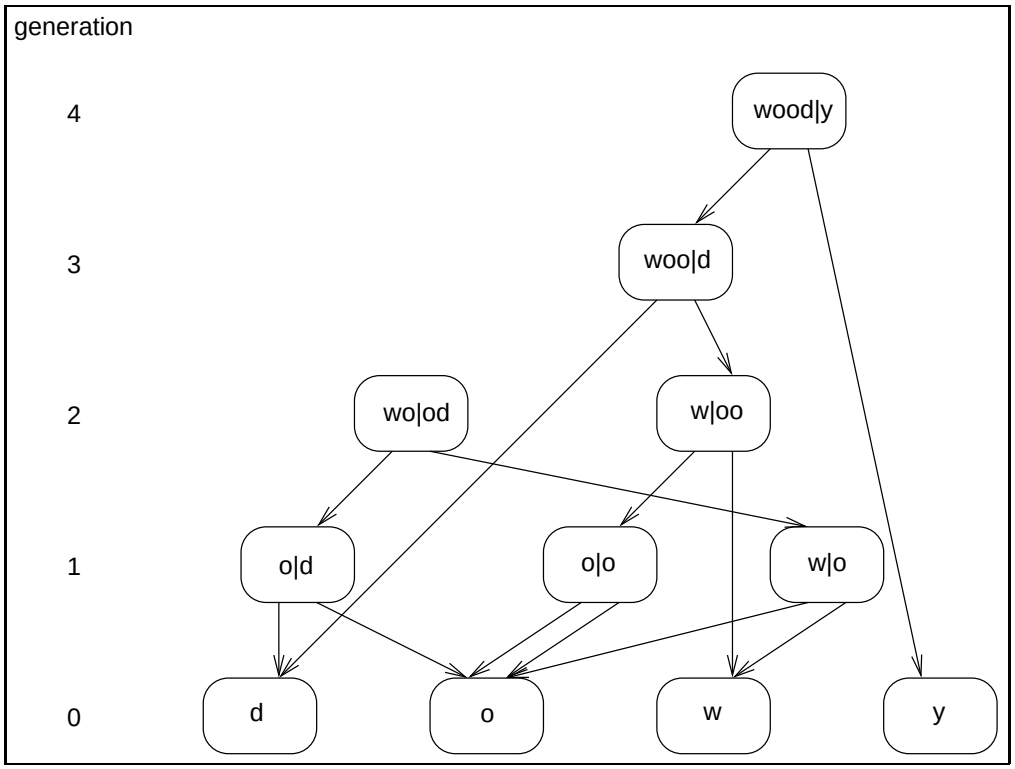


Figure 5.4: The phrase hierarchy of Figure 5.3(c), showing the two symbols that each symbol depends on. Symbols are arranged with the primitives along the bottom, and the highest generation phrase at the top. Two phrases are not used by any other symbol, "wo|od" and "wood|y".

The amount of memory that this phase takes is proportional to the size of the input and output phrase hierarchies. Enough space is required to keep $\mathcal{P}_1$, $\mathcal{P}_2$, and $\mathcal{P}^*$ in memory. To minimise duplication in memory, the final phrase hierarchy can be drawn from $\mathcal{P}^*$. Additional space is also required for the expansion of primitives, while the concatenation and renumbering of the sequences can be performed without keeping any symbols in memory. The space requirements of phase 1 is an improvement to Re-Pair because the memory usage is proportional to the size of the phrase hierarchy, and not the length of the initial sequence.

Excluding the costs of sorting, phase 1 operates through a series of linear passes over the phrase hierarchies. First, the phrase hierarchies are merged a generation at a time by inserting symbols from $\mathcal{P}_1$ into a hash table and then comparing each symbol in $\mathcal{P}_2$ with the hash table entries. When the two initial phrase hierarchies are merged, each symbol is

expanded by performing another pass over the $|\mathcal{P}^*|$ symbols. Once the symbols are sorted, a third linear pass over $\mathcal{P}^*$ is required to eliminate duplicates. A final sort by generation number and chiastic slide value prepares the phrase hierarchy for interpolative coding. The initial sequences are renumbered and concatenated, with buffering required only for minimising disk accesses. Combining two phrase hierarchies requires time proportional to $|\mathcal{P}_1| + |\mathcal{P}_2| + 2|\mathcal{P}^*| + |\mathcal{S}'|$, not including the two sorts that are performed. This amount of time is required for each of the $\lceil \log_2 b \rceil$ merges of a compressed message of b blocks.

The aim of phase 1 is to merge the phrase hierarchies of every block into a single, unified phrase hierarchy, while also removing any duplicate symbols. The relative sizes of the initial, intermediate, and final phrase hierarchies can be summarised as follows: $\max(|\mathcal{P}_1|, |\mathcal{P}_2|) \leq |\mathcal{P}'| \leq |\mathcal{P}^*| \leq |\mathcal{P}_1| + |\mathcal{P}_2|$. After $\lceil \log_2 b \rceil$ iterations of phase 1, all of the phrase hierarchies have been merged and all of the sequences have been concatenated. Effectively, phase 1 has merged all of the blocks in the reduced message and the phrase hierarchy can be browsed.

While phase 1 makes phrase browsing feasible and improves compression effectiveness, even better compression can be obtained. For example, in the final sequence in Figure 5.3(d), the pair of symbols [8 3] in $\mathcal{S}'$ can be replaced by a single symbol, [9]. The next section describes phase 2 of Re-Merge, which shifts the attention from the phrase hierarchy to the sequence, to try to catch such redundancies.

5.3 Identifying Old Symbol Pairs

After the phrase hierarchies have been merged, pairs of symbols may appear in the concatenated sequence that are now candidates for replacement. For example, if $\alpha\beta$ exists twice in block 1, but once in block 2, it would have been recognised as a phrase in the first block, but not in the second. The concatenated sequence needs to be checked to ensure no pairs of symbols exist that are already represented by symbols in the merged phrase hierarchy.

Identifying symbol pairs in the initial sequence that already have a corresponding rule for replacement is the motivation for phase 2. Phase 2 reduces the length of the

sequence, but makes no changes to the phrase hierarchy. As with Re-Pair, the goal of this phase is to minimise the length of the sequence. Since phase 2 is separated from the coder, no guarantees can be made that reducing the length of the sequence also improves compression effectiveness. However, phase 2 improves the usefulness of the phrase browsing interface. Suppose a user is examining a symbol γ, which expands to the pair $\alpha\beta$. Phase 2 ensures that more occurrences of this pair of symbols can be located with the phrase browsing system. Since this phase does not alter the merged phrase hierarchy $|\mathcal{P}'| = |\mathcal{P}_1|$. But as the aim of phase 2 is to reduce the length of the initial sequence, $|\mathcal{S}'| \leq |\mathcal{S}_1|$.

Two methods of performing phase 2 are explored in this section. The first method, phase 2a, is biased towards efficiency, while the second method, phase 2b, favours compression effectiveness.

Efficient Sequence Reduction

Phase 2a of Re-Merge determines replacements by performing a single pass through the initial sequence $\mathcal{S}_1$. As each symbol is processed in the pass, it is considered in combination with the immediately preceding symbol, and a replacement made if the pair exists in the phrase hierarchy.

The steps for phase 2a are shown in Algorithm 5.1. First, a hash table of symbols is constructed, with all phrases in the phrase hierarchy hashed according to their two components, and collisions handled by chaining. Then, each symbol in $\mathcal{S}_1$ is processed as follows. When a symbol α_i from the sequence is read, it is combined with its immediately preceding symbol α_{i-1}. The pair $\alpha_{i-1}\alpha_i$ is searched in the hash table. If a matching phrase γ is found, then a replacement is made. Then γ is combined with its preceding symbol, the one that preceded α_{i-1}. This continues until a replacement is no longer possible. Then, the next symbol in $\mathcal{S}_1$ is read and the process repeats. Phase 2a completes when the end of $\mathcal{S}_1$ has been reached.

The memory requirements of phase 2a include the phrase hierarchy, without symbols expanded, and a buffer for the initial sequence. Since the replacement γ is at least one

Algorithm 5.1: Algorithm for RE-MERGE phase 2a.

```
   input   : P_1 and S_1
   output  : P' and S'
 1 foreach symbol in P_1 do
 2     insert symbol into hash table using its components
 3 for i < |S_1| do
 4     replacement ← TRUE
 5     while replacement = TRUE do
 6         γ ← searchHashTable(α_{i-1}, α_i )
 7         if γ found then
 8             α_{i-1} ← γ
 9             i ← i − 1
10             replacement = TRUE
11         else
12             replacement = FALSE
13 P' ← P_1; S' ← S_1
14 return (P', S')
```

generation higher than α and β, the maximum number of consecutive replacements cannot be more than the maximum symbol generation in the phrase hierarchy. Therefore, a buffer whose size is equal to at least the maximum symbol generation is sufficient. As experiments later show, applying character-based RE-PAIR on blocks from 508 MiB of text produces a phrase hierarchy whose highest symbol generation is 31.

With respect to execution time, phase 2a performs two main operations. First, all of the symbols in the phrase hierarchy are inserted into the hash table. Second, every symbol is combined with its preceding symbol for searching in the hash table. If we assume that the size of the hash table is selected so that the length of the chains are about 1, then accessing it during creation or searching requires constant time. So, the first operation can be performed in $O(|\mathcal{P}_1|)$ time. The second task requires $|\mathcal{S}_1|$ hash table look-ups when no replacements are made. If the number of replacements made is taken into account, then note that the most substitutions an input symbol can participate in cannot be more than the highest symbol generation. That is, if the highest symbol generation is g, then the second task runs in $O(g|\mathcal{S}_1|)$. After phase 2a completes, the number of searches into the

hash table can be given as $|\mathcal{S}_1| + (|\mathcal{S}_1| - |\mathcal{S}'|) \leq g|\mathcal{S}_1|$. A single replacement during phase 2a reduces the length of $|\mathcal{S}_1|$ by one symbol and adds one search operation into the hash table. As the sequence usually has more symbols than the phrase hierarchy, the execution time of phase 2a is $O(g|\mathcal{S}_1|)$.

This phase of Re-Merge is able to locate all pairs of symbols in the sequence that can be replaced. Phase 2a is fast and memory efficient, as demonstrated in experiments later in this chapter.

Effective Sequence Reduction

While phase 2a locates all pairs of symbols in the sequence produced by phase 1, the choices made may not be the best. A replacement now may prevent an even better replacement later that ends up reducing the overall length of the sequence. As an alternative to phase 2a, phase 2b optimally re-parses the sequence using the merged phrase hierarchy as a reference point. Phase 2b should not be used in conjunction with phase 2a, as doing so achieves no benefit.

Phase 2b is based on work by Katajainen and Raita [1989] and Klein [1997], and involves the re-parsing, or recompression, of the original message using phrases from a dictionary. Like the previous investigations in recompression, phase 2b converts the problem of compression to that of determining the single source shortest path in a directed acyclic graph (DAG). Unlike the earlier work, phase 2b is tailored specifically for Re-Pair, in that it exploits the structure in the phrase hierarchy and sequence.

Phase 2b constructs a graph G with a set of vertices V connected by a set of edges E denoted as $G(V, E)$. Each vertex $v_i \in V$ represents the ith primitive in the original message. An additional vertex is added to V as a *sink node*. For a message of n primitives, V has $n + 1$ vertices.

Edges are attached to the vertices based on the strings in the phrase hierarchy. Vertices are examined in the order dictated by the original message, with the sink node being last. A directed edge $e_j \in E$ is drawn from v_i to v_{i+k}, for $k > 0$, if a symbol exists in the phrase hierarchy which, when expanded, coincides with the k primitives starting from v_i up to,

Phrase hierarchy index	Expanded phrase
0	d
1	o
4	od
5	oo
2	w
6	wo
7	woo
8	wood
9	woody
3	y

Table 5.2: The final phrase hierarchy of Figure 5.3, sorted in alphabetical order. The indexes in the first column refer to the positions of the symbols in the unsorted phrase hierarchy.

and including, v_{i+k-1}. Since every primitive in the message exists in the phrase hierarchy, every vertex has at least one edge leading to the next vertex, and the graph is guaranteed to be connected. The vertex representing the last primitive has an edge leading to the sink node v_{n+1}. Every edge has a weight of one.

After all possible edges are added to E, a new sequence $\mathcal{S}'$ is created by finding the shortest path from v_1 to the sink node v_{n+1}, thereby minimising the total number of edges traversed. Because the edges have unit cost, the shortest path can be determined through breadth-first search [Cormen et al., 2001], and represents the optimal parsing of the message using the phrase hierarchy, if optimality is measured by the number of edges spanned.

Figure 5.5 shows a graph for the merged phrase hierarchy and the beginning of the decoded message from Figure 5.3. That phrase hierarchy has been reproduced in Table 5.2, sorted in alphabetical order. In Figure 5.5, the optimal parsing for the first 7 primitives consists of the two symbols 7 and 8, obtained by traversing the dashed edges.

If the phrase hierarchy is sorted, edges can be selected for E through binary search. For example, suppose the vertex v_1 in Figure 5.5 is currently being processed. Let the string starting from this vertex, "woowoodw...", be denoted as τ_1. A binary search for τ_1 in the phrase hierarchy would return "woo" as the longest match. By reducing the

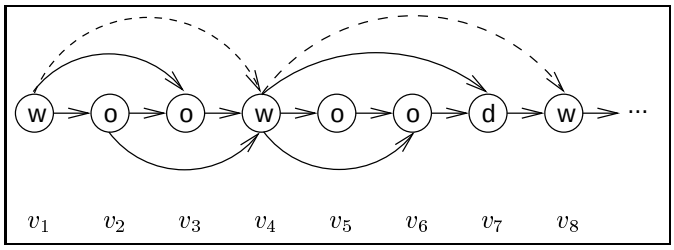


Figure 5.5: A graph representation of part of the message from the example of Figure 5.3. The symbols in the final phrase hierarchy of that figure allow edges to be added. The shortest path through these 8 vertices is shown as dashed lines.

number of significant primitives in τ_1, other edges from v_1 can be found, until the shortest edge of length 1 has been located. This process must be repeated for every vertex in the graph.

The cost of finding just the longest match for every vertex in a graph derived from a message of n primitives is $O(n \log_2 |\mathcal{P}_1|)$ string comparisons. A slight improvement is possible when an index into the phrase hierarchy is set up for the $|\Sigma|$ primitives to constrain the binary search. For example, the longest match for "woowoodw..." must exist between "w" and "x", assuming both exist as primitives in the phrase hierarchy. In the worse case, if every symbol begins with the same primitive, then such a data structure would neither improve nor significantly worsen the execution time. While not implemented for the experiments later, the size of the index could increase by indexing on more than one primitive.

Phase 2b applies binary search with this index structure for primitives, while also exploiting other characteristics of the phrase hierarchy, and of the symbols in the initial sequence. There are four main steps to Re-Merge phase 2b, which are described in the next few paragraphs. The first step adds links to each symbol in the phrase hierarchy. The second step expands the initial sequence into primitives to form V. Then, edges are added to E in the third step. Finally, $\mathcal{S}'$ is output by determining the shortest path through the vertices.

Additional links for the phrase hierarchy

Phase 2b assumes that each symbol in the phrase hierarchy has been expanded into primitive strings and then sorted on these strings. Three types of links are added to each symbol in the phrase hierarchy in preparation for phase 2b. The three link types for each symbol lead to a *primary prefix symbol*, a *secondary prefix symbol*, and a list of *extended symbols*.

A symbol β is a prefix of another symbol α, if $|\beta| \leq |\alpha|$ and the expanded string of β coincides with the first $|\beta|$ primitives of α. A symbol β is the *primary prefix symbol* of α if β is the longest prefix symbol in the phrase hierarchy for α. A symbol β is the *secondary prefix symbol* of α if β is the longest prefix of α, starting from α's second primitive. The primary and secondary prefix symbols of a primitive are the empty symbol, λ. For example, in the phrase hierarchy of Table 5.2, the symbol "wood" has "woo" and "oo" as its primary and secondary prefix symbols, respectively. As no two symbols in the phrase hierarchy expand to the same string after phase 1, every symbol is guaranteed to have exactly one symbol that serves either role.

The symbol β is an *extended symbol* of α, if two conditions hold. First, α must be the primary prefix symbol of β. Second, for some primitive δ, no symbol which is lexically smaller than β exists which has the string $\alpha\delta$ as a prefix, where the juxtaposition of symbols represents string concatenation. According to this definition, the string $\alpha\delta$ does not have to exist in the phrase hierarchy, but this is possible if β is longer than α by one primitive. For example, in Table 5.2, "wood" is an extended symbol of "woo", where $\delta =$ "d".

A symbol has a list of zero to at most $|\Sigma|$ extended symbols. An extended symbol is recorded as a pair, of the form (δ, β), and ordered in the list according to δ. The pair (δ, β) states that, for the current symbol α, the primitive δ can be appended to it, and the lexically smallest symbol in the sorted phrase hierarchy with this string as a prefix is β. When the list of extended symbols is non-empty, then a special pair of the form (ϵ, β) is added to the end. This pair, called an ϵ-pair, indicates that in the lexically sorted phrase hierarchy, no symbol has α as a primary prefix symbol at or after β. Also, the symbol

Phrase hierarchy index	Expanded phrase	Primary prefix symbol	Set of extended symbols
...			
999	wo	847	[("o", 1000), ("r", 1009), (ϵ, 1121)]
1000	wood	999	[("b", 1001), ("c", 1002), (ϵ, 1008)]
1001	woodblock	1000	
1002	woodcarving	1000	
1003	woodchip	1000	
1004	woodchuck	1000	[("s", 1005), (ϵ, 1006)]
1005	woodchucks	1004	
1006	woodcraft	1000	
1007	woodcut	1000	[("t", 1008), (ϵ, 1009)]
1008	woodcutter	1007	
1009	work	999	
...			

Table 5.3: Binary search on the phrase hierarchy using extended symbols. The numbering for the index (first column) is referred to by the third and fourth columns. In the second column, every symbol has been expanded into primitives, and the entire phrase hierarchy has been sorted on this column.

immediately before β must have α as a primary prefix symbol.

Table 5.3 further clarifies the definition of extended symbols by demonstrating how they are used. The table shows a section of a sorted phrase hierarchy from the symbol "wo" to "work", with the symbols expanded into strings. Also shown are every symbol's primary prefix symbol and list of extended symbols. Suppose τ_1 = "woodcutters" and, through methods described below, the current prefix for τ_1 is the string "wood", but whether a longer one exists is unknown. Since the first four primitives of "wood" match with τ_1 and the next primitive in τ_1 is available, the longest match must exist between "woodc" and the lexically largest symbol that has "wood" as a prefix symbol and is followed by a "c". These two boundaries can be established through two consecutive extended symbol pairs for the symbol "wood". While a binary search within the search space allows a longer prefix to be found for τ_1, locating the longest prefix requires this process to be repeated. Rather than applying binary search on the entire phrase hierarchy, extended symbols permit sections of the phrase hierarchy to be isolated based on the primitives that have already been compared.

The steps performed for the example of Table 5.3 is as follows. The extended symbols of "wood" indicate that a "c" can be appended to it. The first such symbol is symbol 1002, "woodcarving". The last symbol that has "wood" as a primary prefix symbol and "c" as the fifth primitive is "woodcut", which is one symbol before the ϵ-pair to "woodcutter". A binary search between these two symbols, shown shaded in Table 5.3, is performed next. Within this interval, all of these strings have the same first five primitives. However, not all of them share the same primary prefix symbol. The longest match within this search interval is "woodcut", but it is still not the longest match for τ_1. An attempt to extend "woodcut" using the next primitive, "t" is made. It succeeds and a search interval of one symbol (1008) is found. This symbol cannot be extended further, so the longest match for τ_1 = "woodcutters" is "woodcutter".

As an example of primary prefix symbols, secondary prefix symbols, and sets of extended symbols, Table 5.4 shows these links for the phrase hierarchy of Table 5.2. The first two columns of the figure show the phrase hierarchy indexes and expanded strings. The next two columns show the primary and secondary prefix symbols. In the last column, the lists of extended symbols are shown for symbols that are primary prefix symbols of another symbol. In case an ϵ-pair points outside of the phrase hierarchy, an extra symbol has been added at position 10 of the phrase hierarchy.

Phase 2b requires the phrase hierarchy to be sorted by strings of expanded symbols. Then, symbols are linked through a series of passes. Primary prefix symbols are assigned first by comparing each symbol with its immediately preceding symbols. When searching, the first prefix symbol located serves as the primary prefix symbol, due to the sorted order. While it is possible for a symbol to be compared with every symbol before it, in practice, a match is usually found after a few strings have been checked. A second pass is done to attach symbols to the lists of extended symbols, using the primary prefix symbol links. Each symbol adds or updates an extended symbol pair to the list associated with its primary prefix symbol. A third pass makes use of the other two types of links to assign the secondary prefix symbols. With the phrase hierarchy sorted according to expanded symbols, consecutive symbols are expected to have the same, or similar, secondary prefix

Phrase hierarchy index	Expanded phrase	Primary prefix symbol	Secondary prefix symbol	Set of extended symbols
0	d	λ	λ	
1	o	λ	λ	[("d", 4), ("o", 5), y(ϵ, 2)]
4	od	1	0	
5	oo	1	1	
2	w	λ	λ	[("o", 6), (ϵ, 7)]
6	wo	2	1	[("o", 7), (ϵ, 8)]
7	woo	6	5	[("d", 8), (ϵ, 9)]
8	wood	7	5	[("y", 9), (ϵ, 3)]
9	woody	8	5	
3	y	λ	λ	
10	(End of phrase hierarchy)			

Table 5.4: The additional symbol links required for the phrase hierarchy before starting phase 2b. Primary and secondary prefix symbols are shown in the third and fourth columns, after the expanded symbols. These symbols do not exist for primitives, and are indicated by the empty symbol, λ. In the last column, sets of extended symbols are either empty, or made up of at least two pairs, with the last pair being of the form (ϵ, α). Note that the pair $(\epsilon, 7)$ of symbol 2 points to symbol 7 and not 10. While symbols 7 to 9 start with "w", none of them have a primary prefix symbol of 2.

symbols. This observation assists in the creation of this third link, in the absence of re-sorting the phrase hierarchy from each string's second primitive.

The three types of links add $6|\mathcal{P}_1|$ words of memory to the phrase hierarchy of phase 1. Every symbol in the phrase hierarchy has exactly one primary prefix symbol and one secondary prefix symbol, each stored in one word of memory. An extended symbol pair requires two words of memory, and there cannot be more than $|\mathcal{P}_1|$ of them. As proof of this claim, note that one of the conditions for α to be an extended symbol of β is that β is the primary prefix of α. Since α has an unique primary prefix symbol, each symbol in the phrase hierarchy participates as an extended symbol only once. Finally, for each non-empty set of extended symbols, there must be an extra ϵ-pair to end the list of pairs. Each of these require two words of memory.

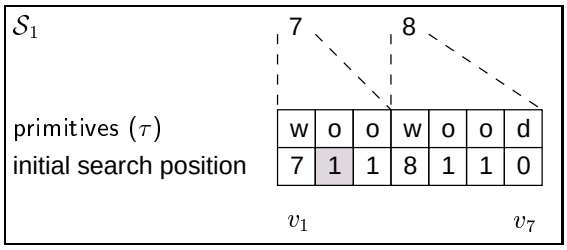


Figure 5.6: Example decoding of $\mathcal{S}_1$ into vertices, represented as an array. Each vertex keeps track of the primitive and an initial starting position.

Decoding the sequence

After the symbol links have been established for the phrase hierarchy, the next step is to expand the symbols in $\mathcal{S}_1$ into τ. For a message of n primitives, this operation requires memory proportional to n. Alternatively, $\mathcal{S}_1$ can be decoded a symbol at a time so that the length of τ is at least equal to the length of the longest symbol in the phrase hierarchy, α_L.

While expanding the sequence into τ, the original symbols in $\mathcal{S}_1$ are maintained to improve search times later. Each vertex is augmented with an *initial search position*. The initial search position at vertex v_i is a symbol which is guaranteed to be the lower bound in a binary search for the longest prefix for τ_i. For example, if τ_1 is the string "woowoodw...", then "w" is one possibility for v_1. However, a longer initial search position is possible if $\mathcal{S}_1$ is taken into account. When a symbol is decoded into primitives for τ_i, the symbol may not represent the longest match, but it may still be a better match than a single primitive. This is illustrated in Figure 5.6 using the final phrase hierarchy and sequence from Figure 5.3. The first two symbols of $\mathcal{S}_1$ are 7 and 8. The primitives at the beginning of the expanded symbols acquire these 2 symbols as initial search positions, as shown in the figure.

The remaining initial search positions are initialised with symbols representing single primitives. However, each are later replaced by the secondary prefix symbol of the preceding vertex, if it is longer. For example, in Figure 5.6, after the longest edge representing "woo" is added to v_1, its secondary prefix symbol is "oo". This symbol replaces the initial search position at v_2, which is the primitive "o" (shown shaded).

Figure 5.6 also demonstrates how only the minimum number of symbols need to be decoded for τ_i. If the current vertex being processed is v_1, then since $|\alpha_L| = 5$ for Table 5.4, only two symbols need to be decoded to create τ_1 with a length of 7 primitives. Additional symbols are decoded as phase 2b progresses through τ.

Adding edges to the graph

After enough of the sequence has been decoded so that $|\tau_i| \geq |\alpha_L|$, edges are added to v_i, starting with the longest one. The longest edge from v_i is also the primary prefix of τ_i.

Let the symbol β represent the best prefix symbol so far and δ be the primitive at position $|\beta| + 1$ in τ_i. The symbol β is initialised with the initial search position of v_i, and is concatenated with δ for searching in the extended symbols of β. If the search is unsuccessful, then β is the longest match. But if the search is successful, two consecutive symbol pairs are retrieved, where the first one is the pair (δ, α). These two pairs, α_{left} and α_{right}, form an area within the phrase hierarchy for binary search. If the binary search fails, the current version of β is the longest primary prefix. If the search succeeds, then a longer string is returned. The process repeats with this longer string by first extending it with its next primitive in τ_i.

Once the longest edge has been determined and added, all shorter edges are located by repeatedly following prefix symbol links until the empty symbol (λ) is reached.

Determining the shortest path

The last step of phase 2b requires the calculation of the shortest path through the graph. Earlier, the number of symbols in the sequence that have to be decoded was reduced by noticing that only the first $|\alpha_L|$ primitives in τ_i are significant. Another observation divides the expanded sequence into segments in order to prevent the entire message from being kept in memory.

A *segment* is defined as being a sequence of vertices which begins and ends with *cut vertices* (also called *choke points*). A cut vertex is a vertex which has edges starting or ending at it, but no edges that bypass it. The first vertex v_1 and the sink node v_{n+1} are

both examples of cut vertices. Other cut vertices also exist in most sequences and can be taken advantage of.

Since no edges bypass a cut vertex, the shortest path through the set of vertices must go through that vertex. Therefore, if the graph G is processed in segments, no loss in the optimal parsing of the message occurs. The shortest path can be determined a segment at a time, rather needing to have all of V in memory.

Figure 5.7 demonstrates how the first few primitives of Figure 5.6 are handled using the phrase hierarchy and symbol links of Table 5.4. Three pointers to the vertices are maintained: *min*, *max*, and i. The vertex at position i is currently being examined. The *min* pointer represents the beginning of the current segment, while *max* points to the vertex that is reached by the longest edge. Whenever the vertex currently being examined meets *max*, a cut vertex has been found, as in Figure 5.7(d).

An examination of Figure 5.7 reveals an alternative method of performing phase 2b, which has not been attempted. Currently, at each step, the vertex at position i is examined to determine all of the edges that start from i. Another way of performing phase 2b is to determine all edges that end at i, instead. This method of conducting phase 2b has the potential of being more memory efficient. As edges are added, others may be removed if a better way of reaching the current vertex i has been found. At least one change to the current phase 2b algorithm is required in order to use this proposed version. While phase 2b requires the phrases in the hierarchy to be sorted in the forward direction, this proposed version would require all of the strings to be reversed, and then sorted.

The segment from *min* to *max* is processed by finding the shortest path through the segment. The shortest path through a segment is determined by using breadth-first search, which has a running time of $O(|V| + |E|)$ [Cormen et al., 2001]. After the shortest path has been found, the symbols that represent that path are added to the final sequence $\mathcal{S}'$. At this point, the memory taken up by the segment is released before starting on the next segment with i and *min* starting from *max*.

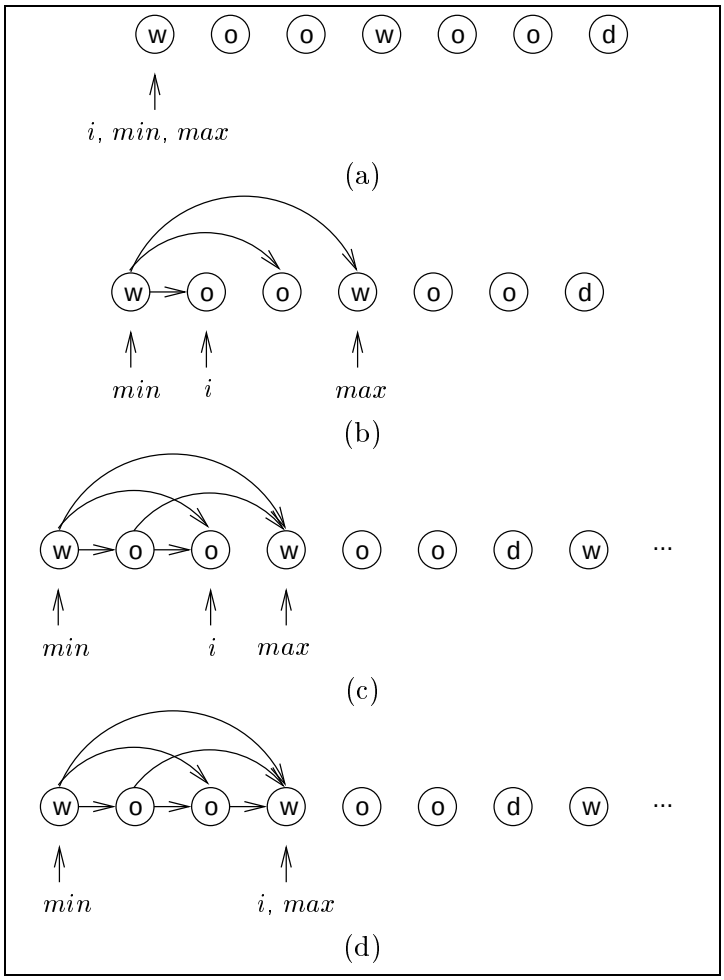


Figure 5.7: Examining vertices to locate the next cut vertex. The pointer i indicates the vertex that is being considered. When i coincides with the pointer max, a cut vertex has been found, and the next segment is from min to max. Note that when $|\tau_i|$ is less than the length of the longest symbol, more symbols need to be decoded, as in the lower two figures.

Analysing Phase 2b

The algorithm for phase 2b is shown in Algorithm 5.2. This algorithm excludes the sorting of the phrase hierarchy by expanding symbols and the addition of symbol links Algorithm 5.2 requires $\mathcal{P}_1$, $\mathcal{S}_1$, and the sorted phrase hierarchy $\mathcal{P}_s$ as input. At the end, $\mathcal{P}'$ and $\mathcal{S}'$ are produced by the algorithm.

The main loop, starting at line 3, continues until the initial sequence $\mathcal{S}_1$ has been exhausted. The sequence is decoded a symbol at a time into primitives on line 4 to

Algorithm 5.2: Algorithm for RE-MERGE phase 2b.

input : $\mathcal{P}_1$, $\mathcal{P}_s$, and $\mathcal{S}_1$
output : $\mathcal{P}'$ and $\mathcal{S}'$

```
1  α_L ← longest symbol in P_1
2  min ← 0; i ← 1
3  while not end of S_1 do
4      decode and expand next symbol in S_1 and append the expansion to τ_i
5      while |τ_i| > |α_L| do
6          β ← initial search position at v_i
7          while longest match not found do
8              δ ← primitive at v_{i+|β|}
9              (α_left, α_right) ← findExtendSymbol (βδ)
10             if α_left = λ then break
11             depth ← |β|
12             β_temp ← binarySearch (τ_i, P_s, α_left, α_right, depth)
13             if β_temp = λ then break
14             β ← β_temp
15         while β ≠ λ do
16             addEdge (β, v_i)
17             update max
18             β ← primary prefix symbol of β
19         update initial search position of v_{i+1}
20         i ← i + 1
21         if i = max then
22             path ← findShortestPath (v_min, v_max)
23             append path to S'
24             min ← max
25 P' ← P_1
26 return (P', S')
```

form the string τ_i, where i is the position of the primitive being considered. Decoding temporarily stops on line 5 whenever τ_i is at least as long as the longest symbol in the phrase hierarchy, α_L. Then, edges are added to v_i through repeated iterations of symbol extensions and binary search (lines 5 to 18). Before proceeding with vertex v_{i+1}, its initial search position is updated with the secondary prefix symbol of v_i on line 19. Next, a check is performed on line 21 to determine whether i is equal to max. If so, then the end of a segment has been found, and the shortest path through that segment is calculated. The symbols that represent this path is appended to the output sequence, $\mathcal{S}$'. After all of $\mathcal{S}_1$

has been processed, $\mathcal{P}'$ is simply a copy of $\mathcal{P}_1$, since phase 2b does not change the phrase hierarchy.

Sorting the phrase hierarchy requires $O(|\mathcal{P}'| \log_2 |\mathcal{P}'|)$ string comparisons, while the addition of the three types of symbol links is performed via a series of linear passes through the phrase hierarchy, as mentioned earlier.

The time needed for the remainder of phase 2b, as shown in Algorithm 5.2, can be calculated by dividing the algorithm into three main operations. If an assumption is made that only two cut vertices are found throughout the graph (v_1 and v_{n+1}), then these three tasks can be considered as if they exist in isolation. Finding the longest edge for all vertices requires a binary search at every vertex in the graph G. So, this runs in $O(n|\mathcal{P}'| \log_2 |\mathcal{P}'|)$ time. This analysis assumes that secondary prefix symbols and extended symbols fail to improve recompression efficiency, without factoring in the overhead cost of searching through lists of extended symbols. Adding shorter edges to a vertex once the longest one has been found requires constant time due to the primary prefix symbol links. Because of this, the second task runs in $O(n)$ time. Finally, as mentioned earlier, finding the shortest path through the segment takes $O(|V| + |E|)$ time, where $|V|$ and $|E|$ are the number of vertices and edges in the graph. Alternatively, this time can be re-written as $O(n + |E|)$. Summing all of these parts of phase 2b results in a running time whose dominant term is $O(n|\mathcal{P}'| \log_2(|\mathcal{P}'|))$.

The three extra links contribute an additional $6|\mathcal{P}'|$ words of memory to the phrase hierarchy. So that this phase runs efficiently, symbols may be expanded completely or with a recently-used buffer, as described for Des-Pair in Section 3.1 on page 48. If the entire message is treated as a single segment, then the expanded sequence requires $2n+|E|$ words of memory. Two words of memory are required for the primitive and initial search position of each vertex. Furthermore, a word of memory is required for each edge added to the graph. Since the message occupies more space than the phrase hierarchy in practice, phase 2b operates in space linear to the length of the message.

However, this analysis of the time and space of phase 2b assumes that no internal cut vertices are found, and that secondary prefix symbols and extended symbols are unhelpful.

Later, experiments demonstrate that this is not the case. Both the running time and space requirements of phase 2b have been overestimated in this analysis. While most time is still spent adding edges to the graph, the frequency with which cut vertices appear ensure that most of the space is attributed to the phrase hierarchy, and not the message. This is shown in the experiments later in this chapter.

Comparison with Related Work

Phase 2b of Re-Merge closely follows the work by Katajainen and Raita [1989] and Klein [1997]. Katajainen and Raita provides a description of recompression that is applicable to any system, regardless of how the dictionary is created. The extended trie structure of Aho and Corasick [1975] is built in order to search for the longest match. While phase 2b aims to minimise the length of the sequence, the system by Katajainen and Raita presumes that codewords are already assigned to each dictionary entry and edge. As a result, the compression ratio of the sequence is minimised rather than the sequence length. This is in contrast to Re-Merge, which follows the same design paradigm as Re-Pair of separating the model from the coder, a choice that was necessary because Re-Pair replaces pairs and builds the phrase hierarchy simultaneously. However, with the phrase hierarchy fixed in phase 2b, assigning weights to edges corresponding to codeword lengths could lead to small gains in compression effectiveness. Aspects of the implementation by Katajainen and Raita which are similar to phase 2b include the use of cut vertices, and a top-level index for primitives.

The recompression technique of Klein [1997] is more specific and is described in the context of the LZ77 family of compression algorithms. It introduces a method of pruning which reduces the number of vertices and edges maintained without any loss in recompression effectiveness. If memory limitations exists for phase 2b, then these pruning techniques can be adopted. The experiments described below do not make use of them.

Phase 2b deviates from previous work by taking advantage of the prior knowledge about the phrase hierarchy and applying binary search to it. Katajainen and Raita created a trie data structure because their algorithm was general and no presumptions could be

made about the dictionary. In our case, it is guaranteed that every symbol in the phrase hierarchy of phase 2b can be broken down into primitives, which all exist in the dictionary. The relationships between symbols permit the definitions of the three types of symbol links. Also, the phrase hierarchy is static throughout phase 2b, an assumption not available to Klein. More importantly, the RE-PAIR phrase hierarchy creates many opportunities for cut vertices during compression, and complex pruning techniques are not necessary.

The segmentation of the message through cut vertices ensures that the memory usage of phase 2b is limited by the phrase hierarchy, and not the sequence. One problem with the algorithm is whether the additional symbol links yield enough benefit for binary searching to justify them. This question is addressed later during experiments with phase 2b when a comparison with a more simplistic method is made.

5.4 Identifying New Symbol Pairs

Regardless of whether phase 2a or phase 2b is applied, the resulting sequence contains no pair of symbols with a replacement in the phrase hierarchy. But it is possible that there are pairs of symbols in the new sequence which are not recorded in the phrase hierarchy. For example, if $\alpha\beta$ exists only once in each of blocks 1 and 2, no replacement symbol would have been added to the phrase hierarchy by RE-PAIR or any other earlier phase of RE-MERGE. The purpose of phase 3 is to identify new symbol pairs that occur twice or more, and to replace them with a fresh symbol.

To achieve this purpose, phase 3 further reduces the length of the sequence. As with the second phase of RE-MERGE, a decrease in compression ratio cannot be guaranteed due to the separation of the coder from RE-PAIR. For each unique pair of symbols found which occur twice or more, an addition is made to the phrase hierarchy. Another intention of phase 3 is to improve the quality of the phrase hierarchy, by creating longer phrases.

Several methods of performing phase 3 on the input sequence $\mathcal{S}_1$ are conceivable. The first method simply applies RE-PAIR to $\mathcal{S}_1$. But even after the application of phases 1 and 2, the length of $\mathcal{S}_1$ going into phase 3 may still be too long, and impossible to process as a single RE-PAIR block. Alternatively, $\mathcal{S}_1$ can first be divided into blocks. However, it

is possible that no amount of iterations on $\mathcal{S}_1$ will reduce the sequence to a length that can ultimately be processed entirely within memory by Re-Pair. Instead of these two, a more direct approach is taken by phase 3.

As was the case in the other phases in Re-Merge, the amount of memory occupied by the chosen method for phase 3 depends on the size of the phrase hierarchy, as opposed to the length of the sequence. The phrase hierarchy is retained in memory while four complete passes are performed on $\mathcal{S}_1$. In the first pass, pairs of symbols are maintained in a probabilistic filter implemented as a bit vector, B. For every pair of symbols in the initial sequence, a set of independent hash functions are applied, identifying a set of bits in B, which are all turned on. Subsequent occurrences of the same pair are identified when every bit in the set are already on. In this case, the symbol pair is added to a hash table, H. Before completing the first pass, the bit vector's memory is released.

There is no attempt in B to resolve collisions, and each bit is either on or off. So, even with the best set of independent hash functions, it is possible for false hits to occur in B that result in a symbol pair being added to H after a single occurrence in $\mathcal{S}_1$. A second pass through $\mathcal{S}_1$ confirms which of the entries in H are valid replacements. For each entry, a counter indicates whether the corresponding pair of symbols has been seen only once, or seen more than once. Like Re-Pair, care must be taken to prevent inflating the count for overlapping pairs; a problem which is easily solved by keeping track of the previous pair of symbols at all times. Once the second pass completes, the entries in the hash table are examined so that symbol pairs that have been seen only once are culled.

The third pass replaces symbol pairs in $\mathcal{S}_1$ according to H, and adds symbols to the phrase hierarchy as each new replacement is performed. Unlike Re-Pair, replacements are performed in a strictly left-to-right manner, starting from the beginning of the sequence, regardless of pair frequency. Since most lower generation phrases have already been found by Re-Pair, the majority of the pairs located by phase 3 are expected to be infrequent, and associated with a high generation. Only a small impact on the overall quality of the phrases is expected, even though phase 3 does not pair symbols recursively. But if recursive pairing is necessary, then multiple iterations of phase 3 over the sequence can be

made, similar to the compression system RAY by Cannane and Williams [2001].

Finally, a fourth pass over the sequence is needed because of the requirements of phrase hierarchy encoding. As the new symbols may be introduced into any generation, except for the generation of primitives, the phrase hierarchy needs to be built one more time. Rebuilding requires the entire phrase hierarchy to be sorted by generation, each symbol assigned a chiastic slide value, and then each generation interpolative coded. After the phrase hierarchy is encoded, the fourth pass renumbers the sequence $\mathcal{S}_1$ to ensure that the symbols refer to the final phrase hierarchy $\mathcal{P}'$.

Phase 3 operates in time linear in the length of the initial sequence $\mathcal{S}_1$ excluding the decoding and encoding of the phrase hierarchy. To be precise, the first three passes scan a sequence of length $|\mathcal{S}_1|$, while the fourth pass scans a sequence of length $|\mathcal{S}'| < |\mathcal{S}_1|$. In addition, the second pass requires a scan over the hash table to remove pairs that were falsely identified.

The separation of phase 3 into passes reduces the amount of memory used. For example, if the bit vector B was not used, then the hash table H could occupy as much memory as there are symbols in $\mathcal{S}_1$. Instead, the memory usage of phase 3 is dominated by the size of the phrase hierarchy, which does not require symbols to be expanded or inter-linked, unlike phases 1 and 2b. The sequence is read directly from disk for each pass of phase 3 so that it is unnecessary to keep any part of it in memory.

Another factor that affects the memory usage of phase 3 has yet to be clarified. The number of entries in H is at least equal to the number of replacements made, but could be higher depending on the number of false duplicates that arise during the first pass. The chance of a false duplicate depends on the size of the bit vector B and the number of independent hash functions. Let the size of the bit vector be $|B|$ and the number of independent hash functions be k. The highest chance for a false duplicate occurs when all of $\mathcal{S}_1$ has been processed. The probability of any particular bit in B still being off after $k|\mathcal{S}_1|$ bits have been randomly turned on is $(1 - 1/|B|)^{k|\mathcal{S}_1|}$. Thus, the probability that a

	$8\|\mathcal{S}_1\|$	$16\|\mathcal{S}_1\|$	$32\|\mathcal{S}_1\|$
1	11.75%	6.06%	3.08%
2	4.89%	1.38%	0.37%
3	3.06%	0.50%	0.07%
4	2.40%	0.24%	0.02%
5	2.17%	0.14%	0.01%
6	2.16%	0.09%	0.00%
7	2.29%	0.07%	0.00%

(a)

	$8\|\mathcal{S}_1\|$	$16\|\mathcal{S}_1\|$	$32\|\mathcal{S}_1\|$
1	11.75%	6.06%	3.08%
2	4.89%	1.38%	0.37%
3	3.06%	0.50%	0.07%
4	2.40%	0.24%	0.02%
5	2.17%	0.14%	0.01%
6	2.16%	0.09%	0.00%
7	2.29%	0.07%	0.00%

(b)

Figure 5.8: Percentage calculations for the chance of obtaining a false duplicate using Equation 5.1. On the left, $|\mathcal{S}_1| = 61{,}734$ and on the right, $|\mathcal{S}_1| = 61{,}734{,}292$. The number of hash functions (on the left side of each table) varies from 1 to 7, while the size of B, measured in bits along the top, varies from 8 bits per symbol up to 32 bits per symbol.

random selection of k bits in B are all on, is given by

$$\left(1 - \left(1 - \frac{1}{|B|}\right)^{k|\mathcal{S}_1|}\right)^k \tag{5.1}$$

Equation 5.1 assumes that the first pass through $\mathcal{S}_1$ has completed. With the sparser distribution of bits turned on in B while the sequence is scanned, this equation represents an upper bound on the probability of obtaining a false duplicate overall.

Figure 5.8 substitutes various values for $|B|$, k, and $|\mathcal{S}_1|$ to provide actual probabilities, represented as percentages. The size of the bit vector depends on the length of the initial sequence. Also, because of efficiency in accessing individual bits, the various sizes of the bit vector were chosen so that they were byte-aligned. For example, in the tables, $8|\mathcal{S}_1|$ represents a bit vector with a size equal to 8 bits (or 1 byte) for each symbol in the initial sequence. The number of independent hash functions range from 1 to 7.

Percentage calculations with two sequence lengths are shown in Figure 5.8, with one being 1,000 times longer than the other. The sequence length of the table on the right is 61,734,292 symbols, a number taken from Table 5.6, later in this chapter. The negligible difference between the two tables show that Equation 5.1 is relatively insensitive to large changes in the sequence length, because of the way that $|B|$ is chosen as a function of $|\mathcal{S}_1|$.

As expected, as B increases in size relative to $|\mathcal{S}_1|$, the chance of a false duplicate decreases. Moreover, as the first column in each table of Figure 5.8 show, as the number

of hash functions increase, the probabilities initially decrease, but ultimately increase again.

Based on these calculations, phase 3 has been implemented with three hash functions and a bit vector of size $8|\mathcal{S}_1|$ bits, giving a false duplicate percentage of roughly 3.1%. This means that at the end of the scanning of the sequence $\mathcal{S}_1$, the chance that a single additional pair of symbols would yield a false positive is about 3.1%. During the processing of $\mathcal{S}_1$, the chance of a false positive is less. The number of slots in the hash table H can be either fixed, or chosen based on the length of $\mathcal{S}_1$.

5.5 Changes for Punctuation-Aligned Re-Pair

In the previous sections, the three phases of Re-Merge have been discussed in the context of character-based Re-Pair. The reason for this presentation was to assist in the understanding of the examples throughout the discussion. However, as described in Chapter 4, word-based or punctuation-aligned Re-Pair are preferred for the purpose of phrase browsing. No changes to Re-Merge are required for word-based Re-Pair. However, punctuation-aligned Re-Pair makes use of punctuation flags, which have to be handled appropriately.

Recall from the previous chapter that the main difference between character-based Re-Pair and word-based Re-Pair, is that the set of primitives is larger. And the difference between word-based Re-Pair and punctuation-aligned Re-Pair is that symbols within the word sequence have been flagged. A flagged symbol indicates that, in the message, the word token is followed by one of the punctuation marks identified by punctuation-aligned Re-Pair.

While no significant changes are necessary for phase 1, minor changes are require for the remaining phases in order to accommodate punctuation flags. The rules governing punctuation-aligned Re-Pair are presented in Table 4.8 on page 93. Phase 2a should stop the recursive pairing of symbols if any of these rules have been violated. A rule violation occurs if the left component is flagged, but the right component is not. As with Re-Pair, this modification ensures that non-flagged tokens are paired with flagged tokens

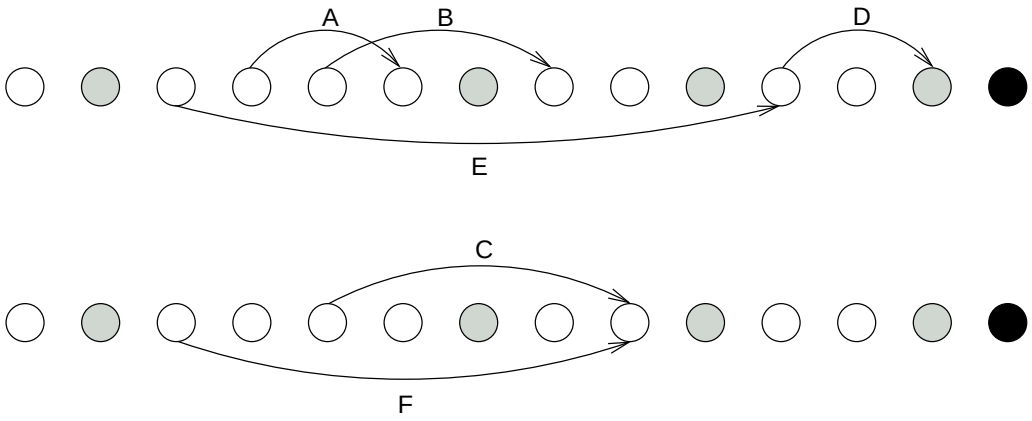
only if the flagged token is the right component. Likewise, recursive pairing should also cease if both components are flagged, but the symbol before the left component is not.

When punctuation-aligned Re-Pair is combined with phase 2b of Re-Merge, primitives which have been flagged need to be treated differently, as shown in Figure 5.9. In particular, certain edges in the directed graph constructed in phase 2b are disallowed, depending on the location of the flagged primitives. Flagged primitives are represented as *minor cut vertices* in G. Whether edges are allowed depends on the table of Figure 5.9(c). Figure 5.9(a) and Figure 5.9(b) together constitute an example for phase 2b with punctuation-aligned Re-Pair.

Of the six rules of Figure 5.9(c), the two rules preceded with an asterisk (*) indicate cases when an edge is not allowed. The remaining four rules indicate situations when an edge is permitted. Which rule is applied depends on three conditions: the vertex type before the tail of the edge, the vertex type before the head of the edge, and whether a minor cut vertex has been bypassed by an edge. Note that for the two cases where edges are disallowed, the edge must have passed over a cut vertex. So an edge of length 1 that leads to the immediately adjacent vertex is never deleted, and G is still guaranteed to be connected. The end result after applying these rules is similar to the constraints as described above for phase 2a.

An example assists in justifying these conditions. Figure 5.9(a) presents a message derived from the "Woodchuck" message used as an example in earlier chapters, with four punctuation marks added. Figure 5.9(b) contains two graphs. The top graph illustrates edges which correspond to the four rules which permit edges, while the bottom graph shows the two cases when edges are not permitted. The vertices, each representing a word token, have been coloured so that the four lightly shaded vertices are flagged primitives, and the darkly shaded vertices are sink nodes. Edges are labelled with letters, indicating rules from Figure 5.9(c). The primitives traversed by the edge are listed in the last column of Figure 5.9(c).

As the example shows, any sequence of primitives between minor cut vertices is permissible (edges A and D). If punctuation marks are seen as marking off English phrases,



(a) Sample message

(b) Graphs with valid and invalid edges

	Graph edge	Vertex before tail	Vertex before head	Pass a minor cut vertex?	Example
	A	W	W	No	a woodchuck
	B	W	P	Only once	woodchuck would chuck,
*	C	W	..... Otherwise		woodchuck would chuck, if
	D	P	W	No	could chuck
	E	P	P	Yes	as a woodchuck would chuck, if a woodchuck,
*	F	P	W	Yes	as a woodchuck would chuck, if a

(c) All possible combinations of edges

Figure 5.9: The rules that dictate the types of edges that can be removed by phase 2b of Re-Merge for punctuation-aligned Re-Pair. Of the six rules at the bottom, only the two that are preceded by an asterisk (*) disallow edges. Each rule is illustrated through one of two example graphs (b) which correspond to the sample message at the top (a).

then permitted edges can be viewed as edges which do not break a phrase. For edge B, three primitives are combined, with only the final primitive immediately followed by a punctuation mark. Two phrases are combined by edge E, and neither of them break any surrounding phrases.

The changes to the third phase of Re-Merge resembles that of phase 2a. Only pairs permitted by phase 2a are permitted by phase 3. These rules are applied during all passes

	WSJ508			NEWS		
	compression ratio (bpc)	encoding time (s)	decoding time (s)	compression ratio (bpc)	encoding time (s)	decoding time (s)
GZIP	2.941	170.0	20.4	2.963	323.3	40.3
BZIP2	2.105	447.3	155.8	2.145	871.5	304.3
PPMD	1.637	800.9	821.5	1.674	1,600.8	1,641.0

Table 5.5: Compression effectiveness and execution times for the compression of WSJ508 and NEWS using GZIP, BZIP2, and PPMD. These results are taken directly from Table 2.5 (page 41) and Table 2.6 (page 41).

of phase 3, not just the replacement pass.

5.6 Experiments

Experiments were conducted with two documents which are larger than the 20 MiB considered in previous chapters. The first document is WSJ508, also used earlier in this chapter. The second document is NEWS, already mentioned in Chapter 2. The NEWS collection consists of 1,000 MiB of news articles from the Wall Street Journal (WSJ) and the Associated Press (AP) with SGML markup. The first 508 MiB of the NEWS collection is WSJ508.

The test machine for these experiments is again a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache, with all execution times averaged over three trials run on an otherwise idle machine. Table 5.5 presents the results of using the GZIP, BZIP2, and PPMD systems on these two documents, running on the same test machine. These results are from Table 2.5 (page 41) and Table 2.6 (page 41). The -9 option was specified for GZIP and BZIP2, while PPMD was tested using 255 MiB of memory and a seventh order model.

The experiments in this section are first reported for character-based Re-Pair for WSJ508, and then for punctuation-aligned Re-Pair for NEWS. In both cases, the block size of Re-Pair is 20,971,520 symbols, in order to provide some consistency with previous experiments.

As shown in Figure 5.2 on page 114, five combinations of the three phases of Re-Merge are possible, labelled method A to E. As a consequence of the choices made during

the implementation of Re-Merge, the phrase hierarchy is encoded at the conclusion of every Re-Merge phase, and decoded at the beginning. However, the encoding of the modifiers and the reduced word sequence is applied after all necessary block merging phases have been completed. In these experiments, entropy coding is handled by shuff, as was also the case in previous chapters. Each stream is processed by shuff as a single block, except for the reduced sequence created by Re-Pair which contains b blocks. In this scenario, each of the b blocks are encoded by shuff individually.

Some statistics about the phrase hierarchy and reduced sequence after Re-Pair and the five methods of Re-Merge are shown in Table 5.6. Most notable are changes between columns of the table. From Re-Pair to method A (Re-Merge phase 1 only), the size of the phrase hierarchy decreases dramatically as the blocks are merged. The maximum symbol generation in a block (gen) drops from 31 to 28 as duplicates are removed and the symbol generations collapse. Methods B and C, which end with Re-Merge phases 2a and 2b respectively, make no changes to the phrase hierarchy, but the sequence lengths are shortened. As expected, since phase 2b is tailored for effectiveness in reducing the length of the sequence, method C achieves a slightly shorter sequence than method B. Likewise, when these two methods are followed by phase 3 (method D for phase 2a and method E for phase 2b), method E is able to produce a shorter sequence with less phrases. With `WSJ508` and character-based Re-Pair, Re-Merge offers no changes to the maximum symbol generation in a block, nor the length of the longest symbol ($|\alpha_L|$). Not shown in the table, the average distance between cut vertices for phase 2b was 66.4 primitives.

Compression ratios for character-based Re-Pair and the five combinations of Re-Merge are reported in Table 5.7. The baseline values for character-based Re-Pair are in line with those presented in Table 5.6. Between the column "None" and the column marked method A, a drop in the size of the phrase hierarchy occurs. Methods B and C improve compression, without changing the cost of the phrase hierarchy. The compression ratios of methods B and C improve again after phase 3, producing the results in columns D and E, respectively. In these last two columns, the overall compression effectiveness again improves, but at the cost of a larger phrase hierarchy.

	None	Re-Merge Method				
		A	B	C	D	E
$\|\Sigma\|$	2,326	94	94	94	94	94
$\|\rho\|$	7,946,135	2,790,238	2,790,238	2,790,238	4,817,719	4,164,166
$\|\mathcal{S}'\|$	49,515,068	49,515,043	44,676,862	42,179,580	39,776,086	38,955,830
gen	31	28	28	28	28	28
$\|\alpha_L\|$	4,970	4,970	4,970	4,970	4,970	4,970
b	26	1	1	1	1	1

Table 5.6: Statistics for Re-Merge with character-based Re-Pair on the `WSJ508` file. The column marked "None" represents no block merging performed, and Σ, ρ, and $\mathcal{S}'$ represent the primitives, phrases, and final sequence, respectively. The row marked "gen" is the maximum symbol generation in any block, while $|\alpha_L|$ is the length of the longest phrase, expressed in primitives. The last row, b, is the number of blocks.

	None	Re-Merge Method				
		A	B	C	D	E
Phrase hierarchy	0.223	0.088	0.088	0.088	0.151	0.136
Sequence (shuff)	1.583	1.643	1.565	1.546	1.494	1.482
Total	1.806	1.731	1.653	1.635	1.645	1.618

Table 5.7: Compression ratios for character-based Re-Pair and Re-Merge for `WSJ508`, expressed in bits per character relative to the original size of the file.

The results for method E have been expanded into a graph as Figure 5.10, clearly separating the five iterations of phase 1 of Re-Merge, and then the gain accrued from phase 2b and phase 3. Each iteration of phase 1 gradually decreases the size of the phrase hierarchy.

The compression, decompression, and block merging times for character-based Re-Pair with `WSJ508` are shown in Table 5.8. The time for phase 1 includes all 5 iterations necessary to transform the 26 blocks into a single block. Of interest is the fact that phase 2b takes 10 times longer than phase 2a. However, when phase 3 follows either (methods D and E), the time required is similar, regardless of which version of phase 2 has been chosen. In comparison to Re-Pair and Re-Merge, the time taken for sequence coding with shuff is small. As for decoding, note that regardless of which type of encoding has been chosen, the decoding requirements is small and always between 70 and 80 seconds. While phase 2b requires an average of 3,276.5 seconds, a simpler approach that eliminated secondary prefix symbols and extended symbols required an average of 5,994.5 seconds.

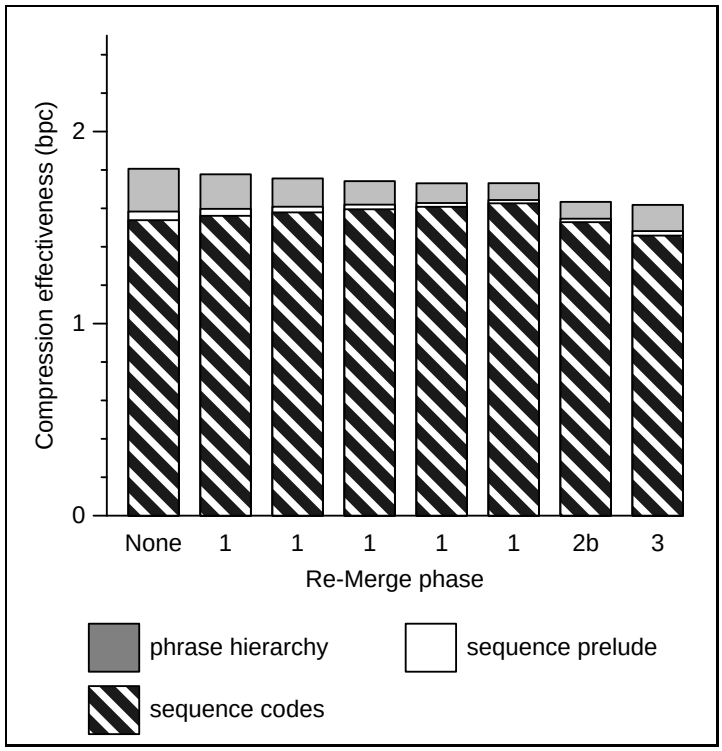


Figure 5.10: Compression ratios of character-based Re-Pair and method E of Re-Merge for `WSJ508`. Each vertical bar represents the initial compression with Re-Pair, or a single application of Re-Merge. Initially, there are 26 blocks, which requires phase 1 five passes to combine into a single block. Then, phase 2b is applied, followed by phase 3.

This less principled approach retained an index to the primitives and primary prefix symbols, though.

The results from applying punctuation-aligned Re-Pair to `NEWS` are similar to those of character-based Re-Pair to `WSJ508`. The first step in punctuation-aligned Re-Pair is the pre-processing stage with Pre-Pair. When a Re-Merge configuration has been chosen, the only difference between it and any other combination is the execution times and compression ratios of the word sequence. So, values related to the lexicons and the modifiers are constant. These fixed values are shown in Table 5.9 and re-appear in future tables in this chapter under the heading of "Other".

Some phrase hierarchy and sequence statistics pertaining to punctuation-aligned Re-Pair for `NEWS` are shown in Table 5.10. Re-Pair creates nine blocks from the 168,721,097 symbols of the word sequence. As with character-based Re-Pair, between the column

	None	Re-Merge Method				
		A	B	C	D	E
Encoding time						
Re-Pair	2,323.3	2,323.3	2,323.3	2,323.3	2,323.3	2,323.3
Re-Merge 1		453.0	453.0	453.0	453.0	453.0
Re-Merge 2a			394.4		394.4	
Re-Merge 2b				3,276.5		3,276.5
Re-Merge 3					222.2	202.5
shuff	39.2	32.1	31.0	30.6	34.3	32.7
Total	2,362.5	2,808.4	3,201.7	6,083.4	3,427.2	6,288.0
Decoding time						
shuff	9.3	8.1	8.3	8.9	9.3	9.2
Des-Pair	62.9	61.9	63.3	65.8	68.2	68.9
Total	72.2	70.0	71.6	74.7	77.5	78.1

Table 5.8: Compression times for Re-Merge with character-based Re-Pair on `WSJ508`. In the first column, "None" represents no block merging performed, using the output from Re-Pair directly. Times are in CPU seconds on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache, and averaged over three trials.

	Compression (bpc)	Encoding time (s)	Decoding time (s)
Pre-Pair	N/A	724.9	142.5
Streams:			
Word lexicon	0.005	1.2	0.7
Non-word lexicon	<0.001	<0.1	<0.1
Case-folding modifiers	0.240	16.2	7.1
Stemming modifiers	0.377	18.3	7.5
Non-word modifiers	0.342	16.6	7.3
Total	0.964	777.2	165.1

Table 5.9: Compression ratios and execution times for the streams of Pre-Pair, not including the word sequence stream, when applied to `NEWS`. The lexicons have been front-coded and then compressed using bzip2. The two streams of modifiers and the non-word sequence have been compressed with shuff, using a block size of 20,971,520 symbols. Execution times have been averaged over three trials.

"None" and method A, the phrase hierarchy's size decreases. Methods B and C both reduce the length of the sequence, with method C being more effective. Likewise, method E is an improvement to method D in both the final sequence length and the number of phrases required to achieve that length. With `NEWS`, there are no changes to the maximum

	None	Re-Merge Method				
		A	B	C	D	E
$\|\Sigma\|$	848,016	336,739	336,739	336,739	336,739	336,739
$\|\rho\|$	9,916,387	6,554,510	6,554,510	6,554,510	8,275,000	7,940,620
$\|\mathcal{S}'\|$	67,097,856	67,097,848	63,709,684	61,734,292	59,932,869	58,685,281
gen	25	25	25	25	25	25
$\|\alpha_L\|$	1,348	1,348	1,348	1,348	1,348	1,348
b	9	1	1	1	1	1

Table 5.10: Statistics for Re-Merge with punctuation-aligned Re-Pair. The column marked "None" represents no block merging performed. Primitives, phrases, and the final sequence are abbreviated as Σ, ρ, and $\mathcal{S}'$. The row marked "gen" is the maximum symbol generation in any block, while $|\alpha_L|$ is the length of the longest symbol, expressed in primitives. The last row, b, is the number of blocks.

	None	Re-Merge Method				
		A	B	C	D	E
Phrase hierarchy	0.163	0.116	0.116	0.116	0.143	0.138
Sequence (shuff)	1.145	1.181	1.159	1.153	1.137	1.131
Other	0.964	0.964	0.964	0.964	0.964	0.964
Total	2.272	2.261	2.239	2.233	2.244	2.233

Table 5.11: Compression ratios for Re-Merge with punctuation-aligned Re-Pair. In the second column, "None" represents no block merging performed, using the output from Re-Pair directly. In the sixth row, "Other" represents the compression ratio of the other streams, as shown in Table 5.9.

generation in a block and the length of the longest symbol. The average length of a segment during phase 2b was 5.5 primitives, much less than for character-based Re-Pair.

The compression ratios of Table 5.11 reflect the levels for character-based Re-Pair, with one notable exception. While phase 3 has reduced the length of the sequence, the compression ratios of methods D and E are worse than those of methods B and C. To be precise, the compression ratios of the sequence for methods D and E have improved, but not enough to compensate for the increase in size of the phrase hierarchy.

The execution times for punctuation-aligned Re-Pair and Re-Merge are presented in Table 5.12. For the total times, the costs of encoding and decoding the modifiers have been taken into account in the rows marked "Other". The execution time difference between methods are similar to the differences between methods for character-based Re-Pair. And, as before, the decoding times are fairly consistent, whether Re-Pair alone

	None	Re-Merge Method				
		A	B	C	D	E
Encoding time						
Other	777.2	777.2	777.2	777.2	777.2	777.2
Re-Pair	1,634.3	1,634.3	1,634.3	1,634.3	1,634.3	1,634.3
Re-Merge 1		636.7	636.7	636.7	636.7	636.7
Re-Merge 2a			1,142.2		1,142.2	
Re-Merge 2b				2,645.4		2,645.4
Re-Merge 3					312.1	288.8
shuff	55.0	51.4	50.5	50.0	53.4	52.3
Total	2,466.5	3,099.6	4,240.9	5,743.6	4,555.9	6,034.7
Decoding time						
shuff	13.3	12.3	12.4	12.9	13.7	13.5
Des-Pair	52.8	53.3	54.4	55.5	57.7	58.3
Other	165.1	165.1	165.1	165.1	165.1	165.1
Total	231.2	230.7	231.9	233.5	236.5	236.9

Table 5.12: Compression times for Re-Merge with punctuation-aligned Re-Pair for the file `NEWS`. In the first column, "None" represents no block merging performed, using the output from Re-Pair directly. The rows labelled "Other" represent the total compression times of the other streams, as shown in Table 5.9. Times are in CPU seconds on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache.

has been applied, or followed by a Re-Merge method.

The elementary approach to phase 2b was also applied, and was actually found to be marginally faster. Without secondary prefix symbols and extended symbols, 2,569.7 seconds was taken by Re-Merge, as opposed to 2,640.9 seconds. The time to construct these symbol links for phase 2b was 38.7 seconds, on average. There was no gain in time by adding these links partly due to the generally shorter symbols than character-based Re-Pair. The longest symbol for punctuation-aligned Re-Pair had 1,348 word tokens, while the longest symbol for character-based Re-Pair consisted of 4,970 characters. In comparison, the extra symbol linking time for character-based Re-Pair was 9.2 seconds, which was easily justifiable due to the improvements in speed. While the phrase hierarchy produced by punctuation-aligned Re-Pair was larger, since the primitives represent words, there was less similarity between consecutive symbols in the phrase hierarchy sorted by expanded strings. So, if memory constraints exists, then choosing the simpler version

of phase 2b would save $6(|\Sigma| + |\rho|)$ words of memory for punctuation-aligned Re-Pair

5.7 Appending to a Compressed Document

The success of Re-Merge as a post-processor for Re-Pair is due to its memory usage. While Re-Pair requires enough space to hold the entire message, each phase of Re-Merge occupies memory which is proportional to the size of the phrase hierarchy. In the experiments described above, `WSJ508` and `NEWS` could be processed by Re-Merge on a test machine with 1 GiB of memory.

Problems with Re-Merge arise if the phrase hierarchy of the reduced message is larger than the amount of available memory. This section proposes a solution which involves the creation of a phrase hierarchy from a subset of the message to act as training data. The size of the subset must be small enough so that when it is post-processed with Re-Merge, the phrase hierarchy can be kept within memory. This phrase hierarchy is then used to compress the remainder of the message through a technique similar to phase 2b. Hence, the time and space requirements for appending text is same as for phase 2b.

Appending text to a compressed document presents two problems. First, the phrases created with the training data may not be representative of the rest of the message. As more text is processed with the static phrase hierarchy, compression is expected to degrade. Second, Re-Merge requires the set of primitives to be known ahead of time. Regardless of how much of a message has been seen, the appearance of a novel primitive can always occur. The structure of the phrase hierarchy and the method with which it is encoded prevents symbols to be inserted in the middle. Whenever a new symbol is added to the phrase hierarchy, the structure must be re-built and the symbols in the sequence renumbered, as with phase 3 of Re-Merge.

The experiments described below respond to the first problem by demonstrating how the size of the training data affects overall compression effectiveness. Appending text with Re-Merge was implemented specifically for character-based Re-Pair. As a result, the set of primitives is limited to the 256 characters of the Latin-1 character set. After

the experiments, a proposed solution for word-based and punctuation-aligned Re-Pair is discussed.

Appending text with Re-Merge is similar to the semi-static version of Ray called XRay [Cannane and Williams, 2002]. The XRay algorithm creates a dictionary using a portion of the message as training data. Then, this training data is applied on-line to the remainder of the message. After the training phase, Huffman codes are assigned to the dictionary entries, so that phrase selection afterwards is based on the lengths of the corresponding codewords. A window of text is examined at a time, and phrases are chosen based on the assignment of a penalty for bypassing a replacement. In contrast to XRay, Re-Merge continues to separate the phrase selection heuristic from the entropy coder for the reduced word sequence, so that phrases are selected based on how much the sequence length can be reduced. As with phase 2b, each primitive from the second part of the message is placed in a node in a DAG, and the shortest path through is determined by following edges representing phrase hierarchy symbols.

In order to measure the performance of appending text, character-based Re-Pair was used to compress `WSJ508`. Various sizes of training data were taken from the front of `WSJ508` and partitioned into blocks of 20 MiB each. These blocks were merged using method E of Re-Merge. That is, all of the blocks were combined through multiple iterations of phase 1, followed by phase 2b, and then phase 3. The remainder of `WSJ508` is then appended to the end of the reduced training data. An alternative experimental framework would obtain the training data from random sections within `WSJ508`. While the current implementation of Re-Merge would support this approach, it was not used because the articles within `WSJ508` are ordered chronologically. Appending text gives the illusion that new documents are being added to the collection over time.

Table 5.13 provides statistics from these experiments after all of `WSJ508` has been processed. For each training size ranging from 20 MiB to 400 MiB, the number of primitives, the number of phrases, the length of the final sequence, the length of the longest phrase in primitives, and the maximum generation are presented. The last row of the table is obtained from earlier experiments with character-based Re-Pair and `WSJ508` (see the col-

Training size (MiB)	$\|\Sigma\|$	$\|\rho\|$	$\|\mathcal{S}'\|$	$\|\alpha_L\|$	gen
20	256	322,432	71,240,604	1,257	21
40	256	552,784	65,724,383	1,257	22
80	256	948,903	60,802,497	2,084	21
120	256	1,302,459	58,221,318	4,970	21
160	256	1,647,723	55,121,237	4,970	23
240	256	2,278,342	52,039,645	4,970	24
320	256	2,905,923	44,730,578	4,970	27
400	256	3,467,728	40,738,832	4,970	28
508	94	4,164,166	38,955,830	4,970	28

Table 5.13: Statistics for appending text with character-based RE-PAIR on `WSJ508`. The first column indicates the size of the training data. Then, Σ, ρ, and $\mathcal{S}'$ represent the primitives, phrases, and final sequence, respectively. The column marked $|\alpha_L|$ is the length of the longest phrase in primitives, and "gen" is the maximum symbol generation.

umn marked "method E" in Table 5.6). Those experiments are equivalent to making all of `WSJ508` the training data.

As Table 5.13 shows, as more training data is used, the number of phrases, the maximum phrase length, and the maximum generation increase. Also, since the phrase hierarchy contains more symbols, the length of the sequence becomes shorter. Note that the number of primitives is always fixed at 256. In the last row, when all of the message is seen, the space allocated to generation 0 is equal to the number of distinct primitives that appear in the message.

Figure 5.11 presents the compression levels achieved as a function of the size of the training data. The compressed sizes are calculated from the merged training data, and again once the rest of `WSJ508` has been appended. In both cases, the reduced sequence was encoded with SHUFF as a single block. As expected, overall compression effectiveness improves as the training set increases in size. The best compression ratio on the graph is achieved when the initial training size is equal to the size of `WSJ508`. However, with just 16% of `WSJ508` used as training data, the compression effectiveness achieved is only 18% worse than applying RE-PAIR and RE-MERGE on the entire file.

The time required to compress `WSJ508` is presented in Table 5.14. As the size of the training data increases, the time required to apply both RE-PAIR and method E of

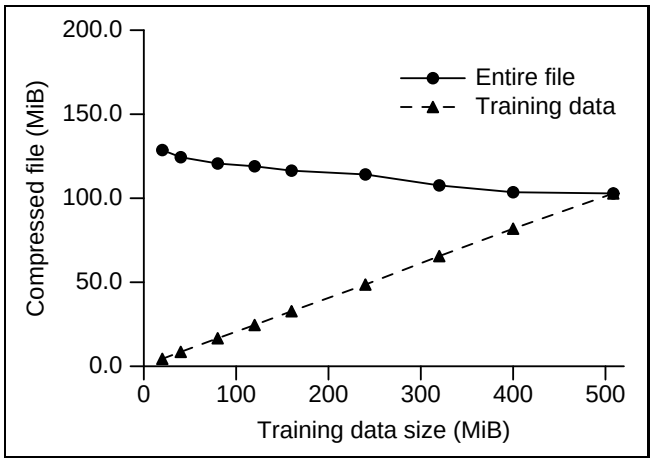


Figure 5.11: Compression effectiveness of Re-Merge when appending text to various sizes of training data using character-based Re-Pair and the `WSJ508` test file. The initial block size for Re-Pair is 20 MiB. The solid line represents the compression effectiveness of the entire file, while the dashed line indicates the size of the compressed training data.

Re-Merge also increases. While Re-Merge is not necessary for the training size of 20 MiB, it was applied in order to remove any duplicates in the phrase hierarchy. The time required to append the remaining text does not increase in a similar manner, though. After a certain training size, the cost of appending is offset by the amount of text left to add. The time required to append text peaks when the training size is 80 MiB. The time taken to encode the reduced sequence with shuff is similar throughout the experiment. Regardless of the costs of appending and sequence encoding, the dominant times belong to Re-Pair and method E of Re-Merge. Because of this, the total time increases with the training size.

The decompression times for appending text are given in Table 5.15. The decoding time of shuff and the phrase expansion time of Des-Pair both increase as the training data becomes larger. This increase is due to the larger phrase hierarchy, despite the sequence becoming shorter.

The addition of text to a compressed message serves two purposes. The first, as mentioned already, is to enable documents which yield a large phrase hierarchy to be processed. The second purpose is to support dynamic collections, where new documents are added to the repository throughout its lifetime. The coding mechanism provided

Training size (MiB)	Re-Pair	Re-Merge (method E)	Append text	shuff	Total
20	88.7	114.6	2,662.3	31.6	2,897.2
40	180.0	186.3	2,937.7	31.3	3,335.3
80	363.0	428.1	3,165.7	31.5	3,988.3
120	546.4	708.5	3,126.0	31.6	4,412.5
160	732.3	981.4	2,986.1	31.8	4,731.6
240	1,102.5	1,610.9	2,457.6	32.0	5,203.0
320	1,467.2	2,246.2	1,891.2	31.6	5,636.2
400	1,834.2	2,954.3	1,200.5	31.6	6,020.6
508	2,323.3	3,932.0	—	32.7	6,288.0

Table 5.14: Encoding times for character-based Re-Pair and appending with Re-Merge for various sizes of training data. Times are in CPU seconds and averaged over three trials on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache.

Training size (MiB)	shuff	Des-Pair	Total
20	7.0	56.0	63.0
40	7.3	57.4	64.7
80	7.7	59.7	67.4
120	7.8	61.3	69.1
160	8.1	62.2	70.3
240	8.3	62.9	71.2
320	8.6	64.4	73.0
400	8.7	65.8	74.5
508	9.2	68.2	78.1

Table 5.15: Times for decoding a compressed message made with character-based Re-Pair with various sizes of training data. Times are in CPU seconds and averaged over three trials on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache.

by shuff does not allow periodic additions to the compressed sequence, and the solution adopted by the experiments is to re-encode the sequence after any changes. An alternative is to apply shuff in blocks, and make each extension a block. Otherwise, support of both aims for character-based Re-Pair is complete.

The problem becomes more complicated for word-based and punctuation-aligned Re-Pair, since no restriction is placed on the number of unique words. When a novel word appears, it must be added to the word lexicon. However, since the positions in the lexically sorted word lexicon are used to create the word sequence for Re-Pair, both streams must

be recompressed. A similar problem occurs when a novel non-word symbol appears in the message.

Moffat et al. [1997] discusses several solutions for compressing dynamic document databases, two of which can be adapted for Re-Pair. The first idea adds an escape symbol which signals to the decoder that the following is a novel word, specified using a secondary model, such as characters. While this idea permits dynamic document compression, the loss of the word boundaries enforced by Pre-Pair make it unsuitable for browsing. The second method proposed by Moffat et al. would also introduce an escape primitive to the phrase hierarchy, except that the escape symbol forces the decoder to consult an auxiliary word lexicon. This secondary word lexicon is unsorted, and no limit is placed on its size. However, because of the compact nature of Re-Pair's phrase hierarchy, these words cannot be used to form phrases. One way of circumventing this problem is to extend the escape mechanism to phrases. Whenever an escape is encountered while expanding the reduced word sequence, the auxiliary phrase hierarchy is consulted instead.

The ideal method of compression for the unstructured auxiliary phrase hierarchy is undetermined at this time and appending of text with word-based or punctuation-aligned Re-Pair requires further investigation.

5.8 Summary

This chapter has described Re-Merge as a post-processing step to Re-Pair. When the source message is too large, it must be compressed as independent blocks by Re-Pair and then merged with Re-Merge. Re-Merge is feasible because each phase requires memory proportional to the size of the phrase hierarchy. This is in contrast to Re-Pair, whose memory usage is related to the length of the message. Even so, a message could be so large that its corresponding phrase hierarchy exceeds the amount of memory available. This problem was considered for character-based Re-Pair. Adapting this solution for word-based and punctuation-aligned Re-Pair is reserved for future work.

The block merging techniques of this chapter serve three major goals. The first goal is to enable phrase browsing by combining multiple phrase hierarchies into a single one. The

second goal is to improve compression effectiveness through the removal of duplicates in the phrase hierarchy and the identification of old and new symbol pairs for replacement. In satisfying the second goal, a third objective emerges, since the quality of the symbols in the phrase hierarchy is improved as more symbol pairs are replaced.

The experiments in this chapter have shown that to achieve these goals, no single best combination of Re-Merge phases exists. The three phases of Re-Merge have been organised into five methods which illustrate the trade-offs between program execution time and compression effectiveness. All five methods satisfy the first goal by outputting a unified phrase hierarchy. However, achieving the second and third targets requires considerable additional computation time. From the perspective of phrase browsing, a reduced message that has been post-processed by any of the five methods of Re-Merge can be browsed. Which method should be chosen depends on the amount of time available, and on the compression effectiveness and phrase hierarchy quality that is required.

The results from the experiments with punctuation-aligned Re-Pair and `NEWS` have been summarised in Figure 5.12. The graph plots the compression effectiveness of Re-Pair and the five methods of Re-Merge against their throughput (number of bytes processed per second).

The emphasis in this thesis is on phrase browsing, and for the remainder of this investigation method E for punctuation-aligned Re-Pair is the preferred approach. While this method is expensive with respect to compression speed, decompression time is unaffected by this decision.

Chapters 3, 4, and 5 have not considered sequence and modifier coding. The coding of these two types of streams is addressed in Chapter 6, by first considering the problems inherent in the method chosen so far (Huffman coding with shuff) and then by proposing two alternative coding schemes.

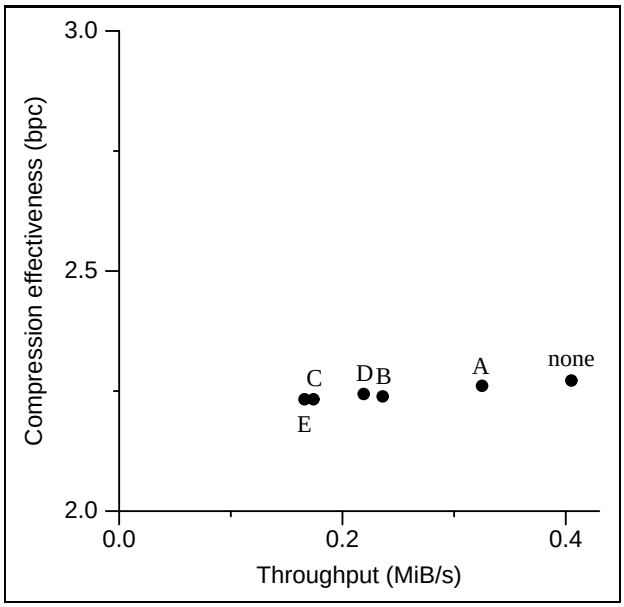


Figure 5.12: Graph of compression effectiveness versus throughput for the previous section's experiments with punctuation-aligned Re-Pair on the `NEWS` file for the five methods of Re-Merge. Times are for encoding, and are measured on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache.

Chapter 6

Coding Considerations

Chapter 2 explained how compression systems can be thought of as three separate components: modelling, probability estimation, and coding (see Figure 2.1, page 17). In the chapter that followed, the Re-Pair algorithm was demonstrated as a method of reducing the length of a message through the recursive replacement of character digrams. Later, pre-processing and post-processing stages were introduced which, when working with Re-Pair, provided a combination of tools for parsing a document into words, reducing the length of the sequence of word tokens, and finally merging blocks so that larger messages can be handled. The compression aspects from Chapter 3 to Chapter 5 have placed emphasis on the modelling and probability estimation components of a compression system.

A relatively small amount of attention was given to the third component: coding. The goal of this chapter is to rectify that omission by considering several methods of coding, and concluding with a selection of coders which follows the theme of this thesis. At the end of this chapter, a coding scheme which provides a suitable balance between compression and browsing for punctuation-aligned Re-Pair is described.

The chapter is structured as follows. Section 6.1 summarises the thesis so far with respect to coding. Section 6.2 and 6.3 propose two coding mechanisms. The first builds on the Huffman coder that has already been used throughout this thesis, while the second relaxes the requirements of entropy coding altogether in order to achieve more efficient phrase browsing. The effectiveness and efficiency of these two coding mechanisms are described in Section 6.4. Section 6.5 concludes this chapter with an overview of the

advantages and disadvantages of the coding methods that have been considered.

6.1 Coding for Compression and Phrase Browsing

The coding techniques employed in previous chapters have been biased towards compression in that they favour compression effectiveness over other more pragmatic factors. This chapter introduces alternatives which attempt to compromise between compression and the special needs of phrase browsing. Before examining these alternatives, a brief review of the previous chapters with respect to coding is appropriate.

The coding component of a compression system encodes the output of the probability estimator into a bit stream for transmission. Improved compression effectiveness is achieved when the number of bits required is reduced. The least number of bits possible for losslessly encoding the message is defined by the self-information (or self-entropy) of the message, as covered in Chapter 2. An approach which attempts to achieve a compression ratio close to the self-information is an entropy coder.

While there are several factors to consider when deciding on a coder for a compression system, the use of an entropy coder implies that some importance has been given to compression effectiveness. The need to permit phrase browsing in the compressed message shifts the importance to decoding time and searchability. In any interactive system, a timely response to the user is essential. Sacrificing a controlled amount of compression effectiveness might be justified, if a shorter response time can be assured.

In the case of punctuation-aligned (or word-based) Re-Pair, seven streams are produced by Pre-Pair, Re-Pair, and Re-Merge. These streams serve different purposes, but they can be divided into two broad categories based on their coding requirements for phrase browsing. There are the data streams that need to be decoded completely in order to be useful, and there are those that need to be only partially decoded if only a fraction of the original message is to be presented.

The Phrase Hierarchy and Lexicons

The first category of streams includes the word lexicon, the phrase hierarchy, and the non-word lexicon. The two most important streams for phrase browsing are the word lexicon and the phrase hierarchy. The word lexicon lists every unique word token found in the document, while the phrase hierarchy contains information about the relationships formed between the word tokens as they form longer phrases. Separate from these two streams is the non-word lexicon, which is not required for phrase browsing. However, the commonality that exists between all three of these streams is that they are encoded so that compression effectiveness is favoured. As a result, the compression mechanisms applied require that they must be completely decoded in order to be useful.

The two lexicons are encoded with front-coding, and can then be processed by a standard compression tool such as BZIP2. The two-dimensional phrase hierarchy is mapped to a single dimension via the chiastic slide before being encoded with interpolative coding. These two methods of encoding are biased towards compression effectiveness. But the strings involved have to be fully decoded to support phrase browsing, so this approach is acceptable. When these compression methods are applied, these three streams were small in comparison to the remaining four. For example, the 1,000 MiB of `NEWS` was processed with punctuation-aligned Re-Pair and method E of Re-Merge in Section 5.6, these three streams represent 6.4% of the total compressed message, or 0.144 bits per character with respect to the original message (see Table 5.9 on page 147 and Table 5.11 on page 148).

The Modifiers and Reduced Sequence

The other four streams possess a much simpler structure than the phrase hierarchy or the lexicons. Each stream is composed of symbols which can be arbitrarily decoded without examining neighbouring symbols. A symbol is represented as a number, which refers to an entry in the non-word lexicon, a set of instructions to reverse stemming or case folding, or a symbol in the phrase hierarchy.

Strictly speaking, in each stream, a number (or reference) does not appear independently of its neighbours. References in the reduced word sequence do not appear at random with equal probability because after RE-PAIR (and RE-MERGE), no pair of adjacent symbols appear twice or more in the entire sequence. One could also argue that the appearance of certain combinations of case folding modifiers precludes the same or some other type immediately after. For example, if the first letter of a word token is in upper case, it is probably signalling the beginning of a new sentence, and one would expect the previous non-word to include a period, and the next few words to be entirely in lower case. When the modifier streams were first discussed in Chapter 4, results from experiments with higher-order models were shown (Table 4.10 on page 97). However, in the interest of obtaining a modifier from any position in the stream without decoding everything before it, a zero-order model was advocated in Chapter 4. Because of this, coding was handled by the Huffman coder, SHUFF.

Two entropy coding schemes for a sequence of references were discussed in Chapter 2. They were arithmetic coding and Huffman coding, with available implementations being UINT and SHUFF, respectively. Chapter 2 and Chapter 3 demonstrated that UINT achieved better overall compression effectiveness, but required more time for both encoding and decoding. Moffat and Zobel [1992] argued that arithmetic coding is unsuitable for an information retrieval system due to the decoding time involved, which translates to waiting time for the user. Instead, Huffman coding was employed and was shown to yield a small loss in compression effectiveness, while providing an estimated improvement of 40 times over an implementation of arithmetic coding by Witten et al. [1991]. Following the work by Moffat and Zobel, UINT was abandoned as a potential coding candidate in a phrase browsing system with RE-PAIR. The systems detailed since Chapter 3 have only considered SHUFF as the coder for RE-PAIR's modifier streams and reduced sequence.

Unfortunately, applying the Huffman coder SHUFF to these four streams does not meet the needs of phrase browsing. To illustrate this, a description of how the streams need to be accessed by a phrase browser is necessary. The phrase hierarchy and the word lexicon allow the user to manoeuvre around the primitives and phrases, with the exact

steps performed being the topic of Chapter 7. Phrase browsing ceases when the user has decided on a symbol of interest, α. The user will then examine the contexts in which α occurs. Within the reduced word sequence, α may appear directly or indirectly, as part of a symbol in a higher generation. That is, the browsing system must translate the symbol α into a set of symbols $\mathcal{C} = (\alpha_1, \alpha_2, ..., \alpha_k)$ where $\alpha = \alpha_1$, but α_2 to α_k are higher-generation symbols which contain α. Precise details about the search through the phrase hierarchy is provided in the next chapter, but can be summarised as a recursive search for all symbols which contain α. The important point for this chapter is that a search for every symbol in $\mathcal{C}$ is done next on the reduced word sequence.

The locations of the symbols of $\mathcal{C}$ are determined and recorded in the set $\mathcal{R}$, along with some suitable context for each. One plausible context may be the 10 symbols before and after each location. Results in $\mathcal{R}$ are interactively shown to the user by using the phrase hierarchy and the word lexicon to translate the reduced word sequence references to words.

If the displayed result does not satisfy the user's information need, further clarification may be required. The initial crude result displays stemmed, case folded word tokens with non-word tokens removed. To improve visual appeal, the initial display might add a space character between each word token. Ultimately, further refinement of the displayed result through the decoding of the three modifier streams is required, and if the ith symbol in $\mathcal{R}$ ($\mathcal{R}_i$) is being displayed to the user, the corresponding modifiers for the symbol and its context need to be located and applied.

There is an important distinction between the purposes of the reduced word sequence and the three modifier streams. A set of symbols are *searched* for in the reduced word sequence, based on specified symbol values. The modifier streams are then synchronised to the same locations. The symbols in the modifier streams are located by their positions, and not their values. Also, locating modifiers for $|\mathcal{R}|$ contexts is unnecessary, since not every result in $\mathcal{R}$ may require clarification. So, one approach to the problem of modifier look-up is to obtain one position and its context at a time, as required.

With these points in mind, the next two sections look at ways of coding the four

streams to enable the necessary searching and synchronisation operations.

6.2 Huffman Coding in Blocks

Motivated by the discussion of compression effectiveness in Chapter 4, the application of several compression systems on the three modifier streams was considered and the costs in time and benefits in space were assessed. While compression effectiveness was good, there are two drawbacks to the approaches with respect to retrieval. First, because the user is waiting for the results, decoding should be fast. While previous results with dictionary-based systems such as GZIP and RE-PAIR have shown that decoding can be done efficiently, the time required cannot be ignored. Second, each time the modifier stream is accessed, only one position is sought. Once that symbol has been found, only it and its neighbouring symbols are required. Decoding the entire stream from the beginning until the symbol of interest seems wasteful.

If time is crucial, then a flat binary code can be employed, as shown in Table 2.4 on page 31. That is, each symbol in each of the modifier streams is encoded as an L-bit integer. The parameter L is chosen so that no symbol in the stream is larger than 2^L. Two passes over the stream are required for encoding, with the first pass used to determine L, and the second pass for encoding each symbol. Since every symbol is fixed at L bits in width, any symbol can be retrieved provided L is specified to the decoder. Retrieval is fast and is only limited by the physical characteristics of the storage medium.

In this section, a third alternative is presented which compromises between these extremes. Compression is achieved by encoding each stream directly with a zero-order Huffman coder (SHUFF). Unlike in earlier chapters, each stream is partitioned into blocks before being processed. In Chapter 5, a message was divided into blocks by RE-PAIR because of limitations in memory. Now, separating the streams of phrase numbers into blocks of B symbols serves a second purpose – to allow any arbitrary block to be decoded by creating a partial index I, as shown in Figure 6.1. Each entry in the index points to the beginning of the corresponding block of Huffman coded symbol numbers.

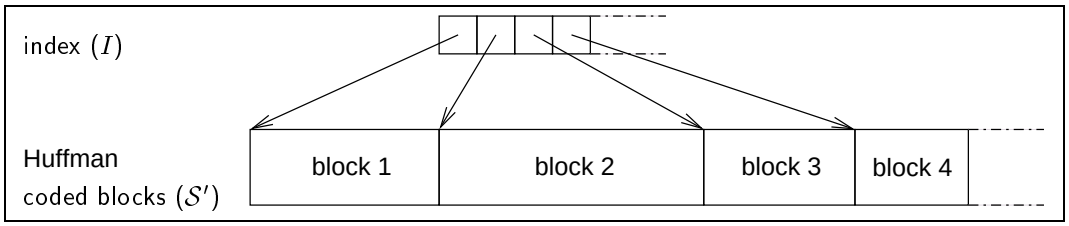


Figure 6.1: The Huffman coding approach with an index which leads to the beginning of each block. Each block is self-contained, and includes the same number of coded symbols.

Algorithm 6.1: Algorithm for Indexed SHUFF.

input : $\mathcal{S}$, block size B
output : $\mathcal{S}'$, index I

```
1 i ← 0; S' ← {}; I ← {}
2 for i < |S| do
3     current_block ← encodeShuff(S[i...i + B − 1])
4     append |current_block| to I
5     append current_block to S'
6     i ← i + B
7 return (S', I)
```

The algorithm for this approach, called *indexed* SHUFF, is shown in Algorithm 6.1. Each block of B symbols is encoded with SHUFF in isolation and appended to the output stream, $\mathcal{S}'$. The size of each compressed block is appended to the index I.

The number of blocks created for a stream of n symbols is $\lceil n/B \rceil$. When a symbol is being sought, the fixed block size ensures that a position p in the stream must be in block $\lceil p/B \rceil$. If line 4 of the algorithm is changed so that I contains cumulative compressed block sizes, then only two index entries need to be obtained in order to find the beginning and ending of the corresponding block. Otherwise, every index entry up until the two being sought need to be decoded. Within the compressed stream ($\mathcal{S}'$), position p plus some context to the left and right is retrieved. If the size of the window of symbols is smaller than B, at most two adjacent blocks need to be accessed from disk, in addition to the entries in the index.

It is important to note that the SHUFF coder has the ability to divide an input stream into blocks, according to options provided by the user. However, the starting locations

of each block are invisible to the decoding program, preventing any block from being processed without decoding all previous blocks.

Since each block of B symbols are compressed individually, indexed SHUFF allows the input stream to be read in blocks of B symbols at a time. Indexed SHUFF is expected to take longer than compressing the entire stream with SHUFF as a single block because of the number of disk accesses necessary. Experiments later in this chapter survey the effect various choices for B have on time and space.

6.3 Byte-Aligned Coding in Blocks

The reduced word sequence needs to support a search operation and not just the seek mechanisms described in the previous section. For efficiency reasons, it is preferred that the sequence be searched while in compressed form. Decoding the sequence before searching requires extra time. Moreover, it may be more efficient to search through a compressed message because it is smaller than its corresponding uncompressed form.

Motivated by these observations, Amir and Benson [1992] described the problem of searching a message in compressed form and labelled it as the *compressed matching problem*. Since then, searching in a compressed message has received a significant amount of attention, with the searching mechanism generally being directly linked to the compression algorithm employed. Some recent work in this area include searching in data encoded with Huffman coding [Klein and Shapira, 2001, Takeda et al., 2002] and searching in messages which have been compressed with LZ78 [Navarro et al., 2001] and SEQUITUR [Mitarai et al., 2001].

The problem considered by Klein and Shapira is that of searching in a Huffman coded text. As Huffman codewords are variable length, a match may be a false one if it is not aligned on a codeword boundary. The solution proposed by Klein and Shapira extends earlier work which showed how Huffman codes tend to resynchronise after errors [Klein and Wiseman, 2000]. If a possible match is found at position i, then their algorithm jumps back a constant number of bits and decodes all of the bits up until position $i-1$ with a Huffman tree. Probability analysis combined with experiments demonstrate that the

quality of the search results improves as the size of the section of bits decoded increase. However, their method is unsuitable for the reduced sequence because several patterns may need to be searched simultaneously. The number of symbols in $\mathcal{C}$ can be large, depending on the symbol selected by the user, and the number of other symbols that rely on it. Ideally, a search method should make a single pass through the reduced sequence, regardless of the size of $\mathcal{C}$.

Takeda et al. [2002] described a method of searching compressed documents without prior modification. They also demonstrated their technique on multi-byte character texts as well as semi-structured texts, and showed that any prefix-free code, including Huffman codes, could be searched. The basis of their idea was to combine synchronisation with string searching by constructing a pattern matching machine, which is then run on the entire file.

Manber [1997] and de Moura et al. [2000] looked at byte-aligned codewords to improve efficiency and to avoid the problem of aligning with codeword boundaries. Decoding byte-aligned codewords is fast, since repeated bit operations like bit shifting and bit masking are eliminated. Manber devised a mechanism for compressing and searching ASCII text which resembles the pairing of Re-Pair. Since ASCII is limited to only 128 symbols, frequent pairs of symbols were replaced non-recursively until the compressed message had 256 unique symbols. By converting the search pattern in a similar fashion, the pattern is searched for directly in the compressed message using a string-matching technique such as Boyer and Moore [1977]. Alternatively, de Moura et al. implemented Huffman coding with a radix-256 code. That is, the nodes in the generated Huffman code tree had an out-degree of 256, instead of 2 for a binary code. Building upon this idea, a Tagged Huffman code was used for compression and searching. Tagged Huffman coding represented words as groups of 7 bit codewords (radix-128), with an eighth bit reserved for indicating the first byte of a codeword, to ensure alignment while searching. In their experiments, Tagged Huffman coding offered compression effectiveness which was around 5 bits per symbol worse than binary Huffman coding. However, overall search times required about half of the time of a direct search on the uncompressed message. More recently, Brisaboa

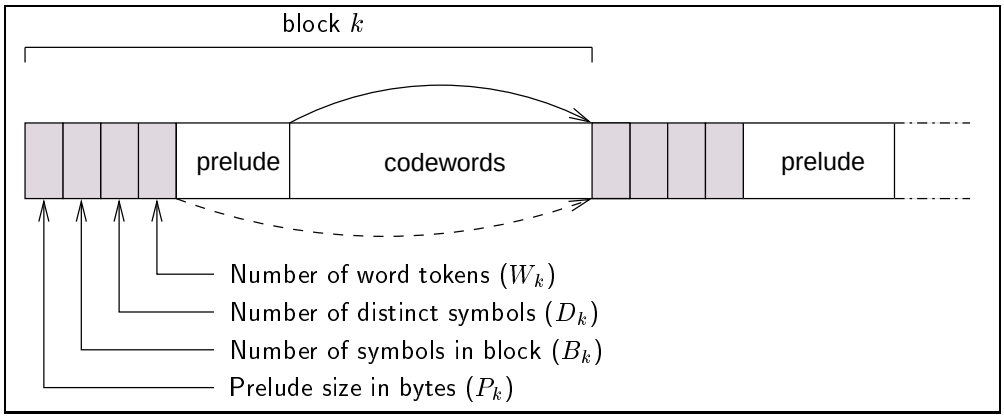


Figure 6.2: One Re-View block and part of the next one. A Re-View block is composed of four integers in the header (shown shaded), a prelude, and a section of codewords. The arrow at the top of the figure indicates the potential to skip the codewords of a block if no match is known to exist in that block.

et al. [2003] extended this work by generalising Tagged Huffman codes to (S, C)-Dense Codes. (S, C)-Dense Codes consist of continuers and stoppers. Stoppers are bytes which indicate the last byte of a codeword. Other bytes are continuers. Tagged Huffman coding is a special case when there are 128 stoppers and 128 continuers. Brisaboa et al. varied the size of these two sets of codes to determine the combination which yielded the best compression ratio for various document collections.

The entropy coding requirements for the reduced word sequence are further eased compared to this previous work, in favour of retrieval. A coder called Re-View has been developed which operates with a corresponding codeword length limit L. Blocks are created so that each block has at most 2^L distinct values. Byte-alignment is achieved by restricting L to be either 8 or 16. These two variants are denoted as Re-View$_8$ and Re-View$_{16}$, respectively.

A sketch of a Re-View block and part of the following block is shown in Figure 6.2. Each block consists of a prelude and codewords, similar to a shuff block. The prelude specifies the 2^L different symbol values that exist in the block. At the beginning of each block, four values are also required, shown shaded in the figure.

The four values in the header provide information about the entire block to improve the search time of Re-View. In addition to the byte-alignment of codewords, the infor-

mational headers allow a block to be bypassed when none of the symbols being sought ($\mathcal{C}$) are known to exist inside. After phrase browsing, a user may have honed in on a phrase which is highly descriptive, and exists in only certain parts of the document collection. Areas in the collection which do not contain the phrase should be skipped. This function is provided by the last three of the shaded values of Figure 6.2. In order to determine whether or not a symbol in $\mathcal{C}$ exists in a block, the prelude must be decoded. If no symbols in $\mathcal{C}$ are found, then the block of codewords may be skipped, as shown by the solid, curved arrow in the figure. There are B_k symbols in block k, which translates to $B_k(L/8)$ bytes. The number of distinct values in the block (D_k) is required to aid in decoding the prelude, and is 2^L for every block except for the last one. The number of original word tokens covered by a block is specified in the header as W_k, and is necessary for synchronisation with the modifier streams. After RE-PAIR, the number of word tokens represented by a block is equal to or greater than the number of symbols it contains. When a block is skipped, not only does the number of phrase symbols that have been passed need to be known (B_k), but the number of original word tokens as well. Then, within a block, as symbols are read, a count of the number of word tokens skipped and read since the beginning of the stream is maintained, so that each word token's corresponding modifier can be determined accurately. In order to support this requirement, both encoding and decoding of a reduced word sequence requires the lengths (in number of word tokens) of each symbol in the phrase hierarchy.

In later experiments, RE-VIEW is applied to the modifier streams as well, to assist in its understanding. As with indexed SHUFF, every block, except for the one which contains the position being sought, needs to be skipped. The first value in the header indicates the size of the prelude in bytes to allow both the prelude and the codewords of a block to be skipped, as shown by the dashed line of Figure 6.2. The block of interest can be located by summing B_k for every block. In order to bypass the prelude and codewords of the current block k, the number of bytes is calculated as $P_k + B_k(L/8)$.

The algorithm for RE-VIEW is presented in Algorithm 6.2. RE-VIEW processes a stream ($\mathcal{S}$) by scanning it sequentially while keeping track of the number of distinct

Difference	Nibble-aligned codes
1 – 8	0 – 7
9 – 135	80 – FE
≥136	FF

Table 6.1: Mapping differences to nibble-aligned codes. The codes are specified in hexadecimal.

symbol values seen so far (D_k). The distinct symbol values are maintained in an array *symbol_map*, whose size is proportional to the number of distinct symbols in the entire sequence (the size of the alphabet, $|\Sigma|$). Once 2^L different values have been seen (line 9), a block is created. The distinct symbols that appear in the block are mapped to L-bit integers on line 14, and recorded in the prelude. The prelude is buffered so that P_k can be placed first within a block. The codewords are produced by mapping the symbol values to L-bit integers, by consulting the *symbol_map* array on line 22. The symbols in the block are represented as fixed L-bit codewords, and retain L-bit alignment throughout the block.

While *symbol_map* is scanned, each symbol is recorded in the prelude on line 13. Symbols are encoding as differences from the preceding symbol using nibble-aligned codes (4-bit codes). The mapping of differences to nibble-aligned codes is specified in Table 6.1. In the second column of the table, the codes employed are specified using hexadecimal.

The main data structures employed by RE-VIEW are the sequence and the *symbol_map* array, requiring a total of $|\mathcal{S}|+|\Sigma|$ words of memory. Three main operations are performed by RE-VIEW on these two structures. When a block is being built, two passes are performed over the reduced sequence. The first pass inspects the sequence in order to identify 2^L distinct values. The second pass maps each symbol to an L-bit codeword. Also, in order to assign symbols to codewords, a single pass over the *symbol_map* structure is necessary for each of the K blocks created. Since *symbol_map* has $|\Sigma|$ entries, K linear passes may be costly. If an index of size 2^L is created so that only the symbols in *symbol_map* need to be looked at, then the linear search can be eliminated. For each block, this index needs to be sorted at a constant cost of $2^L \log_2 2^L$. So, the RE-VIEW algorithm runs in $O(2|\mathcal{S}| + K(2^L \log_2 2^L) + K2^L)$.

Algorithm 6.2: Algorithm for RE-VIEW.

input : $\mathcal{S}$, length of symbols in $\mathcal{P}$, alignment L in bits
output : $\mathcal{S}'$

```
1  S' ← {}; P_k ← 0; B_k ← 0; D_k ← 0; W_k ← 0; block_start ← 0
2  initialise symbol_map
3  for i < |S| do
4      W_k ← W_k + |α_i|
5      B_k ← B_k + 1
6      if symbol_map[α_i] = 0 then
7          symbol_map[α_i] ← 1
8          D_k ← D_k + 1
9      if D_k = 2^L then
10         code ← 0; j ← 0
11         while code < D_k do
12             if symbol_map[j] = 1 then
13                 write α_j to prelude buffer
14                 symbol_map[j] ← code
15                 code ← code + 1
16             j ← j + 1
17         P_k ← size of prelude buffer
18         write P_k, B_k, D_k, W_k to S'
19         write prelude buffer to S'
20         j ← block_start
21         for j < i do
22             write symbol_map[α_j] to S'
23         P_k ← 0; B_k ← 0; D_k ← 0; W_k ← 0; block_start ← i
24         re-initialise symbol_map
25         k ← k + 1
26 return (S')
```

When a set $\mathcal{C}$ is searched for in the compressed sequence, the prelude of every block must be decoded. If at least one symbol in $\mathcal{C}$ exists in the current block k, then the corresponding codewords are also decoded. Otherwise, the set of codewords is skipped using the value B_k in the header. As is demonstrated shortly, since the prelude is about 36.1% of the compressed file, a scan for a rare phrase involves processing at least one-third of the sequence. If the limited amount of decoding involved in processing every prelude cannot be accepted, then one solution is to create a higher-level index, similar to the GLIMPSE system [Manber and Wu, 1994]. The GLIMPSE retrieval system partitions

a file system into blocks. The index indicates, for every term in the file system, the blocks in which it occurs. Once the blocks of interest have been determined, each block is sequentially searched.

Several potential enhancements to RE-VIEW are possible. Disk access can be reduced by physically separating the preludes and the codewords into different files. Then, disk access is concentrated on the prelude file. Another modification would build a top-level index on blocks so that the preludes of blocks which do not contain the symbol of interest are avoided entirely. Such an index would be kept in memory. Extending this last idea to an inverted index on symbols is also possible. None of these options were explored in the experiments reported in the next section since the search times obtained were acceptable.

6.4 Experiments

To investigate the compression effectiveness of the modifier streams and the reduced word sequence, experiments were conducted using Huffman coding, built on the SHUFF implementation, RE-VIEW, and an even simpler binary code. The efficiency for both skipping to a precise location and searching are also examined. Test files were taken from the punctuation-aligned RE-PAIR experiments with method E of RE-MERGE on the `NEWS` test data, as described in Chapter 5. The compression results from that chapter were presented in Table 5.9 (page 147) for the modifiers and Table 5.11 (page 148) for the reduced word sequence. In those tables, the modifiers and the reduced word sequence were coded with SHUFF as a single block. The two tables in Chapter 5 also indicate that the combined compression ratio of the word lexicon, non-word lexicon, and the phrase hierarchy was 0.143 bits per character. The remaining 4 streams occupied 2.090 bits per character.

Additional information pertaining to the four streams is listed in Table 6.2. The last column in the table lists the self-entropy of each file. Note that because these values are relative to the streams themselves and not to the size of `NEWS`, they are not comparable to the compression ratios achieved in Table 5.9 and Table 5.11.

	Symbol range	Distinct values	Number of symbols	Self-entropy
case folding modifier	0 to 65,536	634	168,721,097	1.303
stemming modifier	0 to 7,864,320	541	168,721,097	2.204
non-word modifier	0 to 11,332	11,333	168,721,097	2.054
reduced word sequence	0 to 8,277,359	6,232,453	58,685,281	18.659

Table 6.2: Additional information about the modifier streams and the reduced word sequence from method E of Re-Merge with punctuation-aligned Re-Pair on NEWS. The column headed self-information is the zero-order self-information for the frequency distribution of the symbols in that file, measured for each file in bits per symbol relative to the size of that file.

The other three columns of the table provide the range of symbols, the number of distinct values, and the total number of symbols of each stream. Note that the case folding and stemming mechanism employed by Pre-Pair permit 2^{16} possible case folding values and 2^{23} possible stemming values. In practice it appears that even a large collection only has a small number of distinct values used. In contrast, Pre-Pair assigns non-word modifier values in sequential order, starting from 1, with 0 reserved for the zero-length non-word symbol. For the non-word modifier stream, every symbol within the range appears. Since there is a one-to-one correspondence between a word token and its modifiers, the three modifier streams have the same number of values in them.

Compared to the other three streams, the reduced word sequence is shorter, because Re-Pair and Re-Merge have been applied. No pair of adjacent symbols occurs twice in this sequence, unless one of the restrictions imposed by punctuation-aligned Re-Pair has been violated. As a result, the self-entropy of the stream is significantly higher than the others. Accordingly, the number of distinct values and the symbol range are also high. The symbol range is simply the number of symbols in the phrase hierarchy (see Table 5.10 on page 148), with a special end of block marker. After recursive pairing, some symbols in the phrase hierarchy do not appear directly in the reduced word sequence, but instead have been absorbed by higher generation symbols. That is why the number of distinct symbols is less than the number of entries in the phrase hierarchy.

There are two sets of experiments that are performed in this section. The first set considers the three modifier streams, while the second set examines the reduced word

sequence in isolation. In both scenarios, the compression ratios are reported, as are the times for compression and full decompression. The times for seek or search operations are also reported, whichever is applicable. When necessary, the compression ratios are reported in bits per character (bpc) relative to the size of `NEWS`, and in bits per symbol (bps) relative to the number of symbols in the file itself. The number of bits per character is comparable with all compression ratios reported in previous chapters, while the number of bits per symbol is important when examining the column labelled "self-entropy" in Table 6.2.

Program execution times in this chapter are reported as both CPU time and elapsed time. CPU time is a measure of the amount of computation performed by the program. All execution times reported in previous chapters have been CPU costs. But because this chapter also considers the amount of time a user must wait for a response, efficiency is also reported as elapsed time (also called "real time"). Elapsed time includes both user time and the time taken by the system for the program. Since all programs are executed with no data in memory beforehand, the time needed to access the disk is included in the elapsed time. All times, both CPU and elapsed, are averaged over three trials unless otherwise indicated.

Table 6.3 and Table 6.4 summarise the experiments with the modifiers. Each of the three modifier streams was compressed with indexed SHUFF with block sizes increasing by a factor of 8 from from 4 KiB up to 16,384 KiB. While not the intended purpose of RE-VIEW, RE-VIEW$_L$ was tested with $L = 8$ and $L = 16$ as a preview for the second set of experiments later. A flat binary coder (FLAT-BINARY) was also applied so that each symbol was encoded as a fixed-width binary code whose length was based on the maximum symbol value in the stream. The FLAT-BINARY system treated each stream as a single block.

Generally, as the block size for indexed SHUFF increased, compression effectiveness improved, as shown in Table 6.3. However, the decrease is small, especially between the larger block sizes. Also, the difference in compression ratio between the smallest block size and the largest block size shown is no more than 0.052 bits per character. Similar

Block size (× 1024 symbols)	Compression ratio bpc	bps	Number of blocks
case folding modifiers			
4	0.259	1.608	41,192
32	0.243	1.508	5,149
256	0.240	1.494	644
2,048	0.240	1.492	81
16,384	0.240	1.493	11
FLAT-BINARY	2.736	17.000	1
RE-VIEW$_8$	1.288	8.004	161
RE-VIEW$_{16}$	2.575	16.004	161
stemming modifiers			
4	0.429	2.667	41,192
32	0.388	2.414	5,149
256	0.379	2.358	644
2,048	0.378	2.347	81
16,384	0.377	2.346	11
FLAT-BINARY	3.701	23.000	1
RE-VIEW$_8$	1.487	9.240	479
RE-VIEW$_{16}$	2.642	16.417	161
non-word modifiers			
4	0.370	2.298	41,192
32	0.346	2.148	5,149
256	0.342	2.123	644
2,048	0.341	2.119	81
16,384	0.342	2.126	11
FLAT-BINARY	2.253	14.000	1
RE-VIEW$_8$	1.291	8.020	1,690
RE-VIEW$_{16}$	2.575	16.004	161

Table 6.3: Compression effectiveness of indexed SHUFF, RE-VIEW, and FLAT-BINARY for the modifier streams. The second column lists the compression ratios of each method relative to the size of `NEWS` in bits per character relative to the original file. In the third column, compression levels in bits per symbol relative to the number of symbols in that stream are reported, with the best values for each stream highlighted. The number of blocks produced are shown in the last column.

conclusions can be drawn from the column titled "bps" when compared with the self-entropy of each file (Table 6.2).

The implementation of RE-VIEW first creates an initial buffer of 1,048,576 symbols, preventing any block from being larger than the buffer. The low variability in case folding modifier values means that after RE-VIEW has seen just over one million symbols, less

than 2^8 distinct symbols have been found. As the number of distinct values increases in first the stemming modifiers, and then the non-word modifiers, more blocks are made by RE-VIEW$_8$. However, as RE-VIEW$_{16}$ still only creates 161 blocks in both cases, no buffer has 2^{16} or more distinct values. Compression effectiveness of both types of RE-VIEW are unacceptable, though quite expected. RE-VIEW$_L$ can never achieve compression effectiveness that is better than L bits per symbol.

The ideal method for encoding the modifier streams depends on efficiency as well as effectiveness, as shown in Table 6.4. The second and third columns of the table list the encoding and decoding times of each of the methods, averaged over three trials, and represented as CPU time. In order to demonstrate the random access abilities, 100 random positions were selected from the possible 168,721,097. During the design of the experiments, it was noted that access times varied depending on the position in the stream that is being accessed. However, if the same 100 positions were used by all of the experiments, then the access times between systems can be compared. Moreover, since the access times were short, Table 6.4 presents the total time for accessing all 100 positions in the same order. These times are reported as elapsed time and CPU time in the fourth and fifth columns of the table. All times have been averaged over three trials. In the entries labelled "—", less than 0.1 seconds was noted, a time which cannot be reliably measured.

As less blocks are created, compression and decompression efficiency improves. RE-VIEW is faster than both indexed SHUFF and FLAT-BINARY. The encoding times of FLAT-BINARY are equivalent to the largest block size tested for indexed SHUFF, while decoding is slightly worse than indexed SHUFF with a 256 KiB block size. The time required for random access is fastest for the FLAT-BINARY coder, followed by the two RE-VIEW systems, and then indexed SHUFF in increasing block size.

The second set of experiments looked at encoding the reduced word sequence. Three methods of searching within the word sequence were compared.

The first method applied version 2.4.2 of GNU GREP [Free Software Foundation, 2000] on a simplified version of `NEWS`. The GREP program searches for a set of one or more strings and returns the lines which match. Including GREP in the experiments allows

Block size (× 1024 symbols)	Compression		Random access	
	Encoding time (CPU sec)	Decoding time (CPU sec)	Elapsed time (s)	CPU time (s)
case folding modifiers				
4	270.6	444.5	2.3	1.6
32	47.1	64.8	2.0	1.3
256	24.4	18.7	5.2	3.0
2,048	20.7	12.7	26.3	14.7
16,384	19.7	11.2	194.8	110.1
FLAT-BINARY	15.3	28.9	—	0.1
RE-VIEW$_8$	8.3	3.3	1.5	0.1
RE-VIEW$_{16}$	8.7	3.5	2.3	0.1
stemming modifiers				
4	916.5	445.1	2.1	1.6
32	141.1	66.0	2.2	1.5
256	36.8	19.1	5.1	3.0
2,048	23.1	12.4	26.7	15.3
16,384	21.0	11.3	200.2	113.6
FLAT-BINARY	17.0	37.6	—	0.1
RE-VIEW$_8$	9.0	4.1	2.0	0.4
RE-VIEW$_{16}$	8.7	4.0	3.1	0.3
non-word modifiers				
4	261.5	443.5	2.1	1.5
32	45.1	65.0	2.2	1.3
256	24.8	18.8	5.1	2.9
2,048	20.9	13.1	26.5	15.2
16,384	19.7	11.3	198.3	113.2
FLAT-BINARY	14.7	25.2	—	0.1
RE-VIEW$_8$	8.4	2.7	1.9	0.3
RE-VIEW$_{16}$	8.7	3.7	2.6	0.1

Table 6.4: Compression efficiency of indexed SHUFF, RE-VIEW, and FLAT-BINARY for the modifier streams of `NEWS`. All times are measured in seconds, with the best values shaded. Compression and decompression times are CPU times averaged over three trials. Results from randomly accessing 100 positions in each stream have also been repeated three times each. These times are shown as elapsed time and user time in the fourth and fifth columns, respectively. Times indicated by "—" are less than 0.1 seconds.

comparisons between searching for a string and searching for a reference to a string in the phrase hierarchy. The `NEWS` document must be simplified in order to minimise the number of differences between these two tasks. The modified `NEWS` document contains

every symbol in the reduced word sequence expanded into word tokens. No modifiers are applied, and a single space is inserted between each word token. In order to allow GREP to return lines which match, linefeeds were inserted after each SGML closing tag. As long as no line that is searched contains a SGML closing tag, this change ensures that no search string is missed because it straddles multiple lines. One minor problem with this arrangement occurs when a search string appears twice or more in the same line. In this case, GREP simply returns one copy of the line, and does not continue searching the remainder. Hence, these modifications to the `NEWS` document are not perfect, but do provide a reasonable estimate for direct searching. The modified version of `NEWS` is 860 MiB in size.

While GREP is used to search for a particular string, the string may exist in several symbols in the phrase hierarchy. The second and third methods are applied to the reduced word sequence so that a set of symbols can be searched for simultaneously. The second method assumes the sequence is compressed with FLAT-BINARY or SHUFF (without an index) using various block sizes, with the word sequence fully decompressed for each search. The third method compresses the sequence using RE-VIEW$_{16}$ with searching performed using the partial-decoder approach described earlier in Section 6.3. Note that RE-VIEW$_8$ cannot be sensibly applied to the word sequence because of the large number of distinct symbols.

Table 6.5 presents compression and decompression results with SHUFF, RE-VIEW$_{16}$, and FLAT-BINARY. Larger block sizes were selected for SHUFF than for indexed SHUFF in order to favour compression effectiveness and decompression time. As Table 6.4 showed, the overhead of processing many small blocks with indexed SHUFF had an unfavourable effect on the decompression time. Block sizes for SHUFF ranged from 1,048,576 symbols per block up to the entire stream as a single block (∞).

The compression ratios are listed in the second and third columns of the table in bits per character relative to the source text and bits per symbol relative to the file of phrase numbers, respectively. As the SHUFF block size increases, compression effectiveness improves. The exception occurs with the last block size. Enlarging the block size to ∞

Method	Compression ratio		Number of blocks	Compression time	
	bpc	bps		Encoding (s)	Decoding (s)
SHUFF block size					
1,048,576	1.183	21.144	56	104.5	25.1
2,097,152	1.166	20.827	28	87.3	22.9
4,194,304	1.151	20.570	14	75.9	21.1
8,388,608	1.139	20.355	7	64.9	18.7
16,777,216	1.132	20.219	4	56.3	17.0
20,971,520	1.131	20.199	3	53.6	16.2
33,554,432	1.127	20.138	2	48.3	15.1
∞	1.131	20.213	1	52.6	13.9
FLAT-BINARY	1.287	23.000	1	5.8	13.1
RE-VIEW$_{16}$	1.402	25.044	672	64.9	7.9

Table 6.5: Compression statistics for the reduced word sequence of `NEWS`, using SHUFF, FLAT-BINARY, and RE-VIEW$_{16}$. All times are reported in CPU seconds and averaged over three trials, with the best values shaded.

reduces the total prelude size, but the coded message increases in size. The reason for this unexpected result is that the reduced word sequence, and the `NEWS` file from which it was derived, is not uniform. Recall that `NEWS` is composed of several years' worth of news articles from two different sources, with the Wall Street Journal articles at the beginning and the Associated Press articles at the end. The change in document style and the change in article years was the price paid in order to create a sufficiently large document for experiments in this thesis. The FLAT-BINARY coder attains a compression level which is about 3 bits per symbol worse than SHUFF. RE-VIEW$_{16}$ achieves compression which is 5 bits per symbol worse than SHUFF; not a surprising result given that RE-VIEW$_{16}$ is not an entropy coder and a prelude must be transmitted for each of the 672 blocks. The size of the encoded reduced word sequence with RE-VIEW$_{16}$ was 175.2 MiB, of which the values in the headers (W_k, D_k, B_k, and P_k) and the preludes made up 63.3 MiB. The large preludes ensure that RE-VIEW$_{16}$ yields compression effectiveness which is also worse than FLAT-BINARY.

Following the column of block count are the encoding and decoding times, in seconds of CPU time, averaged over three trials. Generally, encoding time with SHUFF decreases as the block size increases, except for the "∞" block size. Decoding time also decreases

for large block sizes as fewer preludes need to be decoded. The FLAT-BINARY coder is the fastest system for encoding and is faster than SHUFF for decoding, irrespective of block size. RE-VIEW$_{16}$ requires as much time to encode as SHUFF with a block size of 8 million symbols. But RE-VIEW has the lowest decoding time out of all the systems tested.

More important than decoding time is searching time, which was the topic for this set of experiments. To quantify the searching speeds, each of GREP, decompressed searching, and RE-VIEW$_{16}$ were used to search for symbols selected at random from the phrase hierarchy. Decompressed searching includes full decoding with SHUFF, followed by a direct search on the reduced word sequence. Since an infinite block size was employed by SHUFF, all the times for decompressed search include 16.0 seconds of elapsed time and 13.9 seconds of CPU time.

The symbols searched for were selected based on certain criteria. Only symbols which directly appeared in the reduced word sequence and were not components of any higher generation symbol in the phrase hierarchy were chosen. These restrictions ensure that the comparison was as fair as possible. Of the 8,277,359 symbols in the phrase hierarchy, 4,376,175 symbols were deemed valid.

Experiments were conducted from two different perspectives, based on the fact that not all of the valid symbols are equal. The first experiment highlights the fact that symbols occur with different frequencies. The four million valid symbols were divided into 3 equal groups based on frequency in the reduced word sequence: high, medium, and low. The second set of experiments was based on symbol length, in word tokens (or primitives). The valid symbols were divided into three groups based on their lengths when expanded, as shown in Table 6.6. Also shown in this table is the number of invalid symbols (second column). Valid symbols were divided based on their lengths and then merged to create three roughly equal symbol length groupings, as shown in the third and then the fourth columns of the table. In the first group, valid symbols of length 1 or 2 primitives are included. The second group contains valid symbols of lengths 3 or 4 primitives. All remaining valid symbols are in the third group.

Figure 6.3 and Table 6.7 present the results from the first set of experiments, based on

Symbol lengths (words)	Distribution		Size of group
	Invalid candidates	Valid candidates	
1	133,389	203,349	915,908
2	615,548	712,559	
3	804,927	1,149,698	2,040,548
4	623,040	890,850	
5	415,727	507,765	1,419,719
6	275,483	281,307	
7	192,309	156,154	
8	141,607	101,003	
9	106,609	62,752	
10+	592,544	310,738	
Total	3,901,184	4,376,175	

Table 6.6: Distribution of invalid and valid symbol for the experiments with the reduced word sequence. Phrase hierarchy symbols are separated based on length (in primitives), with the valid ones merged to form three symbol length groupings.

symbol frequency in the reduced word sequence. For each grouping, 1, 10, or 100 symbols were searched for, selected at random. The same symbols were searched three times and the time required was averaged, producing the elapsed and CPU times shown in the figure.

As more symbols are searched simultaneously, GREP requires more time. However, the frequency of the symbols has a relatively small effect on the elapsed and CPU times. There is a noticeable difference between elapsed and CPU times for GREP, due to the number of disk accesses. Meanwhile, decompressed searching requires the same amount of elapsed and CPU times, regardless of the number of symbols sought or the frequency of the symbols. Of the three systems shown in the figure, RE-VIEW$_{16}$ achieves the best search times. As RE-VIEW$_{16}$ searched for more symbols at the same time, more time is required. The effect of symbol frequency on search time begins to be noticeable when as many as 100 symbols are searched for. This result reflects the data displayed in the last two columns of Table 6.7. The third column indicates the number of blocks that were fully decoded. When searching with RE-VIEW$_{16}$, every prelude must be decoded, but when the symbols being sought are not in the block, the rest of the block can be skipped. As a consequence of this property, the number of bytes read can never be less than the

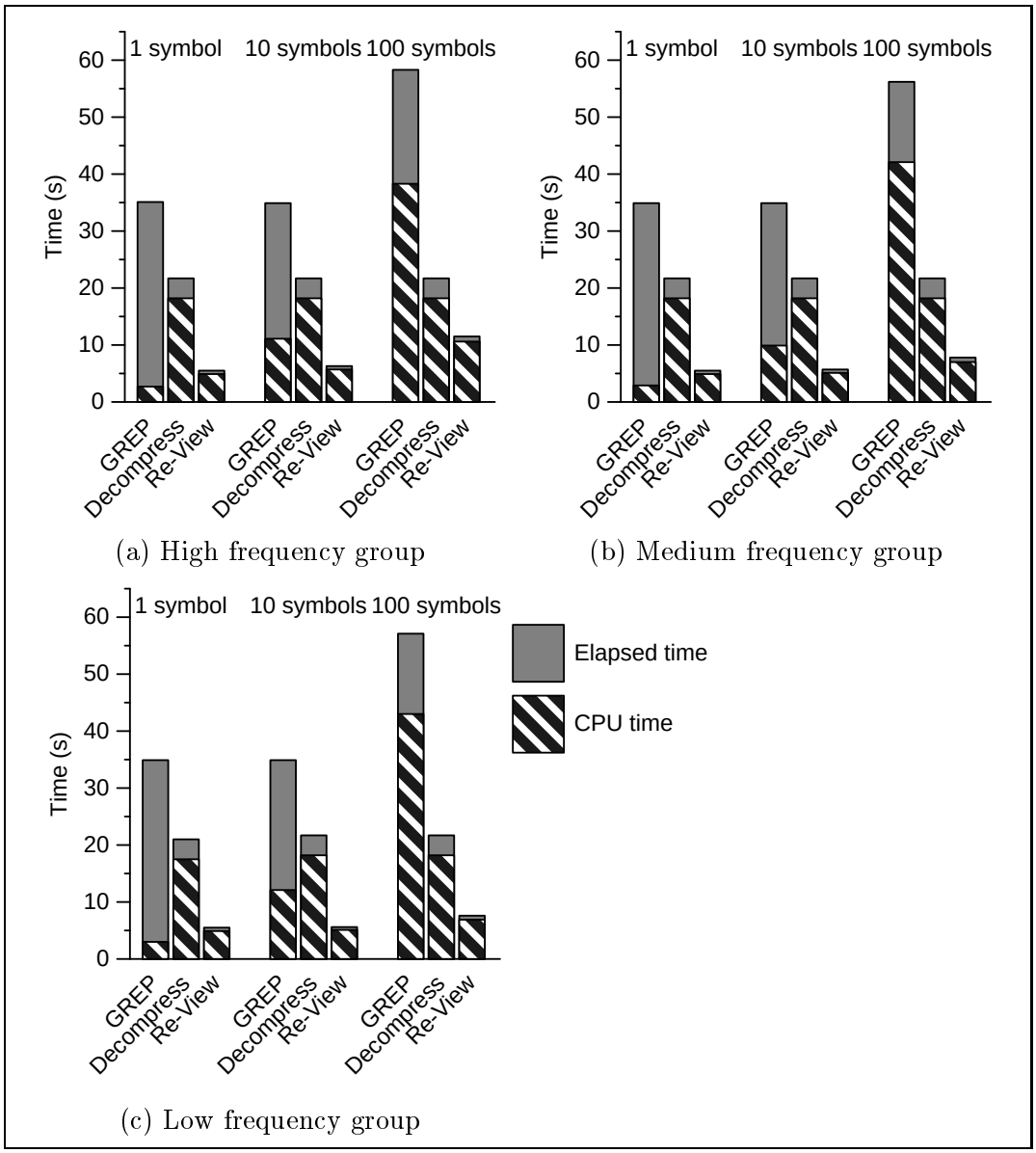


Figure 6.3: Search times with GREP, decompressed search, and RE-VIEW$_{16}$ for the reduced word sequence based on frequency groupings. The elapsed times and CPU times are reported in seconds and averaged over three trials with the same test symbols.

combined size of all of the preludes, which is 63.3 MiB. And, as the fourth column shows, as more blocks need to be fully decoded, more bytes need to be read. So, when either more symbols or more frequent symbols are searched for, the number of blocks that are fully decoded and the number of bytes read increases accordingly.

Frequency group	Symbols searched	Number of blocks decoded	Number of bytes read
High	1	4	63.9 MiB
High	10	58	73.0 MiB
High	100	423	134.0 MiB
Medium	1	2	63.6 MiB
Medium	10	18	66.3 MiB
Medium	100	161	90.0 MiB
Low	1	1	63.4 MiB
Low	10	14	65.6 MiB
Low	100	144	87.4 MiB

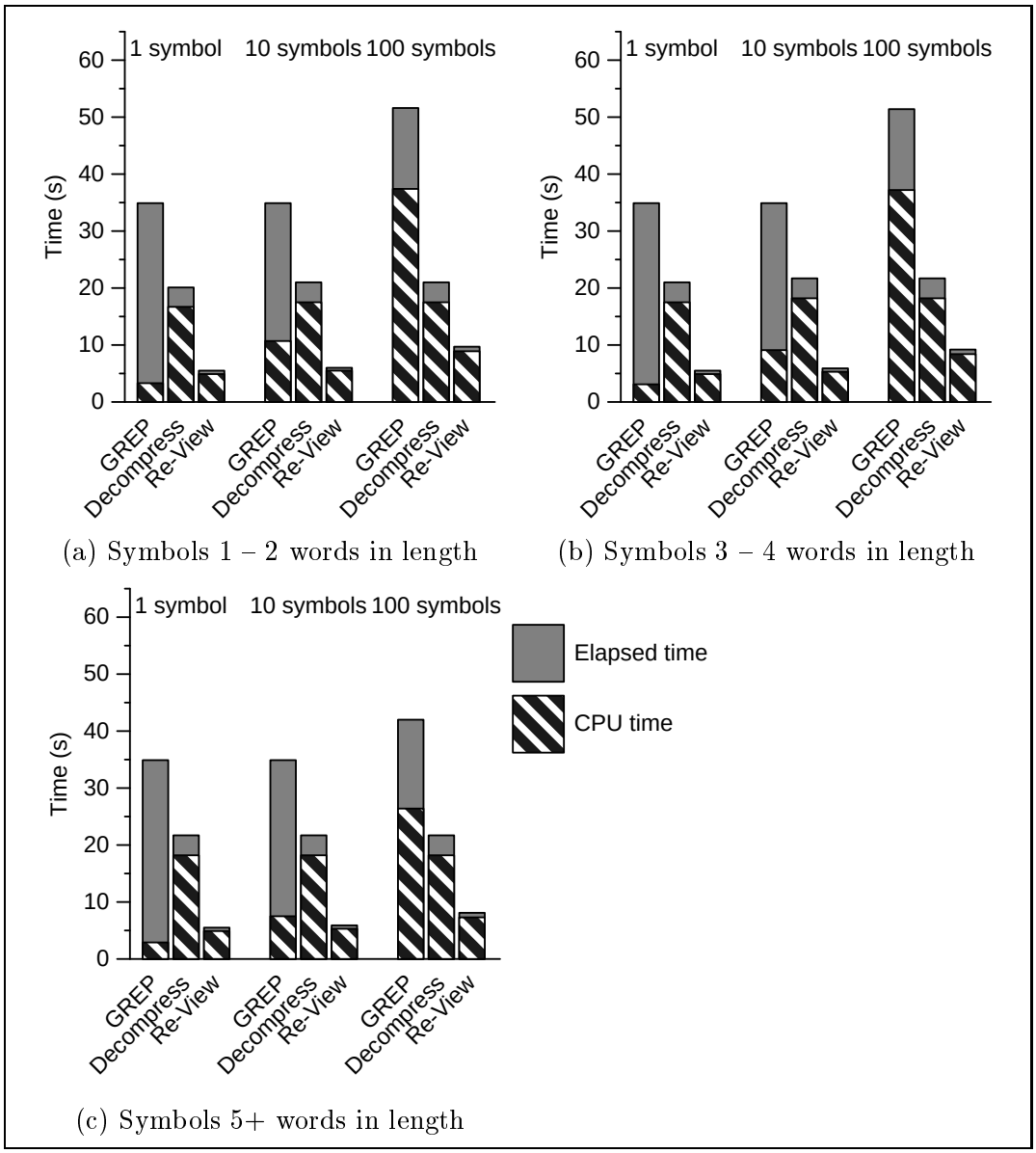
Table 6.7: Search efficiency with RE-VIEW$_{16}$ for the reduced word sequence based on frequency groupings. The number of blocks fully decoded and the number of bytes read in order to complete the searching are shown in the third and fourth columns.

The results from the second set of experiments with the reduced word sequence are contained in Figure 6.4 and Table 6.8. If Figure 6.3(c) is compared with Figure 6.4(c), it would appear that GREP requires less time as symbols which expand to more words are sought. However, symbol length does not have a noticeable effect on decompressed searching or RE-VIEW$_{16}$, according to these graphs.

Table 6.8 shows how less blocks are requested as symbol length increases. But this trend may be caused by the fact that symbols with more words are expected to occur less frequently.

In conclusion, this section has looked at the coding requirements of the three modifier streams and the reduced word sequence. Indexed SHUFF produced the best compression effectiveness when sufficiently large block sizes were chosen. While the compression and random access times offered by RE-VIEW and FLAT-BINARY for the modifiers were good, indexed SHUFF with small block sizes gave comparable results for skipping to arbitrary locations within the streams.

Three compression methods were examined for the reduced word sequence. As with the modifier streams, SHUFF provided the best compression effectiveness. While the FLAT-BINARY coder also gave better compression levels than RE-VIEW$_{16}$, RE-VIEW$_{16}$ was found



(a) Symbols 1 – 2 words in length

(b) Symbols 3 – 4 words in length

(c) Symbols 5+ words in length

Figure 6.4: Search times with GREP, decompressed search, and RE-VIEW$_{16}$ for the reduced word sequence based on symbol length groupings. The elapsed times and CPU times are reported in seconds and averaged over three trials with the same test symbols.

to be more efficient than SHUFF and FLAT-BINARY for decompression and searching. In these experiments, a modified version of `NEWS` was used by GREP, while decompressed searching assumed SHUFF with an infinite block size. The times for decompressed searching and RE-VIEW$_{16}$ were not affected by symbol frequency or symbol length, but both

Symbol length group	Symbols searched	Number of blocks decoded	Number of bytes read
1 – 2	1	2	63.6 MiB
1 – 2	10	41	70.2 MiB
1 – 2	100	298	113.0 MiB
3 – 4	1	2	63.6 MiB
3 – 4	10	30	68.3 MiB
3 – 4	100	261	107.0 MiB
5+	1	2	63.6 MiB
5+	10	32	68.6 MiB
5+	100	181	93.7 MiB

Table 6.8: Search efficiency with RE-VIEW$_{16}$ for the reduced word sequence based on symbol length groupings. The number of blocks fully decoded and the number of bytes read in order to complete the searching are shown in the third and fourth columns.

conditions had a noticeable effect on the number of blocks fully decoded and the number of bytes read from disk by RE-VIEW$_{16}$. Furthermore, as more symbols are searched for simultaneously, both RE-VIEW$_{16}$ and GREP required more time.

6.5 Selecting Coders

This chapter has examined coders for the three modifier streams and the reduced word sequence stream. The decoding requirements of these four streams for phrase browsing are quite different from those that apply to the phrase hierarchy and the two lexicons. Within these four streams, different decoding specifications also emerge. The modifier streams require random access, whereas the reduced word sequence is searched.

The naive approach to coding these four streams is to simply encode each reference as a 32-bit integer. Retrieval time is fast for both random access and searching, and the difficulty is allocating enough storage. Referring to Table 6.2, such an approach would require 4.310 bits per character for only these streams. If this level of compression is acceptable, then such a coding scheme can be considered.

At the other extreme, a compression system such as BZIP2 can be applied to the modifier streams, as explored in Chapter 4. If such an approach was employed, then the

streams must be fully decoded before being used, yielding unfavourable response times.

As a balance between these extremes, indexed Huffman coding is a reasonable choice for the modifier streams. As the streams are required less frequently, if blocks are created with an index for each block, then at most two blocks are decoded at any one time for seeking. The ideal random access times occur when a block size of 32,768 symbols is selected. If this block size is applied, the combined compression ratio of the three modifier streams is 0.977 bits per character.

The reduced word sequence must be searchable and RE-VIEW$_{16}$ combines byte-alignment for the codewords with the ability to skip blocks when it has been determined that no symbol of interest occurs within the block. The disadvantages of this approach are that all of the preludes must be decoded as part of every search, and that compression effectiveness is not as good as if an entropy coder like SHUFF had been used. One improvement to RE-VIEW$_{16}$, which was not explored, would be to create a second-level index so that each token in the file is associated directly with a list of the blocks in which it occurs.

RE-VIEW$_{16}$ achieves better search times than GREP and decompressed searching. But the GREP system still performs remarkably well given that the search criteria were strings instead of sets of symbols. Compression of the reduced sequence with RE-VIEW$_{16}$ requires 1.402 bits per character.

So, through the techniques described in this chapter and the encoding methods decided earlier for the phrase hierarchy and lexicons, the compression effectiveness for all seven streams is 2.523 bits per character. Table 6.9 updates Table 4.16 on page 106 with the coding mechanisms examined in this chapter. The next chapter takes these files and shows how they can be used to construct a phrase browsing tool.

Output from PRE-PAIR	**word lexicon**	word sequence	
Purpose	phrase browsing		searching for symbols
Model	front coding and BZIP2	RE-PAIR	
Coder		**phrase hierarchy** chiastic slide and interpolative coding	**reduced word sequence** RE-VIEW$_{16}$

(a) Words

Output from PRE-PAIR	**non-word lexicon**	**non-word modifiers**	**case folding modifiers**	**stemming modifiers**
Purpose	skipping to symbols			
Model	front coding and BZIP2	indexed SHUFF	indexed SHUFF	indexed SHUFF
Coder				

(b) Non-words and modifiers

Table 6.9: The seven streams produced by PRE-PAIR and RE-PAIR. The purpose of each stream, and the compression methods employed are taken from Table 4.16 on page 106, updated with the coding mechanisms in this chapter.

Chapter 7

Phrase Browsing

The main focus of the previous chapters has been the problem of transforming a message into a more economical form which is also suitable for phrase browsing. Decisions had to be made with respect to the level of word-based pre-processing necessary and the efficiency of coders. All of these issues were considered in the context of the sender of the compressed message. In this chapter, that compressed message is viewed from the point of view of its receiver, in the form of a phrase browser.

The phrase browser is a tool which hides the complexity and size of the compressed message from the user. The browser described in this chapter not only allows a user to gain an understanding of a document through its phrases, but it also provides a searching facility. As alluded to in Chapter 6, after phrase browsing ends and a phrase of interest has been found, the contexts in which the phrase appears may be needed. The phrase browser searches for these locations and also allows the user to refine the search result by incorporating case, stemming, and non-word information. Continuing with the naming scheme adopted by this thesis, the phrase browser is called Re-Phine, in that both the phrase browsing and the subsequent searching are opportunities for the user to continually refine the initial query.

The remainder of this chapter is structured as follows. The steps embodied by the phrase browser are further elaborated in Section 7.1. In order to efficiently browse the phrase hierarchy, a graph data structure is required. This data structure is described in Section 7.2. A demonstration of Re-Phine is given in Section 7.3. Then, Re-Phine is

compared and contrasted with related work and traditional IR techniques in Section 7.4, including an assessment of the quality of phrases made available to the user.

7.1 Browsing and Searching Steps

The RE-PHINE system encompasses both phrase browsing and context searching. The precise steps that are available to the user in order to achieve these aims depend heavily on the seven streams produced during the compression process. Recall that the seven streams were the phrase hierarchy, the word lexicon, the non-word lexicon, the reduced word sequence, the case folding modifiers, the stemming modifiers, and the non-word modifiers.

Implementation details about RE-PHINE are covered in the next section. In this section, only a general overview of RE-PHINE is provided, starting with Figure 7.1. Each rectangular box represents an action performed within RE-PHINE. Boxes with dashed borders are actions performed by RE-PHINE, whereas solid boxes represent actions performed by the system in direct response to the user's request. A shaded cylinder represents one of the seven streams available to RE-PHINE, with dashed lines symbolising file access. The sequence of steps commences at the top left-hand corner of the figure. An explicit end to the sequence of steps is not shown because the user can stop phrase browsing at any time.

In the first step of the figure, RE-PHINE decodes the phrase hierarchy, the word lexicon, and the non-word lexicon, and maintains all three within memory. Next, a directed graph (DG) is built using the symbols of the phrase hierarchy, to permit efficient searching. Details about the structure of the graph is given in the next section. The third step is interactive phrase browsing. In browsing the phrase hierarchy, the user interactively traverses the DG until a symbol of interest, α say, has been isolated.

In the fourth step, the symbol of interest is used to identify all other symbols in the DG which contain α. When that search completes, a set of symbols $\mathcal{C}$ has been established. Next, $\mathcal{C}$ is input to the fifth step, in which the RE-VIEW$_{16}$ decoder is used to locate every context in which any symbol in $\mathcal{C}$ occurs. The RE-VIEW$_{16}$ decoder returns a result set, $\mathcal{R}$. Suppose a symbol β contains the symbol α as a component. In order to avoid reporting

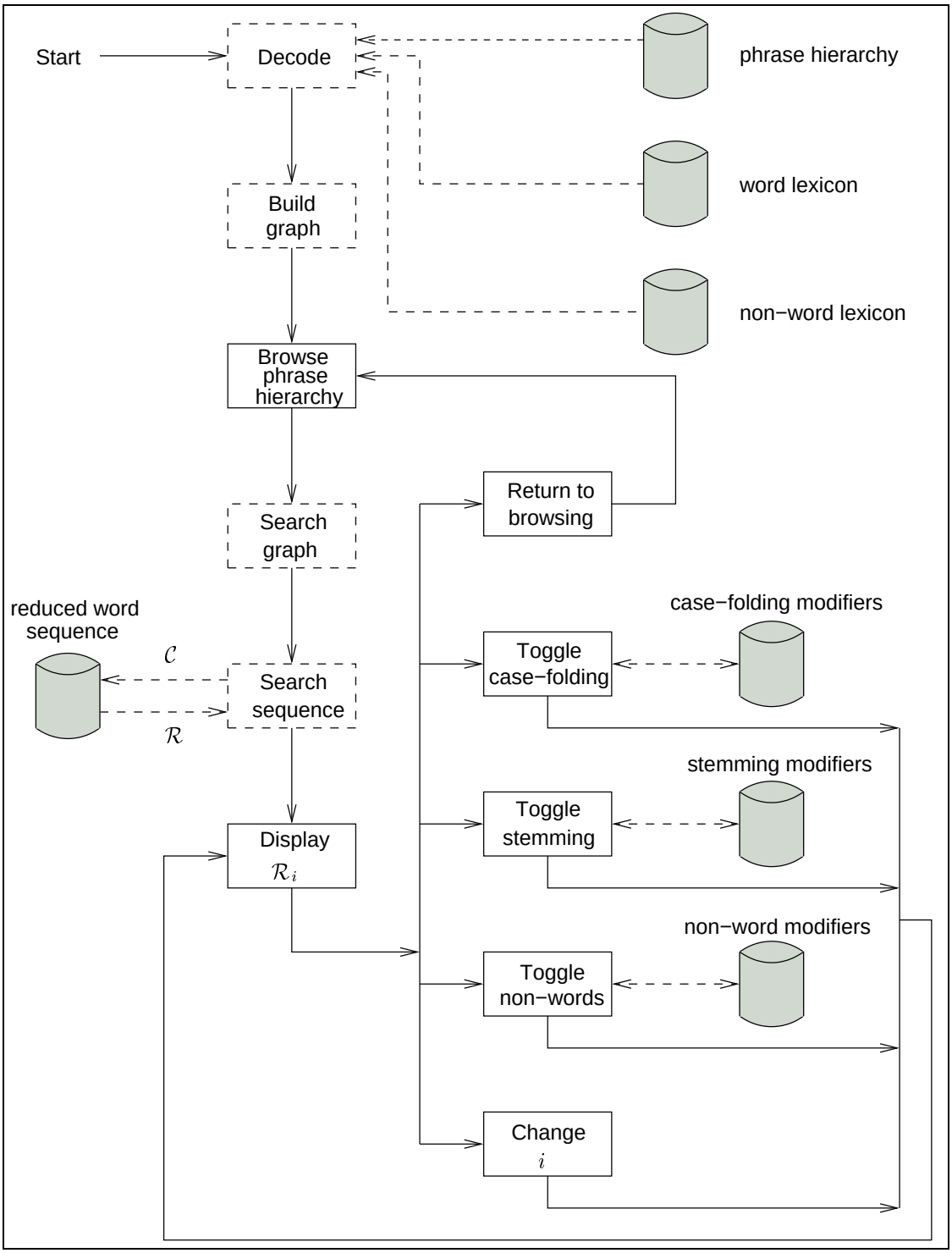


Figure 7.1: The sequence of steps performed by RE-PHINE during phrase browsing. Dashed boxes indicate actions performed by the system, while solid boxes require input from the user. Shaded cylinders represent one of the seven streams accessible by RE-PHINE, with dashed lines from them representing file access. The user can exit the system at any time.

the same location multiple times, when an occurrence of β has been found in the reduced word sequence, only β is reported. The size of $\mathcal{R}$ is denoted as $|\mathcal{R}|$, and members are enumerated from $i = 1$ to $i = |\mathcal{R}|$.

The first result's context, $\mathcal{R}_1$, is displayed to the user. At this point, the user may choose any combination of five actions. These actions are: return to phrase browsing; toggle case folding modifiers, stemming modifiers, or non-word insertion for $\mathcal{R}_1$; or examine the next or previous result. It is assumed that the result set is a circular list, so that the previous result to $\mathcal{R}_1$ is $\mathcal{R}_{|\mathcal{R}|}$. The three actions related to the modifiers initiate seek operations to the corresponding positions in their respective compressed streams. After the modifiers have been retrieved, $\mathcal{R}_i$ is re-displayed to the user. The application of modifiers can be toggled in any order. The first time modifiers are required, access to the streams are performed. Assuming requested modifiers are buffered in memory, subsequent requests do not need to access the file again. Finally, if the contexts that were obtained were inadequate in satisfying the user's need, any part of a context can be used as input into the phrase browsing part of Re-Phine.

Later in this chapter, the entire phrase browsing process is detailed through a brief example which demonstrates the choices available to the user. But first, in the next section, the directed graph required for phrase browsing is explained.

7.2 Phrase Hierarchy as a Directed Graph

Once the phrase hierarchy has been decoded, a directed graph (DG) is constructed. Every symbol in the phrase hierarchy is assigned to a separate node in the DG. Each node contains six directed edges connecting it to other nodes, as shown in Figure 7.2. In this figure, the symbols are based on character-based Re-Pair instead of punctuation-aligned Re-Pair in order to simplify the discussion. The current node is represented by the symbol "w|o", with the vertical bar (|) separating the symbol into its two components. Two of the six pointers are *child pointers*, and refer to a symbol's structure These pointers were described in Chapter 5 as part of phase 1 of Re-Merge, but were called components at that time. The remaining four pointers are collectively called *navigational pointers*.

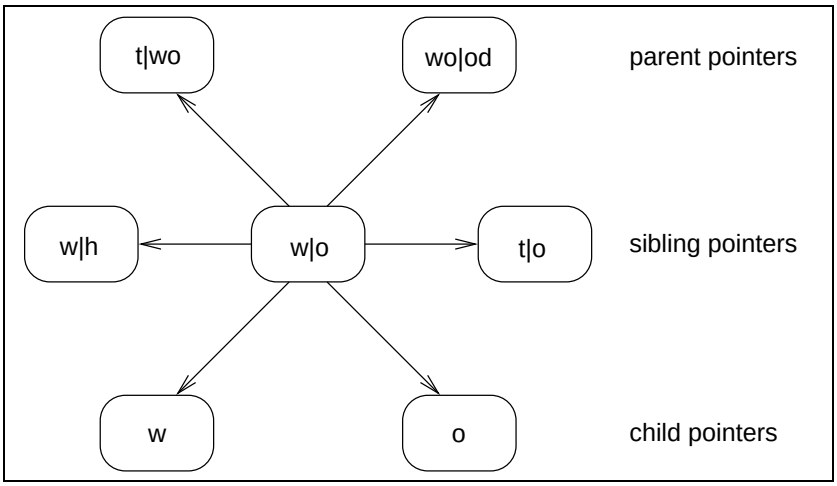


Figure 7.2: The six pointers used by Re-Phine to link nodes representing symbols in the phrase hierarchy into a directed graph. The symbols in the example are based on character-based Re-Pair, for illustrative purposes only. The node of interest is in the centre, with the symbol "w|o". A vertical bar (|) separates the expanded symbols into left and right components. Names of each pair of pointers are provided to the right of the figure.

Navigational pointers are added for the purpose of browsing and have not been required in the previous discussion.

The purpose of each pointer is illustrated in the figure. Pointers appear in pairs and are designated as being "left" or "right". The four navigational pointers are in two pairs – the *sibling pointers*, and the *parent pointers*.

In the example figure, the current symbol is in the centre, with its six pointers listed in full. All of the other nodes in the graph have six pointers each as well, but the outgoing edges are omitted in the figure for simplicity. The centre symbol is "w|o", with left and right components (or children) of "w" and "o", respectively. The left sibling symbol of "w|o" is another symbol which also has "w" as a left child. A chain is formed from the centre node, with all symbols with "w" as a left child being linked together with their left sibling pointers. Within a chain, the nodes are unsorted, so any node can serve as the front of the list. Each chain ends with a final *empty node*. The child pointers of all primitives also lead to empty nodes. Similarly, the right sibling pointer from "w|o", leads to another symbol with "o" as a right child.

One important operation is to obtain the entire chain when the current symbol is any

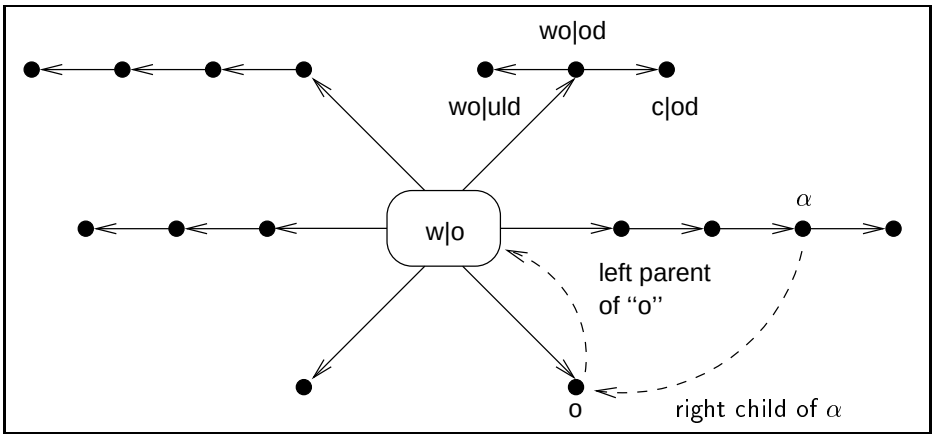


Figure 7.3: The directed graph structure created by RE-PHINE to link symbols together, at a larger scale than Figure 7.2. The node of interest is "w|o" in the centre. The dashed arrows shows how an entire chain of symbols can be obtained from an arbitrary symbol α within the chain. Also shown are two siblings of the symbol "wo|od".

symbol along that chain. The child and parent pointers help solve this problem. Parent pointers directly lead to symbols which are a single *extension* away from the current symbol. An extension occurs when a symbol is juxtaposed to the current symbol on either side, arriving at another symbol in the phrase hierarchy. For example, in Figure 7.2, the right parent pointer from "w|o" allows a right extension to the symbol "wo|od". Figure 7.3 shows Figure 7.2 at a larger scale to demonstrate how an entire chain of symbols can be obtained from anywhere in the list. The "w|o" is at the head of the list of right siblings, and the current symbol is α. Both of these symbols have the same right component. Every symbol along that chain has a right child link to the primitive "o". The front of the chain can be reached by taking the right child pointer from α, followed by the left parent link of "o", as shown by the dashed lines in the figure. This is possible provided that graph construction ensures that every node's parent pointer leads to the beginning of a chain of parents. If a symbol does not have a parent, the parent is represented as an empty node.

The graph structure created by RE-PHINE resembles the data structure employed by MK10 [Wolff, 1975] for the recursive pairing of symbols for discovering sentence structure. Each symbol in the dictionary created by MK10 has major and minor links. A major forwards link is the same as a left child pointer in RE-PHINE's graph, while a major

backwards link is identical to a right child pointer. Minor links operate in a similar fashion to the sibling pointers of Re-Phine.

Equipped with this graph, five operations are available to the user. Assuming the user's currently selected symbol is "w|o" of Figure 7.2 and Figure 7.3, the five operations are as follows. First, the user can decompose the symbol into its two components through the child pointers. Second, if the right parent pointer from "w|o" is used, followed by all left sibling pointers until the empty node is reached, then all right extensions of "w|o" can be retrieved. In Figure 7.3, this operation would obtain the set of two symbols, {"wo|od", "wo|uld"}. Likewise, a third operation is to extend "w|o" to the left. The union of these last two sets results in a fourth function, extending "w|o" in both directions.

The fifth and final operation is reserved for decoding, instead of phrase browsing. Decoding requires identification of all symbols which contain the current symbol α, either directly or indirectly. The located symbols are denoted in Figure 7.1 as $\mathcal{C}$. In order to find all symbols in the phrase hierarchy graph which contain α, a breadth-first search with the navigational pointers is sufficient. Let β represent a symbol which is an ancestor of α. If β has only one occurrence of α, then all of its parents does as well. But α must appear in either its left or its right component, which can be determined based on how β was reached. So, only one of the sibling pointers need to be followed. If β has multiple occurrences of α, with at least one in each component, then during the breadth-first search, β is reached twice. Care must be taken to ensure that when β is reached the second time, only the other sibling pointer is traversed, and not the parent pointers. If this strategy is followed, then the nature of the navigational pointers ensures that all symbols which include "w|o" are included.

Construction of the DG requires $O(|\Sigma| + |\rho|)$ time, for a phrase hierarchy of $|\Sigma|$ primitives and $|\rho|$ phrases. Since the chiastic slide and interpolative coding have been used to code the phrase hierarchy, the phrase hierarchy is already in increasing generation order, and within each generation, in increasing chiastic slide value order. As no symbol is in the same generation as either of its two components, constructing the graph from the first primitive symbol in sequential order results in graph construction resembling a bottom-

up approach. If the phrase hierarchy is stored in memory as an array, and if each of the six graph pointers are represented as array indexes, then locating a symbol's components requires $O(1)$ time. For each phrase, an insertion is performed into the appropriate parent pointers of each of its two components, so that each node is added at the front of its two chains of siblings in $O(1)$ time. That is, a single linear pass through the phrase hierarchy is sufficient to construct the graph.

The memory space required for the phrase hierarchy is linear in the number of symbols in the phrase hierarchy. The graph structure adds an additional 6 words of memory per symbol. Also, the phrase hierarchy may have every symbol expanded into primitives, or symbol expansion may be deferred until the symbol is required. The choice made depends which is more important: memory space or response time. Experiments demonstrating the advantages and disadvantages of phrase hierarchy expansion were reported in Table 3.2 on page 49. In the implementation of Re-Phine described in this chapter, a buffer of recently decoded symbols is maintained and consulted before decoding a symbol. This approach was also employed in Re-Pair's decoder, Des-Pair.

7.3 Phrase Browsing Example

The Re-Phine system performs two main tasks: phrase browsing and decoding symbol contexts for presentation. Examples of both tasks are presented in Figures 7.4 and 7.5. The aspects of Re-Phine related to the graphical user interface were implemented with version 2.0.2 of the GTK+ library[1].

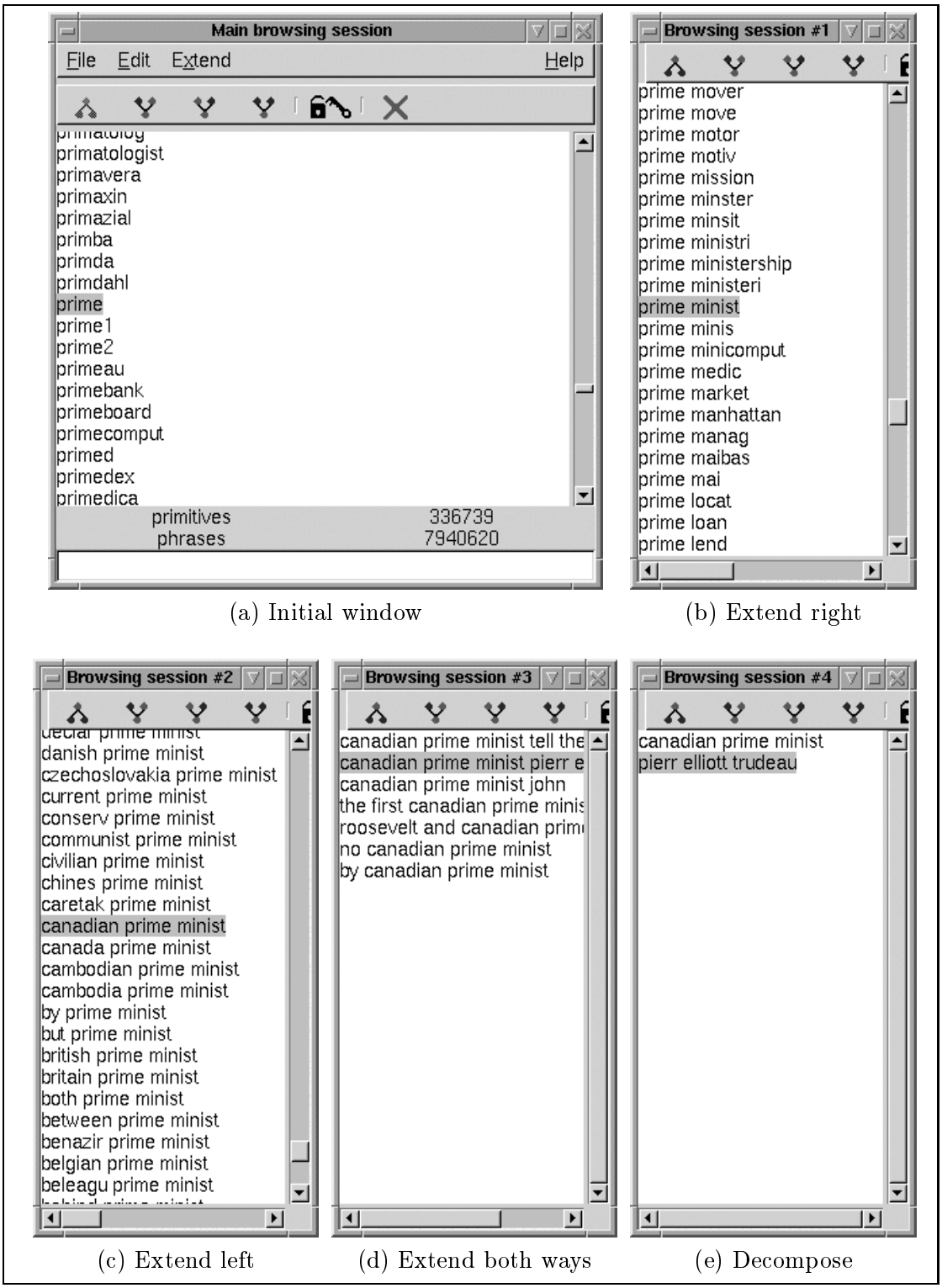
The data for the example is `NEWS`, processed with punctuation-aligned Re-Pair exactly as was described in the experiments of Chapter 5. The sequence of word tokens in those experiments were partitioned into blocks of 20,971,520 symbols each, and then merged using method E of Re-Merge (Section 5.6). Modifier and sequence coding were done with indexed shuff and Re-View$_{16}$ respectively.

In the previous section, the choices available to the user for phrase browsing from

[1] Available from `http://www.gtk.org/`.

the current symbol were discussed. The user also needs to be given a starting point to browse from. In RE-PHINE, the user is first presented with the document's entire lexicon, as shown in Figure 7.4(a). The extensive list can be daunting, and one solution is to filter words, discussed later in the chapter. In Figure 7.4(a), the user has selected the word "prime". The first symbol is extended to the right in Figure 7.4(b), and the phrase "prime minist" has been selected. While phrases in this window are ordered according to their locations in the DG, Jones and Paynter [1999] and Paynter et al. [2000] describe systems where phrases can be ordered according to the number of documents in which they appear, or their frequency in the collection. While details about the implementation of these two features are not available, it is expected that either the additional information is stored by the system, or calculated at run-time. Also note the variations on the phrase "prime minist" due to the different spellings of the second word. While stemming and case folding have reduced the size of the phrase hierarchy, it is still possible that two seemingly equivalent words stem differently. The chosen phrase is extended to the left and the phrase "canadian prime minist" has been highlighted in Figure 7.4(c). As this window shows, unfortunately, the words "Canadian" and "Canada's" stem to different words. The identified phrase of Figure 7.4(c) is extended in both directions, resulting in Figure 7.4(d) and its 7 phrases. Of these 7 phrases, 3 are right extensions and 4 are left extensions to the phrase "canadian prime minist". The selected phrase is broken down into its two components through its child pointers, as shown in Figure 7.4(e). This phrase browsing example commenced with the word "prime" and, in four steps, arrived at the name of a Canadian prime minister. In RE-PHINE, the refinement of a query term is done interactively with the help of the phrase hierarchy's graph.

A sample decoding session is illustrated in Figure 7.5. The selected phrase of Figure 7.4(e) is the starting point for the decoding. The first of 17 occurrences of the phrase "pierr elliott trudeau" is given in Figure 7.5(a). The results are presented to the user in the order in which they occur in the original document. In the first window, the phrase exists in a longer phrase, which has been marked. A fixed context of 10 symbols in either direction of the highlighted symbol has been selected for RE-PHINE. In the absence of



(a) Initial window (b) Extend right

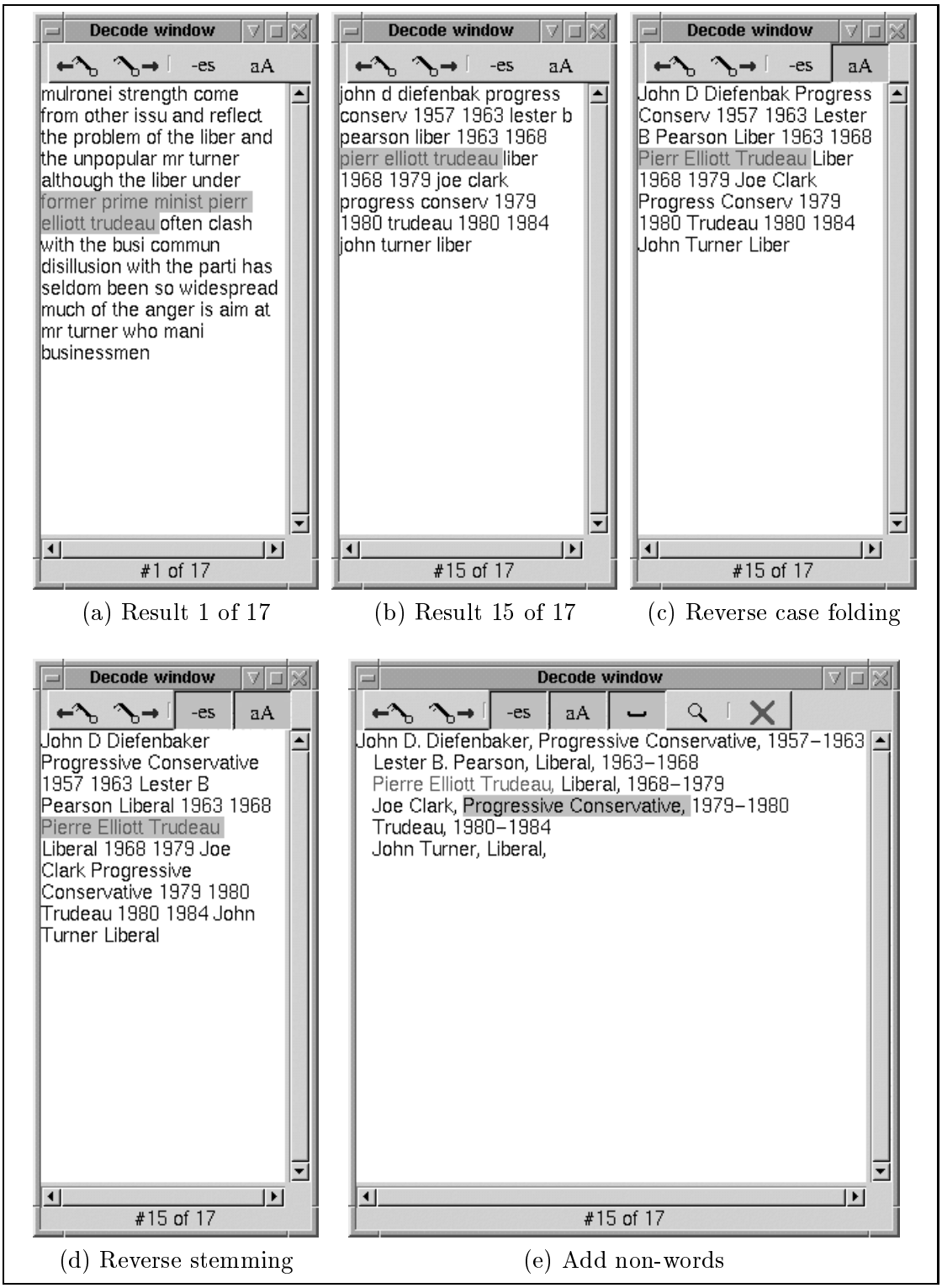
(c) Extend left (d) Extend both ways (e) Decompose

Figure 7.4: A sample phrase browsing session of the NEWS document.

detailed information about the non-words, a space character is inserted between each word token, with linefeeds added as appropriate. Even though the words in this window have been stemmed and case folded, reasonable guesses can be made with most of the words. In Figure 7.5(b), the user has skipped ahead to the 15th result. As this section of text may be of interest, the case folding is first reversed in Figure 7.5(c), and then the stemming modifiers are applied in Figure 7.5(d). The corresponding modifiers are located by the system and only the small number of required modifiers are transmitted to the RE-PHINE window. Modifiers can be toggled in any order, and allow the user to progressively change the displayed context. Finally, in Figure 7.5(e), the non-words are inserted into the text and the window is enlarged for improved display. The text exists in the original `NEWS` document as shown.

At this stage, the user's information need may be satisfied. Alternatively, the user may decide to select the phrase "Progressive Conservative" as the new starting point for phrase browsing. Then, the starting point for the browsing session becomes the phrase "progress conserv".

RE-PHINE applies the modifiers only when requested by the user. This approach takes advantage of the fact that the modifier streams are encoded separately. At least two other ways of handling modifiers are possible, which have not been implemented for RE-PHINE. If the cost of decoding and transmitting the modifiers is acceptable, the user can be given the option to reverse all modifiers by clicking one button. This feature can be added by making a small change to the RE-PHINE interface. Another option is to display initial word tokens with default stems and case information applied. That is, the most frequent stemming and case folding modifier is recorded for each word in the lexicon during pre-processing with PRE-PAIR. The additional disk space required is $2|\Sigma|$ integers, uncompressed, for a compressed document collection of $|\Sigma|$ primitives. When phrases are first displayed (for example, in Figure 7.5(a)), these default modifiers can be applied immediately without any intervention from the user. While the phrases presented may occasionally be linguistically incorrect, they should be valid most of the time. These default modifiers can be transmitted as needed, or once at the beginning of



(a) Result 1 of 17 (b) Result 15 of 17 (c) Reverse case folding

(d) Reverse stemming (e) Add non-words

Figure 7.5: A sample decoding session of the NEWS document, continuing from Figure 7.4(e).

the browsing session. A similar idea was adopted by Paynter et al. [2000]. The aim of both of these alternatives is to make the interface more user-friendly by minimising the amount of exposure users have to case folded, stemmed words.

7.4 Evaluating Re-Phine

Information retrieval systems are generally evaluated in terms of efficiency and effectiveness. Efficiency is measured as the amount of time required to return a response to the user. Effectiveness is an assessment of the extent to which the set of results satisfies the user's information need.

Suppose an IR system indexes the words in a collection of n documents, and that for some set of queries, the number of documents that are relevant is known. The effectiveness of the IR system for the query is determined by the number of documents the system returns which are relevant. Two standard methods of quantifying effectiveness are *precision* and *recall*. These and other methods of judging retrieval effectiveness are described by various sources, including Korfhage [1997], Baeza-Yates and Ribeiro-Neto [1999], and Witten et al. [1999].

Precision and recall are expressed on a scale from 0 to 1. Precision is the fraction of returned results that are relevant, while recall is the proportion of all relevant documents that are returned. If a user poses a query for which the set of relevant documents in the collection is r, and the set of documents returned by the system is a, then precision and recall are defined as follows:

$$\text{Precision} = \frac{|r \cap a|}{|a|} \tag{7.1}$$

$$\text{Recall} = \frac{|r \cap a|}{|r|} \tag{7.2}$$

Ideally, the effectiveness of Re-Phine should be evaluated in a similar manner. However, significant differences between Re-Phine and traditional IR systems prevent any straightforward comparison.

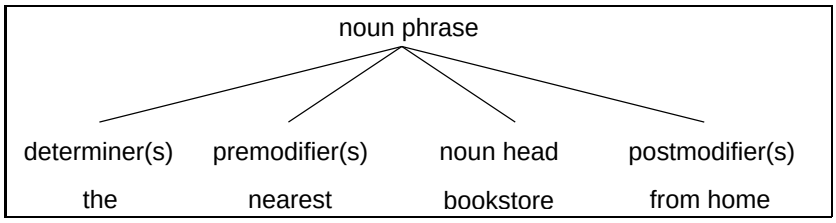


Figure 7.6: Definition of a noun phrase, adapted from Greenbaum [1996, pg. 209].

Instead, RE-PHINE is evaluated in three steps. In the first step, an overview of past work related to phrase browsing is summarised. Second, RE-PHINE is analysed in detail in the context of related work and traditional query methods. Some possible improvements to RE-PHINE are also proposed. Finally, the phrase hierarchy used by RE-PHINE is compared to noun phrases formed through natural language processing techniques.

Related Work

Previous work in the area of phrase browsing can be classified based on the method employed for drawing phrases from a document. Phrases can be obtained through the semantic meaning of the words using natural language processing techniques, or statistically, by applying a mechanism similar to RE-PAIR.

Most work with natural language processing (NLP) techniques are related to tagging words according to their parts-of-speech (that is, noun, verb, etc.) and forming phrases according to the rules of grammar. A significant amount of work has concentrated on *noun phrases*. A noun phrase is a linguistic unit. In it, a noun is both required and designated as the most semantically important word and, as a result, is assigned as the phrase's head. Figure 7.6 shows an example of a noun phrase, divided into its various parts.

While some retrieval systems treat a query as being made up of individual words, occasionally, a query should be seen as a set of one or more noun phrases. If a user posed the query "University of Melbourne", ideally, documents about other universities or the city of Melbourne are not of interest. In fact, the seemingly unimportant word "of" is crucial for this phrase, since "University in Melbourne" has a completely different meaning. Kirsch [1998] provided some insight into the Internet search engine Infoseek and

noted that most users submitted queries which were noun phrases and that queries were, on average, 2.2 words in length.

Several researchers have made use of noun phrases for their phrase-based IR systems. The CLARIT system [Evans and Zhai, 1996] identified both simplex and complex noun phrases for the purpose of improved precision and recall in information retrieval, without the need for training data. The difference between a simplex noun phrase and a complex noun phrase is that a simplex noun phrase omits all prepositional phrases (the postmodifier in Figure 7.6). Anick and Vaithyanathan [1997] developed the PARAPHRASE system which partitioned documents into clusters and summarised each cluster with noun phrases for the purpose of browsing. A variant of the Brill tagger [Brill, 1994] labelled the words according to their parts-of-speech. LINKIT by Wacholder et al. [2001] identified noun phrases, which were subsequently displayed by the INTELL-INDEX interface. A part-of-speech tagger was used to determine simplex noun phrases, which were then grouped according to their heads. As a result, phrases such as "the university bookstore" was associated with the phrase "a bookstore".

Extraction of keyphrases (document keywords in the form of phrases) with machine learning techniques was investigated by Frank et al. [1999] and embedded within a system dubbed KEA. Several interfaces have been built on top of KEA, including KNILES, PHRASIER, and KEYPHIND. KNILES and PHRASIER uses the phrases identified by KEA to link documents together via hyperlinks [Jones, 1999, Jones and Paynter, 1999]. Browsing phrases identified by KEA are supported by the KEYPHIND system [Gutwin et al., 1999]. KEYPHIND removes all but twelve phrases for each document in a collection and constructs three types of indexes with the phrases: a word-to-phrase index, a phrase-to-document index, and a document-to-phrase index. A user study was conducted which compared the usability of KEYPHIND to a traditional IR system based on ranked queries. The results of the study showed that KEYPHIND was preferred for certain browsing tasks, such as collection evaluation. A more recent user study [Jones and Paynter, 2001] with KEA and keyphrases looked at computer science papers and compared the phrases selected by KEA with those specified by the authors of each paper. The study's conclusion was that the

phrases selected by KEA were almost as good as ones chosen by the authors.

Phrase selection through statistical methods is more relevant to RE-PHINE than through semantic methods. The PHIND system of Nevill-Manning et al. [1997] selected phrases using the SEQUITUR compression mechanism, which is similar to RE-PAIR and was discussed in detail in Chapter 3. Like word-based RE-PAIR described here, SEQUITUR was modified to operate on word tokens in order to ensure word alignment. Punctuation marks were removed and words were case folded to lower case. Word stems were handled by the phrase browser so that if the word "book" was chosen, a search for the word with different endings was performed. Follow-up work on a browsing system built on top of SEQUITUR was described by Nevill-Manning et al. [1999]. In order to browse large document collections, a disk-based browsing system is proposed. The phrase hierarchy is placed on disk with an inverted index. Smaller auxiliary data structures are kept in main memory for accessing the phrase hierarchy.

Williams et al. [1999] describe an index structure called a *nextword index* which provides explicit support for phrase querying. This multi-level data structure is an index to pairs of words that appear in the document collection. The word lexicon forms the top-level of the index. Each word in the lexicon has a nextword list that, together, indicates the word pairs that exist in the collection. Every word pair has a position list as the third and final level in the nextword index. The position list specifies the documents and the document positions where the pairs of words occur. Subsequent extensions to this work include compaction techniques for improving storage requirements [Bahle et al., 2001a], and a description of how nextword indexes can be used for mono-directional phrase browsing [Bahle et al., 2001b]. The nextword index for a transformed version of `WSJ508` occupied 56% of the collection size. When a query phrase with more than two words is processed, it must be partitioned into pairs of words for look-up into the index. The position lists obtained using the word pairs need to be combined in order to find locations of the original, longer phrase. Bahle et al. [2001b, 2002] showed how the word pairs should be selected in order to optimise querying time, and then implemented a retrieval system which combined a nextword index with a word-level IR system. It was shown that the

addition of the nextword index provided more efficient retrieval when the first word is a common word.

Several methods combine semantics with statistics for phrase selection. Wolff [1980] tagged words according to their parts-of-speech, and replaced frequently occurring pairs of word classifications with a recursive pairing mechanism similar to Re-Pair. Paynter et al. [2000] improved on Phind by combining Sequitur with Kea. A Brill tagger was combined with Kea and applied to the documents in order to identify boundaries between noun phrases. Next, Sequitur was applied to combine words into phrases, with constraints enforced to prevent any crossing over the boundaries.

A separate, yet related area to retrieval with phrases is that of locality-based indexing (or proximity indexing [Baeza-Yates and Ribeiro-Neto, 1999, pg. 101]). Locality-based indexing creates an index at the granularity of individual word tokens instead of documents. When multiple terms are included in the query, the results are chosen not only because of query frequency, but also the distance between query terms within the documents. Some past works with locality-based retrieval include those of Buckley et al. [1995], Clarke et al. [1995], Hawking and Thistlewaite [1995], and de Kretser and Moffat [1999].

Analysing Re-Phine

The Re-Phine system of this chapter can be analysed in terms of its efficiency, effectiveness, and similarity and differences with other work. With respect to the efficiency of Re-Phine, the time required by decompression has already been presented. Experiments in Chapter 5 showed that the total decoding time for `NEWS` was 236.9 seconds, when shuff was used as the coder (Table 5.12 on page 149). For the purpose of Re-Phine, less time is required since only the phrase hierarchy and the two lexicons are fully decoded. Decoding the two lexicons required 0.8 seconds (Table 5.9 on page 147). The total time required for phrase hierarchy decoding, including graph construction by Re-Phine was 12.3 seconds, averaged over three trials on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache. Experiments in Chapter 6 showed that searching for symbols in the reduced word sequence encoded with Re-View$_{16}$ is fast, but depends on the number of

symbols searched for (see Figures 6.3 and 6.4 on pages 182 and 184, respectively). The values in these graphs provide a good estimate of the amount of time required to search a sequence coded with RE-VIEW$_{16}$. Finally, locating 100 positions in a stream of modifiers with indexed Huffman coding requires no more than 2.2 elapsed seconds if a block size of 32,768 bytes was chosen, shown in Table 6.4 on page 177.

In summary, 13.1 CPU seconds is required by RE-PHINE to decode the phrase hierarchy and the two lexicons. Approximately 10 elapsed seconds is necessary for RE-VIEW$_{16}$ to search the compressed sequence for 100 different symbols. The modifiers at a certain location in a stream can be found in less than 0.1 seconds.

It is more difficult to assess the effectiveness of RE-PHINE, or phrase browsing in general. The main feature of phrase browsing is the user-driven navigation between symbols in the phrase hierarchy. This interface experience cannot be easily quantified. If the searching phase of RE-PHINE is assessed using precision as the evaluation metric, then problems similar to those encountered with Boolean queries need to be addressed, since every result returned contains the symbol being sought, and precision is 1.0.

On the other hand, the proportion of relevant results (recall) can be measured, but in a slightly different manner than in the traditional definition. Unlike with a Boolean query or an exhaustive string search, recall is not guaranteed to be 1.0, because of the way in which the pairing process of RE-PAIR breaks up phrases. For example, suppose part of a message contains the five symbols: $\alpha\beta\delta\beta\delta$. Suppose also that the frequency of $\alpha\beta$ throughout the message is higher than the pair $\beta\delta$, and that the replacements made are ① $\rightarrow \alpha\beta$ followed by ② $\rightarrow \beta\delta$. With these replacements, the segment $\alpha\beta\delta\beta\delta$ reduces to ①δ②. As a consequence, if the symbol ② is selected during phrase browsing, then only the second occurrence of it in this segment is found. The first occurrence has been split by the replacement with symbol ①, and is not reported to the user.

Experiments were conducted with the first 20 MiB of `NEWS` (`WSJ20`), compressed with punctuation-aligned RE-PAIR, to evaluate the average recall. Unlike results from Table 4.12 on page 99, phase 1 of RE-MERGE has been applied, removing 16 duplicates in the phrase hierarchy. For each symbol in the phrase hierarchy, the frequency of the

Symbol length	Number of symbols in phrase hierarchy ($\mathcal{P}$)	Distribution of recall values		
		Median recall	Mean recall	Standard deviation
1	39,637	1.000	1.000	0.000
2	79,693	0.750	0.706	0.295
3	60,825	1.000	0.794	0.260
4	25,245	1.000	0.862	0.225
5	10,932	1.000	0.917	0.184
6	4,998	1.000	0.938	0.163
7	2,895	1.000	0.954	0.141
8	1,686	1.000	0.954	0.144
9	1,180	1.000	0.957	0.136
10+	4,454	1.000	0.955	0.136
Overall	231,545	0.975	0.904	0.168

Table 7.1: Recall of the symbols in the `WSJ20` phrase hierarchy. Median recall values for each symbol length are in the third column, followed by the mean, and the standard deviation.

symbol and all other higher generation symbols that contained it were compared with the actual frequency in the word sequence. The results from these experiments are presented in Table 7.1.

In Table 7.1, the 231,545 distinct symbols formed from `WSJ20` were separated into ten symbol groupings based on their expanded length, measured in word tokens. Since the zero-length word token for separating a long non-word token is of no interest, it does not appear in the table.

The second column of the table shows the size of each grouping. The majority of symbols were 2 or 3 word tokens in length. The remaining three columns indicate some gross statistics for the distribution of recall values for these symbols: the median, the mean, and the standard deviation. Generally, results were worse for symbols which represented 2 to 4 word tokens. For the remaining categories, the average recall was at least 0.9 and their corresponding medians meant that at least half of the symbols had a recall of 1.000. This result is further reinforced by the standard deviation ranging from 0.000 to 0.184. The average recall and standard deviations were less satisfactory for symbols lengths of 2 to 4. When the symbol length is 2, the median recall was only 0.750.

The results from the table can be interpreted as follows. Short phrases are more

difficult to accurately find than primitives and long phrases. For example, in `WSJ20`, the phrase "prime ministr", can be easily split depending on the words immediately before and after it. Note, however, that the most common phrase that includes each word is exactly the one most likely to have a high recall. And, as phrases get longer, since their underlying components are more frequent, RE-PAIR does not separate them.

One anomaly is not shown in the experiments with RE-PHINE. Even though precision across the set of symbols in the phrase hierarchy is always 1.0, not every symbol can be searched. In particular, while RE-PHINE is able to locate every symbol chosen through phrase browsing, many combinations of words do not exist in the phrase hierarchy and cannot be searched for. In Figure 7.4(c), the phrase "prime minist" was extended to the left, and one phrase found was "canadian prime minister". However, the phrase "canadian prime" does not exist in the phrase hierarchy, and therefore, cannot be searched for by RE-PHINE. The problem is the symbol pairing order employed by RE-PAIR (and RE-MERGE) makes no attempt to place every multi-symbol combination into the phrase hierarchy. This point is expanded further later in this chapter.

Phrases for the dictionary are selected by RE-PAIR and RE-MERGE based solely on symbol frequency, and in this sense, RE-PHINE is closely related to SEQUITUR's phrase browser, PHIND. Nevill-Manning et al. [1997] identified the phrase boundary problem with the phrase hierarchies produced by SEQUITUR; a problem which also affects the phrase hierarchies of RE-PAIR. The phrase boundary problem refers to two symbols which expand to identical strings, but have differing compositions. For example, it is possible that "canadian prime | minist" and "canadian | prime minist" become two separate symbols in the phrase hierarchy, creating no problems for compression, but confusing the browsing process. On the other hand, phase 1 of RE-MERGE explicitly addresses this problem when phrase hierarchies are merged. Recall that phase 1 expands each symbol into a string of primitives for comparison. Symbols that represent duplicate strings are removed.

Another drawback with RE-PHINE is apparent in the first screen of Figure 7.4(a), which presents a list of the word tokens in the document collection. Such an exhaustive list is similar to a Boolean or ranked query system which lists every word found in the collection,

Case fold	Stem	Unique primitives	Average length (chars)	Percentage of primitive extensions				Unique phrases
				Left only	Right only	Both	None	
No	No	533,205	8.00	10.2%	11.3%	18.5%	60.0%	8,464,282
Yes	No	448,278	8.10	9.6%	11.4%	18.8%	60.2%	8,269,436
No	Yes	401,448	7.34	10.3%	11.7%	18.2%	59.8%	8,147,217
Yes	Yes	336,739	7.46	9.7%	11.8%	18.1%	60.4%	7,940,620

Table 7.2: The effect case folding and stemming has on the phrase hierarchy created from punctuation-aligned Re-Pair for NEWS with method E of Re-Merge. The last row corresponds to the application of both case folding and stemming of word tokens, and reflects the arrangement used for the experiments with NEWS in this chapter.

and is analogous to a book index containing every word found in the book. The advantage of Re-Phine for phrase browsing is not the word lexicon, but the relationships formed between words in order to combine into longer phrases. Nevertheless, such an opening window is distracting to users. Subsequent lists also suffer from the same problem, but to a lesser degree. The words in the first window and phrases in the remaining windows of Figure 7.4 could be filtered so that unhelpful symbols are removed.

Case folding and stemming by Pre-Pair reduce the size of the word lexicon and phrase hierarchy, while also improving browsing by providing one word which represents several similar ones. Chapter 4 showed the effect stemming and case folding had on the the word lexicon of WSJ20 (Table 4.4 on page 87). Table 7.2 updates those results for NEWS. Recall that the rules for stemming only apply to words in lower case. In the third row of the table, if the part of a word in upper case is examined by the stemming algorithm, and no case folding is applied first, then the word would be left unchanged. For example, "Searching" stems to "Search", while "SearchinG" does not. The third column of the table shows the number of primitives in the phrase hierarchy, while the last column indicates the number of phrases. Case folding and stemming has an effect on the size of the word lexicon, confirming the results for the smaller WSJ20 file. Moreover, the average lengths of words in the lexicon are slightly longer than the values listed in Table 4.4, as indicated in the fourth column of the table.

In the fifth, sixth, seventh, and eighth columns of the table, the set of primitives has been divided into four groups based on whether or not each one can be extended. In all

cases, about 60% of the symbols do not participate as a component of a higher generation symbol. However, simply eliminating these symbols from the opening window of Re-Phine is too extreme. If a word token fails to become part of a phrase, it does not mean that the word is not interesting or is infrequent.

Nevill-Manning et al. [1997] noticed a similar problem with the phrase browser Phind. The solution adopted was to filter words and phrases based on word categories. Words were labelled as *rare*, *interesting*, or *common* based on a set of frequency thresholds. When the phrase hierarchy is explored, an expanded phrase is shown only if it differs from the previous phrase by at least one interesting word. For example, if "book" is an interesting word, but "the" is a common word, then the expansion of "book" to "the book" would not be displayed. The reason is that adding a common word to the original word does not add any value for the user. However, "book" can be expanded directly to "the new book", provided the word "new" is also an interesting word. The drawback of this heuristic is that a common word may still be of interest to users, and a rare word does not necessarily represent a typographical error.

At the centre of the problem with Sequitur, and Re-Pair as well, is that selecting phrases based on frequency generates too many phrases, with many differing by only a common word like "the". Paynter et al. [2000] proposed the solution of combining Sequitur with Kea, since Kea was known to select too few phrases by itself. Likewise, the combination of Re-Pair with a mechanism similar to Kea may improve the number of phrases presented by Re-Phine. A more detailed look at the phrases created by Re-Pair for Re-Phine is provided next.

Comparing Re-Phine with Noun Phrases

Another part of Re-Phine that is worth evaluating is the usefulness of the phrases presented to the user. The quality of the phrases is not related to precision or recall because the primary concern is the type of phrases available for browsing, and not the result from searching.

Wolff [1980] produced phrases through a recursive pairing mechanism called MK10. In MK10, pairs are chosen based on the part-of-speech categories assigned to the word tokens. The quality of the phrases was compared to the phrases selected by a human linguist. In a similar fashion, the phrases made available by Re-Pair can be compared with the ones identified by a reference program employing natural language processing (NLP) techniques.

As with other IR systems based on NLP techniques, the basis for these experiments is that a user is more interested in a complete noun phrase such as "prime minister" than a broken phrase like "Canadian prime". The aim of the experiments is to report on the probability that a symbol in the phrase hierarchy is a noun phrase, and the probability that noun phrases exist in the phrase hierarchy. Noun phrases are identified through an NLP technique called Link Grammar [Sleator and Temperley, 1991] embodied in a program by the same name, available from `http://www.link.cs.cmu.edu/link/`. The latest version of the software is 4.1, dated August 2000.

The Link Grammar system processes a sentence at a time. In the first step, individual words are assigned to part-of-speech categories by consulting a static dictionary. The categories of novel words are guessed through a technique the authors call "morpho-guessing". Words have links to both sides for combining with other words in the sentence. For example, the two words "the book" can merge because of a right-facing "D+" link on the word "the" and a corresponding left-facing "D−" link on the noun "book". The "D" in both cases symbolise a determiner such as "the" and "an". An edge is drawn between these two words to indicate that they are compatible with each other. When a grammatical sentence is correctly parsed, a valid *linkage* is said to be formed. Several requirements must be satisfied in order to obtain a linkage. One of these requirements is that the graph formed must be connected, with all of the words in the sentence reachable.

Subsequent work on the system [Grinberg et al., 1995] permits *null links* and post-processing of valid linkages with a phrase parser for identifying constituents such as noun phrases and verb phrases. If a sentence cannot be successfully parsed, then words in the sentence are removed incrementally until a successful parsing can be found. When the

sentence is finally parsed, each removed word introduces a null link into the linkage. When a linkage for a sentence has been established, the post-processing phrase parser combines words to form constituents of one or more words. On-line documentation for the phrase parser[2] reports that the constituents determined by the parser were 75% correct on the Penn Treebank text[3], which is newspaper text already annotated linguistically.

A sample parsing of a headline from `WSJ20` is shown in Figure 7.7. In the figure the headline is shown in part (a). The LINK GRAMMAR system considers it a valid sentence since two linkages are identified, as shown in parts (b) and (c). In Figure 7.7(a), the "G" link is used to pair "South" with "Korea". A "G" link connects proper nouns together, identified by the upper case letters at the beginning of each of these words. Similarly, a "G" link is inserted between "Current" and "Account". The second linkage case folds "South" so that the first "G" link is converted to an "A" link. An "A" link connects a adjective with the noun it follows. The "Ss" link pairs a subject with a verb, while "Ost" edges connect a transitive verb to a singular object. So, LINK GRAMMAR incorrectly treats "'s" as a verb in both linkages. Linkages are scored so that ones with less null links or a lower edge length total are displayed first. The constituents for the linkage in (b) is presented in Figure 7.7(d). At the beginning of each constituent is its classification. The abbreviation "VP" denotes a verb phrase, while "NP" is a noun phrase.

As with previous work in IR and NLP, the focus of the phrase comparison is with noun phrases. The number of unique phrases that LINK GRAMMAR found in `WSJ20` was compared with RE-PHINE's phrase hierarchy. Since only grammatical sentences can produce linkages, the test data of news articles were filtered so that sequences of words that are not expected to give many noun phrases were omitted. First, SGML tags were removed and multiple consecutive whitespaces collapsed into a single space. Sentences were given to LINK GRAMMAR one at a time. Then, the constituent analysis was determined by the phrase parser.

As with the earlier experiments to measure recall, the `WSJ20` document was used as

[2] Available at `http://www.link.cs.cmu.edu/link/ph-explanation.html`.
[3] The Penn Treebank project is at `http://www.cis.upenn.edu/~treebank/`.

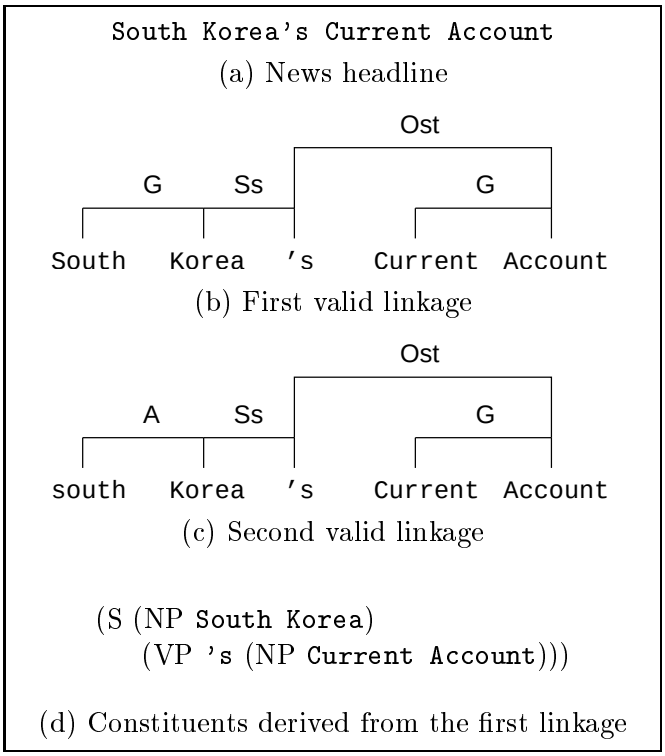


Figure 7.7: An example parsing of a headline from WSJ20 using the Link Grammar system. The headline is shown along the top as part (a). Two linkages are determined by Link Grammar, which are shown in (b) and (c). The linkage from (b) is post-processed for constituent identification, yielding the output in (d). Noun phrases are marked with "NP".

test data. The computing resources of the Link Grammar program were quite high. It required just over 5 hours to process the first 1 MiB of WSJ20 on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache test machine. This time included the above steps, starting from the SGML tag filtering, and ending with the identification of constituents. Note that the amount of work performed by Link Grammar is more than what is necessary to identify noun phrases. Other systems, such as the Keyphind browser of Gutwin et al. [1999], simply categorise words according to their parts-of-speech, and then form noun phrases based on a set of fixed rules. Even so, Gutwin et al. required 4 days to process 1 GiB of text on a Pentium 233 MHz machine running Linux. In contrast, Link Grammar must first successfully find a linkage for a sentence, and only when this is done, can it isolate the noun phrases.

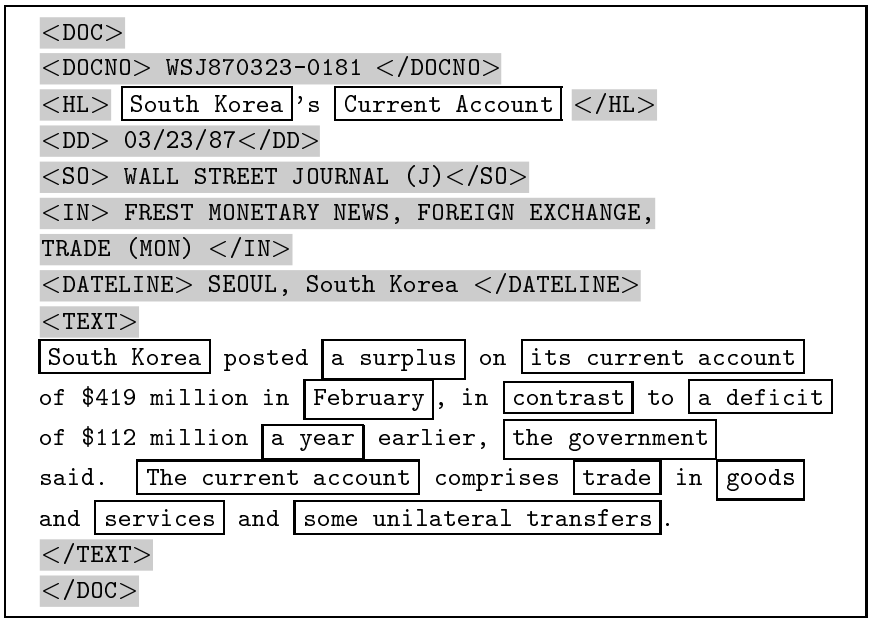


Figure 7.8: An example article from the WSJ20 document, with noun phrases indicated with outlined boxes. Shaded boxes represent parts of the article that were removed prior to applying Link Grammar. The article has been formatted with linefeeds for improved display.

After Link Grammar completes, constituents which are noun phrases, and are not composed of other phrases, were recorded. The noun phrases were case folded and then stemmed with the Porter stemming algorithm employed by Pre-Pair. In order to comply with the 16 character per word limitation imposed by Pre-Pair, spaces were inserted into long words. Finally, a list of all of the distinct noun phrases in WSJ20 was prepared.

Figure 7.8 illustrates how a news article in WSJ20 was parsed, with linefeeds altered to improve display. Text that was stripped from the article is shown shaded. All SGML tags are removed, as well as text within most tags. Only text between opening and closing <HL> and <TEXT> tags were retained, since the headline and the body of the article has the best chance of providing complete sentences. The noun phrases found by Link Grammar for this example are shown in boxes. The headline used as an example in Figure 7.7 was taken from the article shown in Figure 7.8.

The Link Grammar system successfully found 147,294 grammatical sentences in WSJ20. Within these sentences, 744,721 noun phrases were identified, of which 216,661 were unique after stemming and case folding. These unique phrases were divided into

Phrase length	Number of phrases in hierarchy ($\mathcal{P}$)	Number of unique noun phrases (NP)	Size of $\mathcal{P} \cap$ NP	Probability $\mathcal{P}$ has noun phrase	Probability noun phrase in $\mathcal{P}$
1	39,637	13,172	13,172	0.332	1.000
2	79,693	78,527	21,692	0.272	0.276
3	60,825	72,492	6,350	0.104	0.088
4	25,245	32,419	1,578	0.063	0.049
5	10,932	12,304	397	0.036	0.032
6	4,998	4,939	144	0.029	0.029
7	2,895	1,696	32	0.011	0.019
8	1,686	662	8	0.005	0.012
9	1,180	247	2	0.002	0.008
10+	4,454	203	1	0.000	0.005
Total	231,545	216,661	43,376	0.187	0.200

Table 7.3: Quantifying the quality of symbols in the phrase hierarchy of `WSJ20` by calculating the intersection of noun phrases, as identified by Link Grammar, with the phrase hierarchy. Symbols have been divided into 10 groups, based on length (in words). The second and third columns provide insight into the number of symbols (or noun phrases). The intersection of these two columns are reported in the fourth column. The last two columns indicate the proportion of phrases in $\mathcal{P}$ which are noun phrases (fourth column divided by the second column), and the probability a noun phrase also being in $\mathcal{P}$ (fourth column divided by the third column), respectively.

groups based on their lengths, in words, and compared with the phrases available to Re-Phine, which were also grouped according to length. The phrases for Re-Phine are identical to the ones reported in Table 7.1. Information about these two sets of phrases are shown in Table 7.3. The sizes of the sets of phrases are listed by lengths in the second and third columns. Note that noun phrases are only found if a valid linkage has been determined. In the fourth column of this table, the number of phrases common to the phrase hierarchy and in the set of noun phrases has been tabulated. The probability of a symbol in $\mathcal{P}$ being a noun phrase is given in fifth column. The reverse, the probability of randomly selecting a noun phrase which is in the phrase hierarchy, is listed in the last column of the table.

As Table 7.3 shows, the majority of phrases in the phrase hierarchy and noun phrases from Link Grammar are 2 or 3 words in length. The overlap between the two sets is around 28% for phrases of length two for both measures, a somewhat disappointing result. Worse, as the phrases increase in length, there is even less correlation with the phrase

hierarchy, as the last two columns show. This difference is partly due to the methods used by Re-Pair and Link Grammar to build phrases. The Link Grammar system operated on sentences individually, so phrases were guaranteed to be aligned on sentence boundaries. Also, SGML markup did not appear in any of the sentences, because of the preliminary filtering. On the other hand, while punctuation-aligned Re-Pair encourages alignment with punctuation, phrases can still span sentences, since they are not processed individually. Furthermore, since Re-Pair was applied on the original version of `WSJ20`, any phrase that contained markup would not have a corresponding phrase from Link Grammar. The analysis in Table 7.3 is based solely on the existence of phrases, and takes no account of the frequency. Because Re-Pair finds common phrases, it is likely that the scores would be considerably better if the probabilities were assessed over the sequence instead of over the phrase hierarchy.

Making phrases available in the phrase hierarchy is only half of Re-Phine's purpose since matching contexts must also be located in the reduced sequence. Using the same methodology as established for Table 7.1, the recall of the phrases that exist in both the phrase hierarchy and the list of noun phrases is presented in Table 7.4. This time, rather than presenting recall values for all symbols in the phrase hierarchy, only the ones that were also noun phrases identified by Link Grammar are considered. The second column of this table lists the number of noun phrases that were also in the phrase hierarchy, copied from the fourth column of Table 7.3. In Table 7.4, as with the earlier experiment, the median recall value for each phrase length group is reported. The mean and standard deviation of each set of recall values is also shown.

The results from Table 7.4 mirror those of Table 7.1, but note the small size of the sample for phrases of length 7 or more. Again, the average recall is above 0.7, and more than half of the symbols had perfect recall. The standard deviation is wider and affects more groupings than in Table 7.1.

Phrase length	Size of $\mathcal{P} \cap$ NP	Distribution of recall values		
		Median recall	Mean recall	Standard deviation
1	13,172	1.000	1.000	0.000
2	21,692	0.800	0.720	0.298
3	6,350	1.000	0.787	0.271
4	1,578	1.000	0.845	0.242
5	397	1.000	0.831	0.264
6	144	1.000	0.849	0.250
7	32	1.000	0.853	0.258
8	8	1.000	1.000	0.000
9	2	1.000	1.000	0.000
10+	1	1.000	1.000	0.000
Overall	43,376	0.980	0.889	0.158

Table 7.4: Recall effectiveness of the noun phrases in the phrase hierarchy for `WSJ20`. The median has been calculated based on phrase length (in words) in the third column, with the mean, and the standard deviation indicated in the last two columns.

7.5 Tools for Retrieval

The Re-Phine system allows users to perform two inter-related tasks. The first of these is phrase browsing. Starting from a list of the distinct words found in the document, the user can either extend the current symbol by adding symbols to the left or to the right, or decompose the current symbol into its two constituents. A graph structure with one symbol from the phrase hierarchy at each node was established for the purpose of phrase browsing. Each node has six pointers for navigation and defining its structure.

Once the user has found a phrase of interest, the graph structure is consulted again to determine the complete set of phrases containing the chosen one. This set of symbols, $\mathcal{C}$, is the basis for the search in the reduced word sequence to determine all of the locations at which any symbol in $\mathcal{C}$ appears, along with a suitable context for each. The second job of Re-Phine is to interactively display this set of results one by one to the user. Each result consists of word tokens which have been stemmed and case folded. The non-words that follow each word token are also initially unavailable. Each presented result can be refined by the user by requesting modifiers from the three modifier streams, in any combination. If at this stage, another symbol of interest has been isolated, it can be passed back to the

phrase browsing part of Re-Phine.

In this chapter, Re-Phine has been demonstrated with a document collection 1,000 MiB in size on a 933 MHz Pentium III with 1 GiB RAM and 256 KiB on-die cache. Before considering larger collections, note that only three of the seven streams available to Re-Phine must reside in memory. The four remaining streams are compressed in blocks so that only parts of them are read from disk into memory. The phrase hierarchy and the two lexicons occupy 1.8% of disk space compared to the size of `NEWS`, and all of them must be accessible by Re-Phine in decoded form. The complexity and size of the phrase hierarchy ensures that it requires the most memory during browsing after it has been fully decoded.

Based on these observations, several approaches may be selected in order to browse larger collections. First, Section 5.7 showed how Re-Merge can be extended so that documents are appended to a compressed representation while keeping the phrase hierarchy static. Experiments demonstrated this technique with character-based Re-Pair, and an analysis of the changes necessary for punctuation-aligned Re-Pair was discussed. In particular, a method of handling novel words is required. Second, either the maximum generation or the maximum symbol length can be limited so that the phrase hierarchy only contains moderately long phrases. Third, while all word tokens must exist in the phrase hierarchy as primitives, additional heuristics, such as those offered by Kea and Link Grammar, may be used when building the phrase hierarchy. Instead of reducing the number of phrases shown by Re-Phine, this approach would prevent certain symbols from being added to the phrase hierarchy. Finally, any combination of these three methods can be used in order for larger document collections to be compressed by Re-Pair and Re-Merge, and subsequently browsed.

The Re-Phine implementation has not reached the stage where it could sensibly be made the subject of a user study. As with Sequitur, too many symbols exist in Re-Pair's phrase hierarchy, and some filtering is necessary to improve usability. The removal of symbols through NLP techniques has been trialled by other IR systems, and it is likely that Re-Phine would be more feasible if similar techniques could be employed. An

extension of this idea is to rank symbols using some metric so that those that are expected to be the most useful are presented at the top. Since there are reasons why users would want frequent and and infrequent phrases, simply sorting symbols by frequency would not help.

User studies have been conducted to quantify the usefulness of phrase browsing retrieval systems [Gutwin et al., 1999, Jones and Paynter, 2001], with some results being in favour of phrase browsing, and some less conclusive. Wacholder and Nevill-Manning [2001] concluded that more work is still required in the area of phrase browsing evaluation with users. But even if the suggested changes were adopted for Re-Phine, it is expected that a comparative user study would not favour Re-Phine. While providing users with a phrase browser was one of the aims of this development, the necessary tension between browsing and compression inevitably means that Re-Phine is not as careful as a dedicated phrase searching mechanism. One necessary improvement appears to be achieving higher recall. It is possible that the solution lies in closely regulating word-based Re-Pair's phrase selection heuristic. As word-aligned Re-Pair showed, preventing certain pairs of symbols from joining does not necessarily degrade compression effectiveness (see Figure 4.5 on page 103).

The benefits to compression are easier to quantify than the benefits of phrase browsing. Since the latter quantity cannot be measured, it is also difficult to assess when a reasonable compromise between the two has been achieved. Instead, the goal of this thesis has been to identify areas where compression and retrieval effectiveness and efficiency are lost, so that one can be adjusted in favour of another, as necessary.

Re-Phine is built on top of a compression mechanism, a structure which limits its ability to browse and retrieve data. Throughout this thesis, the amount of compression effectiveness lost in favour of faster retrieval times has been highlighted as a means of compromising between the two competing sets of constraints. In this chapter, the current drawbacks with Re-Phine represent the amount of retrieval functionality lost in favour of supporting the lossless compression of a document collection with the underlying mechanism. Employing NLP techniques with Re-Phine may be the ideal solution for

compensating for RE-PHINE's loss in retrieval effectiveness. As mentioned above, at the same time, NLP techniques may also reduce the number of symbols that reside in memory during browsing.

Regardless of the improvements made to RE-PHINE, phrase browsing in general is one of many IR techniques. This chapter has also provided a glimpse into other querying techniques, including ranked queries, proximity queries, and Boolean queries. The system a user chooses depends on factors such as how familiar the user is with the document, whether query terms include phrases, and whether the order and the positions of the words in a phrase matter. As Gutwin et al. [1999] pointed out, no single IR method is perfect and the most viable solution might be to provide "several search tools that can make up for each others' shortfalls".

This chapter concludes the discussion in this thesis about the balance between compression and phrase browsing. Before summarising the thesis in Chapter 9, the next chapter discusses a separate topic related to the compression of small HTML files.

Chapter 8

Compression of HTML Documents

This chapter deals with the compression of documents published on the World Wide Web using the HyperText Markup Language (HTML) [Raggett et al., 1999]. While the HTML standard is a subset of the SGML standard, HTML documents have many similarities with the SGML documents considered for most of this thesis. The `NEWS` document is 1,000 MiB in size, but artificially constructed by concatenating news articles together. When the compression systems outlined in previous chapters were applied to `NEWS`, the best compression ratio achieved was 1.674 bpc for PPMD (Table 2.5 on page 41).

Typical HTML files cannot be simply concatenated together and compressed if they are to be accessed on an individual basis. While HTML files are as brief as short news articles, news articles do not form relationships with other articles, except for their chronological order. In addition, "flattening" the HTML files on a web server into a single document would lose the intricate hierarchical structure created by hyperlinks. On the other hand, most compression systems attain better compression effectiveness as more text is available as context.

In the case of the smaller HTML files, context exists through hyperlinks. This chapter describes an investigation into a scheme under which HTML files from a medium-sized web server are compressed for storage and transport by exploiting the way they are accessed, either with or without hyperlinks.

The rest of this chapter is laid out as follows. In Section 8.1, related work by other researchers is summarised. Then, in Section 8.2, an overview of the data collected from the web server is given. In the context of this data, the experimental framework is described in Section 8.3, followed by the results in Section 8.4. Some conclusions about this work is drawn in Section 8.5.

8.1 Related Work

The two areas covered by this chapter are the reduction of bandwidth through compression and the detection of context in web pages. Both of these areas have been studied extensively by others.

Several techniques have been devised which reduce the bandwidth requirements of HTML documents. Delta encoding (for example, see Mogul et al. [1997, 1998]) assumes that users usually request a recent version of a previously downloaded file. Instead of re-sending the entire file, a difference between the latest file and the version of the file in the client's cache is sent. Nielsen et al. [1997] considered the benefits of lossy compression of HTML files by case folding tags without affecting the web browser's ability to render the document. Nielsen et al. also demonstrated that high-level compression of documents with the ZLIB library [Gailly and Adler, 1995] was more effective than the compression systems built into the set of modems that they considered.

On the World Wide Web, the current version of the HyperText Transfer Protocol (HTTP/1.1) [Gettys et al., 1999] supports compression through two header fields. The `Accept-Encoding` field allows the browser to notify the web server of its support for one of a set of encodings, as defined by the HTTP/1.1 standard. These encodings are primarily used for compression, and include both GZIP and DEFLATE. If the web server is able to support the specified encoding, then a compressed version of the requested document is sent to the client, and the `Content-Encoding` header field is embedded within the response.

Support for these headers have been implemented by the `mod_gzip` module[1] for the Apache web server[2], used by approximately 63% of the web sites on the Internet, according to a September 2003 survey conducted by Netcraft[3]. The current version of `mod_gzip` (1.3.26.1a) delivers compressed documents from a cache, but can recompress files during a request when newer versions are found.

With respect to hyperlinks, Brin and Page [1998] applied hyperlinks to IR by using them to judge the relevance of documents on the Web. Mizuuchi and Tajima [1999] established a set of rules to determine the most likely path a user would take to reach a certain HTML file. Li et al. [2000] later applied a set of rules to a web site in order to divide it into a set of logical domains, with each domain relevant to the same topic.

The experiments in this chapter are concerned with the compression of HTML files based on the order in which they are accessed by users.

8.2 Data Overview

Rather than developing a compression mechanism embedded within multiple web browsers and a web server, a simulation was performed. The simulation made use of access logs and web pages from the medium-sized web server of the University of Melbourne's Department of Computer Science and Software Engineering[4]. The web server used version 1.1 of HTTP.

Access logs lasting one week were taken, starting from 12.15 a.m. on 17 July 2000 until 12.15 a.m. on 23 July 2000. The access logs are in the Apache Common Log Format[5]. Each entry in the log records the client that accessed the server, the file requested, and the date and time of the request. While version 1.1 of HTTP supports persistent connections, the log file does not indicate this. Because of this, it is not possible to determine from

[1] `http://sourceforge.net/projects/mod-gzip/`
[2] `http://httpd.apache.org/`
[3] `http://news.netcraft.com/`
[4] `http://www.cs.mu.oz.au/`
[5] Described at `http://httpd.apache.org/docs/logs.html`

File type	Number of requests	Total volume (MiB)	Average (KiB)
Local requests			
HTML text	55,976	626.4	11.5
images	20,884	73.6	3.6
other	1,871	76.4	41.8
External requests			
HTML text	48,343	443.1	9.4
images	83,585	304.5	3.7
other	5,739	637.4	113.7
Total	216,378	2,161.5	10.2

Table 8.1: Breakdown of requests by origin and file type, 17 July 2000 to 23 July 2000 inclusive.

File type	Number of files	Total size (MiB)	Average (KiB)
HTML text	68,361	721.0	10.8
images	16,840	300.0	18.2
other	17,513	2,233.8	130.6
Total	102,714	3,254.8	32.4

Table 8.2: Snapshot of a web site, 24 July 2000.

the log file whether two consecutive requests from the same client was made by the same person, or by two different people using the same computer.

To prepare these logs for our investigation, the entries in the access logs were separated based on whether the client was local or external to the department, and then, whether the document requested was an HTML file, an image, or another file type. The results from this breakdown are summarised in Table 8.1.

A snapshot of the web site was taken on 24 July 2000, soon after the access logs were acquired. The snapshot was also split based on file type, yielding Table 8.2. For each file type, the number of files, the total file size, and the average file size are indicated.

The data described in these two tables forms the basis of the simulation. Almost 40% of the total requests were made from within the same network as the web server. Since transmission costs of local requests are relatively less than external requests, these

local requests have been omitted from the simulation. Furthermore, as the topic of this investigation is HTML files, the 443.1 MiB of HTML files requested from an external client is the focus of the work. The remaining externally requested files are used in the simulation, but are of lesser importance.

The size of the HTML files requested reflects their average size on the web server. In both cases, the average HTML file size was around 10 KiB. According to Table 8.2, most of the files on the web server were HTML files, but they occupied less than one quarter of the total disk space. The files labelled "other" include software packages (often in large archives) and documents created in proprietary file formats.

8.3 Experimental Framework

The purpose of the simulation was to establish the extend to which compression effectiveness could be improved, using the logs of requested files as an indication of typical access sequences. Suppose x^t, y^t, and z^t are HTML files at time t. If a user obtains x^i and then y^j, where $i < j$, then the straightforward approach to achieving bandwidth reduction is to compress each file individually. However, after x^i has been transmitted, the web server can improve the compression of y^j by making use of x^i, a file that is now assumed to exist on both sides of the communications channel. The compressed representation of y^j given x^i (denoted as, $c(y^j|x^i)$) is transmitted instead of $c(y^j)$ alone. The compression system is said to be "primed" with x^i before y^j is processed.

Because relatively little structure exists in the access logs, heuristic approximations have to be made with regard to whether two requests from the same client were made by the same user[6]. Also, a limit on the difference between the times i and j was imposed in order to reduce the chance that the first file, x^i, had not changed. A time interval, τ, was introduced into the simulation, which is the most time a user can take to read a document and move on to the next one. A *traversal* occurs when a client visits x^i and then y^j, provided the difference between i and j is constrained to be less than τ. In the sequence

[6] On a computer where all users share the same cache, this distinction between clients and users is less relevant, though.

of web page accesses by a client, if the interval between any two consecutive requests is less than τ, then a *browsing session* for that client has been found.

There are two subtle points to note with respect to these definitions. A valid traversal does not require a hyperlink to be followed, even though the motivation for this work was the hyperlinks within HTML files. This simplification is required because the access logs only indicate the files that were accessed, and not whether they were accessed with hyperlinks, bookmarks, or by manually entering them into a browser. Furthermore, the files within a browsing session can be of any type, but the experiments later are concerned with compression of HTML files within the session. So, the compression of y^j can be primed with x^i, even though many non-HTML files may be sent between times i and j. The only restriction is that no more than τ minutes has passed between any two adjacent file requests between x^i and y^j.

In the experiments of this chapter, τ was fixed at 5 minutes. The access logs were separated into browsing sessions based on this value of τ, and in Figure 8.1, only HTML files are reported. The graph indicates the percentage of HTML file requests for various browsing session lengths, measured in number of HTML documents. There were 21,067 browsing sessions found, and with 48,343 HTML files requested overall, with each session containing 2 HTML files, on average. Approximately 30% of the requests were to one-off documents with no traversal prior or following. The remaining 70% of the requests were part of browsing sessions with other HTML files. In one browsing session, there were 2,163 HTML files requested, presumably as part of a search engine crawl. As for the suitability in choosing 5 minutes for τ, if the threshold was extended to 60 minutes, then the percentage of page requests that were part of a browsing session with only one HTML file falls from 30% to 20%. Given this result, it would seem that 5 minutes is a plausible value.

Instead of using the most recently sent HTML file for priming, the web server and the web browser can also share a file made exclusively for priming. Of course, this priming file must be sent by the web server to every new client which accesses it, and the cost of its transmission must be accounted for in any analysis.

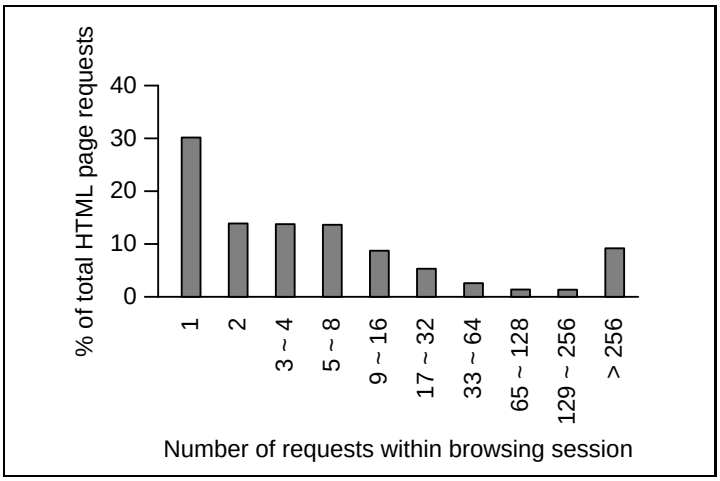


Figure 8.1: Percentage of page requests as a function of browsing session length, in number of HTML files, for $\tau = 5$ minutes.

Of the compression mechanisms discussed in this thesis, GZIP was selected as the basis for these experiments for two reasons. First, earlier results in Chapters 2 and 3 have shown that GZIP is relatively fast compared to other systems surveyed – an important factor when the user is waiting in front of a web browser. More importantly, the sliding window mechanism employed by GZIP makes priming possible with little modification, in comparison to other systems. The GZIP system simply compresses the document after the window of phrases has been initialised with the priming text. This method of priming has also been suggested by Witten et al. [1999, pg. 391] for algorithms related to LZ77. Also, priming was investigated with more complex compression systems by Teahan [1998].

There is a one important difference between the priming text for GZIP and the training data employed by RE-MERGE when appending text (see Section 5.7 from page 150). The phrase hierarchy is not altered by RE-MERGE while text is being added. In contrast, the phrases produced by GZIP from the priming text are updated as compression progresses through the file.

Two dictionary files were constructed for the sole purpose of priming GZIP. These two files contain the most frequently occurring words from all of the HTML files on the web server, in decreasing frequency order. After being compressed with the `-9` option of GZIP, these two files were 1 KiB and 10 KiB in size. Uncompressed, these two files were 1,583

```
HREF</HTML></A><TITLE></TITLE><P><HTML>http</BODY>
wwwThis<BR><HR></H1></UL><UL><LI></EM><EM>generate
d<HEAD></HEAD>that2000footerEST<STRONG></STRONG>NA
MESubject</SMALL><SMALL>sortedDateemailhypermailDT
DtrailerDOCTYPEPUBLICFromthisnextthreadisosentisor
```

Figure 8.2: The first 250 bytes of the priming dictionary file, which compresses to 10 KiB. Linebreaks have been introduced to allow display.

bytes and 17,736 bytes in size, respectively. Figure 8.2 shows the first 250 bytes of the larger file.

8.4 Experimental Results

The experiments demonstrate the reduction in the number of bytes transferred by priming the compression mechanism. However, compressing each file before delivery to the client adds a time overhead to file transmission. Each file must be compressed by the web server, transferred, and then decompressed by the client. The impact of this can be minimised if files are compressed while the web server is idle, and before they are requested. Alternatively, compressed files can be placed in a cache of recently requested files. The second of these methods was chosen for the simulation.

The simulation measures the bandwidth and the disk space required at the server. Bandwidth is quantified as the number of bytes sent to the client, while the disk space is the extra overhead required for the cache. It is assumed that all of the web pages exist uncompressed on disk at the server. As pages are requested, they are compressed, sent to the client, and copies of the compressed files are left in the cache, in the hope they can be used again. For the purpose of the simulation, the cache is initially empty and never cleared, so that the total disk space taken is the size of the cache at the end of the simulation.

Nine methods of processing HTML files were evaluated. Compression was executed with GZIP, using its `-9` option. Two methods, NO-COMPRESSION and INDEPENDENT, form the baseline for the experiments. The NO-COMPRESSION method assumes documents are

sent as plain text, while the INDEPENDENT method compresses each requested HTML document individually, as a stand-alone file.

Then, several other methods were considered. The COMMON-DICTIONARY method compresses each file with either of the two standard dictionary files as priming text (1 KiB or 10 KiB), assuming that the dictionary files are available to the client for "free" and do not need to be explicitly sent. For this reason, the COMMON-DICTIONARY method is unrealistic and the TRANSMITTED-DICTIONARY corrects that by adding the cost of sending the respective dictionary files (in compressed form) for every browsing session. The ONE-PRIOR and the ALL-PRIOR methods utilise the browsing patterns of the users. The ONE-PRIOR method compresses each file, priming GZIP in each case using the previously requested file in the browsing session. The first file in any browsing session is compressed without any priming. The ALL-PRIOR method uses all HTML files in the browsing session for priming. Finally, the ALL-AVAILABLE method is the most optimistic method, since it combines the COMMON-DICTIONARY method with the ALL-PRIOR method. The ALL-AVAILABLE method assumes that a standard dictionary file is available for free and that it can be combined with all previously requested HTML files in that browsing session to prime the compressor.

The results from the simulation are presented in Table 8.3 as percentages. The amount of disk space occupied by the HTML files with no compression is 100%, since it is assumed an uncompressed version of the web pages must exist. The bandwidth taken when no compression is applied is 100%. With each of the remaining compression methods, the disk space usage increases and the bandwidth decreases.

The INDEPENDENT method shows that applying GZIP to files individually already reduces the bandwidth requirements to 27.9%. The disk space rises only by 4.7%, because only a small proportion of the publicly available HTML files were accessed in the simulation window of 7 days. Applying either of the dictionaries to prime GZIP slightly reduces both the disk space and bandwidth requirements in comparison to the INDEPENDENT method. Of course no change in disk space exists between the COMMON-DICTIONARY and the TRANSMITTED-DICTIONARY methods. However, when the more realistic situation of

Method	Disk space (% uncompressed)	Network bandwidth (% uncompressed)
NO-COMPRESSION	100.0	100.0
INDEPENDENT	104.7	27.9
COMMON-DICTIONARY, 1 KiB	104.5	27.1
COMMON-DICTIONARY, 10 KiB	104.6	27.2
TRANSMITTED-DICTIONARY, 1 KiB	104.5	31.5
TRANSMITTED-DICTIONARY, 10 KiB	104.6	71.2
ONE-PRIOR	109.0	24.5
ALL-PRIOR	–	23.9
ALL-AVAILABLE, 10 KiB	–	23.5

Table 8.3: The extra disk space required and the bandwidth saved for each of the compression strategies employed. The simulation window spanned 7 days in July 2000.

transmitting the dictionaries is considered, the bandwidth requirements rises from 27.1% to 31.5% for the 1 KiB dictionary, and from 27.2% to 71.2% for the 10 KiB dictionary. The overall cost of transmitting the dictionaries is not recovered - a consequence of the large number of short browsing sessions, and the effectiveness of GZIP when dealing with moderate-length files.

The ONE-PRIOR method is more effective than the previous methods, yet the gain is still marginal. Unlike TRANSMITTED-DICTIONARY, no explicit dictionary needs to be sent to the client, and for browsing sessions with only one HTML file, the file is compressed without a priming file. Since the remaining two methods in Table 8.3 require multiple files for priming, maintaining a cache appears difficult due to the many file combinations that could exist. For this reason, disk space consumption is not reported. The bandwidth required by these two methods continues to decrease in comparison with ONE-PRIOR because on average, the 32 KiB sliding window of GZIP can hold more than one HTML file or one dictionary file for priming. Only the experiment with the 10 KiB standard dictionary is shown for the ALL-AVAILABLE method. It achieves the best result with respect to bandwidth, but this is partly because the dictionary is assumed to be available without charge.

Some of the bandwidth results from the simulation are presented in more detail in Figure 8.3. In the figure, the cumulative bandwidth is plotted against the number of

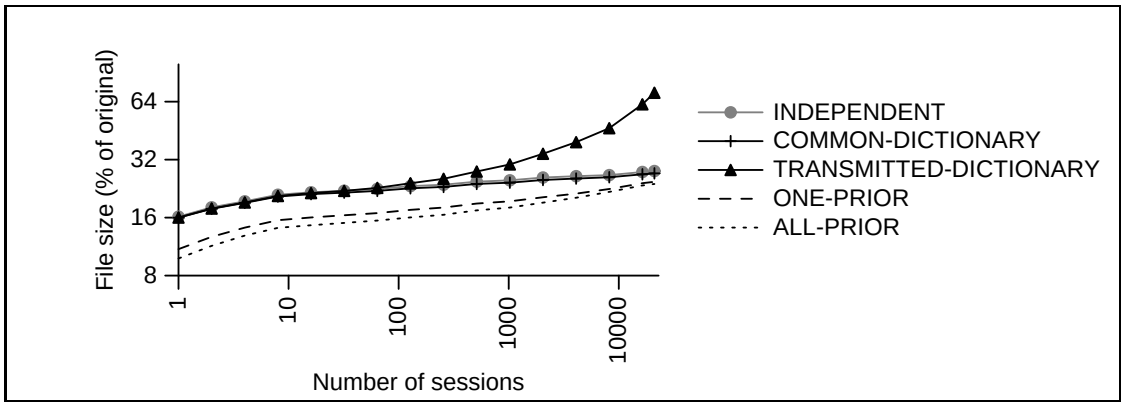


Figure 8.3: The cumulative bandwidths achieved with each of the compression methods. The browsing sessions have been ordered from left to right in decreasing length of session, with length assessed as the number of pages requested.

browsing sessions, ordered according to decreasing length of session. The right-hand end of the graph corresponds to the bandwidth results in Table 8.3. At the left end of the graph, long browsing sessions enable the ONE-PRIOR and the ALL-PRIOR methods to perform significantly better than the other three methods. As the length of the browsing sessions decrease, this advantage slowly evaporates. Furthermore, when the short browsing sessions are introduced, the cumulative bandwidth of TRANSMITTED-DICTIONARY rises sharply, and the cost of sending the priming dictionary outweighs the benefits obtained from using it.

8.5 Summary

This chapter has shown how the order users visit web pages combined with a compression system such as GZIP can achieve small additional reductions in bandwidth requirements. This idea was demonstrated through a simulation with a web server and its corresponding access logs. Additional experiments showed how a static dictionary of the most frequently occurring words on the web server also improved compression, but only if the dictionary could be assumed to be static, and available free of charge. Ultimately, the high cost of transmitting the dictionary in the many short browsing sessions was too high to justify such an approach as a general solution. The simulation has also demonstrated that GZIP

alone is a good technique which reduces bandwidth requirements to 27.9%.

The experiments have also demonstrated the tension that exists between bandwidth, processing time, and storage. Bandwidth can be reduced by priming the compression system with the previously requested file. But the extra latency in compressing each file as it is requested may be unacceptable to users, who already wait impatiently during network delays. Disk space can be used to alleviate this problem by compressing all files beforehand. However, if HTML files are compressed in the context of even just one previously requested file, then a quadratic number of file variants need to be maintained. The large number of document versions is required because a user can arrive at a HTML file through any other file on the web site, or by manually entering its location in the web browser. A possible compromise would be to maintain a cache of recently requested documents, in their various compressed forms, which in the simulation was assumed to be one week. Nevertheless, it seems unlikely that all three factors can be simultaneously reduced.

Future extensions to this work, include the possible development of a compression mechanism specifically for HTML documents that offers improved compression ratios. However, unlike syntax-directed compression systems (for example, Katajainen et al. [1986]), such a system must be capable of accepting syntactically incorrect data, as HTML documents are often poorly structured and only rarely are error free. Also, while experimental results have shown that priming with a standard dictionary is problematic due to the added cost of sending it, it may be possible to separate a web server into logical domains and then to create a unique dictionary for each domain.

Chapter 9

Summary

This thesis details the compromises required in order to satisfy the two opposing tasks of compression and phrase browsing. While a compression system reduces the size of a document by removing any redundancies, a retrieval system generally augments redundant information to the document in order to facilitate efficient access. These two document management methods usually exist in isolation; but this thesis considers the costs and benefits of combining them. Throughout, the compromise between retrieval and compression has translated into a balance between disk space and retrieval time.

Chapter 2 began with an overview of the foundations of text compression, and continued by listing three complete compression systems (GZIP, BZIP2, and PPMD) and two coding mechanisms (arithmetic coding and Huffman coding) as examples. In Chapter 3, a fourth compression program called RE-PAIR [Larsson and Moffat, 2000] was introduced. Character-based RE-PAIR, as it was later called, reduces the length of a document (or message) by recursively replacing the most frequently occurring pair of symbols with a new symbol until no pair of adjacent symbols exists twice or more in the message. RE-PAIR produces a phrase hierarchy which is encoded using chiastic slide and interpolative coding, and a sequence whose coding is varied.

The RE-PAIR system forms the basis of the retrieval system in this thesis because of two other properties. As a dictionary-based compression mechanism, the decoding time is fast, an important concern for interactive retrieval systems. Also, the phrase hierarchy is both compact and rich with information. After encoding, it occupies only 0.223 bpc compared

to the original data, but contains information about the characters in the document and the relationships they form to make partial or complete words and phrases. A retrieval system based on browsing these symbols has the advantage of not requiring any of the auxiliary data structures to be permanently stored. Chapter 3 concluded by noting three drawbacks to character-based RE-PAIR, which are addressed by the enhancements that form the main body of this thesis.

Chapter 4 added a pre-processing step to RE-PAIR called PRE-PAIR. The PRE-PAIR system separates a message into two alternating streams of word tokens and non-word tokens. Each of these streams is accompanied by a lexicon. Word tokens are also case folded and stemmed in the interest of reducing the size of the word lexicons, resulting in a total of seven streams. Pairing rules are augmented to RE-PAIR to prevent phrases from straddling punctuation marks. Combining all of these additions results in a system called punctuation-aligned RE-PAIR. The phrase hierarchy, coupled with the word lexicon, now contains the words that occur in the document and the relationships they form to make phrases.

In order to tackle the problem of memory usage, Chapter 5 first described how Larsson and Moffat [2000] segments a large message into blocks. Then, RE-MERGE was described as a method of combining compressed blocks in phases. An extension to RE-MERGE which allows text to be appended to a previously compressed document is also presented.

Chapter 6 looks at alternative encoding mechanisms for the reduced word sequence and the three modifier streams (case folding, stemming, and non-words). The reduced word sequence is encoded with a byte-aligned coder called RE-VIEW, while the modifiers are encoded with an indexed variant of SHUFF. The decision to switch to these methods from SHUFF was based on improved access times rather than compression effectiveness.

Attention shifted from compression to phrase browsing in Chapter 7. The RE-PHINE system enables the symbols that make up the phrase hierarchy to be browsed, and the contexts in which they occur to be retrieved. Symbols can be extended, or broken into their components. Once a symbol of interest has been isolated, the reduced word sequence is searched for the symbol in question, and the contexts are displayed one-by-one. Further

refinement through the inclusion of case folding, stemming, and intervening non-word characters is also possible.

Chapter 8 takes a detour from the primary theme of the thesis by looking at the issue of web page delivery and compression.

9.1 Putting it all together

The systems mentioned from Chapter 3 to Chapter 7 work in concert to provide document storage and browsing. Together, they are nicknamed RE-STORE, and are summarised in Figure 9.1. In the figure, boxes indicate the major components that comprise RE-STORE, while lines between them indicate the flow of data. On the right side of the figure, the matching chapter of each system is shown.

As the pieces of RE-STORE came together, the loss in compression effectiveness in order to satisfy browsing has been noted throughout the thesis. Overall, the compression effectiveness of the `NEWS` document has worsened from 1.674 bpc with PPMD to 2.523 bpc with punctuation-aligned RE-PAIR. Similarly, by building a retrieval system on top of a compression mechanism, drawbacks become apparent. For example, the browser is unable to accurately judge the usefulness of symbols in the phrase hierarchy, and resorts to displaying all of them instead. This is the equilibrium point between compression and browsing for RE-STORE.

Opportunities exist for future work that may end up bringing compression and browsing closer together. For example, as Figure 4.5 on page 103 shows, the modifiers add almost 1.0 bpc to the compressed message when encoded independently with SHUFF. Compression levels would probably improve if these three streams were compressed together, so that the context for any one stream is formed from the two remaining ones. While compression levels closer to character-based RE-PAIR might then be attained, the effect on browsing would need to be re-assessed.

Section 7.5 gave few directions for possible improvements to phrase browsing. One suggestion is filtering primitives and phrases from RE-PHINE so that only useful ones are shown. Alternatively, a ranking scheme can be adopted which shows interesting symbols at

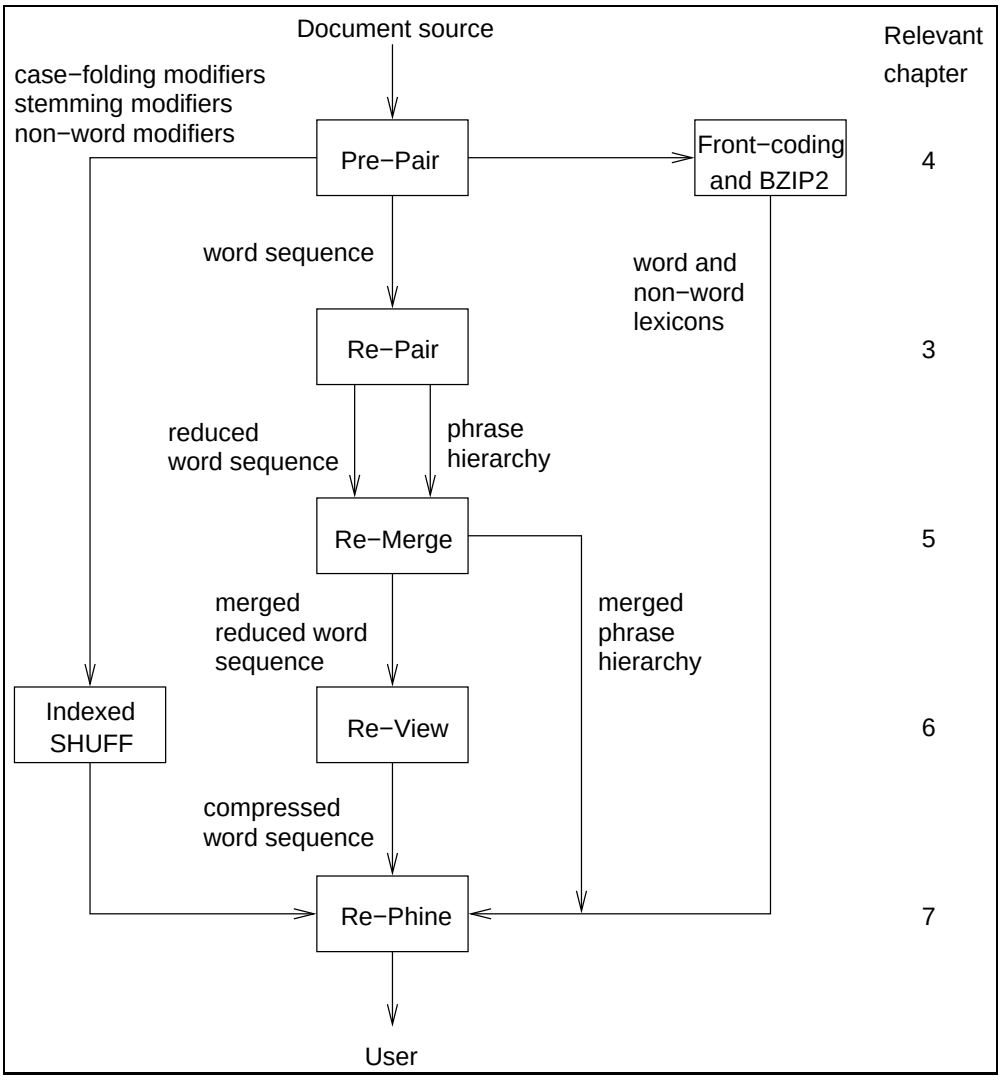


Figure 9.1: A sketch of the overall RE-STORE system, with relevant chapters indicated.

the top of a RE-PHINE window. Another important direction for RE-STORE is to modify the appending phase of RE-MERGE (Section 5.7 on page 150) to allow novel words to be handled. Then, documents larger than 1,000 MiB can be processed.

9.2 Towards the Future

Compression and information retrieval are both well-studied areas in the field of document management. However, the successful coupling of these two areas offers new opportunities

for future work. Effective storage and improved accessibility ensure that collected data can accumulate while still remaining useful.

But is there room in information retrieval for phrase browsing? This question can be divided into two parts. First, is there a need for phrase browsing? Also, are there challenges in developing phrase browsing?

A closer look at how people approach new books will help in addressing the first question. Indexes and table of contents help people expedite their search for information between book covers. However, the words and phrases that lie on these pages also allow the reader to browse the book, essentially for free. No extra amount of work needs to be expended by the author or the publisher to facilitate browsing. If electronic documents are extensions of books, then it is expected that the tools already available for the latter will be useful for the former.

So, the problem for phrase browsing is to adapt concepts from printed matter to electronic media. Solutions employed by index-based systems cannot be replicated for phrase browsing. While metrics such as precision and recall have been established for querying, the effectiveness of phrase browsing is more difficult to measure. Precision and recall indicate how well a system has performed and how it compares to other systems. While phrase browsing can be assessed through user studies, precise work in this area is notoriously difficult. Also, the techniques and data structures for index-based systems differ from what is needed in a phrase browsing system.

In an ideal situation, an information retrieval system should provide a selection of tools which can appeal to a variety of users. Index-based querying and phrase browsing are two possible tools, and just as this thesis has looked at combining compression with phrase browsing, another area of work may lie in combining multiple information retrieval techniques into a single, succinct system.

Not only are tools merging to form more complex systems, but the format of data is changing as well. While this thesis has presented techniques for browsing compressed document collections in SGML markup, the documents have been treated like flat text. As more documents are published electronically with SGML or XML markup, compressing

and browsing them using the tags is becoming another important direction for the future. The structural information in these document formats might improve compression levels and allow users to selectively browse parts of the documents.

Appendix A

Notation

Symbol	Definition
α, β, δ, γ	A symbol
$\|\alpha\|$	Length of α in primitives after expansion
ⓐ, ①	A replacement symbol introduced by Re-Pair
ω	String of symbols
Σ	Alphabet of message (for Re-Pair, the primitives)
ρ	Set of new symbols created by Re-Pair (phrases)
$\|$	Division between left and right component of an expanded Re-Pair phrase
␣	A space character
$\mathcal{P}$, $\|\mathcal{P}\|$	The dictionary created by Re-Pair (phrase hierarchy), and size
$\mathcal{P}_1$, $\mathcal{P}_2$	Initial phrase hierarchies before block merging
$\mathcal{P}'$	Final phrase hierarchy after block merging
$\mathcal{S}$, $\|\mathcal{S}\|$	The sequence of references to a Re-Pair dictionary (sequence), and length
$\mathcal{S}_1$, $\mathcal{S}_2$	Initial sequences before block merging
$\mathcal{S}'$	Final sequences after block merging
$\mathcal{C}$	Set of symbols searched for in the compressed word sequence
$\mathcal{R}$	Set of results returned from search in the compressed word sequence
1 KiB	1 kibibyte = 2^{10} bytes = 1,024 bytes
1 MiB	1 mebibyte = 2^{20} bytes = 1,048,576 bytes
1 GiB	1 gibibyte = 2^{30} bytes = 1,073,741,824 bytes

Appendix B

Porter Stemming Algorithm

The Porter stemming algorithm was originally described by Porter [1980] and later reprinted in 1997. The algorithm is used in the word-based and punctuation-aligned Re-Pair variants to stem words to their root form by removing word suffixes. Chapter 4 provides details of the different methods of selecting phrases for Re-Pair and how the word-based pre-processor, Pre-Pair, incorporates the algorithm. This appendix provides information specific to the stemming algorithm, with emphasis on the modifiers for stemming reversal.

The version of the algorithm employed by Pre-Pair transforms an English word into its root form by applying a sequence of six independent steps for suffix removal. Step 0 is the first step and consists of a single action. Step 1 is composed of multiple sub-steps (1a to 1c), unlike Steps 2, 3, and 4. And step 5 consists of two sub-steps (5a and 5b), applied one after the other. Each step consists of rules which either remove characters from the end of the word, or change characters at the end of the word, or do a combination of both. A rule is applied if a certain set of conditions is met. Some possible conditions include a match on the suffix, or a minimum number of consonant and vowel combinations before the stem.

Some changes to the traditional implementation of the Porter stemming algorithm were necessary for the implementation found in Pre-Pair. The differences are as follows:

1. If a word ends in "logi", then the final "i" is removed. (New rule for step 2)

2. If a word ends in "bli", then the final "i" is changed to an "e". (Updated rule for step 2.)

3. Words of length 3 characters or less, or which are SGML tags that end with a ">" character are not processed.

4. Seven contractions which include the apostrophe are removed in step 0.

The first two changes were suggested by Porter on the web page created for the algorithm, located at `http://www.tartarus.org/~martin/PorterStemmer/`. The last two changes were added specifically for PRE-PAIR. The third change prevents short words from being stemmed further and improves overall efficiency. In an SGML document, word tokens which end with a ">" character would not be stemmed according to the algorithm. The modification to the algorithm allows PRE-PAIR to identify tags immediately, without attempting to stem them. The last change introduces a step 0 to the stemming mechanism in PRE-PAIR for the removal of certain suffixes which include an apostrophe. The seven contractions identified by step 0 are some of the most common ones in English [Flick and Millward, 1993, pg. 217]. In particular, they include contractions where a verb occurs with the word "not", or when a pronoun is followed by an auxiliary verb. In other cases not covered by these rules, the apostrophe is treated as a non-word during parsing. For example, "o'clock" contains two word tokens, with the first one being the letter "o" by itself. As an example of the apostrophe handling, the word "king's" is first treated as a single word token. Step 0 removes the apostrophe and the "s", since the "s" is too short to be useful for phrase browsing. The alternative would result in "king" and "s" being processed as two separate word tokens, to the detriment of any subsequent phrase searching.

In this investigation, stemming is required for compression, and must be reversible. To achieve reversibility, a modifier is stored along with the stemmed word to indicate which rules must be applied to invert the stemming process. While the stemming process requires knowledge about the length of the word token and the characters which precede the stem, the inverse is more mechanical and applies the steps from 5b to 0 without checking the

Step	Number of Rules	Number of Bits
5b	2	1
5a	2	1
4	20	5
3	8	3
2	15	4
1c	2	1
1b	7	3
1a	3	2
0	8	3
Total:	67	23

Table B.1: The steps in PRE-PAIR's version of the Porter stemming algorithm. The steps are listed in the order they are used when decoding. The second column lists the number of rules in each step. The number of bits required to express which rule was used in that step is shown in the third column. In total, a 23-bit modifier is required for each word token to indicate how to reverse the stemming.

word. Moreover, in the algorithm, several rules within a step remove the same suffix, but based on different conditions on the characters preceding the suffix. Since the inverse of all of these rules add the same suffix back, they can be combined into a single modifier. The first rule of every step indicates that the step was not used. Only one rule is applied per step.

Table B.1 lists the steps in the order in which they are applied when decoding. Since each step is independent of other steps, a modifier for a word token is 23 bits in length. The table also lists the number of rules per step. In a few cases, the number of possible rules that can be represented by the bits allocated exceeds the actual number of rules. For example, 5 bits for step 4 can account for 32 rules. While the size of the modifier could be reduced by merging steps, the steps were separated due to encoding and decoding efficiency reasons. Chapter 6 takes a closer look at the distribution of stemming modifiers when coding the stream of stemming modifiers is examined.

The remainder of this appendix shows the decoding rules for each step. Each rule applies changes to the current word, ω. In the following tables, most rules appear in the following form: ωe $\rightarrow$ ωional. This means that the letter "e" which ends the word is replaced with "ional". Other rules may simply append the suffix on to the word, without

Step	Value	Word (ω)
		commun
5b	0	commun
5a	0	commun
4	5	communic
3	1	communicate
2	7	communication
1c	0	communication
1b	0	communication
1a	2	communications
0	0	communications

Table B.2: Example of transforming the word "commun" back to the original word "communications" using its corresponding modifier, starting from step 5b. No changes are applied to the word whenever rule 0 is found.

Value	Rule
0	No change to ω
1	Double final letter in ω

(b) Step 5b requires 1 bit.

Value	Rule
0	No change to ω
1	$\omega \rightarrow \omega$e

(a) Step 5a requires 1 bit.

Figure B.1: Porter stemming algorithm – step 5.

any prior changes to the word.

As an example of how stemming is reversed, suppose the initial stemmed word token is "commun" and its corresponding modifier is: 0, (2, 0, 0), 7, 1, 5, (0, 0) . Step 0 is represented by the first "0", and parentheses have been inserted around the composite steps 1 and 5 to improve legibility. Table B.2 demonstrates how the modifier is used to recover the original word, "communications", starting from step 5b.

Value	Rule
0	No change to ω
1	$\omega \rightarrow \omega$al
2	$\omega \rightarrow \omega$ance
3	$\omega \rightarrow \omega$ence
4	$\omega \rightarrow \omega$er
5	$\omega \rightarrow \omega$ic
6	$\omega \rightarrow \omega$able
7	$\omega \rightarrow \omega$ible
8	$\omega \rightarrow \omega$ant
9	$\omega \rightarrow \omega$ement
10	$\omega \rightarrow \omega$ment
11	$\omega \rightarrow \omega$ent
12	$\omega \rightarrow \omega$ion
13	$\omega \rightarrow \omega$ou
14	$\omega \rightarrow \omega$ism
15	$\omega \rightarrow \omega$ate
16	$\omega \rightarrow \omega$iti
17	$\omega \rightarrow \omega$ous
18	$\omega \rightarrow \omega$ive
19	$\omega \rightarrow \omega$ize

Step 4 requires 5 bits.

Figure B.2: Porter stemming algorithm – step 4.

Value	Rule
0	No change to ω
1	$\omega \rightarrow \omega$ate
2	$\omega \rightarrow \omega$ative
3	$\omega \rightarrow \omega$ize
4	$\omega \rightarrow \omega$iti
5	$\omega \rightarrow \omega$al
6	$\omega \rightarrow \omega$ful
7	$\omega \rightarrow \omega$ness

(a) Step 3 requires 3 bits.

Value	Rule
0	No change to ω
1	ωe $\rightarrow \omega$ional
2	$\omega \rightarrow \omega$al
3	ωe $\rightarrow \omega$i
4	$\omega \rightarrow \omega$r
5	$\omega \rightarrow \omega$li
6	ωe $\rightarrow \omega$ation
7	ωe $\rightarrow \omega$ion
8	ωe $\rightarrow \omega$or
9	$\omega \rightarrow \omega$ism
10	$\omega \rightarrow \omega$ness
11	$\omega \rightarrow \omega$iti
12	ωe $\rightarrow \omega$iti
13	ωle $\rightarrow \omega$iliti
14	$\omega \rightarrow \omega$i

(b) Step 2 requires 4 bits.

Figure B.3: Porter stemming algorithm – steps 3 and 2.

Value	Rule
0	No change to ω
1	ωi $\rightarrow$ ωy

(c) Step 1c requires 1 bit.

Value	Rule
0	No change to ω
1	$\omega \rightarrow \omega$ed
2	$\omega \rightarrow \omega$d
3	Double the final letter in ω and then apply $\omega \rightarrow \omega$ed
4	$\omega \rightarrow \omega$ing
5	ωe $\rightarrow$ ωing
6	Double the final letter in ω and then apply $\omega \rightarrow \omega$ing

(b) Step 1b requires 3 bits.

Value	Rule
0	No change to ω
1	$\omega \rightarrow \omega$es
2	$\omega \rightarrow \omega$s

(a) Step 1a requires 2 bits.

Figure B.4: Porter stemming algorithm – step 1.

Value	Rule
0	No change to ω
1	$\omega \rightarrow \omega'$d
2	$\omega \rightarrow \omega'$m
3	$\omega \rightarrow \omega'$s
4	$\omega \rightarrow \omega'$t
5	$\omega \rightarrow \omega'$ll
6	$\omega \rightarrow \omega'$re
7	$\omega \rightarrow \omega'$ve

Step 0 requires 3 bits.

Figure B.5: Porter stemming algorithm – step 0.

Bibliography

A. V. Aho and M. J. Corasick. Efficient string matching: An aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, June 1975. [135]

American National Standards Institute. *Information Systems - Coded Character Sets - 7-Bit American National Standard Code for Information Interchange (7-Bit* ASCII*)*. 1997. Document number: ANSI INCITS 4-1986 (R1997), formerly ANSI X3.4-1986. [14]

A. Amir and G. Benson. Efficient two-dimensional compressed matching. In J. A. Storer and M. Cohn, editors, *Proc. 1992 IEEE Data Compression Conference*, pages 279–288. IEEE Computer Society Press, Los Alamitos, California, March 1992. [166]

P. G. Anick and S. Vaithyanathan. Exploiting clustering and phrases for context-based information retrieval. In *Proc. 20th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 314–323, 1997. [203]

A. Apostolico and S. Lonardi. Off-line compression by greedy textual substitution. *Proc. IEEE*, 88(11):1733–1744, November 2000. [66]

F. S. Awan and A. Mukherjee. LIPT: A lossless text transform to improve compression. In *Proc. Information Technology: Coding and Computing*, pages 452–460, 2001. [84]

R. Baeza-Yates and B. Ribeiro-Neto. *Modern Information Retrieval.* ACM Press, 1999. [6, 201, 205]

D. Bahle, H. E. Williams, and J. Zobel. Compaction techniques for nextword indexes. In *Proc. 8th International Symposium on String Processing and Information Retrieval*, pages 33–45, San Rafael, Chile, November 2001a. IEEE Computer Society Press, Los Alamitos, CA. [204]

D. Bahle, H. E. Williams, and J. Zobel. Optimised phrase querying and browsing in text databases. In M. Oudshoorn, editor, *Proc. 24th Australasian Computer Science Conference*, pages 11–19, Gold Coast, Australia, January 2001b. [204]

D. Bahle, H. E. Williams, and J. Zobel. Efficient phrase querying with an auxiliary index. In K. Järvelin, M. Beaulieu, R. Baeza-Yates, and S. H. Myaeng, editors, *Proc. 25th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 215–221, Tampere, Finland, August 2002. [204]

T. C. Bell. Better OPM/L text compression. *IEEE Transactions on Communications*, COM-34:1176–1182, December 1986. [21]

T. C. Bell, J. G. Cleary, and I. H. Witten. *Text Compression.* Prentice Hall, 1990. [8, 26]

T. C. Bell and I. H. Witten. Greedy macro text compression. Technical Report 87/285/33, Department of Computer Science, University of Calgary, Calgary, Alberta, Canada, 1987. [26]

T. C. Bell and I. H. Witten. The relationship between greedy parsing and symbolwise text compression. *Journal of the ACM*, 41(4):708–724, July 1994. [27]

J. Bentley and D. McIlroy. Data compression with long repeated strings. *Information Sciences*, 135:1–11, 2001. [65]

J. L. Bentley, D. D. Sleator, R. E. Tarjan, and V. K. Wei. A locally adaptive data compression scheme. *Communications of the ACM*, 29(4):320–330, April 1986. [75]

T. Berners-Lee. WWW: Past, present, and future. *IEEE Computer*, 29(10):69–77, 1996. [4]

C. L. Borgman. *From Gutenberg to the Global Information Infrastructure: Access to Information in the Networked World.* MIT Press, 2003. [6]

R. S. Boyer and J. S. Moore. A fast string-searching algorithm. *Communications of the ACM*, 20(10):762–772, October 1977. [5, 167]

E. Brill. Some advances in rule-based part of speech tagging. In *Proc. 12th National Conference on Artificial Intelligence*, pages 722–727. AAAI Press, 1994. [203]

S. Brin and L. Page. The anatomy of a large-scale hypertextual Web search engine. In P. Thistlewaite and H. Ashman, editors, *Proc. 7th International World Wide Web Conference*, pages 107–117, April 1998. [223]

N. R. Brisaboa, A. Fariña, G. Navarro, and M. F. Esteller. (S, C)-dense coding: An optimized compression code for natural language text databases. In M. A. Nascimento, editor, *Proc. 10th International Symposium on String Processing and Information Retrieval*, pages 122–136, Manaus, Brazil, October 2003. [167, 168]

C. Buckley, A. Singhal, and M. Mitra. New retrieval approaches using SMART: TREC 4. In D. K. Harman, editor, *Proc. Fourth Text REtrieval Conference (*TREC-*4), National Institute of Standards and Technology Special Publication 500-236*, pages 25–48. Department of Commerce, National Institute of Standards and Technology, November 1995. [205]

M. Burrows and D. J. Wheeler. A block-sorting lossless data compression algorithm. Technical Report 124, Digital Equipment Corporation, Palo Alto, California, May 1994. [27]

M. Caflisch. The ampersand. Available on-line at: `http://www.adobe.com/type/topics/theampersand.html`., 2003. [8]

A. Cannane and H. E. Williams. General-purpose compression for efficient retrieval. *Journal of the American Society for Information Science and Technology*, 52(5):430–437, 2001. [66, 138]

A. Cannane and H. E. Williams. A general-purpose compression scheme for large collections. *ACM Transactions on Information Systems*, 20(3):329–355, July 2002. [151]

N. Chomsky. On certain formal properties of grammars. *Information and Control*, 2: 137–167, 1959. [62]

C. L. A. Clarke, G. V. Cormack, and F. J. Burkowski. Shortest substring ranking (multi-Text experiments for TREC-4). In D. K. Harman, editor, *Proc. Fourth Text REtrieval Conference (*TREC-*4), National Institute of Standards and Technology Special Publication 500-236*, pages 295–304. Department of Commerce, National Institute of Standards and Technology, November 1995. [205]

J. G. Cleary and W. J. Teahan. Unbounded length contexts for PPM. *Computer Journal*, 40(2/3):67–75, 1997. [26, 29]

J. G. Cleary and I. H. Witten. Data compression using adaptive coding and partial string matching. *IEEE Transactions on Communications*, 32(4):396–402, April 1984. [25]

T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms.* The MIT Press, second edition, 2001. [33, 123, 131]

T. M. Cover and J. A. Thomas. *Elements of Information Theory.* John Wiley & Sons, Inc., 1991. [15]

J. L. Dawson. Suffix removal and word conflation. *Bulletin of the Association for Literary and Linguistic Computing*, 2(3):33–46, 1974. [84]

O. de Kretser and A. Moffat. Effective document presentation with a locality-based similarity heuristic. In *Proc. 22nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 113–120, August 1999. [205]

E. S. de Moura, G. Navarro, N. Ziviani, and R. Baeza-Yates. Fast and flexible word searching on compressed text. *ACM Transactions on Information Systems*, 18(2):113–139, April 2000. [75, 167]

S. Deorowicz. Improvements to Burrows-Wheeler compression algorithm. *Software - Practice and Experience*, 30(13):1465–1483, November 2000. [30]

P. Deutsch. DEFLATE compressed data format specification version 1.3 (RFC 1951). Source code available from `ftp://ftp.uu.net/graphics/png/documents/zlib/zdoc-index.html`, May 1996. [36]

P. Elias. Universal codeword sets and representations of the integers. *IEEE Transactions on Information Theory*, 21(2):194–203, March 1975. [31, 32]

D. A. Evans and C. Zhai. Noun-phrase analysis in unrestricted text for information retrieval. In *Proc. ACL-96, 34th Annual Meeting of the Association for Computational Linguistics*, pages 17–24, 1996. [203]

P. Fenwick. Block sorting text compression. In K. Ramamohanarao, editor, *Proc. 19th Australasian Computer Science Conference*, pages 193–202, Melbourne, January 1996a. [30]

P. Fenwick. The Burrows-Wheeler transform for block sorting text compression: Principles and improvements. *Computer Journal*, 39(9):731–740, September 1996b. [30]

P. M. Fenwick. Ziv-Lempel encoding with multi-bit flags. In J. A. Storer and M. Cohn, editors, *Proc. 1993 IEEE Data Compression Conference*, pages 138–147. IEEE Computer Society Press, Los Alamitos, California, March 1993. [21]

P. Ferragina and G. Manzini. Opportunistic data structures with applications. In *Proc. IEEE Symposium on Foundations of Computer Science*, pages 390–398, 2000. [69]

J. Flick and C. Millward. *Handbook for Writers: Canadian Edition.* Harcourt Brace Jovanovich Canada Inc., second edition, 1993. [242]

W. B. Frakes. Introduction to information storage and retrieval systems. In Frakes and Baeza-Yates [1992], pages 1–12. [84]

W. B. Frakes and R. Baeza-Yates, editors. *Information Retrieval: Data Structures and Algorithms.* Prentice Hall, 1992. [6, 251]

E. Frank, G. W. Paynter, I. H. Witten, C. Gutwin, and C. G. Nevill-Manning. Domain-specific keyphrase extraction. In *Proc. International Joint Conference on Artificial Intelligence*, pages 668–673, 1999. [203]

Free Software Foundation. GNU GREP. `http://www.gnu.org/software/grep/grep.html`, 2000. version 2.4.2. [176]

Free Software Foundation. ZGREP (wrapper for GREP program), 2001. Adapted from a version by Charles Levert <`charles@comm.polymtl.ca`>. [42]

P. Gage. A new algorithm for data compression. 12(2), 1994. [66]

J.-L. Gailly. GZIP program and documentation. Source code available from `http://www.gzip.org/`, 1993. [36]

J.-L. Gailly and M. Adler. ZLIB program and documentation. Source code available from `ftp://ftp.uu.net/pub/archiving/zip/zlib/`, 1995. [37, 222]

J. Gettys, J. Mogul, H. Frystyk, L. Masinter, P. Leach, and T. Berners-Lee. *Hypertext Transfer Protocol* – HTTP*/1.1 (Request for Comments 2616)*. World Wide Web Consortium, June 1999. Available from: `http://www.w3.org/Protocols/`. [222]

D. Gottlieb, S. A. Hagerth, P. G. H Lehot, and H. S. Rabinowitz. A classification of compression methods and their usefulness for a large data processing center. In *Proc. National Computer Conference*, pages 453–458. AFIPS Press, May 1975. [88]

S. Greenbaum. *The Oxford English Grammar*. Oxford University Press, 1996. [92, 202]

D. Grinberg, J. Lafferty, and D. Sleator. A robust parsing algorithm for Link Grammars. Technical Report CMU-CS-95-125, Carnegie Mellon University, School of Computer Science, 1995. [211]

C. Gutwin, G. Paynter, I. Witten, C. Nevill-Manning, and E. Frank. Improving browsing in digital libraries with keyphrase indexes. *Decision Support Systems*, 27(1–2):81–104, November 1999. [203, 213, 219, 220]

D. K. Harman. Overview of the second Text REtrieval Conference (TREC-2). *Information Processing and Management*, 31(3):271–289, May 1995. [39]

D. Hawking and P. Thistlewaite. Proximity operators – so near and yet so far. In D. K. Harman, editor, *Proc. Fourth Text REtrieval Conference (*TREC-*4), National Institute of Standards and Technology Special Publication 500-236*, pages 131–144. Department of Commerce, National Institute of Standards and Technology, November 1995. [205]

S. Heinz, J. Zobel, and H. E. Williams. Burst tries: A fast, efficient data structure for string keys. *ACM Transactions on Information Systems*, 20(2):192–223, April 2002. [88]

R. N. Horspool and G. V. Cormack. Constructing word-based text compression algorithms. In J. A. Storer and M. Cohn, editors, *Proc. 1992 IEEE Data Compression Conference*, pages 62–71. IEEE Computer Society Press, Los Alamitos, California, March 1992. [75]

P. G. Howard. *The Design and Analysis of Efficient Lossless Data Compression Systems.* PhD thesis, Brown University, Providence, RI, 1993. [26]

D. A. Huffman. A method for the construction of minimum-redundancy codes. *Proc. Inst. Radio Engineers*, 40(9):1098–1101, September 1952. [32]

IEC Technical Committee (TC) 25. *IEC 60027-2, Letter symbols to be used in electrical technology - Part 2: Telecommunications and electronics.* International Electrotechnical Commission (IEC), second edition, November 2000. Paraphrased at: `http://physics.nist.gov/cuu/Units/binary.html`. [36]

International Organization for Standardization. *Information processing — Text and Office Systems — Standard Generalized Markup Language (*SGML*) (ISO 8879).* Geneva, 1986. [39]

International Organization for Standardization. *Information technology – 8-bit single-byte coded graphic character sets – Part 1: Latin alphabet No. 1 (ISO 8859-1).* Geneva, 1998. [14]

R. Y. K. Isal and A. Moffat. Parsing strategies for BWT compression. In Storer and Cohn [2001], pages 429–438. [75, 102]

R. Johnsonbaugh. *Discrete Mathematics*. Macmillan Publishing Company, 1993. [33]

S. Jones. Design and evaluation of Phrasier, an interactive system for linking documents using keyphrases. In *Interact 99: Seventh IFIP Conference On Human-Computer Interaction*, pages 483–490, September 1999. [203]

S. Jones and G. Paynter. Topic-based browsing and querying within a digital library using keyphrases. In *Digital Libraries '99 The Fourth ACM Conference on Digital Libraries*, pages 114–121, August 1999. [197, 203]

S. Jones and G. W. Paynter. Human evaluation of Kea, an automatic keyphrasing system. In E. A. Fox and C. L. Borgman, editors, *Proc. First ACM/IEEE-CS Joint Conference on Digital Libraries*, pages 148–156. ACM Press, 2001. [203, 219]

J. Katajainen, M. Penttonen, and J. Teuhola. Syntax-directed compression of program files. *Software - Practice and Experience*, 16(3):269–276, March 1986. [16, 232]

J. Katajainen and T. Raita. An approximation algorithm for space-optimal encoding of a text. *Computer Journal*, 32(3):228–237, 1989. [122, 135]

B. W. Kernighan and M. D. McIlroy, editors. *UNIX Programmer's Manual*, volume 1. Bell Telephone Laboratories, Inc., seventh edition, January 1979. On-line version: `http://plan9.bell-labs.com/7thEdMan/`. [5]

S. Kirsch. Infoseek's experiences searching the internet. *SIGIR Forum*, 32(2):3–7, September 1998. [202]

S. T. Klein. Efficient optimal recompression. *Computer Journal*, 40(2/3):117–126, 1997. [122, 135, 136]

S. T. Klein and D. Shapira. Pattern matching in Huffman encoded texts. In Storer and Cohn [2001], pages 449–458. [166]

S. T. Klein and Y. Wiseman. Parallel Huffman decoding. In J. A. Storer and M. Cohn, editors, *Proc. 2000 IEEE Data Compression Conference*, pages 383–392. IEEE Computer Society Press, Los Alamitos, California, March 2000. [166]

D. E. Knuth. Dynamic Huffman coding. *Journal of Algorithms*, 6(2):163–180, June 1985. [34]

D. E. Knuth, J. H. Morris, and V. R. Pratt. Fast pattern matching in strings. *SIAM Journal of Computing*, 6(2):323–350, 1977. [69]

R. R. Korfhage. *Information Storage and Retrieval.* John Wiley & Sons, Inc., 1997. [6, 201]

G. Kowalski. *Information Retrieval Systems: Theory and Implementation.* Kluwer Academic Publishers, 1997. [6, 74, 84]

A. Kreitzman. The history of shorthand. *Journal of Court Reporters (On-Line)*, July/August 1999. Available at: `http://www.ncraonline.org/jcr/9907/9907_shorthand.htm`. [7]

L. L. Larmore and D. S. Hirschberg. A fast algorithm for optimal length-limited Huffman codes. *Journal of the ACM*, 37(3):464–473, July 1990. [34]

N. J. Larsson and A. Moffat. Offline dictionary-based compression. *Proc. IEEE*, 88(11): 1722–1732, November 2000. [9, 10, 43, 45, 50, 53, 54, 56, 58, 61, 76, 233, 234]

M. Lesk. *Practical digital libraries : books, bytes, and bucks.* Morgan Kaufmann, 1997. [6]

W.-S. Li, O. Kolak, Q. Vu, and H. Takano. Defining logical domains in a web site. In F. Shipman, D. Hicks, and P. Nuernberg, editors, *Proc. 11th ACM Conference on Hypertext and Hypermedia*, pages 123–132. ACM Press, May 2000. [223]

M. Liddell and A. Moffat. Incremental calculation of minimum-redundancy length-restricted codes. In J. A. Storer and M. Cohn, editors, *Proc. 2002 IEEE Data Compression Conference*, pages 182–191. IEEE Computer Society Press, Los Alamitos, California, April 2002. [34]

J. B. Lovins. Development of a stemming algorithm. *Mechanical Translation and Computational Linguistics*, 11(1–2):22–31, 1968. [84]

U. Manber. A text compression scheme that allows fast searching directly in the compressed file. *ACM Transactions on Information Systems*, 15(2):124–136, April 1997. [66, 69, 167]

U. Manber and S. Wu. GLIMPSE: A tool to search through entire file systems. In *Usenix Winter 1994 Technical Conference*, pages 23–32. Usenix Association, 1994. [171]

S. Mitarai, M. Hirao, T. Matsumoto, A. Shinohara, M. Takeda, and S. Arikawa. Compressed pattern matching for SEQUITUR. In Storer and Cohn [2001], pages 469–478. [166]

Y. Mizuuchi and K. Tajima. Finding context paths for web pages. In K. Tochtermann, J. Westbomke, U. K. Wiil, and J. J. Leggett, editors, *Proc. 10th ACM Conference on Hypertext and Hypermedia*, pages 13–22. ACM Press, February 1999. [223]

A. Moffat. Word-based text compression. *Software - Practice and Experience*, 19(2): 185–198, February 1989. [75, 83]

A. Moffat. Implementing the PPM data compression scheme. *IEEE Transactions on Communications*, 38(11):1917–1921, November 1990. [25]

A. Moffat, R. M. Neal, and I. H. Witten. Arithmetic coding revisited. *ACM Transactions on Information Systems*, 16(3):256–294, July 1998. Source code available from `http://www.cs.mu.oz.au/~alistair/arith_coder/`. [35, 38]

A. Moffat and L. Stuiver. Binary interpolative coding for effective index compression. *Information Retrieval*, 3(1):25–47, July 2000. [58]

A. Moffat and A. Turpin. On the implementation of minimum redundancy prefix codes. *IEEE Transactions on Communications*, 45(10):1200–1207, October 1997. Source code available from: `http://www.cs.mu.oz.au/~alistair/mr_coder/`. [34, 37]

A. Moffat and A. Turpin. *Compression and Coding Algorithms.* Kluwer Academic Publishers, 2002. [8, 36]

A. Moffat and R. Wan. Re-Store: A system for compressing, browsing, and searching large documents. In *Proc. 8th International Symposium on String Processing and Information Retrieval*, pages 162–174, San Rafael, Chile, November 2001. IEEE Computer Society Press, Los Alamitos, CA. [iv]

A. Moffat and J. Zobel. Coding for compression in full-text retrieval systems. In J. A. Storer and M. Cohn, editors, *Proc. 1992 IEEE Data Compression Conference*, pages 72–81. IEEE Computer Society Press, Los Alamitos, California, March 1992. [162]

A. Moffat, J. Zobel, and N. Sharman. Text compression for dynamic document databases. *IEEE Transactions on Knowledge and Data Engineering*, 9(2):302–313, March/April 1997. [155]

J. C. Mogul, F. Douglis, A. Feldmann, and B. Krishnamurthy. Potential benefits of delta-encoding and data compression for HTTP. In M. Steenstrup, editor, *Proc. ACM SIGCOMM'97 Conference: Applications, Technologies, Architectures, and Protocols for Computer Communication*, volume 27 of *Computer Communication Review*, pages 181–194, New York, October 1997. ACM Press. See Mogul et al. [1998] for errata. [222]

J. C. Mogul, F. Douglis, A. Feldmann, and B. Krishnamurthy. Errata for 'Potential benefits of delta encoding and data compression for HTTP'. *ACM SIGCOMM Computer Communication Review*, 28(1):51–55, 1998. [222, 257]

B. Moore, editor. *The Australian Concise Oxford Dictionary.* Oxford University Press, third edition, 1997. [7]

G. Navarro, T. Kida, M. Takeda, A. Shinohara, and S. Arikawa. Faster approximate string matching over compressed text. In Storer and Cohn [2001], pages 459–468. [166]

M. Nelson and J.-L. Gailly. *The Data Compression Book.* M & T Books, second edition, 1996. [8]

C. G. Nevill-Manning and I. H. Witten. Compression and explanation using hierarchical grammars. *Computer Journal*, 40(2/3):103–116, 1997. [10, 67, 75, 89]

C. G. Nevill-Manning and I. H. Witten. On-line and off-line heuristics for inferring hierarchies of repetitions in sequences. *Proc. IEEE*, 88(11):1745–1755, November 2000. [67]

C. G. Nevill-Manning, I. H. Witten, and G. W. Paynter. Browsing in digital libraries: A phrase-based approach. In *Proc. 2nd ACM Conference on Digital Libraries*, pages 230–236, 1997. [10, 68, 204, 208, 210]

C. G. Nevill-Manning, I. H. Witten, and G. W. Paynter. Lexically-generated subject hierarchies for browsing large collections. *International Journal of Digital Libraries*, 2 (2–3):111–123, September 1999. [204]

H. F. Nielsen, J. Gettys, A. Baird-Smith, E. Prud'hommeaux, H. W. Lie, and C. Lilley. Network performance effects of HTTP/1.1, CSS1, and PNG. In M. Steenstrup, editor, *Proc. ACM SIGCOMM'97 Conference: Applications, Technologies, Architectures, and Protocols for Computer Communication*, volume 27 of *Computer Communication Review*, pages 155–166, New York, October 1997. ACM Press. [16, 222]

I. Opie and P. Opie. *The Lore and Language of Schoolchildren.* Oxford University Press, 1959. [18]

A. J. Palay and M. S. Fox. Browsing through databases. In R. N. Oddy, S. E. Robertson, C. J. van Rijsbergen, and P. W. Williams, editors, *Information Retrieval Research*, pages 310–324. Butterworth & Co. Ltd., 1981. [6]

G. W. Paynter, I. H. Witten, S. J. Cunningham, and G. Buchanan. Scalable browsing for large collections: a case study. In *Proc. 5th ACM Conference on Digital Libraries*, pages 215–223, 2000. [197, 201, 205, 210]

I. Pitman. *A History of Shorthand.* Isaac Pitman and Sons, 1891. [8]

M. F. Porter. An algorithm for suffix stripping. *Program*, 14:130–137, 1980. Reprinted in Readings in Information Retrieval, pages 313–316, 1997. Web site created for algorithm: `http://www.tartarus.org/~martin/PorterStemmer/`. [84, 241, 242]

D. Raggett, A. Le Hors, and I. Jacobs, editors. *HyperText Markup Language* – HTML*/4.01*. World Wide Web Consortium, December 1999. Available from: `http://www.w3.org/TR/html401/`. [221]

J. Rissanen. Generalised Kraft inequality and arithmetic coding. *IBM Journal of Research and Development*, 20:198–203, May 1976. [35]

J. Rissanen and G. G. Langdon, Jr. Universal modeling and coding. *IEEE Transactions on Information Theory*, 27(1):12–23, January 1981. [16, 26]

M. Rodeh, V. R. Pratt, and S. Even. Linear algorithm for data compression via string matching. *Journal of the ACM*, 28(1):16–24, January 1981. [21]

F. Rubin. Experiments in text file compression. *Communications of the ACM*, 19(11): 617–623, November 1976. [65, 66]

K. Sadakane. A fast algorithm for making suffix arrays and for Burrows-Wheeler transformation. In J. A. Storer and M. Cohn, editors, *Proc. 1998 IEEE Data Compression Conference*, pages 129–138. IEEE Computer Society Press, Los Alamitos, California, March 1998. [30]

D. Salomon. *Data Compression: The Complete Reference*. Springer-Verlag, second edition, 2000. [8, 33]

K. Sayood. *Introduction to Data Compression*. Morgan Kaufmann Publishers, 1996. [8]

R. Sedgewick. *Algorithms in C, Parts 1-4: Fundamentals, Data Structures, Sorting, Searching*. Addison-Wesley Publishing Co., third edition, 1997. [20]

J. Seward. BZIP program and documentation. Source code available from `http://sources.redhat.com/bzip2/`, 1996. [37]

J. Seward. On the performance of BWT sorting algorithms. In J. A. Storer and M. Cohn, editors, *Proc. 2000 IEEE Data Compression Conference*, pages 173–182. IEEE Computer Society Press, Los Alamitos, California, March 2000. [30, 37]

J. Seward. Space-time tradeoffs in the inverse B-W transform. In Storer and Cohn [2001], pages 439–448. [30, 37]

C. E. Shannon. A mathematical theory of communication. *The Bell System Technical Journal*, 27(3):379–423,623–656, July 1948. [14, 15, 26]

Y. Shibata, T. Kida, S. Fukamachi, M. Takeda, A. Shinohara, T. Shinohara, and S. Arikawa. Byte pair encoding: a text compression scheme that accelerates pattern matching. Technical Report DOI-TR-CS-161, Kyushu University, Department of Informatics, April 1999. [69]

D. D. Sleator and R. E. Tarjan. Self-adjusting binary search trees. *Journal of the ACM*, 32(3):652–686, July 1985. [88]

D. D. K. Sleator and D. Temperley. Parsing English with a Link Grammar. Technical Report CMU-CS-91-196, Carnegie Mellon University, School of Computer Science, October 1991. [211]

J. A. Storer. *Data Compression: Methods and Theory.* Computer Science Press, 1988. [8]

J. A. Storer and M. Cohn, editors. *Proc. 2001 IEEE Data Compression Conference*, March 2001. IEEE Computer Society Press, Los Alamitos, California. [254, 256, 257, 260, 262]

J. A. Storer and T. G. Szymanski. Data compression via textual substitution. *Journal of the ACM*, 29(4):928–951, October 1982. [18, 65]

M. Takeda, S. Miyamoto, T. Kida, A. Shinohara, S. Fukamachi, T. Shinohara, and S. Arikawa. Processing text files as is: Pattern matching over compressed texts, multibyte character texts, and semi-structured texts. In *Proc. 9th International Symposium on String Processing and Information Retrieval*, pages 170–186. Springer-Verlag, September 2002. [166, 167]

W. J. Teahan. *Modelling English Text.* PhD thesis, University of Waikato, New Zealand, 1998. [227]

P. Tischer. A modified Lempel-Ziv-Welch data compression scheme. In *Australian Computer Science Conference*, pages 262–272, Geelong, February 1987. Deakin University. [24]

L. Todd. *Cassell's Guide to Punctuation.* Cassell & Co., 2001. [7]

A. Turpin and A. Moffat. Housekeeping for prefix coding. *IEEE Transactions on Communications*, 48(4):622–628, April 2000. Source code available from: `http://www.cs.mu.oz.au/~alistair/mr_coder/`. [34, 37]

A. Turpin and W. F. Smyth. An approach to phrase selection for offline data compression. In M. Oudshoorn, editor, *Proc. 25th Australasian Computer Science Conference*, volume 24, pages 267–273. Australian Computer Society Inc., Sydney, Australia, January 2002. [66]

J. S. Vitter. Design and analysis of dynamic Huffman codes. *Journal of the ACM*, 34(4): 825–845, October 1987. [34]

N. Wacholder, D. K. Evans, and J. L. Klavans. Automatic identification and organization of index terms for interactive browsing. In E. A. Fox and C. L. Borgman, editors, *Proc. First ACM/IEEE-CS Joint Conference on Digital Libraries*, pages 126–134. ACM Press, 2001. [203]

N. Wacholder and C. Nevill-Manning. The technology of phrase browsing applications. *SIGIR Forum*, 35(1):16–20, Spring 2001. [6, 219]

R. Wan and A. Moffat. Effective compression for the web: Exploiting document linkages. In M. E. Orlowska and J. F. Roddick, editors, *Proc. 12th Australasian Database Conference*, pages 68–75. IEEE Press, February 2001a. [iv]

R. Wan and A. Moffat. Interactive phrase browsing within compressed text. In W. B. Croft, D. J. Harper, D. H. Kraft, and J. Zobel, editors, *Proc. 24th Annual International*

ACM SIGIR Conference on Research and Development in Information Retrieval, pages 410–411. ACM Press, September 2001b. [iv]

R. Wan and A. Moffat. Block merging for off-line compression. In A. Apostolico and M. Takeda, editors, *Proc. 13th Annual Symposium on Combinatorial Pattern Matching*, pages 32–41. Springer-Verlag Berlin, July 2002. [iv]

T. A. Welch. A technique for high performance data compression. *IEEE Computer*, 17 (6):8–20, June 1984. [24]

H. E. Williams, J. Zobel, and P. Anderson. What's next? Index structures for efficient phrase querying. In J. Roddick, editor, *Proc. 10th Australasian Database Conference*, pages 141–152, Auckland, New Zealand, January 1999. [204]

A. I. Wirth and A. Moffat. Can we do without ranks in Burrows Wheeler transform compression? In Storer and Cohn [2001], pages 419–428. [29]

I. H. Witten and D. Bainbridge. *How to Build a Digital Library.* Morgan Kaufmann, 2002. [4, 6]

I. H. Witten and T. C. Bell. The zero frequency problem: Estimating the probabilities of novel events in adaptive text compression. *IEEE Transactions on Information Theory*, 37(4):1085–1094, July 1991. [25]

I. H. Witten, T. C. Bell, and C. G. Nevill. Models for compression in full-text retrieval systems. In J. A. Storer and J. H. Reif, editors, *Proc. 1991 IEEE Data Compression Conference*, pages 23–32. IEEE Computer Society Press, Los Alamitos, California, April 1991. [162]

I. H. Witten, A. Moffat, and T. C. Bell. *Managing Gigabytes.* Morgan Kaufmann, second edition, 1999. [5, 6, 8, 26, 28, 74, 201, 227]

I. H. Witten, R. M. Neal, and J. G. Cleary. Arithmetic coding for data compression. *Communications of the ACM*, 30(6):520–541, June 1987. [35]

J. G. Wolff. An algorithm for the segmentation of an artificial language analogue. *British Journal of Psychology*, 66(1):79–90, 1975. [65, 194]

J. G. Wolff. The discovery of segments in natural language. *British Journal of Psychology*, 68:97–106, 1977. [65]

J. G. Wolff. Language acquisition and the discovery of phrase structure. *Language and Speech*, 23(3):255–269, 1980. [205, 210]

J. Ziv and A. Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23(3):337–343, May 1977. [19]

J. Ziv and A. Lempel. Compression of individual sequences via variable rate coding. *IEEE Transactions on Information Theory*, 24(5):530–536, September 1978. [22]